

—— // VI MLRS // ——

vimlrs

v0.2.1

THE COMPANION

commands · expressions · the machine underneath

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	3
Chapter 1 — The Pitch	4
Chapter 2 — Hello, vimlrs	9
Chapter 3 — The Command Line and the Expression	14
Chapter 4 — Variables and Their Scopes	18
Chapter 5 — Types and Coercions	25
Chapter 6 — Functions	32
Chapter 7 — Lists, Dictionaries, and Blobs	39
Chapter 8 — Funcrefs, Lambdas, and Closures	45
Chapter 9 — Strings, printf, and Text	51
Chapter 10 — Pattern Matching and the Regex Engine	56
Chapter 11 — Control Flow and Exceptions	61
Chapter 12 — Metaprogramming: :execute, eval, and :source	67
Chapter 13 — What “Faithful Port” Actually Means	71
Chapter 14 — The Machine Underneath	78
Chapter 15 — The Script Cache, AOT, and Deployment	83
Chapter 16 — JSON, MessagePack, and the Outside World	87
Chapter 17 — Tooling: LSP and DAP	92
Chapter 18 — The Standalone Contract: an Editor-Absent Vim	96
Chapter 19 — The Lexer: From Text to Tokens	101
Chapter 20 — The Parser and the eval1...eval7 Ladder	105
Chapter 21 — Lowering: From AST to fusevm Bytecode	110
Chapter 22 — The Bridge, the Disassembler, and VIML_* Builtins	116
Chapter 23 — Numbers, Floats, and the Arithmetic Port	122
Chapter 24 — The Virtual Buffer and the Ex-Command Layer	126

Chapter 25 — The Builtin Catalogue, Family by Family	133
Chapter 26 — The String Functions	137
Chapter 27 — Containers and Numbers	142
Chapter 28 — Files, Buffers, and Lines	146
Chapter 29 — System and Environment	149
Chapter 30 — The Functional Surface I: Testing, Iteration, and Introspection	152
Chapter 31 — The Functional Surface II: Cursor, Registers, Search, and the Command Line	155
Chapter 32 — The Functional Surface III: Windows, Folds, Signs, and the Editor Tail	158
Chapter 33 — Ex Commands, Options, and Special Variables	162
Chapter 34 — Autoload, Runtimepath, and the Sourcing Model	165
Chapter 35 — The Grammar of Failure: Error Codes and Aborting Semantics	169
Chapter 36 — Characters, Bytes, and Cells: the Multibyte Story	171
Chapter 37 — Four Languages, One Engine	174
Chapter 38 — The Shape of a Hot Loop	176
Chapter 39 — A Tour Through the Examples	178
Chapter 40 — One Script, End to End	200
Appendix A — Variable Scopes and Sigils	203
Appendix B — Special v: Variables	204
Appendix C — The Builtin Function Catalogue	205
Appendix D — Glossary	207
Appendix E — Ex-Command and Statement Reference	209
Appendix F — The Vim Regex Dialect	211
Appendix G — Command-Line Flags and Environment	212
Appendix H — The Source File Map	213
Appendix I — Index of Example Scripts	214

Before You Begin

This is not the reference manual. The reference enumerates every one of vimlrs's builtins, every ex-command it understands, every option it honours, and every special `v:` variable it predefines — more than nine hundred function entries alone — and you reach for it when you need to know exactly what `byteidxcomp` returns or which flag `trim()` takes third. This book is the other thing: the one you read in a chair, front to back, to understand not just *what* vimlrs does but *why* it is shaped the way it is.

vimlrs is a faithful Rust port of the Vimscript (VimL) interpreter, lifted out of Neovim's C eval engine and run as an ordinary standalone program. That sentence is true and almost completely fails to describe it. A fuller description: vimlrs is what you get when you decide that a language which has only ever existed *inside* a text editor deserves to be a real interpreter in its own right — and that the way to do it honestly is not to reinvent the semantics from the documentation, but to port the actual C source of Neovim's `eval/*` tree line by line, treating that source as the specification. Then, because a faithful semantics layer still needs a machine to run on, vimlrs does not write its own virtual machine. It lexes and parses VimL to an abstract syntax tree, lowers that tree to `fusevm` bytecode, and lets the shared `fusevm` engine — a bytecode VM with a three-tier Cranelift JIT — execute it. It is the fourth language hosted on that engine, after `zshrs` (zsh), `stryke`, and `awkrs` (awk), and it carries no VM and no JIT of its own.

The book is in five movements. The first is the language itself: the two layers of VimL — the colon-prefixed command line and the expression grammar beneath it — variables and their many scopes, the type system and its coercions, functions and their ranges and varargs, and the data shapes (lists, dictionaries, blobs, funcrefs). The second is pattern matching: Vim's idiosyncratic regular-expression dialect and the engine vimlrs built to reproduce it. The third is the faithfulness story — what “ported from the Neovim C eval engine” actually means in practice, where the port is exact, and where it deliberately diverges because there is no editor underneath. The fourth is the machine underneath: the pipeline from source to bytecode, the two JIT tiers that compile hot numeric loops to native code, the rkyv script cache, the ahead-of-time builder, the disassembler, and the LSP and DAP tooling. The fifth is a long tour through the real example scripts that ship with the crate, walked line by line, followed by reference appendices — the scopes, the special variables, the builtin catalogue, and a glossary.

Everything here is grounded in the actual repository. Every code snippet is copied from a real file — an example program, a test, or the documentation that ships with the crate — and every claim about the machine is traceable to a specific source file. vimlrs is young; its own status badge says *early · in development*, and where a number, a phase, or a behaviour is still provisional this book marks it as such rather than rounding up. vimlrs is the work of MenkeTechnologies, built faithfully on the foundation of Neovim's eval engine and the Vim authors before it, and this book is grounded in that code.

Let's start.

Chapter 1 – The Pitch

Vimscript has a strange distinction among programming languages: hundreds of thousands of people have written it, almost none of them think of themselves as having used a programming language, and until now not one of them could run a `.vim` file the way they run a `.py` or a `.sh` file. Vimscript has only ever existed embedded inside an editor. You write it in your `vimrc`, in plugins, in filetype hooks; it runs because Vim or Neovim is running and has an eval engine wired into its guts. There is no `vimscript` on your `PATH`. There never was. The language is real, mature, and used at enormous scale, and it has spent its entire life as a captive of one application.

It is worth being honest, up front, about how young vimlrs is, because the honesty is part of the project's character and this book inherits it. The status badge says “early · in development,” and the README's status table is candid about what is working, what is in progress, and what is planned — the builtin surface is “partial,” the `funcs.c` table has “~113 ported” of the full set, `:command/autocmd` are listed as planned even though the parser recognizes them, and `:const`'s immutability lock is not yet enforced. This book marks those gaps where it meets them rather than rounding up, and it does so because the project does: every place vimlrs falls short of Vim is visible in the source, in a banner, a stub, a flagged caveat, or an honest return value. The claim vimlrs makes is not that it is already a complete Vim — it is not — but that it is a faithful and growing one, with every gap legible. So when this book says a thing works, it is because a CI-run example asserts it; when it says a thing is provisional, it is because the source says so. The young-project honesty is not a disclaimer to get past; it is the same discipline that makes the faithfulness claim trustworthy, applied to the project's own completeness.

vimlrs is the argument that this is an accident of history rather than a fact of nature. The eval engine — the part of Vim that parses `1 + len(split(s))` and evaluates it, that knows what a List is and how `:let` binds a variable and what `==?` means — is separable from the editor that hosts it. You can take it out, give it a command line, a file argument, and a REPL, and run Vimscript as an ordinary program. That is exactly what vimlrs does, and the README states the claim without hedging: it is “the first compiled **standalone** VimL interpreter, run outside Vim.” The rest of this chapter is about why that is worth doing, and why the two design decisions underneath it — *port the C faithfully* and *lower to a shared bytecode machine* — are the right ones rather than merely convenient ones.

What vimlrs is

vimlrs is a standalone interpreter for VimL, written in Rust, whose language semantics are ported from Neovim's C `eval/*` tree and whose execution happens on the `fusevm` bytecode VM and its Cranelift JIT. The crate is named `zmax-viml`; the binary it produces is `vimlrs`. Unpack that slowly, because each clause carries weight.

“Standalone” means there is no editor. When you run `vimlrs script.vim`, nothing opens a buffer, nothing draws a status line, nothing enters insert mode. The program lexes the file, parses it, compiles it, and runs it, and when it is done the process exits with a status code — zero if every assertion passed, non-zero if an error was raised. This is the whole point, and it has a sharp consequence that Chapter 18 is devoted to: a great deal of Vimscript is *about* the editor — `bufnr()`, `winheight()`, `getbufvar()`, `jobstart()` — and vimlrs has to decide what those mean when there is no buffer, no window, and no event loop. The answer it gives is principled rather than evasive, but the question is unavoidable the moment you take the language out of its host.

“Ported from Neovim's C eval engine” means that the semantics are not reinvented. There is a directory, `csrc/`, that vendors the actual Neovim C source the port mirrors — `eval.c`, `eval.h`, `eval.lua`, `ex_eval.c`,

`ex_eval.h` — and the README is blunt about its status: “the C source is the spec.” When vimlrs needs to know how two values compare, it does not consult the help files and guess; it ports `typval_compare`. When it needs to divide two numbers, it ports `num_divide` and `num_modulus`, which is why integer `/` in vimlrs truncates toward zero the way Vim’s does rather than producing a float. Chapter 13 is the full account of this discipline, but the headline is that fidelity here is mechanical, not aspirational: there is a Rust module that mirrors each C file, uses the exact C identifiers, and cites the source line it came from.

“Lowered to fusevm bytecode” means vimlrs does not interpret the AST by walking it on every run. The library documentation in `lib.rs` is explicit: VimL is the “4th language frontend on fusevm, after zshrs (zsh), strykelang (stryke), and awkrs (awk),” and “like zshrs, it has **no local VM and no local JIT**: source is lexed and parsed into an AST, lowered to fusevm bytecode, and executed by fusevm.” The pipeline, drawn straight from the README, is:

```
VimL source → lexer → parser (AST) → lower to fusevm bytecode → fusevm VM +  
↳ Cranelift JIT
```

Everything to the left of “lower to fusevm bytecode” is code that lives in vimlrs. Everything to the right is the shared engine, pulled from crates.io as the `fusevm` crate with its `jit`, `jit-disk-cache`, and `aot` features turned on. The division of labour is the organizing fact of the whole project, and the next two sections are about why each half is shaped the way it is.

Why faithful, not approximate

There is an easier version of vimlrs that nobody would have to port any C to build. You could read `:help eval`, write a Vimscript-ish interpreter that gets the common cases right, and ship it. It would run FizzBuzz. It would handle most plugins’ configuration blocks. And it would be subtly, unfixably wrong in exactly the places that matter, because Vimscript’s semantics are a thicket of specific, documented, occasionally surprising behaviours that no reimplementer would reproduce by intuition.

Consider a few of the behaviours vimlrs’s own example suite pins down. The length of a string is its *byte* length, so `len('héllo')` is 6, not 5, while `strchars('héllo')` is 5 — and a composing character folds into its base under `byteidx()` but is counted separately under `byteidxcomp()`. The `printf()` implementation supports positional `%N$` arguments that may be reused and reordered, the `#` alternate form that puts `0x/0b` prefixes on hex and binary but suppresses them for a zero value, and the `%g` conversion that picks `%e` or `%f` by exponent and shows six significant digits. The `str2nr()` builtin takes an explicit base and, with a truthy third argument, ignores embedded ‘ digit separators. The `tr()` builtin errors with `E475` when a source character has no corresponding destination character. Integer literals understand octal via a leading `0`, hex via `0x`, binary via `0b`, and the explicit `0o` octal prefix — so `010` is 8 and `0129` is 129. None of these are things a from-scratch interpreter gets right by accident. They are right in vimlrs because the code that implements them was ported from the code that defines them.

The deeper reason to be faithful is that the *value* of a standalone Vimscript interpreter is proportional to how much existing Vimscript it can run unchanged. An approximation is a new language that looks like Vimscript; a faithful port is Vimscript. The compatibility target, as the engineering report puts it, is simply Neovim: “Behaviour tracks the Neovim reference, so scripts that run there are the compatibility target.” That is a target you can only hit by copying the reference, and copying the reference is only tractable because the reference is open C source that can be vendored and mirrored module for module. The faithfulness is not a virtue the project performs; it is the mechanism by which the project is possible at all.

Why lower to fusevm

The second decision — build no VM, lower to fusevm — is the one that separates vimlrs from the obvious shape of a language interpreter. The obvious shape is a tree-walker: parse the source to an AST and evaluate the AST by recursively walking it. Neovim’s own eval engine is not even that; as the AST module’s header notes, “Neovim’s `eval.c` parses and evaluates in one pass over the source string; there is no AST.” vimlrs

could have written a tree-walker over its own new AST and stopped there. It does not, and the reason is the same reason its sibling languages do not.

`fusevm` is a language-agnostic bytecode virtual machine with a three-tier Cranelift JIT, and it is shared. The same engine runs `zsh` under `zshrs`, runs `stryke`, runs `awk` under `awkrs`, and now runs VimL under `vimlrs`. The engineering report draws out what that sharing buys: “JIT and VM improvements land once and benefit `zshrs`, `stryke`, `awkrs`, and `vimlrs` together.” A language that hosts itself on `fusevm` inherits, for free, every optimization anyone makes to `fusevm` — a faster dispatch loop, a better register allocator, a new JIT tier — without writing any of it. The `jit-disk-cache` feature persists compiled native code across runs, so a warm start skips recompilation. The bytecode is architecture-portable, and the Cranelift JIT covers x86-64 and aarch64 on both macOS and Linux. `vimlrs` gets all of this by lowering to a target it does not own.

The architecture portability is worth dwelling on, because it is a property `vimlrs` gets entirely for free and would have been expensive to build. The bytecode the compiler emits is the same on every platform — `Op::Add`, `Op::GetSlot`, `CallBuiltin`, the rest — so a compiled program, or a cached `CompiledProgram`, is not tied to one CPU. The platform-specific step is the JIT, and that is Cranelift’s job, not `vimlrs`’s: Cranelift emits x86-64 machine code on an Intel or AMD machine and aarch64 on an ARM machine, including Apple Silicon, on both macOS and Linux. So the same `.vim` file runs, and its hot loops JIT-compile to native code, on a Linux x86-64 server and an Apple Silicon laptop alike, with no per-platform code in `vimlrs`. The disk JIT cache is keyed appropriately so a cache built on one architecture is not used on another. This breadth — two architectures, two operating systems, all four combinations — is exactly the kind of thing a small project could not afford to build and maintain itself, and `vimlrs` has it because Cranelift and `fusevm` have it. It is the clearest single illustration of the borrow-the-machine thesis: cross-platform native-code generation, the hardest part of a JIT, arrives as a property of the engine `vimlrs` lowers to, leaving `vimlrs` to write only the platform-independent frontend.

The payoff is not theoretical. The README’s status table reports that a 20-million-iteration loop “runs ~15–100× faster with the JIT” than under the interpreter baseline, and that you can watch it happen: `VIMLR5_JIT_STATS=1 vimlrs script.vim` reports how many loop traces were compiled, and `VIMLR5_NO_JIT=1` forces the slow path so you can compare. A tight numeric `while` loop with provably-Number locals trace-JIT-compiles to machine code; an integer expression block-JIT-compiles; float accumulators, native modulo, native bitwise operations, and numeric ternaries all stay on the fast path. Chapter 14 is the full anatomy. The point for now is that a scripting language with Vimscript’s humble surface posts those numbers not because `vimlrs` is clever about loops but because `fusevm` is, and `vimlrs` had the discipline to lower to it rather than to reinvent it. The decoupling of terseness from speed is the conceptual point worth holding onto: Vimscript’s *surface* is humble — line-oriented commands, scope sigils, a small set of types — and that surface is unchanged by the JIT, which operates entirely on the bytecode beneath it. A user writing `while i < n` sees no machinery; the machinery is in how that loop lowers to slots and rotates to a backedge and traces to native code. So the language can be exactly as simple as Vimscript has always been while running, in its hot paths, as fast as compiled code — the two properties live at different layers and do not constrain each other. This is why the speed claims and the faithfulness claims are not in tension: faithfulness is a property of the semantics (ported from C), speed is a property of the machine (borrowed from `fusevm`), and the terse surface sits on top of both, unaffected by either. A humble language, a faithful core, and a fast machine, stacked so that each is free to be the best version of itself.

The four siblings and what sharing buys

`vimlrs` is the fourth language hosted on `fusevm`, and the company it keeps is worth naming because the family is the whole argument for the lower-to-`fusevm` decision. The README lists the siblings: `zshrs` hosts `zsh`, `stryke` is an original Perl-lineage language, `awkrs` hosts `awk`, and a fifth, `elisp`, hosts Emacs Lisp. All five lex and parse their own surface syntax, lower to `fusevm` bytecode, and run on the shared engine; none carries a VM or a JIT of its own. The architectural kinship runs deep — `vimlrs`’s `compile_viml.rs` is “the `vimlrs` analogue of `zshrs`’s `compile_zsh.rs`,” its `fusevm_bridge.rs` “the `vimlrs` analogue of `zshrs`’s `fusevm_bridge.rs`,” its `aot.rs` “the same design as `zshrs`’s `aot.rs`,” its `lsp.rs` “ported from `awkrs`’s `lsp.rs`,” and its `dap.rs` mirrors “`zshrs`’s `dap.rs` architecture.” Each new language is built by following the pattern the

previous ones established, which is why a port of a language as different from zsh as Vimscript can reuse the shape of zsh's frontend wholesale.

The sharing is not just code-reuse at authoring time; it is a permanent leverage on performance. Because all five lower to the same bytecode and run on the same VM, "JIT and VM improvements land once and benefit zshrs, stryke, awkrs, and vimlrs together." A faster dispatch loop, a better register allocator, a new JIT tier, an improvement to the disk cache — any of these, written once in fusevm, appears in every hosted language for free. vimlrs did not write the three-tier Cranelift JIT, the architecture-portable bytecode, the `jit-disk-cache` native-code persistence, or the AOT object emitter; it inherited all of them by lowering to a target it does not own. The cost is the bridge — the `VIML_*` builtin layer and the `typval_T ↔ fusevm::Value` marshalling — but that cost is paid once and buys an entire mature machine, and the machine keeps getting better without vimlrs touching it. This is the economic core of the whole project: own the thing that is unique to your language (the faithful semantics) and borrow the thing that is not (the execution machine), so that your effort goes entirely into the part only you can write.

What standalone actually unlocks

It is worth being concrete about what taking Vimscript out of the editor actually buys, because "runs standalone" sounds like a technical curiosity and it is in fact a change in what the language *is for*. As long as Vimscript could only run inside Vim, it could only do one job: configure and extend the editor. Every Vimscript program was, by definition, a thing that made Vim behave differently, because Vim was the only context it could run in. This is an enormous limitation hiding in plain sight — a Turing-complete language with a large standard library, confined to a single application, unable to be a script in the ordinary sense of a program you run to get a job done. Standalone vimlrs removes the confinement, and the jobs that open up are exactly the jobs scripting languages do: read a file, transform it, write it out; count something in a log; shell out to a command and process its output; serialize some data to JSON; time a computation. A Vimscript program can now be a `#!/usr/bin/env vimlrs` script in a pipeline, doing the work you would otherwise reach for awk, sed, or a quick Python script to do. The language did not change — it is the same Vimscript, same syntax, same builtins — but its *role* changed, from "the configuration language of one editor" to "a general-purpose scripting language that happens to have grown up inside an editor."

The reason this is worth doing, rather than just an interesting thing one can do, is that there is a large population of people who already know Vimscript — who have written it for years in their vimrc and their plugins — and who, until now, could not use that knowledge for anything but configuring Vim. Standalone vimlrs lets that knowledge transfer: the byte-versus-character rules you learned writing a plugin, the regex dialect you internalized, the list and dict idioms you use daily, all become applicable to ordinary scripting tasks. A Vim power user can now write a text-processing one-liner in the language they already think in, rather than context-switching to a different language for tasks that leave the editor. This is the same bet every "X, but standalone" project makes — that a captive language has value waiting to be released the moment it can run outside its host — and the bet is especially strong for Vimscript because its host is so widely used and its captivity so total. There was never a `vimscript` on anyone's PATH; the language was real, mature, used at enormous scale, and entirely unable to run as a program. vimlrs is the argument that this was an accident of history, not a fact of nature, and the standalone binary is that argument made concrete — Vimscript, finally, as a thing you can run.

The shape of the rest of the book

The two decisions — faithful semantics, shared machine — meet at a seam, and the seam is where vimlrs's architecture actually lives. On one side is the `ported` zone: strict one-to-one ports of the Neovim C, with exact names and source citations, the place where VimL's meaning is defined. On the other side is a set of "carve-out" modules that have no C counterpart — the lexer, the parser, the AST, the bytecode compiler, the fusevm bridge, the regex engine, the JSON parser — because Neovim's string-walking interpreter has no lexer, no AST, and no bytecode to port. Every one of those files opens with a banner declaring itself an "EXTENSION — NO `csrc`/ COUNTERPART," so that no synthesis code can ever masquerade as a faithful

port. Chapter 13 walks that boundary in detail; it is mentioned here because it is the single most important structural idea in the codebase, and almost everything else in this book is a consequence of it.

From here the book descends into specifics. The next chapter gets you running: the four ways to invoke vimlrs, your first script, and the texture of the language at its smallest scale. After that we climb through the language proper — the command line, the scopes, the types, the functions, the data — then through pattern matching, then through the machine, and finally through the examples. By the end you should be able to read any `.vim` file vimlrs can run and know not only what it does but why vimlrs runs it the way it does.

Chapter 2 – Hello, vimlrs

The first program in most language books prints a greeting and explains nothing. We will print a greeting and read it closely, because even at this scale vimlrs has decisions worth seeing. But first: how do you run anything at all? vimlrs has four modes, and they map onto the four shapes a Vimscript workload actually takes.

Four ways to run

The command-line driver, defined in `cli.rs`, documents its own modes at the top of the file:

```
vimlrs --expr 'EXPR'      # evaluate one expression, print its value
vimlrs --cmd 'echo ...'   # run one ex command line
vimlrs FILE.vim           # source a script file
vimlrs                   # no args – read-eval-print loop on stdin
```

The `--expr` form (short flag `-e`) evaluates a single expression and prints its value the way `:echo` would. `vimlrs -e '1 + 1'` prints `2`; `vimlrs -e 'len(split("a b c"))'` prints `3`. The driver runs the expression, and — exactly as `:echo` does — if a runtime error fires during evaluation it suppresses the spurious result and lets only the error reach `stderr`, because a failed evaluation has no valid value to show.

The `--cmd` form (short flag `-c`) runs one ex command line: `vimlrs -c 'echo 1 + 1'`. The difference between `-e` and `-c` is the difference between the two layers of the language, which is the whole subject of the next chapter. An expression is a thing that has a value; an ex command is a thing that does something. `1 + 1` is an expression; `echo 1 + 1` is the `:echo` command applied to that expression.

The file form is the one you will use most: `vimlrs script.vim`. The driver seeds the argument list — the standalone counterpart of Vim’s file arglist, which `argv()` and `argc()` read — with the file arguments, then sources each file in order, routing through the bytecode cache so that the second run of a script skips lexing, parsing, and compiling entirely. More on that cache in Chapter 15.

The no-argument form drops you into a REPL. Each line you type is one statement; a bare expression prints its value, while an `:echo` or `:let` runs for effect. The prompt is `vimlrs>`, and the loop is the obvious one — read a line, evaluate it, print a value if there was one, prompt again. It is the fastest way to answer “what does this actually do?” for a single expression, and most of the small snippets in this book can be pasted straight into it.

The four input modes are not arbitrary; they map onto the four shapes a Vimscript workload actually takes, which is why the driver offers exactly these. The `--expr` form is for the one-off computation — “what does `len(split(s))` give for this `s`?” — the role a calculator or a quick `python -c` plays, and it prints the value the way `:echo` would. The `--cmd` form is for the one-off *action* — run one ex command for its effect — the difference being the command/expression seam of the next chapter. The file form is the workhorse, sourcing one or more scripts in order, seeding the arglist so `argv()/argc()` see the files, and routing through the cache so warm runs are fast. And the REPL is for exploration, the fastest way to answer “what does this do?” interactively. The four cover the workload space — compute once, act once, run a program, explore — and a fifth shape, the *pipeline filter* (read `stdin`, transform, write `stdout`), is served not by a dedicated mode but by the combination of the file mode with the `system/readfile/buffer` machinery, since a vimlrs script can read its input and write its output like any shell tool. The driver’s job is to recognize which of the four shapes you want from the arguments and route accordingly, and the recognition is simple because the four shapes are syntactically distinct: a flag, a flag, a filename, or nothing.

Beyond these four there are mode flags that change what the driver does entirely rather than how it reads input: `--lsp` and `--dap` start the language-server and debug-adapter servers (Chapter 17), `--disasm` prints the

fusevm bytecode listing before running (Chapter 14), `--build` bakes scripts into a self-contained executable (Chapter 15), and `--clear-cache` deletes the rkyv script cache. They are all introduced where they belong; for now, the four input modes are enough.

The REPL up close

The no-argument REPL repays a closer look, because it is the fastest way to learn the language and because its behaviour encodes the command/expression split the next chapter is about. You launch it by running `vimlrs` with no file and no flags, and it prompts `vimlrs>` and reads one line at a time. The loop is the obvious read-eval- print: each line is parsed as one statement, and if that statement is a bare expression the loop prints its value the way `:echo` would, while an `:echo`, a `:let`, or a `:call` runs for its effect. So at the prompt `1 + 1` prints `2`, `len(split('a b c'))` prints `3`, `let g:x = 5` binds silently, and `echo g:x` prints `5`. State persists across lines within a session — a function you `:function`-define on one line is callable on the next, a global you set stays set — because the REPL threads one engine state through the whole session, exactly as sourcing a file does. This is what makes the REPL the right tool for answering “what does this actually do?” about a single expression, and it is why nearly every small snippet in this book can be pasted straight into it and checked.

The driver and the exit code

The command-line driver in `cli.rs` is small but it does three jobs worth naming. It parses the mode (which of `-e`, `-c`, a file, the REPL, or one of the server/build flags), it seeds the argument list so `argv()/argc()` see the file arguments, and it sets the process exit code from the run. That last job is the load-bearing one for the example suite: the driver tracks whether Vim’s `did_emsg` flag was set during the run — whether any error was raised — and exits non-zero if so. On the file path it routes each script through the bytecode cache (Chapter 15) before running it, so the second run of a script skips lexing, parsing, and compiling. On the `-e` path it compiles a single expression and prints its value, suppressing the value if evaluation errored so that only the error reaches stderr. The driver is deliberately thin — it is a mode selector and an exit-code setter wrapped around the real machinery — but its exit-code discipline is exactly what turns every self-testing example into a CI-checkable regression test, because a `:throw` that nothing catches sets `did_emsg`, and `did_emsg` set means a non-zero exit, and a non-zero exit means CI fails. The whole self-testing tower of Chapter 11 rests on this one small behaviour of the driver.

The first script

Here is a complete, runnable `vimlrs` program:

```
echo 'hello, world'
```

There is no `main`, no preamble, no import. The line is one ex command, `:echo`, applied to one string expression. (The leading colon is optional at the start of a line; `echo 'hello, world'` and `:echo 'hello, world'` are the same. Vimscript writes the colon in documentation to mark a command, and omits it in scripts.) The optional colon is a small convention with a real history: it descends from the `ex` command line, where you literally typed `:` to enter command mode, so every command was written with its colon. Vim’s documentation kept the colon as a marker — `:echo`, `:let`, `:substitute` — to signal “this is a command, type it at the `:` prompt or in a script,” while scripts themselves drop it because in a script every line is already in command context. `vimlrs` honours both: `echo 'hi'` and `:echo 'hi'` parse identically, the leading colon stripped if present. This is why this book writes builtins and commands with their colons when naming them (`:echo`, `:let`) but shows them without colons in the example snippets — the colon is the documentation convention, its absence the script convention, and the two refer to the same command. Run it with `vimlrs hello.vim` and it prints `hello, world` followed by a newline. `:echo` evaluates each of its space-separated arguments

and displays them; its sibling `:echon` does the same without the trailing newline, and `:echomsg` additionally saves the message in the history.

Strings come in two flavours, and the difference is not cosmetic. A single-quoted string is almost literal — the only escape is `' '` for an embedded single quote — so `'a'b'` is the three-character string `a'b`. A double-quoted string honours backslash escapes: `"\t"` is a tab, `"\n"` a newline, `"\x08"` a backspace. This matters enormously for regular expressions, which are riddled with backslashes: a pattern is almost always written in single quotes so that `'\d\+'` reaches the regex engine as the literal backslash-d-backslash-plus it needs, rather than being mangled by string-escape processing first. Every regex example in the crate uses single quotes for exactly this reason, and the `regex_verymagic.vim` example says so in a comment: “regex patterns use single quotes so `'\v'` is not a string escape.”

Comments and line structure

A comment runs from a double-quote character to the end of the line — but only where a double-quote cannot begin a string, which is the source of one of Vimscript’s oldest gotchas and a thing vimlrs’s lexer has to get right. In the example files, every header block is a run of `"`-prefixed comment lines:

```
" fizzbuzz.vim - classic FizzBuzz, with embedded unit tests.
"
" Demonstrates: :function/:return, :if/:elseif/:else, integer % (native modulo),
" and the built-in assert framework (assert_equal/v:errors).
```

The comment-versus-string ambiguity is one of Vimscript’s oldest traps, and it is worth seeing why it is hard. A `"` can begin a string or a comment, and which one depends on whether a string is legal at that position. After a complete command, `echo 'hi' " a comment` treats the second `"` as a comment because no string can begin there; but `let x = "a string"` treats the `"` as a string because a value is expected. The classic gotcha is a `"` comment after a command that *could* take a string argument — in some positions Vim guesses wrong, which is why seasoned Vimscript authors are wary of trailing comments. vimlrs’s lexer and command splitter share the same understanding of where a `"` can start a string, so they agree on where a comment begins, and the example files lean on `"`-comments freely in their header blocks and between assertions, trusting that the splitter strips them correctly. The line-comment rule is simple to state — a `"` that cannot begin a string runs to end of line — but getting “cannot begin a string” right at every position is the fussy part, and it is the same machinery that decides whether a bar is a separator (Chapter 3), which is why the two concerns live together in the splitter.

A statement ends at the newline. There is no statement terminator — no semicolon — because Vimscript is a line-oriented language descended from the colon command line of `ex`, where each line you typed was one command. Two constructs bend this. The first is the line-continuation backslash: a line whose first non-whitespace character is `\` is glued to the end of the previous line, which is how the example suite writes multi-line list and dict literals:

```
let config = {
  \ 'name': 'vimlrs',
  \ 'version': [0, 1, 0],
  \ 'features': ['jit', 'lsp', 'dap'],
  \ 'standalone': v:true,
  \ }
```

The second is the bar, `|`, which separates two commands that share a line — `let l = [1] | echo l` runs two statements on one line. The bar is genuinely tricky because it is also the regex alternation operator, part of `| |`, and a legal character inside a string, and vimlrs’s lexer is “strings/`| |`/`\`/comment- aware” so that it splits commands only where a bar really is a separator. The bar also drives Vimscript’s one-line block syntax, where an `if`, `while`, or `for` opener and its body and terminator all share a line; Chapter 3 takes that apart.

What “ran successfully” means

Run a vimlrs script and the process exits with a status code, and that code is load-bearing in a way worth establishing now because the entire example suite depends on it. The driver tracks whether any error was raised — whether Vim’s `did_emsg` flag got set during the run — and exits non-zero if so. On top of that mechanism the examples build a convention: every example file is a self-test. It runs assertions with the built-in `assert_equal` / `assert_true` / `assert_match` family, each of which appends a message to the special list `v:errors` on failure rather than aborting, and then ends with an epilogue that inspects `v:errors` and throws if it is non-empty:

```
if !empty(v:errors)
  for e in v:errors
    echo 'FAIL:' e
  endfor
  throw len(v:errors) . ' assertion(s) failed'
endif
echo 'OK: fizzbuzz assertions passed'
```

A `:throw` that nothing catches propagates to the top, sets the error flag, and the process exits non-zero. This is not incidental: the crate’s test harness, `tests/examples.rs`, runs every `examples/*.vim` file in CI and treats a non-zero exit as a failure, so the examples are simultaneously documentation and a regression suite. When this book tours them in the final chapters, remember that each one is a program that asserts its own correctness and that CI checks every day. That is the strongest possible guarantee that the snippets here are real: they are not illustrations of how vimlrs might behave, they are tests of how it does.

Why self-testing examples are the right design

It is worth dwelling on why the examples are structured as self-tests rather than as plain demonstrations, because the choice shapes everything about how this book can use them and it is a genuinely good idea worth understanding on its own terms. A conventional example program shows you what a feature looks like: here is how you write a closure, here is how you call a builtin. It is documentation, and like all documentation it can rot — the language changes, the example does not, and one day the example shows syntax that no longer works or behaviour that no longer holds, and nobody notices because nothing checks it. A self-testing example is different in kind. It does not merely show what a feature looks like; it *asserts what the feature produces*, and it is wired into CI so that if the assertion ever fails, the build goes red. This converts the example from documentation that can rot into a test that cannot, because the moment its assertions stop matching the implementation, CI catches it. The example is documentation and regression test at once, and the two roles reinforce each other: the documentation stays accurate because the test enforces it, and the test stays readable because it is also documentation.

The consequence for this book is direct and worth making explicit, because it is the reason the book can quote so freely. Every code snippet in these pages is copied from an example or a test, and every example asserts its own output and is checked in CI. So when this book says `len('héllo')` is 6 and quotes the assertion `call assert_equal(6, len('héllo'))`, that is not the book claiming a behaviour and hoping it is right — it is the book quoting a test that runs every day and passes. The distinction matters because most technical books illustrate intended behaviour and occasionally get it wrong, drifting from the implementation as both evolve; this book quotes verified behaviour, and the verification is not the book’s own but the project’s CI. The self-testing convention is what makes the difference between “here is how vimlrs is supposed to work” and “here is a test that vimlrs does work, which you can run yourself.” When the later chapters tour the examples, they are touring a regression suite, and the confidence the book places in each snippet is borrowed directly from the confidence CI places in each example. That borrowing is only possible because the examples were built to assert rather than merely to show, and it is the reason the book can ground every claim in a real file rather than in the author’s belief about the implementation.

The epilogue pattern, dissected

The little epilogue every example shares — inspect `v:errors`, print each failure, throw if non-empty — is worth dissecting once, because it is a small tower of three ported mechanisms working together and it is the load-bearing structure under the whole self-testing convention. The first mechanism is `v:errors` as an ordinary list. The assert builtins do not throw on failure; they *append* a descriptive message to `v:errors`, which means a single example can run a hundred assertions and, instead of dying at the first failure, accumulate all hundred results, so that a run reports every failure at once rather than only the first. This is Vim's actual testing convention, and it is why an example can be a dense block of assertions rather than a sequence of guarded checks. The second mechanism is `:throw` as escalation. The epilogue tests whether `v:errors` is non-empty and, if it is, throws — converting the accumulated, non-fatal failures into a single fatal exception that nothing catches. The third mechanism is the exit code. An uncaught `:throw` propagates to the top, sets Vim's `did_emsg` flag, and the CLI driver, reading that flag at the end of the run, exits non-zero. So the chain is: asserts append to a list (non-fatal, accumulating), the epilogue throws if the list is non-empty (escalating to fatal), and the uncaught throw produces a non-zero exit (which CI reads). Three ported pieces — `v:errors` as a list, `:throw` as escalation, `did_emsg` as the exit trigger — compose into a testing harness that needs no test framework beyond the language itself. That a language can be its own test runner this cleanly, with the assert family appending and the exit code reporting, is a small elegance, and it is the mechanism that makes every `examples/*.vim` file simultaneously a program, its own test, and a CI check.

When an expression errors

There is a subtlety in how `:echo` behaves when its argument raises an error, and `error_output.vim` pins it down because it is exactly the kind of thing a careless implementation gets wrong. When a runtime error fires while evaluating an expression, the command is aborted and *no fallback value is produced* — the error reaches stderr and nothing else does. The example asserts that the canonical error-raising expressions raise cleanly:

```
call assert_fails('echo [1,2,3][10]', 'E684')
call assert_fails("echo printf('%d', 3.7)", 'E805')
```

An out-of-range list index is `E684`, and a float passed where `%d` wants an integer is `E805`. The header explains the behaviour that matters at the CLI: “echo `[1,2,3][10]` prints only the E684 message — never a trailing `v:null`; echo `printf('%d', 3.7)` prints only E805, never a trailing -1.” A naive interpreter might catch the error, substitute a default value, and print *that* alongside the message — producing spurious output. vimlrs follows `eval.c/ex_docmd.c`: a failed evaluation has no valid value, so the command that would have displayed it is aborted entirely. This is the same `did_emsg` discipline Chapter 2 opened with, seen from the expression side — an error does not just get reported, it stops the command that raised it, which is why the `-e` driver suppresses the result when evaluation fails and why the assert framework's `assert_fails` can catch an error cleanly without any stray value leaking out.

Chapter 3 – The Command Line and the Expression

Vimscript is two languages stacked on top of each other, and almost every confusion a newcomer has comes from not seeing the seam. The bottom layer is the *expression* language: things that have values — `1 + 1`, `len(x)`, `a ? b : c`, `[1, 2, 3]`. The top layer is the *command* language: the colon commands, the ex-commands, the things that do work — `:let`, `:call`, `:echo`, `:if`, `:for`, `:execute`. The command layer drives; the expression layer computes. vimlrs mirrors this split precisely in its architecture, and seeing the split in the code is the fastest way to internalize it.

Two grammars, two entry points

In vimlrs's AST module (`viml_ast.rs`) there are two top-level types, and they are the two layers made concrete. `Expr` is the expression language: its variants are `Number`, `Float`, `Str`, `List`, `Dict`, `Lambda`, `Var`, `Option`, `Env`, `Register`, `Unary`, `Arith`, `Compare`, `And`, `Or`, `Ternary`, `Coalesce`, `Index`, `Slice`, `Member`, `Call`, `CallExpr`, and `Method`. `Stmt` is the command language: `Echo`, `Let`, `Call`, `If`, `While`, `For`, `Function`, `Try`, `Throw`, `Execute`, `Set`, `Source`, `Unlet`, and a dozen more. Every statement is “one ex-command's worth of work,” as the source comment puts it, and most statements *contain* expressions — an `Echo` holds a `Vec<Expr>`, a `Let` holds a target and an `Expr`, an `If` holds `(Expr, Vec<Stmt>)` arms.

The two CLI flags from the last chapter, `-e` and `-c`, are the two entry points into these two grammars. `-e` parses and evaluates an `Expr` and prints its value. `-c` parses and runs a `Stmt`. This is why `vimlrs -e 'echo 1'` is wrong (`echo 1` is not an expression) and `vimlrs -c 'echo 1'` is right, while `vimlrs -e '1 + 1'` is right and `vimlrs -c '1 + 1'` runs a bare expression statement for its (nonexistent) effect. The flags are not two ways to do the same thing; they are doors into two different languages.

The expression precedence ladder

Vimscript's expression grammar is defined in Neovim's C by a cascade of functions named `eval1` through `eval7`, each handling one precedence level and deferring to the next. vimlrs's parser reproduces this ladder exactly — the AST header notes its shape is “dictated by the `eval1...eval7` precedence ladder so the compiler can lower it to fusevm bytecode” — and knowing the rungs tells you how every expression binds. From loosest to tightest:

- `eval1` — the ternary `cond ? then : else` and the falsy-coalesce `expr ?? expr`. These are the loosest operators; everything else binds tighter.
- `eval2` — logical OR, `||`. Short-circuits, yields `0` or `1`.
- `eval3` — logical AND, `&&`. Short-circuits, yields `0` or `1`.
- `eval4` — comparison: `==`, `!=`, `>`, `>=`, `<`, `<=`, the regex matches `==~` and `!~`, and the identity operators `is` and `isnot`. Crucially, this level carries a *case flag*: a comparison may be suffixed `#` to force case sensitivity or `?` to force case insensitivity, so `==#`, `==?`, `==~#`, `==~?` are all distinct operators, and the bare form follows the `'ignorecase'` option.
- `eval5` — addition `+`, subtraction `-`, and string concatenation `.` / `...`
- `eval6` — multiplication `*`, division `/`, and modulo `%`.

- `eval7` — the unary leaders `!`, `-`, `+`, and the primary atoms: literals, variables, function calls, list and dict literals, parenthesized expressions, and the postfix operators that bind tightest of all — indexing `[i]`, slicing `[a:b]`, member access `.key`, function call `(args)`, and the method arrow `->`.

The comparison level's case flag is the rung most worth dwelling on, because it is pure Vimscript and the `compare.vim` example exercises the whole set:

```
call assert_equal(1, 'ABC' ==? 'abc')
call assert_equal(0, 'ABC' ==# 'abc')
call assert_equal(1, 'abc' ==# 'abc')
call assert_equal(1, 'FooBar' =~? 'foobar')
call assert_equal(0, 'FooBar' =~# 'foobar')
```

`==?` is case-insensitive equality, `==#` is case-sensitive, and a bare `==` would consult `'ignorecase'`. The same three-way choice applies to the regex match `=~`. vimlrs carries the flag from the lexer (`CmpOp` plus `CaseFlag`) through the AST's `Compare` node into the bytecode, so the distinction survives all the way to the comparison the ported `typval_compare` ultimately performs. The identity operators are at the same level: `is` and `isnot` test whether two container values are the *same* List or Dict, not whether they are equal — `compare.vim` shows `a is b` true when `b` was assigned from `a`, but `a isnot c` even though `a == c`, because `c` is an independent list with equal contents.

The unary leaders and the postfix tail

At `eval7`, before the primary atom, come the unary leaders: `!` for logical not, `-` for numeric negation, `+` for numeric coercion. They are represented in the AST as `Unary { op, expr }` with `UnaryOp`: `Not`, `Neg`, or `Plus`. After the atom come the postfix operators, which chain: `d.items[0]` is a member access whose base is `a...` no — it is an index whose base is a member access whose base is `d`. The postfix tail binds left to right and tightest, so `base[index]`, `base[from:to]`, `base.key`, `name(args)`, and `base->name(args)` compose naturally. The method arrow is sugar with a precise meaning: `base->name(args)` calls `name` with `base` inserted as its first argument, so `split(s)->join('-')` is `join(split(s), '-')`. The AST records it as a `Method { base, name, args }` node rather than rewriting it to a `Call`, because the receiver is evaluated once and the spelling carries intent. The method arrow is the readability counterpart to the lambda: where the lambda threads a transformation into a higher-order call, the arrow threads a value through a sequence of calls left to right, in execution order. A chain like `getline(1)->split()->map({i, v -> trim(v)})->join(',')` reads as “get the line, split it, trim each piece, join with commas” — top to bottom, in the order the operations happen — whereas the equivalent nested-call form `join(map(split(getline(1)), {i, v -> trim(v)}), ',')` reads inside-out, the first operation buried deepest. For a multi-stage transformation the arrow is dramatically clearer, which is why it has become idiomatic in Neovim Vimscript, and vimlrs supports it natively as the `Method` node. The receiver being evaluated once matters for correctness as well as readability: `expensive()->f()->g()` calls `expensive()` once and threads its result, where a naive desugaring to `g(f(expensive()))` also evaluates it once but loses the left-to-right spelling. vimlrs keeps the `Method` node distinct precisely to preserve that spelling through to the bytecode, so the arrow is not silently rewritten into a nested call but lowered as the threading operation it is.

The postfix operators chain, and the chaining is left-to-right and tightest-binding, which is what lets a deeply nested access read naturally. An expression like `user.langs[0]` is parsed as an index whose base is a member read whose base is the variable `user` — the `.langs` member read applied first, then the `[0]` index applied to its result. A longer chain like `d.items()[0].name` composes the same way: a member read, a call, an index, a member read, each applied to the result of the one before. The method arrow `->` participates in the chain too, so `split(s)->join('-')` is a `Method` whose base is the `split(s)` call, and `getline(1)->split()->reverse()` threads a value through three calls left to right. Because the postfix tail binds tighter than any binary operator, `a + b.c` is `a + (b.c)`, not `(a + b).c` — the member read attaches to `b` before the addition. This tight, left-associative, chaining postfix is what makes Vimscript's data access read in the order it executes, and the parser's `postfix` routine builds it by looping: it parses a primary atom and then, as long as the next token opens a postfix operator, wraps the result in the corresponding node, so the chain grows outward one operator at a time.

One subtlety the lexer handles here separates `d.key` (member access) from `a . b` (concatenation): both use the dot, and the disambiguation is adjacency. The token records its byte span precisely so that “adjacency checks like `d.key` member access vs `a . b` concatenation” can be made — a dot touching its operands on both sides is member access, a dot with spaces around it is concatenation. This is the kind of fussy, real detail that a faithful frontend has to get right and that you never think about until it is wrong.

Statements and the bar

The command layer is flatter than the expression layer — statements do not have precedence, they run in sequence — but it has its own structure worth naming. A statement is one ex-command. Most lines hold one statement. The bar `|` separates two statements on a line, and the example `oneline.vim` leans on it:

```
let total = 0 | for n in [1, 2, 3, 4] | let total += n | endfor
```

That is four statements on one line: a `:let`, then a `:for` whose body is another `:let` and which closes with `:endfor`. The bar is doing two different jobs here — separating the leading `let` from the `for`, and separating the `for` header from its body and terminator — and both are the “one-line block” syntax. A block opener (`if`, `while`, `for`) and its body and closer can all share a line when the parts are bar-separated:

```
if 1 | let g:msg = 'yes' | endif
if x > 3 | let g:size = 'big' | else | let g:size = 'small' | endif
for n in range(1, 6) | if n % 2 == 0 | call add(evens, n) | endif | endfor
```

The last line nests a one-line `if` inside a one-line `for`, all on one physical line, which the README lists as a supported case: one-line block bars work “incl. after a leaf command.” The reason this is hard, and the reason vimlrs’s status table calls it out specifically, is that the bar’s meaning is context-dependent — inside a string it is a character, in `||` it is part of an operator, after a backslash in a regex it is alternation — so the command splitter has to be aware of all of those to know where one statement ends and the next begins. When you write `let l = [1] | echo l` and it Just Works, that awareness is why.

The bar, the comment, and the four exceptions

The bar `|` is the most context-dependent character in Vimscript’s surface, and the command splitter that handles it (Chapter 20) is worth understanding from the script author’s side, because knowing the four cases where a bar is *not* a separator explains every one-liner that works. The splitter, mirroring Vim’s `do_one_cmd`, leaves a bar alone in four situations: inside a string (`echo 'a | b'` echoes the literal `a | b`), as part of the `||` operator (`if x || y` is one condition), when backslash-escaped in a pattern (`\|` is regex alternation), and after a `"` that begins a line comment (everything after the comment is text). Everywhere else, a bar separates two commands. This is why `let l = [1] | echo l` is two statements but `let s = 'a|b'` is one, and why the splitter has to track quote state as it scans — it cannot decide whether a given bar is a separator without knowing whether it is inside a string. The comment interaction is the subtlest: a `"` can begin a string or a comment depending on where it appears, so the splitter has to apply the same “can a double-quote start a string here?” logic the lexer uses, which is why the two pieces of bar-handling (splitting in the parser, string-lexing in the lexer) share the same understanding of where a `"` is a comment.

One-line blocks in practice

The one-line block syntax that the bar enables is worth its own treatment, because it is everywhere in real `.vim` code and it is where the bar’s command-separator role does the most work. A block opener — `if`, `while`, `for` — its body, and its terminator can all share one physical line when the parts are bar-separated, and the parser’s source splitter turns each bar-delimited piece into its own logical line before block parsing. So `if 1 | let g:msg = 'yes' | endif` becomes three logical lines — the `if`, the `let`, the `endif` — parsed into one `If` statement. The `oneline.vim` example pushes this to nesting: `for n in range(1, 6) | if`

`n % 2 == 0 | call add(evens, n) | endif | endfor` nests a one-line `if` inside a one-line `for`, all on one physical line, with five bars each playing the separator role. The reason this is hard, and the reason the README calls it out specifically, is that the splitter must get the bar's role right at every position — including “after a leaf command,” where a `let` is followed by a block opener on the same line — so that the `if` and `endif` end up on their own logical lines for the block parser to pair. When you write a one-line conditional and it Just Works, the splitter's quote-, operator-, and comment-aware bar handling is why, and the `oneline.vim` example asserts it across exactly these nested and leaf-command positions.

Why the split matters

It would be possible to paper over the command/expression distinction and let users not think about it, and Vimscript itself sometimes blurs it — `:eval expr` exists precisely to evaluate an expression for its side effects as a command. But the split is real and it pays to see it, because it explains the shape of everything downstream. It is why `:let x = expr` has a command keyword, a target, an equals sign, and an expression, rather than being one uniform thing. It is why `:execute` (which builds a command string from expressions and runs it) and `eval()` (which evaluates an expression string and returns its value) are two different metaprogramming tools rather than one — they cross the seam in opposite directions. And it is why vimlrs's parser has a `parse_stmt` entry the LSP calls per line for diagnostics, and a separate expression parser underneath it. The two languages are stacked, vimlrs keeps them stacked, and once you see the stack the rest of Vimscript stops being a pile of special cases and becomes two small, regular grammars meeting at a well-defined seam.

Where the two-layer design comes from

The command-and-expression stack is not an arbitrary design; it is the fossil record of where Vimscript came from, and seeing the lineage makes the shape feel inevitable rather than odd. Vimscript descends from the colon command line of `ex`, the line editor that predates Vim's visual mode, where everything you did was a command typed after a colon: `:1,5d` to delete lines, `:s/old/new/` to substitute, `:w` to write. The command was the unit of work, and the command line was the interface. When `ex` grew into `vi` and then Vim, and when the need arose to compute values — to test a condition, to build a string, to count something — the expression language was grown *underneath* the command language rather than replacing it, because the command line was the established interface and the expressions were a new capability slotted into it. So you get `:if {expr}`, a command that takes an expression; `:let {target} = {expr}`, a command that binds the value of an expression; `:echo {expr1} {expr2}`, a command that displays expressions. The command is always the outer frame, the thing you type at the colon, and the expression is the inner computation the command operates on. The two-layer stack is what you get when a command-oriented line editor grows an expression language without abandoning its command-oriented heritage, and that is exactly Vimscript's history.

This origin explains features that otherwise look like quirks. It explains why a bare expression is not, by itself, a valid thing to write at the top level the way it is in most languages — you cannot just write `1 + 1` on a line and have it mean something, because the top level is the command layer, and `1 + 1` is not a command; you need `:echo 1 + 1` or `:eval 1 + 1` to wrap the expression in a command. It explains why `:execute` and `eval()` are two separate tools: one goes from expressions up to the command layer (compute a command string, run it as a command), the other goes from a string down to the expression layer (parse a string as an expression, return its value), and they are separate because the two layers are separate and you sometimes need to cross in each direction. And it explains why the line, not the statement, is the natural unit — there is no semicolon terminator because a command was always one line you typed, and the line-continuation backslash and the bar separator are the two mechanisms for bending that one-line-per-command rule when a command needs more than a line or a line needs more than a command. vimlrs reproduces the stack faithfully — two top-level AST types, `Expr` and `Stmt`, two parser entry points, the bar splitter that divides commands — not because the stack is elegant but because it is what Vimscript is, and a faithful port reproduces the language's actual structure, history and all.

Chapter 4 — Variables and Their Scopes

Vimscript variables wear their scope on their sleeve. A name like `g:counter` tells you, before you know anything else, that this is a *global* variable; `l:` is a function-local, `a:` an argument, `s:` script-local, and `b:`, `w:`, `t:`, `v:` are buffer-, window-, tab-, and Vim-predefined scopes. This is unusual — most languages resolve scope by lexical position, silently — and it is one of the features a faithful port has to reproduce exactly, because the scope sigil is not decoration; it changes where a name lives, who can see it, and when it resolves. This chapter is about the scopes, the `:let` command that binds them, and the several other things `:let` can target besides a plain variable.

The eight scopes

`vimlrs`'s lexer recognizes the single-letter scope prefixes directly: a name beginning `a:`, `b:`, `g:`, `l:`, `s:`, `t:`, `v:`, or `w:` “absorbs its `:` and the name after it,” so `g:foo` and `v:true` lex as one identifier token rather than a name, a colon, and another name. The eight scopes divide into three groups by how they behave outside an editor, which is exactly the distinction that matters for a standalone interpreter.

The *always-real* scopes are `g:` (global) and `v:` (Vim-predefined). A global is a global anywhere; `g:counter` works in `vimlrs` exactly as in Vim. The `v:` scope holds the special predefined variables — `v:true`, `v:false`, `v:null`, `v:none`, and the dynamic ones the engine maintains: `v:val` and `v:key` inside `map()/filter()`, `v:errors` for the assert framework, `v:exception` and `v:throwpoint` inside a `:catch`, `v:shell_error` after `system()`, and others like `v:count`, `v:version`, `v:hlsearch`, and `v:searchforward`. You do not assign most of these; the engine does, and your code reads them.

The *function-tied* scopes are `a:` (arguments) and `l:` (locals). They exist only inside a function call. `a:n` is the parameter `n`; `a:000` is the list of extra varargs; `a:0` is their count. `l:name` is a function-local variable — though inside a function a bare name is `l:`, so `let x = 1` and `let l:x = 1` bind the same slot. This equivalence is not a footnote; it is load-bearing for the JIT. The README explains that “explicit `l:name` refs in a function share the bare slot (`l:` is the local scope),” which is why a numeric loop over bare locals can trace-compile: the compiler can prove those locals are Numbers and give them VM slots, whereas a name carrying a `g:/s:/a:` alias stays dict-backed and cannot.

The *editor-tied* scopes are `s:` (script), `b:` (buffer), `w:` (window), and `t:` (tab page). The script scope `s:` is genuinely meaningful in a standalone interpreter — a sourced file has a script identity, so `s:` variables are private to the file that declared them, which is how Vimscript libraries hide their internals. The buffer, window, and tab scopes are the ones with no real referent when there is no editor; Chapter 18 covers what `vimlrs` does with the getters and setters that touch them. For now, note that the sigil is always *legal* even when the thing it names is absent.

A bare scope sigil with nothing after it is itself a value: a Dict of every variable in that scope. The `scopedvars.vim` example pins this down:

```
let g:alpha = 10
let g:beta = 20
call assert_equal(4, type(g:))
call assert_equal(10, get(g:, 'alpha', -1))
call assert_true(has_key(g:, 'beta'))
```

`type(g:)` is 4 — the type code for a Dict — and you can `get()` and `has_key()` against it like any dict. Inside a function, `a:` is the dict of arguments, and it carries the positional pseudo-keys too:

```
function! ArgKeys(one, two)
    return sort(keys(a:))
endfunction
call assert_equal(['0', '000', 'one', 'two'], ArgKeys(1, 2))
```

The keys of `a:` are the named parameters (`one`, `two`) plus `0` (the vararg count) and `000` (the vararg list) — a precise, slightly strange detail that vimlrs reproduces because Vim does.

Why scope-on-the-sleeve

It is worth pausing on how unusual Vimscript's scoping model is, because it runs against the grain of almost every other language a programmer is likely to know, and understanding why it is shaped this way explains a great deal of what follows. In most languages, scope is resolved by *position*: a name refers to the nearest enclosing binding, found by walking outward through lexical blocks, and the resolution is invisible — you write `x` and the compiler silently decides, from where the `x` sits in the source, which `x` it means. This is lexical scoping, and it is so universal that programmers stop seeing it as a choice. Vimscript does the opposite: scope is resolved by *prefix*, a sigil glued to the front of the name that declares, before the compiler knows anything about position, which namespace the name lives in. `g:counter` is global no matter where it appears; `s:helper` is script-local no matter how deeply nested; `a:n` is an argument no matter what blocks surround it. The scope is a property of the name, not of the name's location, and it is written explicitly rather than inferred.

This design has a history and a logic. Vimscript grew inside an editor, where the “program” is not a single compiled unit but a sprawl of separately-sourced files — your `vimrc`, dozens of plugins, filetype hooks, autocommands — all sharing one global namespace and one running engine. In that world, lexical scoping does not help, because there are no enclosing lexical blocks spanning the files; what you need instead is a way to say “this variable is mine, private to my plugin” without a module system, and “this variable is the shared global everyone can see.” The sigils are that mechanism: `s:` carves out a private per-file namespace so two plugins can both use `s:state` without colliding, and `g:` is the shared space they communicate through. The buffer, window, and tab scopes (`b:`, `w:`, `t:`) extend the idea to the editor's data model, attaching a variable's lifetime to a buffer or window rather than a file. The function-tied scopes (`a:`, `l:`) are the one place Vimscript does something lexical-shaped — a function does have an inside and an outside — but even there the prefix is explicit, and a bare name inside a function is `l:` by a rule rather than by position. The whole model is the natural answer to “how do you scope variables in a system made of independently-loaded files sharing one engine,” and it is why a faithful port has to reproduce the sigils exactly: they are not decoration on top of a normal scoping model, they *are* the scoping model, and a script's correctness depends on `s:x` and `g:x` and a bare `x` resolving to three different places.

The scoping model meets the JIT

The scope-on-the-sleeve design has a consequence the rest of this book keeps returning to, and it is worth drawing the connection explicitly here. Because scope is written on the name, the compiler can tell, syntactically, which namespace a variable lives in, without any flow analysis — `g:x` is global, full stop, and `x` inside a function is local, full stop. This syntactic clarity is exactly what makes the slot planner's job possible. When the planner asks “can this local live in a fast VM register rather than a dictionary?”, the first thing it needs to know is whether the name is genuinely a function-local or an alias to some dynamic scope, and the sigils answer that for free: a bare name (or its `l:` spelling, which is the same slot) is a local and a candidate for a slot, while any name carrying a `g:`, `s:`, `a:`, `b:`, `w:`, or `t:` prefix is dict-backed and disqualified. A language with lexical scoping and inferred resolution would have to do real analysis to determine which variables are local and which escape; Vimscript's explicit sigils hand the compiler that answer in the syntax. So the very feature that makes Vimscript's scoping look strange to a newcomer — the prefixes everywhere — is the feature that lets vimlrs prove which locals are safe to put in fast slots, and a numeric loop over bare locals traces to native code

precisely because the language wrote, on every name, which scope it belongs to. The unusual surface and the JIT speed are the same fact seen from two sides: scope-on-the-sleeve is a burden to read and a gift to compile.

The script scope and file identity

The `s:` scope is the one editor-tied scope that is fully meaningful standalone, and it is worth a closer look because it is how Vimscript libraries hide their internals. A sourced file has a *script identity* — vimlrs tracks which file is currently being sourced, the same sourcing-name stack that `expand('<sfile>')` reads (`sfile.vim`) — and an `s:name` variable is private to the file that declared it. Two libraries can each have an `s:counter` without colliding, because each `s:counter` is keyed to its own script. This is genuine information hiding: a library exposes its public functions (named without a scope, in the global function namespace) and keeps its state in `s:` variables that callers cannot reach. A script-local function — one whose name is `s:helper` — is likewise private, callable within the file but not from outside, which is how a library keeps its helpers out of the global namespace. vimlrs implements all of this because a sourced file genuinely has an identity even with no editor: the `:source` command and the autoload mechanism both establish a script context, and the `s:` scope is bound to it. So the library pattern of Chapter 12 — source a file, call its public functions, never see its `s:` internals — works exactly as it does in a Vim configuration, which matters because real `.vim` code leans on `s:` for encapsulation constantly.

:let and its targets

The `:let` command binds a value to a target, and the interesting thing about it is how many kinds of target there are. vimlrs's AST enumerates them in the `LetTarget` type, and each corresponds to a different lvalue Vimscript allows on the left of the equals sign:

- `Var(name)` — a plain (possibly scoped) variable: `let x = ...`, `let g:x = ...`.
- `Option(name)` — an option, via the `&` sigil: `let &ignorecase = 1`.
- `Env(name)` — an environment variable, via `$`: `let $PATH = ...`.
- `Register(char)` — a register, via `@`: `let @x = ...`.
- `Index { base, index }` — an element or member: `let l[i] = v`, `let d['k'] = v`, `let d.k = v`. The base is the full container expression, so nesting like `d['a']['b']` works because `base` can itself be an index.
- `List { names, rest }` — list-unpack: `let [a, b] = list`, or with a trailing remainder `let [a, b; rest] = list`.
- `Range { base, idx1, idx2 }` — a list-range assignment: `let l[1:3] = [...]`.

The index/member form is what makes Vimscript's dicts and lists feel mutable. From `lists_dicts.vim`, the frequency-counter idiom updates a dict slot in place:

```
let freq = {}
for w in words
  let freq[w] = get(freq, w, 0) + 1
endfor
```

vimlrs's status table notes that a Dict element set additionally “fires `dictwatcheradd()` watchers” — the ported `dictwatcheradd/dictwatcherdel` machinery is wired through the element-set path, so a watcher registered on a dict sees the mutation, exactly as in Neovim.

The list-unpack form destructures, and the `;rest` variant captures the tail. This is the same syntax `:for` uses to unpack each item of an iterable, which is why the AST shares a `ForVars` type with `One(name)` and `List(names)` variants: `for [k, v] in items(d)` binds `k` and `v` from each `[key, value]` pair.

Compound assignment

`:let` supports compound operators — `+=`, `-=`, `*=`, `/=`, `%=`, and the string-concatenation `.=` — and vimlrs implements them by desugaring. The status table is precise about it: compound assignment “desugars to `target = target op rhs`, so accumulator loops trace-JIT.” This is a small but consequential decision. Because `let s += i` becomes `let s = s + i` at compile time, the accumulator pattern that every tight loop uses lowers to exactly the same bytecode as the spelled-out form, and the JIT’s loop tracer — which is looking for provably-numeric updates to a slot — sees a plain addition it can compile to a native `fadd` or integer `add`. The `fib` example’s inner loop is built entirely on this:

```
function! FibIter(n) abort
  let a = 0
  let b = 1
  let i = 0
  while i < a:n
    let t = a + b
    let a = b
    let b = t
    let i += 1
  endwhile
  return a
endfunction
```

The `let i += 1` is the loop counter; because it desugars to `let i = i + 1` over a provably-Number local, the whole `while` trace-compiles to machine code.

The desugaring is the detail that makes the compound operators “free” — they cost nothing over the spelled-out form because they *become* the spelled-out form before any lowering happens. The parser reads the operator character just before the `=` and rewrites `target op= rhs` into `target = target op rhs`, so `let s += i` is literally `let s = s + i`, `let s *= 2` is `let s = s * 2`, and `let s .= 'x'` is `let s = s . 'x'` by the time the compiler sees it. This has two consequences. The first is the JIT one: an accumulator written with `+=` lowers to exactly the same bytecode as one written out, so the slot planner sees a plain numeric store and the loop traces — there is no special compound-assignment opcode that the JIT would have to learn. The second is semantic consistency: because `+=` is `= ... +`, it inherits the `+` operator’s behaviour exactly, including the List-concatenation meaning (`let l += [x]` extends a list) and the numeric-versus-string distinction. A from-scratch implementation might give `+=` its own subtly different semantics; vimlrs cannot, because `+=` is not a separate operation at all — it is sugar the parser expands into the operation that already exists. The string-concatenation `.=` is the same: `let msg .= line` is `let msg = msg . line`, inheriting concatenation’s coercion rules. Six compound operators, zero new semantics, zero JIT cost — all because the parser desugars them before anything downstream can tell they were ever written compactly.

`:const`, `:unlet`, and heredocs

Two more `:let`-adjacent commands round out variable binding. `:const` assigns like `:let` — in vimlrs, as the `lists_dicts.vim` comment notes, “immutability is not enforced here,” so `const PI = 3` binds `PI` to `3` and you read it back normally, but vimlrs does not currently reject a later rebind. This is a provisional simplification worth flagging: in Neovim `:const` locks the value, and vimlrs’s parser accepts the keyword and binds the value but does not yet enforce the lock. The honest way to read this is that `:const` is *parsed and bound* but not *enforced* — a `const PI = 3` works as a binding, you read `PI` back as `3`, and a script that uses `:const` for documentation (signalling “I do not intend to change this”) behaves correctly as long as it keeps its own promise. What is missing is the *guard*: Neovim would reject a later `let PI = 4` with E741, and vimlrs currently accepts it. This is exactly the kind of gap the project marks rather than hides — the source comment in `lists_dicts.vim` says “immutability is not enforced here,” and this book repeats the caveat. It is a provisional state, not a permanent divergence: the parser already recognizes `:const`, so enforcing the lock is a matter of adding the guard, not redesigning anything. Until then, `:const` is a binding with documented intent but no teeth, and

a reader should treat it as `:let` with a comment rather than relying on it to prevent reassignment.

`:unlet` removes a variable, and `:unlet!` does so without erroring on a missing name. It takes multiple targets, and each may be a bare name or a container element — the AST’s `UnletArg` has `Name(name)` and `Item { base, index }` variants, mirroring “the two non-name branches of `do_unlet_var()`” in the C.

The interaction with the slot planner (Chapter 21) is one reason a function that `:unlets` a local bails out of slotting: a slotted local lives in a VM register, which has no “removed” state, so a function that might `:unlet` a name keeps that name dict-backed where removal is meaningful. So `:unlet` is both the inverse of `:let` across the same lvalue space — removing a variable, a dict entry, or a list element where `:let` would assign one — and a construct whose mere presence in a function disables the fast-local optimization, because removal and slotting are incompatible. The bang form is the one to reach for when absence is possible: `:unlet! g:cache` clears a variable that may or may not have been set, the defensive form a script uses when it cannot be sure of state.

The `varargs.vim` example shows all three forms:

```
unlet g:tmp
let g:a = 1 | let g:b = 2
unlet g:a g:b
unlet! g:missing g:keep
```

Finally, the heredoc: `:let var =<< [trim] END` reads the lines that follow, verbatim, up to a line equal to the marker, and assigns them as a List of strings. It “mirrors `heredoc_get()` in Vim.” The body is literal — quotes, bars, and punctuation are all just text — and the optional `trim` strips the first body line’s indentation from every line while preserving relative indent. From `heredoc.vim`:

```
let raw =<< EOF
it's got a | pipe
a "quoted" word
END is only the marker when alone
EOF
call assert_equal(["it's got a | pipe", 'a "quoted" word',
    \ 'END is only the marker when alone'], raw)
```

A heredoc can target a Dict entry (`let doc.body =<< END`) like any other `:let` lvalue, and it composes with surrounding code. It is the cleanest way to embed a block of literal text in a script, and the fact that the body ignores Vimscript’s own metacharacters — the bar especially — is exactly what makes it useful for holding data that would otherwise need escaping.

The heredoc’s implementation has a subtlety that explains why its body can be so literal: it is captured *before* the source splitter does any of its work. The parser’s `Lines` cursor (Chapter 20) collapses a heredoc in its very first pass, taking the body lines “*VERBATIM* from the raw source — before continuation-folding or bar-splitting.” So a body line containing `a | pipe` keeps its bar, a line containing `"quoted"` keeps its quotes, and a line that happens to read like a line-continuation or a comment is still just text — because none of the splitting passes that would interpret those metacharacters have run yet when the heredoc body is captured. The `trim` modifier adds one transformation on top: it strips the *first* body line’s indentation from every line, preserving relative indent, so a here-document written indented inside a function body comes out dedented but with its internal structure intact — `heredoc.vim` asserts that `root` and `root2` lose their leading spaces while `child` keeps the two-space step that made it deeper. The `eval` modifier, where supported, additionally evaluates `{...}` interpolations in the body. The whole feature “mirrors `heredoc_get()` in Vim,” and its placement at the front of the pipeline — ahead of every metacharacter-interpreting pass — is the mechanical reason its body is as literal as it is.

List-unpack in depth

The list-unpack target deserves a fuller look than the `LetTarget` enumeration gave it, because `list_unpack.vim` shows it is more capable than a simple multi-assign. The `skip_var_list()` grammar from

Neovim's `vars.c` is what the parser reproduces, and the example walks it from the exact-length case to the `;rest` remainder:

```
let [a, b, c] = [1, 2, 3]
let [a, b] = [b, a]
call assert_equal([2, 1], [a, b])
let [head; rest] = [10, 20, 30, 40]
call assert_equal(10, head)
call assert_equal([20, 30, 40], rest)
```

Three things are worth drawing out. First, the swap `let [a, b] = [b, a]` works because the right-hand side is fully built *before* any binding happens — the list `[b, a]` is constructed from the old values, then deconstructed — so there is no temporary variable and no clobbering. Second, the `;rest` form binds the remainder as a List: `[head; rest]` puts the first element in `head` and “soaks up the remaining elements” into `rest`. The rest binding can match exactly zero remaining elements, in which case it is the empty list — `let [only; nothing] = [99]` leaves `nothing` as `[]`. Third, unpack is element-wise over heterogeneous values, so `let [name, attrs] = ['root', #{uid: 0}]` binds a string and a dict in one statement. And the most common use, from Chapter 11, is destructuring inside a `:for` over pairs:

```
for [k, v] in items({a: 1, b: 2, c: 3})
  let total += v
endfor
```

`items()` yields `[key, value]` pairs and the `[k, v]` target unpacks each, which is why the AST shares its `ForVars::List` machinery with the `:let` list target — the same grammar parses both, so anything you can destructure in a `:let` you can destructure in a `:for`.

Options, environment, and registers as targets

Three of the `:let` targets reach things that are not ordinary variables, and the `options.vim`, `env_vars.vim`, and `registers.vim` examples show each round-tripping. An option is reached through the `&` sigil, both as an rvalue and a `:let` target, and it agrees with the `:set` command:

```
call assert_equal(8, &shiftwidth)
let &shiftwidth = 4
call assert_equal(4, &shiftwidth)
set shiftwidth=8
call assert_equal(8, &shiftwidth)
let &ignorecase = 1
call assert_equal(1, &ignorecase)
```

`&shiftwidth` reads the option, `let &shiftwidth = 4` writes it, and `set shiftwidth=8` is a third surface onto the same value — `setlocal` is a fourth. A boolean option round-trips through `let &opt` as 0 or 1. The environment is the second non-variable target: `let $X = v` sets a variable for the process, `$X` reads it (empty if unset), and `getenv/setenv` are the functional forms, with the faithful distinction Chapter 16 detailed — an unset variable is `v:null` through `getenv()` but `'` through `$X`. The third is the register, reached through `@`:

```
let @a = 'hello'
let @a = @a . ' world'
call assert_equal('hello world', @a)
let @" = 'unnamed'
call assert_equal('unnamed', @")
```

`let @a = ...` writes a register, `@a` reads it, and the unnamed register `@"` works like any other. The payoff Chapter 24 made — that registers are shared with the `:yank/put` commands — is why this matters beyond expression code: `let @a` and `:yank a` touch the same store, so the expression layer and the command layer stay consistent. All three targets — option, environment, register — are first-class `:let` lvalues with their own `LetTarget` variant, which is why the AST needed to enumerate so many kinds of target rather than treating

the left of $=$ as one uniform thing.

Chapter 5 – Types and Coercions

Vimscript has a small, fixed set of value types and a set of coercion rules between them that are specific, occasionally surprising, and — because they are ported rather than reinvented — exactly Vim’s. This chapter walks the types, shows how `type()` names them, and digs into the two coercion behaviours most likely to bite: numeric division and string-to-number conversion. Everything here is grounded in the ported value layer, which the engineering report says was ported first “so the rest of the engine has a faithful substrate to build on.”

The eleven types

`vimlrs`’s ported `typval_defs_h` module defines the `VarType` enum with the same variants Neovim uses: `VAR_UNKNOWN`, `VAR_NUMBER`, `VAR_STRING`, `VAR_FUNC`, `VAR_LIST`, `VAR_DICT`, `VAR_FLOAT`, `VAR_BOOL`, `VAR_SPECIAL`, `VAR_PARTIAL`, and `VAR_BLOB`. The user-visible types — the ones `type()` reports and the ones you actually manipulate — are:

- **Number** — a 64-bit integer. `has('num64')` is true. Integer literals understand bases: `010` is octal 8, `0xff` is 255, `0b101` is 5, `0o17` is 15.
- **String** — a byte string. Its length is its *byte* count, which matters for multibyte text and is covered in Chapter 9.
- **Float** — a double. Rendered like Vim’s `printf("%g")`: six significant digits, so `string(1.0 / 3.0)` is `'0.333333'` and `string(1.0)` is `'1.0'`.
- **List** — an ordered, heterogeneous sequence.
- **Dict** — an *insertion-ordered* map. The ordering is deliberate and is covered below.
- **Funcref** and **Partial** — references to functions, with partials carrying bound arguments. Chapter 8 is devoted to them.
- **Bool** — `v:true` / `v:false`.
- **Special** — `v:null` and `v:none`.
- **Blob** — a byte buffer, written `0z...`. Chapter 7 covers it.

The eleven types divide naturally into three groups by how a script uses them, and the grouping is worth seeing. The *scalars* are Number, Float, String, Bool, and Special — values that are copied by value, compared by content, and that a condition coerces to a truthiness. The *containers* are List, Dict, and Blob — heap values copied by reference, compared by content but distinguishable by identity (`is/isnot`), and mutated in place by the standard library. The *callable*s are Funcref and Partial — values that wrap a function, with the Partial additionally carrying bound arguments or a bound dict. Most of the type system’s interesting behaviour is at the boundaries between these groups: the coercions that turn a String into a Number in arithmetic, the identity-versus-equality distinction that separates two equal Lists, the `VAR_PARTIAL` that unifies the callable forms. A script rarely thinks in terms of all eleven at once; it thinks “is this a number or a string?”, “is this a list or a dict?”, “is this callable?” — and the three groups are exactly those three questions. `vimlrs` carries all eleven as the ported `VarType` variants, but the user-facing surface is the three groups and the `type()` code that names each, which is why `type()` is the first reflection function a script reaches for when it must branch on what it was handed.

The `type()` builtin returns a numeric code for each — the reference documents it as “0 Number, 1 String, 2 Func, 3 List, 4 Dict, ...” — and the codes are exported as the ported constants `VAR_TYPE_NUMBER`, `VAR_TYPE_STRING`, and so on. The `special_none.vim` example uses the codes directly: `type(v:none)` is 7, and `type(v:null) == type(v:none)` because both are the Special type even though they render distinctly.

Booleans and specials

`v:true` and `v:false` are genuine Bool values, not aliases for 1 and 0, though they coerce to 1 and 0 in numeric context. They render as themselves: the reference says `v:true` has “numeric value 1, string value `\\"v:true\\"`.” The `json.vim` example relies on this round-tripping — `'standalone': v:true` encodes to JSON `true` and decodes back to a Bool.

The distinction between a genuine Bool and the Numbers 1 and 0 matters in exactly the places where type is observable. A `v:true` has `type()` code 6 (Bool), not 0 (Number), so a script that branches on `type(x) == type(v:true)` distinguishes a Bool from a number that happens to be 1. It renders as `'v:true'` through `string()`, not as `'1'`, so its serialized form is distinct. And it encodes to JSON `true`, not `1`, so a round trip preserves the boolean-ness — a Number 1 would come back a Number. In *numeric* context, though, `v:true` coerces to 1 and `v:false` to 0, so `v:true + 1` is 2 and `if v:true` takes the true branch — the Bool behaves like its numeric value where a number is wanted, while keeping its own type where the type is observable. This is the same have-it-both-ways design as the specials: a genuine distinct type that coerces transparently in the contexts that want a number, so old code that treats truth as 1 keeps working while new code that wants a real boolean gets one. vimlrs keeps Bool distinct from Number because the ported value layer has a `VAR_BOOL` variant separate from `VAR_NUMBER`, which is why `json.vim` can assert that `v:true` survives a JSON round trip as a Bool rather than collapsing to 1.

The two specials, `v:null` and `v:none`, are both “empty” values that behave like an empty string in most contexts, but they keep distinct identities, and `special_none.vim` makes the difference visible:

```
call assert_equal('v:null', string(v:null))
call assert_equal('v:none', string(v:none))
call assert_equal('[v:null, v:none]', string([v:null, v:none]))
call assert_equal("{\"a': v:none}", string({'a': v:none}))
```

Each renders with its own name, and each keeps that rendering inside lists and dicts. They are `empty()` and they are the Special type, but they are not each other. A from-scratch interpreter would very likely collapse them; vimlrs keeps them apart because the ported encoder does.

The reason to keep `v:null` and `v:none` distinct is not pedantry — they mean different things in the contexts that use them. `v:null` is the absence-of-value that JSON's `null` decodes to and that `getenv()` returns for an unset variable; it round-trips through `json_encode/json_decode` as `null`. `v:none` is Vim's “use the default” sentinel, distinct from “explicitly null,” which matters in argument positions and option contexts where omitting a value and passing null are different. Both are the Special type (code 7), both are `empty()`, and both behave like an empty string in most coercions — but they render with their own names and keep their identity inside containers, so `string([v:null, v:none])` is `'[v:null, v:none]'`, not a collapsed pair. The `special_none.vim` example asserts exactly this non-collapse, and it passes because the ported encoder emits each special by its own name. A from-scratch implementation that mapped both to a single null value would lose the distinction silently, and any script that relied on telling “unset” from “default” — or “JSON null” from “Vim none” — would break in ways that are hard to debug. vimlrs keeps them apart because the C it ports keeps them apart, which is the whole argument for porting in one sentence: the distinctions you would not think to preserve are preserved for free.

Insertion-ordered dictionaries

A Vimscript Dict iterates in a specific order, and vimlrs goes out of its way to match it. The `Cargo.toml` pulls in `indexmap` precisely for this, with a comment explaining why: the dict “keeps insertion order (closest to Vim's observable hashtable iteration, and deterministic for `echo/string()` output).” The engineering report repeats the rationale — `dict` “preserves insertion order to match Vim's observable hashtable iteration and keep `echo / string()` output deterministic.”

This is a fidelity decision with a testability payoff. Because the order is deterministic, `string()` of a dict produces a stable, predictable string, which is what lets the examples assert exact dict renderings:

```
call assert_equal({'vim': 3, 'rust': 2, 'c': 1}, freq)
call assert_equal({'id': 1, 'tag': 'x'}, #{id: 1, tag: 'x'})
```

If the dict iterated in hash order, those assertions would be fragile across platforms and runs. Insertion order makes them reliable, and it happens to match what a user reading `echo`'d output expects.

The choice ripples outward to every place a dict is serialized or compared. `string()` of a dict produces a stable string because the keys come out in insertion order, which is why `string_repr.vim` can assert `string(#{b: 2, a: 1})` is `{'a': 1, 'b': 2}` — note that `string()` actually *sorts* the keys for its output, a second determinism on top of the insertion-order storage, so the rendered form is canonical regardless of insertion order while the *iteration* order (what `keys()`, `items()`, and a `:for` see) is insertion order. `json_encode` relies on the storage order so its output is predictable byte for byte, which is what lets `json.vim` assert the exact encoded string. And `==` over two dicts compares contents regardless of order, so two dicts built by inserting the same pairs in different orders are equal. The `indexmap` crate is what provides all of this: it is an insertion-ordered map with hash-map lookup performance, so vimlrs pays nothing for the determinism — lookups are still $O(1)$, and iteration simply follows the insertion sequence. The `Cargo.toml` comment names the reason it is pulled in precisely: the dict “keeps insertion order (closest to Vim’s observable hashtable iteration, and deterministic for `echo/string()` output).” A fidelity decision and a testability decision turn out to be the same decision, and one crate delivers both.

The VarType enum and the type codes

The type system has a concrete representation worth naming, because `type()`'s numeric codes are not arbitrary — they are the ported `VarType` enum's discriminants. The ported `typval_defs_h` module defines `VarType` with the same variants Neovim uses: `VAR_UNKNOWN`, `VAR_NUMBER`, `VAR_STRING`, `VAR_FUNC`, `VAR_LIST`, `VAR_DICT`, `VAR_FLOAT`, `VAR_BOOL`, `VAR_SPECIAL`, `VAR_PARTIAL`, and `VAR_BLOB`. The user-visible `type()` codes — exported as the ported constants `VAR_TYPE_NUMBER`, `VAR_TYPE_STRING`, `VAR_TYPE_FUNC`, `VAR_TYPE_LIST`, `VAR_TYPE_DICT`, `VAR_TYPE_FLOAT`, `VAR_TYPE_BOOL`, `VAR_TYPE_SPECIAL`, and `VAR_TYPE_BLOB` — are the numbers `type()` returns: 0 for Number, 1 for String, 2 for Func, 3 for List, 4 for Dict, 5 for Float, 6 for Bool, 7 for Special, and 10 for Blob. The codes are stable because they are Vim's, which is why `special_none.vim` can assert `type(v:none)` is 7 and `string_repr.vim` can rely on the numeric codes directly. A Funcref and a Partial both report code 2 (Func) to `type()`, even though they are distinct `VarType` variants internally, because that is how Vim collapses them at the user-visible layer — a faithful detail that a from-scratch implementation, having chosen its own representation, would have to remember to reproduce and probably would not.

Truthiness and the coercion contract

Every value has a truthiness, and the rule is Vim's: a value is true when it is a non-zero Number, and everything else coerces toward that. A Number is true when non-zero; a Float is *not* automatically truthy in a condition (Vim treats a Float in a boolean context as an error in strict mode, and the ported `tv_get_number` governs the coercion); a String coerces to the leading integer it parses to, so `'3abc'` is 3 and `'abc'` is 0, which means a non-empty non-numeric string is *false* in a condition — a famous Vimscript trap that catches programmers expecting Python's non-empty-string truthiness. `v:true` and `v:false` coerce to 1 and 0, and `v:null` and `v:none` coerce to 0. A List or Dict cannot be coerced to a number at all and raises an error in a numeric context (`let x = [] + 1` is the `assert_fails` case `testing.vim` uses). All of this lives in the ported `tv_get_number` and its siblings, so the truthiness of a condition, the value of a string used as a number, and the error on a list-as-number are all exactly Vim's — the conditions in every `:if` and `:while` in this book resolve through the same ported coercion, which is why a loop `while i < a:n` and a guard `if !empty(v:errors)` mean precisely what they mean in Neovim.

Numeric division: the integer trap

Here is a coercion that catches everyone, and that vimlrs gets right because it ported the arithmetic rather than borrowing fusevm's. The README's JIT notes mention it in passing while explaining why modulo can be native but division cannot: "Native integer % ... so modulo loops trace; / stays on the builtin (fusevm div is float, unlike VimL integer /)." That parenthetical is the whole story. In Vimscript, dividing two Numbers does *integer* division — `7 / 2` is 3, not 3.5 — truncating toward zero, exactly as the ported `num_divide` defines. But fusevm's native division is floating-point. So vimlrs deliberately keeps integer `/` on the ported builtin path rather than letting the JIT lower it to a native float divide, because doing the fast thing would produce the *wrong answer*. The faithfulness constraint overrides the performance one: a native float divide is faster and wrong, so vimlrs takes the slower, correct builtin. Modulo, by contrast, has matching integer semantics, so `%` *does* get a native `Op` and modulo loops trace-compile. This is the faithful-port discipline operating at the level of a single operator: every optimization is checked against the ported semantics, and the ones that would diverge are declined.

The deepest lesson of the division trap

The integer-division case is the single clearest illustration of the entire project's design philosophy operating at the smallest possible scale, and it rewards sitting with, because it shows the two halves of vimlrs — faithful semantics and borrowed speed — coming into direct conflict over one operator and the conflict being resolved by a fixed priority rather than a case-by-case judgment. Here is the conflict in full. The borrowed machine, fusevm, has a native division instruction, and using it would be fast — a single machine op rather than a builtin call, and it would let a loop containing division stay on the JIT's fast path. The faithful semantics say Vimscript's `/` on two integers is integer division, truncating toward zero, so `7 / 2` is 3. And fusevm's native division is *float* division, so using it would make `7 / 2` evaluate to 3.5. The fast thing and the correct thing are in direct opposition: the JIT could make division fast only by making it wrong. Every language project faces some version of this conflict, and the tell of a project's real priorities is how it resolves it. A project that secretly valued speed over fidelity would take the native divide and quietly accept that `7 / 2` is 3.5, perhaps rationalizing that "most people expect float division anyway." vimlrs does the opposite without hesitation: it keeps integer `/` on the slower, correct builtin path, declining the optimization, because the faithfulness constraint has absolute priority over the performance one.

What makes this more than a single decision is that it is the *rule*, applied at the granularity of individual operators, and the rule is what gives the JIT its trustworthiness. Modulo goes native, because the ported `num_modulus` and fusevm's integer mod agree on the sign rule, so the optimization is safe. Integer addition, subtraction, and multiplication go native, because they agree. Division does not go native, because they disagree. Each operator is checked, individually, against the question "does the fast native op produce the same result as the faithful builtin?", and it is optimized only if the answer is yes. This is why a script can trust that its loops are both fast and correct: the JIT never trades correctness for speed, because every place it makes something fast, it has first proven that fast and correct coincide for that operation. The division trap is the proof case, the operator where fast and correct visibly diverge and the project visibly chooses correct, and it is the reason the speed claims in this book carry no asterisk — the loops that run fifteen to a hundred times faster are running the *same* computation the interpreter would, because the optimizer declined every shortcut, like the native divide, where the computation would have changed. Own the semantics, borrow the machine, and let the semantics win every conflict: the division trap is that principle compressed into one operator, and it is the cleanest single window into why vimlrs is fast without being unfaithful.

Comparison, coercion, and identity

Comparison is its own small thicket of coercions, and vimlrs gets it right because it ports `typval_compare` rather than reinventing the rules. The `compare.vim` example exercises the case-flag operators of Chapter 3 — `==?` case-insensitive, `==#` case-sensitive, `=~?` a case-insensitive regex match — but it also pins the distinction

between *equality* and *identity*, which is where a from-scratch implementation most often goes wrong:

```
let a = [1, 2]
let b = a
let c = [1, 2]
call assert_equal(1, a is b)
call assert_equal(1, a isnot c)
call assert_equal(1, a == c)
```

`a is b` is true because `b` was assigned *from* `a`, so they are the same list object; `a isnot c` is true because `c` is an independent list built separately; and yet `a == c` is also true, because their *contents* are equal. Identity asks “the same object?” and equality asks “equal contents?”, and Vimscript — faithfully, in vimlrs — lets you ask both, because `typval_compare` implements `is/isnot` as a pointer-identity check on the container and `==` as a structural comparison. The two questions genuinely differ for containers, and conflating them (as a naive interpreter that made `is` an alias for `==` would) would quietly break any code that relies on the distinction — a watcher checking whether a list it was handed is the *same* list it stored, for instance.

The coercion rules inside `==` are ported too, and they are the kind of thing only a port gets right. Comparing a Number to a String coerces per Vim’s rules; the case flag on a string comparison selects sensitivity; and a bare `==` consults `'ignorecase'`, which is why the option is “wired into” the comparison path the same way it is wired into the regex engine. None of this is rederived from the help files. It is `typval_compare`, ported with its exact name and a citation, and the `compare.vim` assertions are the proof that the port reaches the comparison layer intact — equality, identity, the case flags, and the option, all behaving as Vim’s do.

The philosophy of coercion

Coercion is the place where a dynamically-typed language reveals its character, and it is worth stepping back to see what kind of language Vimscript is by the way it coerces. There is a spectrum. At one end sit languages like Python, which coerce almost nothing implicitly: `'3' + 4` is an error, because a string and a number are different kinds of thing and the language refuses to guess which one you meant. At the other end sit languages like Perl and the shell, which coerce aggressively: a string is a number when you do arithmetic to it and a string when you concatenate it, and the language silently does whatever the operator asks, reading a leading number out of a string or stringifying a number as needed. Vimscript sits firmly toward the Perl end, and for the same reason — it is a text-processing language descended from the same tradition, where the data you handle is strings that often represent numbers, and demanding explicit conversion at every boundary would be ceremony that buys nothing. So `'3abc' + 4` is 7 in Vimscript: the `+` wants numbers, so the string is coerced by reading its leading integer, `3`, and the trailing `abc` is discarded. This is the behaviour that makes one-liners terse and the behaviour that occasionally bites, and vimlrs reproduces it exactly because the ported `tv_get_number` is the function that defines it.

But Vimscript’s coercion is not uniformly permissive, and the places it draws lines are as characteristic as the places it does not. It will read a number out of a string, but it will not read a number out of a List — `[] + 1` is an error (E745-family), not some coercion of the list to a count, because a list is structured data and there is no sensible leading number to extract. It will divide two numbers as integers, truncating, but it will refuse a float operand to `%` with E804, because modulo is defined on integers and a float modulo would be a different operation Vim chose not to provide. It treats a non-empty non-numeric string as *false* in a boolean context, because the coercion to boolean goes through the coercion to number (`'abc'` reads as 0, and 0 is false), which surprises programmers who expect the Python or JavaScript rule that a non-empty string is *truthy* — and this is exactly the kind of place where a from-the-help interpreter would guess the more familiar rule and be wrong. The pattern across all of these is that Vimscript coerces freely between the *scalar* types that have a natural numeric or string reading, and refuses to coerce the *structured* types that do not, and within the scalars it picks specific rules — truncating division, integer-only modulo, leading-number string parsing — that are documented behaviours rather than the obvious ones. A faithful port cannot derive these from a principle, because they are not principled in the sense of following from one rule; they are a set of specific decisions the Vim authors made over decades, and the only way to reproduce them is to port the functions that make them.

Why the coercion rules cannot be guessed

The deeper point, and the one that ties this chapter to the fidelity story of Chapter 13, is that coercion is the single richest source of the edge cases that distinguish a faithful interpreter from an approximate one, and it is almost entirely invisible to a casual reading of the language. When you write Vimscript, you do not think about coercion — you write `freq[w] = get(freq, w, 0) + 1` and it works, you write `if len(x)` and it works, and the coercions happen silently underneath. It is only at the edges — the float passed to `%`, the list added to a number, the string with trailing garbage, the negative index, the empty container — that the coercion rules become visible, and those edges are exactly where a reimplementer's intuition fails, because their intuition was formed by the common cases where coercion is invisible. This is why the example suite spends so much of its effort on numeric and string edge cases: `numbers.vim`, `numeric_edge.vim`, `float_literals.vim`, and `strings.vim` are, between them, an enumeration of the coercion decisions that a from-scratch interpreter would get wrong, each pinned to an exact expected value. And it is why this book keeps returning to them as evidence of fidelity: a string function being “mostly right” is hard to assess, but `str2float(' -2.5e3xyz')` being exactly `-2500.0` is a single concrete fact that either matches Vim or does not. The coercion layer is where the rubber of “ported from the C” meets the road of “behaves like Vim,” and the fact that vimlrs's coercions match Vim's down to the trailing-garbage and the sign bit is the most precise evidence the book can offer that the port reaches the layer where Vimscript's character actually lives.

String-to-number and back

Coercion between strings and numbers is governed by ported functions — `str2nr`, `str2float`, and the `vim_str2nr` port in `charset.rs`. `str2nr` reads a leading integer and honours an explicit base, while `str2float` reads a leading float and ignores trailing garbage the way C's `strtod` does. From `numbers.vim`:

```
call assert_equal(3.14, str2float('3.14abc'))
call assert_equal(-2500.0, str2float(' -2.5e3xyz'))
call assert_equal(0.5, str2float('.5'))
call assert_equal(0.0, str2float('abc'))
```

`str2float('3.14abc')` is `3.14` — the parse stops at the first non-float character rather than erroring — and `str2float('abc')` is `0.0`. The `str2nr` side, from `strings.vim`, honours base and the `'`-separator option:

```
call assert_equal(255, str2nr('ff', 16))
call assert_equal(5, str2nr('101', 2))
call assert_equal(-42, str2nr(' -42 '))
call assert_equal(1000000, str2nr("1'000'000", 10, 1))
```

The inverse direction is `string()`, which produces a representation that `eval()` can read back — the reference describes it as “quoting embedded strings (inverse of `eval()`).” That round-trip property, `eval(string(x)) == x`, is the backbone of vimlrs's serialization story and shows up again when we reach JSON in Chapter 16.

The string-to-number conversions are governed by a small family of ported functions, and their differences matter. `str2nr` reads a *leading integer*, honouring an explicit base argument and stopping at the first character that is not a digit in that base, so `str2nr('ff', 16)` is 255 and `str2nr(' -42 ')` is -42 (leading whitespace and sign are handled, trailing garbage ignored). Its third argument, when truthy, ignores embedded `'` digit separators, so `str2nr("1'000'000", 10, 1)` is a million. `str2float` reads a *leading float* with C's `strtod` semantics, which means it accepts a decimal, an exponent, or even a hex float, and stops at the first non-float character — so `str2float('3.14abc')` is 3.14 and `str2float('0x1f')` is 31.0. The implicit string-in-numeric-context coercion is a third path, the `tv_get_number` one of Chapter 5, which is *less* permissive than `str2nr`: it reads a leading integer with no base argument, so a `'0xff'` used directly in arithmetic is 0 (the parse stops at the `x`), which is why `str2nr('0xff')` with no base is also 0 but `str2nr('0xff', 16)` is 255. The three paths — explicit `str2nr`, explicit `str2float`, and implicit coercion — have three subtly different rules, and a script that uses the wrong one gets a subtly wrong number. vimlrs keeps them distinct because the ported functions keep them distinct, and `strings.vim` and `numbers.vim`

assert each path's behaviour separately. The coercions are many and fiddly, but they are not arbitrary: each one is a ported C function with a documented behaviour, and the example suite asserts that vimlrs reproduces it.

Chapter 6 – Functions

Functions are where Vimscript's command and expression layers meet most visibly: you *define* a function with the `:function` command and a block of statements, and you *call* it from within an expression. vimlrs supports the full shape of a Vimscript function — named or autoloading, with fixed parameters, optional defaults, and varargs; with the `abort` attribute; with recursion; and with the function-tied `a:` and `l:` scopes from Chapter 4. This chapter covers definition and call; funcrefs, lambdas, and closures get their own chapter next.

Defining a function

The `:function` command opens a definition that runs to `:endfunction`. vimlrs's AST `Function` statement records the name, the parameter names (without their `a:` prefix), the optional defaults, the body, and a `bang` flag for `function!`. That bang is the most common decoration in real Vimscript: `function!` replaces an existing definition rather than erroring if the name is taken, which is what lets a script be re-sourced without complaint. Nearly every example defines its functions with the bang:

```
function! Fib(n) abort
  if a:n < 2
    return a:n
  endif
  return Fib(a:n - 1) + Fib(a:n - 2)
endfunction
```

The `abort` attribute tells Vim to stop the function at the first error rather than blundering on, and it is idiomatic to write it on every function. The reason it matters is that, without it, a function that hits an error keeps executing its remaining statements — which in a function that computes a result almost always compounds the first error into a cascade of further ones, with the original cause buried. With `abort`, the function halts at the first error and returns, so the error that actually occurred is the one that surfaces. vimlrs implements this through the ported `ex_eval.c/message.c` state: when an error sets the `did_emsg` flag inside an `abort` function, the function's execution stops, and the flag propagates up so the caller sees the failure. This is why every example writes `function! Name(...) abort` — not as ceremony but as the behaviour that makes a failing function fail cleanly. The attribute composes with the exception machinery of Chapter 11: an error in an `abort` function aborts the function the way a `:throw` would unwind it, and a surrounding `:try/catch` can catch the resulting error. So `abort` is the function-level analogue of fail-fast, and writing it everywhere is the discipline that keeps a script's errors legible — the first thing that went wrong is the thing you see, not the tenth consequence of it. The parameter `n` is read inside the body as `a:n` — arguments live in the `a:` scope. The function calls itself, `Fib(a:n - 1) + Fib(a:n - 2)`, demonstrating that recursion works and that a function name is in scope within its own body.

That a function name is in scope within its own body is worth dwelling on, because it reflects how function definitions and the function namespace work. A `:function!` definition registers the name in the function registry *before* the body runs, so by the time `Fib` is first called, `Fib` is a name the engine can resolve — which is why a recursive call inside the body finds the function. The function namespace is separate from the variable namespace: a lowercase bareword like `fib` in a *call* position resolves to a function (builtin or user), while the same bareword in a *value* position resolves to a variable, which is the namespace separation that forces a Funcref-holding variable to be capitalized (Chapter 8). Mutual recursion works for the same reason a self-call does: define `IsEven` and `IsOdd`, each calling the other, and as long as both are defined before either is called, each resolves the other through the registry. The `function!` bang is what makes re-sourcing safe — it replaces an existing definition rather than erroring — so a library that defines `Fib` can be sourced

twice without complaint, the second definition replacing the first. Recursion, mutual recursion, and re-definable functions all fall out of one mechanism: the function name registered in a registry that the call path consults, separate from the variable scope, populated before the body runs.

Function names that contain a `#` are *autoload* names, and they trigger Vimscript's lazy-loading mechanism, which vimlrs implements. Calling `pkg#func()` sources `autoload/pkg.vim` on demand the first time, then runs the call; subsequent calls reuse the loaded function. The `autoload.vim` example builds a throwaway package at runtime to show it:

```
call writefile([
    \ 'function! demo#double(n) abort',
    \ '    return a:n * 2',
    \ 'endfunction',
    \ 'let g:demo_autoloaded = 1',
    \ ], 'autoload/demo.vim')
call assert_equal(10, demo#double(5))
call assert_equal(1, g:demo_autoloaded)
```

The first call to `demo#double` finds no such function, derives the file name from the prefix (`demo#` → `autoload/demo.vim`), sources it, and then dispatches. vimlrs resolves autoload files relative to the current directory, the standalone analogue of Vim's `runtimepath` search.

The autoload mechanism is worth seeing as the lazy-sourcing machine it is. A function name containing a `#` is parsed as an autoload name, and the prefix before the last `#` names the file: `pkg#sub#func` derives `autoload/pkg/sub.vim`. The first time such a name is called and no function of that name is yet defined, the engine derives the file path from the prefix, sources it (which runs its top-level statements and defines its functions), and then dispatches the original call. Subsequent calls find the function already defined and skip the sourcing. This is exactly Vim's `autoload-functions` feature, and it is what lets a large Vimscript codebase defer loading code it might never use — a plugin with fifty autoload functions pays to load only the files whose functions are actually called. The `autoload.vim` example builds a throwaway package at runtime and calls into it, proving the derive-source-dispatch sequence works; the file written to `autoload/demo.vim` is found, sourced, and its `demo#double` dispatched on the first call, with `g:demo_autoloaded` set as a side effect of the sourcing.

Dict-bound functions

A function can be bound to a dictionary, giving Vimscript a form of method dispatch, and vimlrs supports it through the same Funcref machinery the next chapter details. `function({name}, {dict})` produces a Funcref bound to a dict, and inside such a function the special variable `self` refers to the bound dict — so a function stored as a dict entry and called as `d.method()` can read and mutate `d` through `self`. The binding is carried on the `VAR_PARTIAL` value, the same value type that carries a partial's bound arguments, because a dict-bound funcref is just a partial that pins `self` rather than a positional argument. This is the mechanism behind Vim's object-ish dictionaries, where a dict holds both data fields and function-valued fields that operate on the dict. It is a less-used corner of the language than the plain Funcrefs and lambdas of the next chapter, but it is part of the first-class- function story, and vimlrs carries it on the same partial representation rather than inventing a separate binding mechanism — one more case of the ported value layer having exactly the variant the feature needs.

Returning, or not

`:return [expr]` ends a function with an optional value. A function with no `:return`, or a `:return` with no expression, yields the Number `0` — Vim's default. The `:call` command invokes a function and *discards* its return value, which is how you call a function for its side effects: `call add(list, x)` runs `add` for the mutation and throws away the (returned) list. A function call that appears in an expression keeps its value, naturally; the difference is whether the surrounding context is a `:call` statement or an expression position.

This `:call-discards-the-value` rule has a practical consequence worth dwelling on, because it interacts with the in-place-and-return convention of the list functions. When you write `call add(list, x)`, the `:call` discards `add`'s return value — but `add` mutated `list` in place, so the mutation persists even though the returned list is thrown away. This is the idiomatic way to use the mutating functions for their effect: `call add(...)`, `call extend(...)`, `call sort(...)` all run the function for the mutation and discard the (returned) container. By contrast, `let result = add(list, x)` keeps the return value, which is the *same* list — so `result` and `list` are now two names for one list, an aliasing that surprises programmers who expected `add` to return a new list. The distinction between `:call f(...)` (run for effect, discard value) and `let x = f(...)` (run for value) is the same command-versus-expression seam of Chapter 3, applied to function calls, and getting it right is the difference between a script that mutates correctly and one that accidentally aliases. A function with no `:return`, or a `:return` with no expression, yields the Number 0 — Vim's default — so even a pure side-effecting function has a value to discard, and `:call` is the statement that discards it.

Parameters: fixed, default, and variadic

A function's parameter list can mix three kinds of parameter, and vimlrs handles all three. Fixed parameters are the ordinary case. Optional parameters carry a default value, recorded in the AST as `(param index, default expr)` pairs — `func F(a, b = 10)` records `(1, Num(10))`, and the default “is evaluated at call time when the argument is omitted,” in the partially-bound `a:` scope. This matches Vim's `optional-function-argument` feature. The detail that makes optional arguments powerful rather than merely convenient is that the default is *evaluated at call time, in the partially-bound a: scope*, not stored as a constant. So a default can be any expression — a list literal, a function call, a ternary — and it can reference an earlier argument, because by the time it runs, the earlier arguments are already bound in `a:`. The compiler implements this by compiling each default to its own chunk (Chapter 21) that runs in the call prologue when its argument is omitted, after the supplied arguments are bound, which is exactly what makes the partially-bound-scope behaviour fall out naturally. A default that does not reference earlier arguments simply ignores the scope; a default that does `(a:lo + 10, abs(a:n))` reads it. The mechanism is the same in both cases: defaults are code, run at call time, in the scope built so far.

The third kind is varargs, written `...`, which collect all extra arguments. The `varargs.vim` example exercises them both alone and after fixed parameters:

```
function! Sum(...) abort
  let total = 0
  for x in a:000
    let total += x
  endfor
  return total
endfunction
call assert_equal(10, Sum(1, 2, 3, 4))

function! Tag(label, ...) abort
  return a:label . ': ' . a:0 . ' items'
endfunction
call assert_equal('y: 3 items', Tag('y', 1, 2, 3))
```

Inside a variadic function, `a:000` is the List of extra arguments and `a:0` is their count. In `Sum`, every argument is an extra (there are no fixed parameters), so `a:000` holds all of them. In `Tag`, the fixed parameter `label` is consumed first and `a:000` holds only what follows, so `a:0` is 3 for `Tag('y', 1, 2, 3)`. This is exactly Vim's rule, and it is the reason `a:0` and `a:000` showed up as keys of the `a:` dict back in Chapter 4.

The argument scope as a dictionary

There is something worth dwelling on in the fact that `a:` is a genuine *dictionary*, not just a set of named slots, because it reveals a design choice that is characteristically Vimscript and that a faithful port has to reproduce

even though a from-scratch implementation would almost certainly do it differently. In most languages, a function's parameters are slots — named locations the call binds and the body reads, with no reified existence beyond the names. In Vimscript, the parameters live in a dictionary, the `a:` scope, which means you can do things to them that slot-based parameters do not allow: you can iterate the parameters with `keys(a:)`, you can read one by computed name with `get(a:, name)`, you can check `has_key(a:, 'foo')`. The `ArgKeys` example in Chapter 4 sorts `keys(a:)` and gets `['0', '000', 'one', 'two']` — the named parameters plus the two pseudo-keys `0` (the vararg count) and `000` (the vararg list) — which is a slightly strange result, because it exposes the internal bookkeeping of the variadic mechanism as ordinary dictionary keys. A reimplementer who modelled parameters as slots would never produce this; they would have to deliberately build a dictionary view with exactly these keys, including the odd `0` and `000`, to match Vim. vimlrs reproduces it because the ported function-call machinery genuinely populates a dictionary, so the pseudo-keys fall out of the same mechanism that binds the named ones.

This dictionary-as-scope design is consistent across all the scopes — `g:`, `l:`, `s:`, and the rest are all dictionaries, which is why a bare `g:` with nothing after it is a Dict of every global, and why `type(g:)` is 4. The scopes are not abstract namespaces the compiler resolves; they are concrete dictionaries the engine maintains, and a scoped name like `g:foo` is, at bottom, a lookup of the key `'foo'` in the `g:` dictionary. This is slower than slot-based variable access — a dictionary lookup is a hash, not a register read — and it is exactly why the slot planner of Chapter 21 works so hard to lift provably-numeric locals *out* of the dictionary into fast slots: the default, dict-backed representation is the faithful one, and the slot is the optimization layered on top for the cases where it is provably safe. So the dictionary-scope model and the JIT's slot optimization are two layers of the same story — the dictionaries are the semantics, faithfully ported, and the slots are the speed, synthesized on top — and the fact that a script can read `a:` or `g:` as a dictionary is the visible surface of the dict-backed semantics that the JIT then works to bypass where it can.

Three ways to be a function

Vimscript's function model is broader than the named `:function` this chapter has focused on, and it is worth naming the three forms together because they are one of the language's richer corners and they all reduce, in vimlrs, to a small set of ported value types. The first form is the named function, defined with `:function!` and registered by name — the `Fib`, `FizzBuzz`, `Sum` of the examples. The second is the Funcref, a function used as a value: `function('name')` turns a named function or a builtin into a callable value you can store, pass, and call, and a Partial binds some of its arguments ahead of time. The third is the lambda, an anonymous function written inline, `{args -> body}`, which captures its enclosing locals. These look like three different features, and in many languages they would be three different mechanisms, but in vimlrs they collapse onto two ported value types: a function is either registered in the function table by name, or it is a `VAR_FUNC/VAR_PARTIAL` value carrying a reference to a function plus optional bound arguments and a bound dict. A named function is a table entry; a Funcref is a value pointing at a table entry; a Partial is a Funcref plus bound arguments; a lambda is a generated table entry plus a Partial that pins its captures; a dict-bound method is a Partial that pins `self`. The whole first-class-function story — named functions, references to them, partial application, closures, methods — is built from two value types and a function table, which is why the bridge can dispatch all of them through one `call_func` path and why the compiler can desugar a lambda into a generated function plus a `function()` call. The surface is three forms; the substrate is two value types; and the reduction of the rich surface to the small substrate is exactly the kind of thing the ported value layer makes possible, because Vim's own value model already unifies them this way.

Dict functions and the funcref bridge

Functions and the data layer meet in two directions. A function can be stored and passed as a value — a Funcref — which the next chapter is entirely about. And the higher-order builtins `map`, `filter`, `sort`, `reduce`, `call`, and `foreach` accept a function in any of the forms Vimscript allows: a string expression (evaluated with `v:val` bound), a named-function Funcref, or a lambda. The status table notes these work over “lists and dicts; string-expr + funcref,” and the `lists_dicts.vim` example shows the spread:

```
let evens = filter(copy(nums), 'v:val % 2 == 0')
let squared = map(copy(nums), 'v:val * v:val')
let total = reduce(nums, function('Add'), 0)
```

`filter` and `map` here take *string-expression* bodies — `'v:val % 2 == 0'` is a string that the builtin evaluates against each element with `v:val` bound to the element. `reduce` takes a Funcref built by `function('Add')`. That all three forms — string, funcref, and the lambda we will meet next — are interchangeable in these positions is a real Vimscript feature, and vimlrs's filter/map machinery exposes the hooks (`FILTER_MAP_EVAL_HOOK`, `FILTER_MAP_CMD_HOOK`) that let the ported list functions call back into the bytecode evaluator to run whichever form the caller supplied. The fact that the ported `f_map/f_filter` in `list.rs` can invoke compiled VimL is one of the more intricate seams between the ported zone and the synthesis zone, and it is what makes the standard higher-order functions feel native rather than bolted on.

Arity checking

Before a builtin runs, its argument count is validated, and `builtin_arity.vim` shows that vimlrs reproduces Vim's exact error codes for getting it wrong. Vim validates a call against its `funcs[]` table — generated from `eval.lua` — and vimlrs mirrors that table, so a too-few or too-many call is a clean error rather than a crash in the function body:

```
call assert_fails('call get([])', 'E119')
call assert_fails('call abs()', 'E119')
call assert_fails('call abs(1, 2)', 'E118')
call assert_fails('call len([1], 2)', 'E118')
```

Too few arguments is `E119`, too many is `E118`, and the codes are faithful — a script that checks for them gets the right one. The guard applies on every dispatch path, not just direct calls: `call('get', [[]])` raises `E119` through the dynamic `call()` path, and a Funcref built with `function('add')` is arity-checked when invoked. Variadic builtins accept any count past their minimum, so `printf('a')` with no further arguments is fine while `printf` with a format but missing arguments has its own behaviour. This matters more than it first appears: a mis-arity call that reached the function body could read past its arguments and produce garbage or panic, and Vim's contract is that it errors *before* the body runs. vimlrs honours that contract because it carries the same arg-count metadata the reference's signatures are derived from — the same table, recall from Chapter 25, that the `builtin_docs.rs` signatures come from — so the documented signature and the runtime arity check are two views of one source.

The calling convention

It is worth seeing how a function call actually works underneath, because it ties together the `a:` scope, the defaults, and the varargs into one mechanism. The compiler records each function as a `UserFuncDef` (Chapter 21) carrying its name, its parameter names “without the `a:` prefix,” its compiled default-value chunks as (`param index`, `chunk`) pairs, its bang flag, and a `varargs` flag set when the signature ends in `...`. At call time, the prologue binds the arguments into the `a:` scope in order, and three things happen that the earlier sections established piece by piece.

First, each supplied argument binds to its named parameter — `a:name`. Second, if a parameter was omitted and it has a default, the default's compiled chunk runs to produce the value, and crucially it runs “in the partially-bound `a:` scope,” which is why `default_args.vim` can write `function! Span(lo, hi = a:lo + 10)` and have the default see `a:lo` already bound. The defaults are compiled chunks, not stored expressions, so a default is real code evaluated at call time, not a literal pattern. Third, any arguments beyond the named parameters collect into `a:000`, the List of extras, with `a:0` holding their count and each extra also reachable positionally as `a:1`, `a:2`, ... `a:N` (the two stay in sync, as `varargs.vim` asserts) — which is why a pure-varargs `Sum(...)` sees everything in `a:000` while `Tag(label, ...)` sees only what follows the fixed parameter.

This is also where Chapter 4's point about `a:` being a real Dict pays off: the keys of `a:` are the named

parameters plus the pseudo-keys `0` and `000`, because the prologue genuinely populates a dictionary with those entries. And it is where the JIT boundary sits: a callee cannot see the caller's slotted `l:` locals, because they are slots in the caller's frame, not entries in a shared dict — which is exactly why Chapter 14 could say a hot loop traces even when the same function calls helpers. The calling convention keeps each frame's fast locals private, binds the slow `a:` scope as a dict, and runs defaults as code, and those three choices together are what make functions both faithful and fast.

vim9script: the def form and its dialect

Vim 9 introduced a second dialect, vim9script, and vimlrs parses it. A file that opens with the `:vim9script` marker — or any `def ... enddef` body, even one reached from a legacy script — switches the parser into vim9 mode, and `has('vim9script')` answers 1 to match Vim 9.2. The headline is the `def` function form, the vim9 counterpart to `:function`:

```
vim9script
def Add(x: number, y: number): number
    return x + y
enddef
echo Add(2, 3)    " 5
```

Two things distinguish `def` from `:function`. First, parameters are *bare*: inside the body you write `x` and `y`, not `a:x` and `a:y`. vimlrs realises this with the same trick the `{->}` lambda compiler uses — a synthesised `let x = a:x` prologue — so the parameters still bind through the existing `a:` call machinery yet resolve as ordinary bare locals, and optional `= default` values run through the shared function path exactly as they do for a legacy definition. Second, `def` carries type annotations: `x: number`, the `: number` return type, `list<number>`. vimlrs parses those annotations and then discards them — the vim9 type system is deferred, so the types are accepted for syntax but not checked. What the assertions pin is the runtime value, and every vim9 example is binary-verified against Vim 9.2.

The other structural difference is automatic line continuation. In a vim9 region an expression joins the next physical line whenever a `[`, `{`, or `(` is still open, a line ends on a trailing binary operator, or the next line *starts* with an operator, a `->`, a `.`, a `?/:` ternary arm, a closing bracket, or a `|`. Inside the region, `#` comments are dropped. This is why a vim9 `def` can spread a list, a call, or a `?:` chain over several lines with no `\` continuation markers, the way `vim9_def.vim` does throughout.

var, const, and bare reassignment

vim9script replaces `:let` with `var`, and adds `final` and `const`. Each strips an optional `: type` annotation and then declares like a `let`: `var n: number = 10` binds `n` to 10, and the annotation is parsed and ignored. A *type-only* declaration with no initializer — `var s: string`, `var l: list<number>` — default-initializes to that type's zero value: `''` for a string, `0` for a number, `[]` for a list, `{}` for a dict, `false` for a bool, `0.0` for a float, as `vim9_var_assign.vim` pins.

Assignment to an already-declared variable drops the keyword entirely: `n = 5`, `n += 3`, `s .= 'c'`, `d['k'] = 9` are all bare reassignments, the vim9 way. The bracketed list-unpack form works keyword-free too — `[a, b] = expr`, the rhs fully built before any target is bound (so `[a, b] = [b, a]` swaps), and `[head; rest] = list` soaks the remainder into `rest`. A CamelCase name on the left (`Total += 7`) assigns as a variable rather than being read as a user command.

Bare literals, bare-key dicts, and arrow lambdas

Three expression-level changes come with the dialect. Bare `true`, `false`, and `null` are keyword literals equal to `v:true`, `v:false`, and `v:null` — Bool (type 6) and Special (type 7) values, not the undefined names they

would be in legacy Vimscript. Alongside them are the predefined `null_*` constants — `null_string`, `null_list`, `null_dict`, `null_blob`, `null_function`, `null_partial` — each the null value of its type, so `null_string` is an empty String and `null_list` stringifies to `[]`; a real runtime file (`autoload/dist/vim9.vim`) writes `var os_viewer = null_string`, and before these were ported it failed to load with E121.

Dictionary literals change spelling: in a vim9 region `{key: value}` uses *literal* keys, so `{a: 1}` has the string key `"a"`, not the value of a variable `a` (the legacy meaning). Quoted keys (`{'k': 9}`) and `[expr]` computed keys still work, so `{[key]: 1}` evaluates `key` and stringifies it. And the lambda gains a second form: `(params) => body`, distinct from the legacy `{args -> body}`. The arrow form takes `: type` annotations (parsed and discarded) and reads naturally in method chains — `[1, 2, 3]->mapnew(_, v) => v * v)`, `list->sort([a, b] => a - b)` — which is how `vim9_lambda.vim` exercises it.

Finally, definitions may carry the `export` modifier — `export def`, `export var`, `export const`. Editor-less there is no import machinery for it to feed, so the marker has no runtime effect; what matters is that the definition still *registers* (an `export def` body defines a function rather than running as top-level statements).

What vim9script does not yet do

The honest boundary is the same documented-versus-working split the rest of the book draws. vimlrs parses and *runs* vim9script — the marker, `def/enddef`, `var/final/const`, bare literals and bare-key dicts, arrow lambdas, the `null_*` constants, and vim9 line continuation — and every feature above is binary-verified against Vim 9.2. What it does not yet do is the vim9 *type system*: `: type` annotations are parsed for syntax but neither checked nor coerced, so a type mismatch that Vim 9 would reject at compile time runs in vimlrs. `:class/:enum` blocks and `import/export` resolution are likewise deferred — a class or enum block is consumed as a tolerant unit rather than modelled — so a script that leans on vim9 typing or objects is parsed without error but not given those semantics. The dialect is real and the runtime values are faithful; the type layer is the named next step.

Chapter 7 — Lists, Dictionaries, and Blobs

Three container types carry almost all of Vimscript's data: the List, the Dict, and the Blob. The first two you will use constantly; the third, a raw byte buffer, shows up wherever binary data does. This chapter covers their literals, their indexing and slicing, the operations the ported standard library provides over them, and the conversions between them. All three were among the first things ported — the value layer is the “faithful substrate” the rest of the engine is built on — so their behaviour is Vim's down to the edge cases.

List literals, indexing, and slicing

A List is written `[a, b, c]`, heterogeneous and ordered. It indexes with `[i]`, including negative indices from the end, and slices with `[from:to]` where both bounds are inclusive and either may be omitted. From `lists_dicts.vim`:

```
call assert_equal([3, 4, 5], nums[2:4])
call assert_equal('viml', user.langs[0])
```

The slice `nums[2:4]` takes elements at indices 2, 3, and 4 — note the *inclusive* upper bound, which is a Vimscript-ism that trips up programmers used to Python's half-open slices. The AST represents these as `Index { base, index }` and `Slice { base, from, to }` with the bounds as `Options`, so an omitted bound is a real `None` rather than a sentinel.

The List standard library is the ported `list.rs` plus list-shaped functions in `funcs.rs`, and the reference's “List manipulation” chapter catalogues 22 of them: `add`, `copy`, `count`, `deepcopy`, `flatten`, `flattennew`, `foreach`, `index`, `indexof`, `insert`, `mapnew`, `max`, `min`, `range`, `reduce`, `remove`, `reverse`, `slice`, `sort`, `uniq`, and the blob converters `blob2list`/`list2blob`. The `sort` builtin deserves a note: it takes an optional comparator, which may be the string `'n'` for numeric sort, a Funcref, or a lambda. The `lists_dicts.vim` example sorts numerically and then reverses:

```
let desc = reverse(sort(copy(nums), 'n'))
call assert_equal([10, 9, 8, 7, 6, 5, 4, 3, 2, 1], desc)
```

The `copy(nums)` is not decoration. Many of these functions mutate in place and return the same list — `sort`, `reverse`, `map`, `filter`, `add` all do — so the example copies first to keep `nums` itself intact. This in-place-and-return convention is Vim's, and it is why `copy()` and its deep cousin `deepcopy()` are used so liberally in idiomatic code.

The list operations divide cleanly into the mutators and the non-mutators, and knowing which is which is half of using them correctly. The mutators — `add`, `insert`, `remove`, `extend`, `sort`, `uniq`, `reverse`, `map`, `filter`, `flatten` — change their first argument in place and return it, so they must be applied to a copy when the original matters. The non-mutators — `copy`, `deepcopy`, `slice`, `range`, `mapnew`, `extendnew`, `flattennew`, `count`, `index`, `indexof`, `max`, `min`, `reduce`, `foreach` — leave their argument intact and return a fresh value (or a scalar). The naming is a deliberate signal: the `new` suffix on `mapnew`, `extendnew`, and `flattennew` marks the non-mutating variant of a mutating function, so when you want a fresh container you reach for the `new` form and when you want in-place efficiency you reach for the bare form. `reduce` and `foreach` are non-mutating in the sense that they do not rebuild the container — `reduce` folds it to a scalar, `foreach` runs a body for effect and returns the original — though the body they run could mutate the elements. This split

is Vim's, and it is why idiomatic Vimscript is dotted with `copy()`: the mutators are efficient but sharp, and `copy()` is the guard that lets you use them without clobbering data you meant to keep. A script that forgets the guard — `sort(nums, 'n')` instead of `sort(copy(nums), 'n')` — sorts the original out from under whatever else held a reference to it, the aliasing hazard the reference semantics make real.

Sort comparators and stable ordering

The `sort()` builtin deserves a closer look, because its comparator argument is one of the places the three callable forms of Chapter 6 all appear. With no comparator, `sort` orders by Vim's default string comparison; with the string `'n'` it sorts numerically; with `'i'` case-insensitively; and with a Funcref or lambda it uses the caller's comparison, which must return a negative, zero, or positive Number the way `qsort` expects. The `wordfreq.vim` example sorts `(word, count)` pairs by descending count with a named comparator returning `a:b[1] - a:a[1]`, and the `lambdas.vim` example sorts descending with the lambda `{a, b -> b - a}`. The sort is in place and returns the same list, which is why `reverse(sort(copy(nums), 'n'))` copies first. There is one faithful subtlety: Vim's `sort` is stable, preserving the relative order of equal elements, and vimlrs's ported `f_sort` preserves it too — a detail that only matters when you sort by a key that leaves ties, but one a careful comparator-based sort depends on.

Copy, deepcopy, and the reference trap

The distinction between `copy()` and `deepcopy()` is the one that catches programmers who forget that Vimscript containers are reference types. `copy()` makes a *shallow* copy: a new list whose elements are the *same* objects as the original's, so mutating a nested list inside the copy mutates it inside the original too. `deepcopy()` makes a *recursive* copy, cloning nested containers all the way down, so the copy is fully independent. The `is/isnot` identity operators (Chapter 5) are how you observe the difference: after `let b = copy(a)`, `b isnot a` (different outer list) but `b[0] is a[0]` (same nested element), while after `let c = deepcopy(a)`, `c[0] isnot a[0]` too. The `{noref}` argument to `deepcopy` controls how shared references and cycles within the structure are handled — by default it preserves sharing (two references to the same sub-list in the original become two references to one copied sub-list), and with a truthy `{noref}` it copies each occurrence separately. These are exactly Vim's semantics, ported, and they matter because the in-place-and-return convention of the mutating list functions makes accidental shared mutation a real hazard — which is why idiomatic code reaches for `copy()` so often, and `deepcopy()` whenever the structure is nested and the original must be protected.

Reference semantics and why they matter

The reason `copy()` appears so often, and the reason the `is/isnot` operators exist at all, is that Vimscript's containers are reference types, and understanding what that means concretely is worth a paragraph because it is the source of a whole class of bugs that the language deliberately exposes rather than hides. When you write `let b = a` for two scalars — two numbers, two strings — `b` gets a copy of `a`'s value, and changing `b` later does not touch `a`; scalars are value types. But when `a` is a List or a Dict, `let b = a` does *not* copy the container — it makes `b` a second name for the *same* container, so a mutation through `b` is visible through `a` and vice versa. This is reference semantics, and it is the same model Python, Ruby, and JavaScript use for their collections, but it surprises programmers who expect assignment to copy, and it is exactly the hazard the in-place-and-return convention of the list functions makes dangerous: `sort(a, 'n')` sorts `a` in place, so if some other variable `b` is a second name for `a`, that variable's list is sorted too, out from under it. The defensive idiom is to copy before mutating — `sort(copy(a), 'n')` — which makes a fresh list to sort, leaving `a` and anything that shares it intact. vimlrs reproduces reference semantics exactly because the retpool (Chapter 22) gives a container a stable identity that two names can share, and the ported value layer assigns by sharing that identity rather than by copying.

The `is/isnot` operators exist precisely to let a script ask the question reference semantics makes meaningful: not “are these two containers equal?” but “are these two names the *same* container?” The `compare.vim` example pins the distinction with three lists — `a`, `b` assigned from `a`, and `c` built independently with the same contents — and asserts `a is b` (same container), `a isnot c` (different containers), and `a == c` (equal contents). All three are true, and they are not contradictory: identity and equality are different questions, and Vimscript lets you ask both because both matter. A dict watcher checking whether the list it was handed is the one it stored needs `is`; a test checking whether two computations produced equal results needs `==`. A language with only `==` could not express the first question, and a language that conflated `is` with `==` — making `is` an alias that compares contents — would answer the first question wrong, as a from-scratch implementation that did not realize the distinction probably would. vimlrs keeps them distinct because the ported `typval_compare` implements `is` as a pointer-identity check on the container and `==` as a structural comparison, and the `compare.vim` assertions are the proof that the port reaches the layer where identity and equality diverge. The whole apparatus — reference assignment, the copy functions, the identity operators — is one coherent design: containers are shared by reference, you copy when you need independence, and you ask `is` when you need to know whether two names point at one thing. It is more to think about than value semantics would be, and it is exactly Vim’s, ported intact.

Dict literals and the two spellings

A Dict is written `{'key': value, ...}` with string keys, and it iterates in insertion order (Chapter 5). Members read two ways: subscript `d['key']` and the dotted `d.key`, which are equivalent for string keys. There is also a second literal syntax, the *literal-key* dict `#{...}`, where the keys are bare words needing no quotes. The lexer recognizes `#{` as its own `HashBrace` token. From `lists_dicts.vim`:

```
call assert_equal({'id': 1, 'tag': 'x'}, #{id: 1, tag: 'x'})
call assert_equal({'z': 1}, #{z:1})
call assert_equal({'a': 1, 'b': 2}, #{a:1, b:2})
```

The literal-key form has a sharp edge the example calls out: a key works “without a space after `:`”, including single-char and scope-letter keys (which the lexer would otherwise treat as a scope sigil like `a:/s:`.) So `#{a:1}` is the dict `{'a': 1}`, *not* a reference to the `a:` argument scope — the lexer has to know it is inside a `#{ }` literal to read `a:` as a key rather than a sigil. This is the sort of context-sensitivity that makes a Vimscript lexer harder than it looks, and it is exactly the kind of thing the example suite exists to pin down. The `literal_dict.vim` example draws the sharpest version of the distinction by contrasting `#{ }` with `{: }`: `let lit = #{foo: 10}` makes the dict `{'foo': 10}`, but `let expr = {foo: 10}` evaluates `foo` as an *expression* — so if `foo` is a variable holding `'EXPANDED'`, `expr` has the key `'EXPANDED'`, not `'foo'`. The two literal forms are not cosmetic variants; they differ in whether the keys are evaluated. The `#{ }` form is for when your keys are known words you do not want quoted or evaluated — configuration dicts, record-like structures — and the `{: }` form is for when your keys are computed. A digit-leading key like `#{200: 'ok'}` is valid in the literal form precisely because the key is literal text, not an identifier, so it produces the string key `'200'`. vimlrs implements the `#{ }` form through the lexer’s `HashBrace` token and the parser’s literal-dict routine that splits the absorbed scope-colon back apart, which is the machinery that makes `#{a:1}` the dict `{'a': 1}` rather than a reference to the `a:` argument scope.

The Dict standard library — the reference’s “Dictionary manipulation” chapter — is `dictwatcheradd`, `dictwatcherdel`, `extend`, `extendnew`, `get`, `has_key`, `items`, `keys`, and `values`. The `get(d, key, default)` form is the safe read used everywhere: `get(freq, w, 0)` returns the count for `w` or `0` if absent, which is the heart of the frequency-counter idiom. `keys`, `values`, and `items` give you the three views, and `has_key` tests membership.

The `get` function deserves singling out as the universal safe read, because it works across three container types and is the idiom that makes container access robust. `get(dict, key, default)` returns the value for `key` or the `default` if the key is absent; `get(list, index, default)` returns the element at `index` or the `default` if the index is out of range; and `get(blob, index, default)` returns the byte value or the default. In every case it never errors on a missing key or index — it returns the default — which is what distinguishes

it from the subscript syntax `d[key]` or `l[i]`, which *do* error on absence. So `get` is the function you reach for whenever a key or index might not be present: the frequency idiom `get(freq, w, 0)` reads the current count or zero, `get(config, 'timeout', 30)` reads a setting or a fallback, `get(args, 0, '')` reads the first argument or empty. The one faithful edge is that `get` on a *String* is an error, [E1531](#), not a silent default (`index_get.vim`), because a string is not a keyed collection — the safe-read semantics apply to the three keyed containers, not to strings. With `get` for the safe read, `has_key` for the explicit membership test, and the subscript syntax for the read-that-should-exist, a script has three levels of strictness to choose from, and choosing `get` is the robust default whenever absence is possible.

The dictionary standard library in practice

The nine Dict functions compose into a small, complete vocabulary for working with maps, and seeing them together shows how they cover the operations a script actually performs. Reading is `get(d, key, default)` for the safe lookup and `has_key(d, key)` for the membership test, with the dotted `d.key` and subscript `d['key']` syntax for a key known to exist. The three views — `keys(d)`, `values(d)`, and `items(d)` — turn a dict into lists you can iterate, sort, or map over; `items(d)` in particular yields the `[key, value]` pairs that a `:for [k, v]` destructures, the idiom at the heart of iterating a dict. Writing is the index-assignment `let d[key] = value` (which also fires any registered watcher), and merging is `extend(d, other)` in place or `extendnew(d, other)` for a fresh dict. Removing is `remove(d, key)`, which returns the removed value, or `:unlet d.key`. The frequency-counter idiom that recurs through the examples — `let freq[w] = get(freq, w, 0) + 1` — is two of these functions working together: `get` reads the current count or a zero default, and the index assignment writes it back, building a histogram in one line per element. With just `get`, `has_key`, the three views, `extend`, `remove`, and the index-assignment `lvalue`, a script has everything it needs to build, query, merge, and tear down maps — and the watcher pair adds reactive mutation on top, for the rarer case where a dict needs to notify on change.

The higher-order trio over both

`map`, `filter`, and `reduce` work over Lists and Dicts alike, and over both they accept the three callable forms — string expression, Funcref, lambda. Over a Dict, the index passed to the callback is the key rather than a numeric position. The `wordfreq.vim` example builds a frequency table and sorts the pairs with a named comparator:

```
function! ByCountDesc(a, b) abort
    return a:b[1] - a:a[1]
endfunction
call sort(pairs, function('ByCountDesc'))
call assert_equal(['the', 3], pairs[0])
```

The comparator returns a negative, zero, or positive Number the way `qsort` expects, and `function('ByCountDesc')` turns the named function into the Funcref `sort` needs. Everything composes: `uniq(sort(copy(words)))` deduplicates a sorted copy, `len()` counts, and the result is asserted exactly.

Blobs

A Blob is a buffer of bytes, written with the `0z` prefix followed by hex digit pairs. The lexer parses `0z00112233` into the four bytes directly, and `.` may group the pairs for readability. From `blobs.vim`:

```
let b = 0z00112233
call assert_equal(4, len(b))
call assert_equal(17, b[1])
call assert_equal(0z001122, 0z00.11.22)
```

A Blob indexes to the byte value at a position (`b[1]` is 17, the decimal value of hex `11`), slices with byte indices via `slice()` (exclusive end, unlike list slicing's inclusive end), and round-trips to and from a List of byte-value Numbers via `blob2list` and `list2blob`:

```
call assert_equal([0, 17, 34, 51], blob2list(b))
call assert_equal(b, list2blob([0, 17, 34, 51]))
```

The mutating list operations `add`, `insert`, and `remove` work on a Blob's bytes too, and they enforce the byte range — `insert(0x0102, 300)` fails with `E475` because 300 is not a byte. Hex digits are case-insensitive, so `0zdeadbeef` equals `0zDEADBEEF`. Blobs are the least-used of the three containers in day-to-day scripting, but they are exactly what you reach for to hold the result of `readblob()` or to build binary data for `writefile()`, and vimlrs implements their literal syntax, indexing, slicing, and conversions faithfully.

The Blob's operation set is worth collecting, because it is broader than its rare use suggests and because it shares machinery with the List. The literal is `0z` followed by hex pairs, with `.` allowed to group them for readability (`0z00.11.22` equals `0z001122`) and hex digits case-insensitive (`0zdeadbeef` equals `0zDEADBEEF`). Indexing `b[i]` returns the *byte value* as a Number — `0z00112233[1]` is 17, the decimal of hex `11` — and a negative index counts from the end. Slicing uses byte indices, with the `slice()` function's exclusive end (unlike list slicing's inclusive end). The mutating list functions work on a Blob's bytes too: `add`, `insert`, and `remove` operate on bytes and *enforce the byte range*, so `insert(0x0102, 300)` fails with `E475` because 300 is not a byte. And the converters `blob2list` and `list2blob` round-trip a Blob to a List of byte-value Numbers and back. The interesting design fact is that the parser builds a `0z` literal by *desugaring* it to a `list2blob` call over the decoded byte list (Chapter 20), reusing the ported `list2blob` builtin rather than constructing the Blob in the parser — so even the literal syntax routes through the faithful conversion. A Blob is a niche type, but its full vocabulary — literal, byte index, byte slice, range-enforced mutation, list conversion — is present and ported, which matters whenever a script genuinely handles binary data, from a `readblob` of a file to a `writefile` of a constructed buffer.

Searching and folding: `indexof` and `foreach`

Two builtins find and iterate without building intermediate lists, and both are faithful to a `funcs.c` detail worth seeing. `indexof({obj}, {expr} [, {opts}])` returns “the index of the first item for which {expr} is true, or -1,” where the predicate may be a string expression (seeing `v:key/v:val`) or a funcref taking (`key`, `val`). The `indexof.vim` example exercises both forms over a List and a Blob:

```
call assert_equal(2, indexof([10, 20, 30, 40], 'v:val == 30'))
call assert_equal(1, indexof([10, 20, 30], {i, v -> v == 20}))
call assert_equal(2, indexof([5, 5, 5], 'v:key == 2'))
call assert_equal(1, indexof(0z0A0B0C, {i, v -> v == 11}))
```

Over a Blob the predicate sees each byte value, so `indexof(0z0A0B0C, {i, v -> v == 11})` finds the byte `0x0B` at index 1. The faithful detail is in the `startidx` option's edge cases, which differ by type: “a List uses a user index — out of range yields -1; a Blob clamps a very-negative start to 0.” So `indexof([1,2,3], ..., {'startidx': -99})` is -1 while `indexof(0z0A0B, ..., {'startidx': -99})` is 0 — two different out-of-range rules, both ported from the separate `indexof_list` and `indexof_blob` paths in `funcs.c`. A reimplementation would almost certainly give them one shared rule and be wrong on one of them.

`foreach({expr1}, {expr2})` is the side-effecting cousin of `map`: it applies its body to each item “for its side effects” and returns the original container unchanged, which is the right tool when you want iteration without transformation and without a `:for` loop's ceremony. Together with `map`, `filter`, `reduce`, and `indexof`, it completes the higher-order vocabulary over both Lists and Dicts, all accepting the same three callable forms — string expression, funcref, lambda — that Chapter 6 established.

repeat over three types

`repeat({expr}, {count})` is a small function with a faithful three-way behaviour that `repeat.vim` pins down. Over a String it concatenates copies, over a List it repeats the items into a new List, and over a Blob it repeats the bytes:

```
call assert_equal('ababab', repeat('ab', 3))
call assert_equal([1, 2, 1, 2, 1, 2], repeat([1, 2], 3))
call assert_equal(0z010201020102, repeat(0z0102, 3))
call assert_equal('', repeat('ab', 0))
call assert_equal('555', repeat(5, 3))
```

A count of zero or less yields the empty value of the right type — the empty string, the empty list, the empty blob. And the last assertion is the faithful quirk: `repeat(5, 3)` is `'555'`, because a Number argument is stringified first, exactly as Vim does. The pattern across `indexof`, `foreach`, and `repeat` is the one this whole book keeps returning to: each is a small function, each has a specific behaviour over each container type, and each is right in vimlrs because it was ported from the function that defines it rather than guessed from its name.

Chapter 8 – Funcrefs, Lambdas, and Closures

A function in Vimscript is also a value. You can store it in a variable, pass it to `map()`, bind some of its arguments and call the rest later, or write it inline as a lambda that captures the variables around it. This first-class-function story is one of the richer parts of the language, and vimlrs implements all of it — named-function Funcrefs, builtin Funcrefs, Partial with bound arguments, lambdas with closure capture, and the autoload-resolved funcref. This chapter takes them in order of increasing cleverness.

How a configuration language grew first-class functions

It is worth a moment of surprise that Vimscript has first-class functions at all, because nothing about a 1990s editor configuration language demanded them, and tracing how they got there explains the slightly awkward shape they have. A configuration language could have stopped at named functions you call by name — that is enough to extend an editor, and many extension languages never went further. But Vimscript kept growing, and as it grew it accumulated the higher-order builtins — `sort` with a comparator, `map` and `filter` with a transformation, `reduce` with a folding function — and those builtins need a way to be *handed* a function, not just to call one by name. So the Funcref appeared: a value that refers to a function, which you can pass to `sort` so `sort` can call your comparator. Then partial application appeared, because a callback often needs configuration fixed ahead of time, giving the Partial. Then lambdas appeared, because writing a named function for a one-line comparator is tedious, giving anonymous inline functions with closure capture. Each step was a response to a concrete need in extending the editor, and the result is a genuinely capable first-class-function story — but one that grew incrementally rather than being designed up front, which is why it has the seams it has.

Those seams are visible in the awkward corners this chapter will tour. The capitalization rule — a Funcref-holding variable must start with an uppercase letter — exists because the function namespace and the variable namespace are separate, a separation that predates first-class functions and that they had to be fitted into rather than designed around. The string-expression form of `map` and `filter` — passing `'v:val * 2'` as a string — is an older mechanism that predates lambdas and survives alongside them, which is why the higher-order builtins accept three interchangeable callable forms rather than one. The `function()` builtin that constructs a Funcref from a name is more verbose than a language designed for first-class functions would need, a consequence of functions being named-and-called first and made into values later. vimlrs reproduces all of these seams faithfully, because they are part of the language — the capitalization rule, the three callable forms, the `function()` construction — and a port that smoothed them away would be a cleaner language that was no longer Vimscript. The first-class-function story is rich and a little awkward, the richness and the awkwardness both products of a configuration language growing capabilities it was not originally designed for, and the chapter takes the features in order of increasing cleverness precisely because that order roughly retraces the order in which the language grew them.

Funcrefs to named and builtin functions

`function('name')` produces a Funcref — a callable value that refers to the named function. The name may be a user function or, crucially, a *builtin*: the `funcrefs.vim` example's whole point is that “Vim lets you `call()`, `function()` and store a funcref to any builtin ... vimlrs resolves the name to its ported builtin.”

```
let Upper = function('toupper')
call assert_equal('HELLO', Upper('hello'))
call assert_equal('3-4', call('printf', ['%d-%d', 3, 4]))
call assert_equal(5, call('abs', [-5]))
```

`function('toupper')` makes a Funcref to the builtin `toupper`; calling it through the variable `Upper` dispatches to the ported `f_toupper`. The sibling builtin `call(name, arglist)` invokes a function by name with an argument *list*, and it too reaches builtins — `call('printf', ['%d-%d', 3, 4])` runs `printf` with two arguments unpacked from the list. The convention that a Funcref-holding variable is capitalized (`Upper`, `Add`, `Censor`) is a Vimscript requirement, not a style choice: an unscoped variable that holds a Funcref must start with an uppercase letter, because lowercase bare names are reserved for the builtin-function namespace.

Partials

A Partial is a Funcref with some leading arguments already bound. `function('name', [arg, ...])` binds those arguments; the rest are supplied at the call site. From `funcrefs.vim`:

```
let Censor = function('substitute', ['hello world'])
call assert_equal('hell0 w0rld', Censor('o', '0', 'g'))
```

`Censor` is `substitute` with its first argument pinned to `'hello world'`. When you later call `Censor('o', '0', 'g')`, those three become `substitute`'s second, third, and fourth arguments, so the whole call is `substitute('hello world', 'o', '0', 'g')`. Partials are the standard Vimscript tool for currying configuration into a callback, and vimlrs carries them as the `VAR_PARTIAL` value type with the ported `f_funcref/f_function` builtins constructing them. The currying use is the common one: a callback often needs configuration that is known when the callback is *set up* but not when it is *called*, and a partial pins that configuration. A timer callback that needs a buffer number, a sort comparator that needs a key, a map function that needs a multiplier — each is a partial that binds the configuration argument and leaves the rest for the call site. The bound arguments always come *first*, prepended to the call-site arguments, which is why a partial curries from the left — you fix the leading arguments and supply the trailing ones, so `Censor('o', '0', 'g')` becomes `substitute('hello world', 'o', '0', 'g')` with the bound subject in front. This left-currying is exactly Vim's convention, and because the bound arguments live on the `VAR_PARTIAL` value, a partial can be stored, passed, and called like any Funcref, carrying its pinned configuration with it — which is what makes it the right tool for configuring a callback that will be invoked somewhere else, later, by code that does not know the configuration.

Lambdas

A lambda is `{args -> body}`: an anonymous function written inline. vimlrs lexes and parses them natively and, as the AST comment says, “desugars to an anonymous function returning `body`.” The compiler generates a uniquely named `<lambda>N` function for each and folds it into the program's function table. Lambdas are what make `map/filter/sort/reduce` read cleanly, because you no longer need a string expression or a separate named function. The readability gain is real and worth stating: compare `map([1,2,3,4], {i, v -> v * v})` with the string-expression form `map([1,2,3,4], 'v:val * v:val')` and the named-function form that requires defining `Square` elsewhere and passing `function('Square')`. The lambda puts the transformation inline, at the call site, where you read it — no string quoting, no `v:val` magic variable, no jump to a definition. For a one-expression callback, which is the common case, the lambda is the clearest of the three forms, which is why idiomatic modern Vimscript reaches for it. The trade-off is the single-expression constraint: a callback that needs statements must be a named function, so the lambda is the terse choice precisely when the body is small. The three forms remain interchangeable — anywhere a lambda works, a string expression or a Funcref works too — so the choice is about readability, not capability, and the lambda wins it whenever the body fits in one expression. From `lambdas.vim`:

```
call assert_equal([1, 4, 9, 16], map([1, 2, 3, 4], {i, v -> v * v}))
call assert_equal([2, 4, 6], filter([1, 2, 3, 4, 5, 6], {i, v -> v % 2 == 0}))
call assert_equal([3, 2, 1], sort([3, 1, 2], {a, b -> b - a}))
call assert_equal(10, reduce([1, 2, 3, 4], {acc, v -> acc + v}, 0))
```

The first lambda argument in `map/filter` is the index (or key), the second the value — `{i, v -> i . ':' . v}` builds `'0:a'`, `'1:b'` from a list. A lambda can be stored in a variable and called directly (`let Add = {x, y -> x + y}` then `Add(3, 4)`), or with no arguments at all (`call({-> 42}, [])`). The body is a single expression, which is the whole grammar of a lambda — there are no statements inside one, only an expression that becomes the return value.

The single-expression constraint is worth dwelling on, because it shapes how lambdas are used and why they desugar so cleanly. A lambda cannot contain a `:let`, an `:if`, or a loop — its body is one expression — so anything that needs statements must be a named `:function`. This is not a limitation vimlrs imposes; it is Vim’s lambda grammar, and it is exactly what makes the desugaring of Chapter 21 straightforward: a lambda becomes a generated function whose body is `return <the expression>`, with the captured free variables bound as leading parameters. Because the body is one expression, there is no control flow to lower, no local scope to manage beyond the captures — just an expression to evaluate and return. The constraint also pushes you toward the right tool: when a callback fits in one expression, a lambda is the terse choice (`{i, v -> v * v}`), and when it needs statements, a named function with a Funcref is the choice (`function('ByCountDesc')`), and the two are interchangeable in every higher-order position. The grammar is small — `{ optional-params -> expression }` — and its smallness is exactly what lets lambdas be both convenient to write and simple to compile, a single expression that closes over its surroundings and yields a value.

Builtin funcrefs and the dispatch path

The fact that `function('name')` resolves a *builtin* name as readily as a user function is worth dwelling on, because it is a small feat of plumbing. When you write `function('toupper')`, vimlrs resolves `toupper` to its ported builtin and produces a Funcref that, when called, dispatches through the same `call_func` wrapper (Chapter 22) that a direct `toupper(...)` call uses. So `function('toupper')('hi')`, `call('toupper', ['hi'])`, and `toupper('hi')` all reach the identical ported `f_toupper` — three syntactic routes to one implementation. This is why the `funcrefs.vim` example can build a Funcref to any builtin and call it through a variable, and why `call('printf', ['%d-%d', 3, 4])` works: the dynamic `call()` path, the Funcref-variable path, and the direct path converge on the same dispatch. The arity check of Chapter 6 happens on all three, because it lives in the wrapper, so a mis-arity call through a Funcref errors exactly as a direct one would. Making builtins first-class values — storable, passable, partially-bindable — without a separate mechanism for “builtin funcref” versus “user funcref” is the payoff of routing every call through one wrapper: a name is a name, and a Funcref to it works the same whether the name belongs to the ported builtin table or the user-function registry.

Partials as the unifying value

It is worth seeing that a single value type, `VAR_PARTIAL`, carries every form of bound callable in the language: a partial with leading arguments (`function('substitute', ['hello world'])`), a dict-bound function (the `self`-carrying funcref of Chapter 6), a capturing lambda (`function('<lambda>3', [a:n])`, Chapter 21), and any combination of the three. The partial holds the underlying funcref plus an optional bound-argument list plus an optional bound dict, and at call time the engine prepends the bound arguments, supplies the bound dict as `self`, and appends the call-site arguments. This uniformity is why the closure, the curried callback, and the method-bound function all behave consistently — they are the same value type with different fields populated. A language that grew these features separately would likely have three representations and three call paths; vimlrs has one, because the ported value layer has one `VAR_PARTIAL` variant that subsumes them all, and the compiler and bridge build that one variant for every binding form. The `Curry(100)(20)(3)`

nesting of `closures.vim` is three partials, each pinning one captured value, each the same kind of value as a `function('substitute', ['hello world'])` partial — first-class functions, bound arguments, and closures, all one mechanism.

Closures: capture by value

The genuinely subtle part is what a lambda *closes over*. `vimlrs`'s compiler implements lexical capture, and the rule it follows is precise. The `compile_viml.rs` source spells it out where it collects a lambda's free variables: "A lambda closes over the enclosing function's local scope: bare names and the function-tied scopes `a:` (arguments) and `l:` (locals). The dynamic scopes (`g:/b:/w:/t:/v:/s:`) resolve globally when the lambda runs, so they are not captured." Two things follow from this, and the `closures.vim` example tests both.

First, function-local state *is* captured, by value, at the moment the lambda is created:

```
function! Adder(n) abort
  return {x -> x + a:n}
endfunction
let Add5 = Adder(5)
call assert_equal(15, Add5(10))
```

`Adder(5)` returns a lambda that has captured `a:n` as 5. The captured value is fixed at creation, so lambdas made in separate calls are independent — `Const(1)` and `Const(2)` return closures that yield 1 and 2 respectively, with no shared mutable cell. Capture nests, too: `Curry(100)(20)(3)` is 123, each inner lambda capturing the parameter of the layer outside it.

Second, *dynamic* scopes are not captured — they are read when the lambda runs:

```
let g:counter = 10
function! ReadCounter() abort
  return {-> g:counter}
endfunction
let GetIt = ReadCounter()
let g:counter = 99
call assert_equal(99, GetIt())
```

`GetIt` returns `g:counter`, and because globals are not captured, it sees the *current* value 99, not the 10 that was in place when the lambda was made. This distinction — locals captured by value, globals resolved late — is exactly Vim's closure semantics, and getting it right is the difference between a closure implementation that looks correct on a simple test and one that is actually faithful. `vimlrs`'s `collect_free_vars` walks the lambda body, gathers the bare, `a:`, and `l:` names it references, and captures precisely those, leaving the dynamic scopes to resolve at call time. It is one of the most carefully reasoned pieces of the compiler, and the closures example is its proof.

Why the capture rule is exactly this, and not simpler

The closure capture rule — locals and the function-tied scopes captured by value at creation, dynamic scopes resolved late — is the kind of thing that looks arbitrary until you see that any simpler rule would be wrong, and walking through why is the best way to understand what a faithful closure implementation has to get right. The naive rule, the one a first implementation reaches for, is "capture everything the lambda body references." It is simpler to implement and it passes the easy test: `Adder(5)` returns a lambda that reads `a:n`, and capturing `a:n` by value gives the right answer, 15, for `Add5(10)`. But run the late-binding global test and the naive rule breaks. `ReadCounter` returns a lambda that reads `g:counter`; if the implementation captured `g:counter` by value at creation, then changing `g:counter` to 99 after the lambda was made would have no effect, and `GetIt()` would return the captured 10. The naive rule asserts 10. Vim, and `vimlrs`, assert 99, because in Vim a global is not captured — it is resolved when the lambda runs, so it sees the current value. The naive "capture everything" rule is wrong precisely on the dynamic scopes, and the only way to be right is to capture

the function-tied scopes (which exist only during the call and would be gone by the time the lambda runs, so they *must* be captured) while leaving the dynamic scopes to resolve late (because they persist and are meant to be read live).

So the rule is not arbitrary; it falls out of *which scopes still exist when the lambda runs*. A lambda made inside `Adder` and called later runs after `Adder` has returned, so `Adder`'s `a:n` and its locals are gone — if the lambda did not capture them at creation, they would be unreachable, so it must capture them, and capturing by value at creation is the only moment they are available. The global `g:counter`, by contrast, is still there when the lambda runs, and Vim's semantics say a lambda reads the current global, not a snapshot — so capturing it would be both unnecessary and wrong. The rule “capture the scopes that will be gone, resolve the scopes that will remain” is the principled rule, and Vim's specific assignment of which scopes are which (bare and `a:` and `l:` are function-tied and captured; `g:`, `s:`, `b:`, `w:`, `t:`, `v:` are dynamic and late) is what `collect_free_vars` encodes. The `closures.vim` example tests both halves — independent closures from separate `Const` calls proving by-value capture, and the late-binding `g:counter` proving late resolution — because both halves are necessary and a wrong implementation fails one or the other. That a four-test example pins a capture rule this precisely is why this book points to it when asked whether vimlrs's closures are really faithful: the answer is not a claim about the implementation but a CI-checked assertion that the implementation gets the one test a naive version fails.

Currying as repeated capture

The nested-currying test, `Curry(100)(20)(3)` yielding 123, is worth its own look because it shows the capture rule composing through layers, which is the part most likely to break in an implementation that handled single capture but not nesting. `Curry(100)` returns a lambda that, when called with `20`, returns another lambda that, when called with `3`, computes `a:a + b + c`. Three values get captured at three different moments: the outer lambda captures `a:a` as 100 when `Curry(100)` runs; the middle lambda captures the outer's parameter `b` as 20 when the first returned lambda is called with 20; and the innermost adds its own parameter `c` as 3 at the final call. Each capture happens at the creation of the lambda that needs it, and each captures by value the binding that is about to go out of scope, so by the time the innermost lambda runs, it has 100, 20, and 3 available — the first two as captures threaded down through the nesting, the third as its own argument. This is capture *through* layers, and it works because `collect_free_vars` runs at each lambda's creation, gathering exactly the free variables that lambda references, so the middle lambda captures `b` (the outer's parameter, which is a function-tied name in the outer's frame) and the inner lambda captures `a:a` and `b` (both function-tied names it references). The compiler's desugaring turns each lambda into a generated function whose captured names become leading parameters, so a three-layer curry becomes three generated functions, each a partial pinning its layer's captured values — and `Curry(100)(20)(3)` being exactly 123 is the proof that the capture-and-desugar machinery composes correctly through the nesting, not just for a single closure.

The funcref namespace and the capitalization rule

There is a naming rule running through every example in this chapter that is easy to miss until it bites: a variable that holds a Funcref must begin with an uppercase letter, unless it is explicitly scoped. `Upper`, `Add`, `Censor`, `Adder`, `GetIt`, `F` — every funcref-holding variable in `funcrefs.vim`, `closures.vim`, and `funcref_call.vim` is capitalized, and it is not a style choice. The rule is Vim's, and it exists to keep two namespaces apart. A lowercase bareword like `add` or `len` is reserved for the *builtin function* namespace, so if you could write `let add = function('...')` you would shadow the builtin `add` and create an ambiguity the parser cannot resolve. Vim forbids it: an unscoped variable holding a Funcref must start with a capital, or carry a scope sigil (`g:add` is fine, because the sigil disambiguates).

This is why the direct-call syntax of `funcref_call.vim` reads the way it does — `function('toupper')('hi')` calls an *expression*, but `let F = function('toupper') | F('ok')` calls through a capitalized *variable*. It is also why a dict-stored funcref does not need the capital — `d['up']('yo')` works with a lowercase key — because a dict member is reached through the dict, not the

bareword namespace. The rule is small, but it shapes the look of all idiomatic first-class-function code in Vimscript, and vimlrs enforces the namespace boundary exactly as Vim does. When you read a `.vim` file and see a run of capitalized variables, you are looking at its funcrefs; the capitalization is documentation the language requires.

Chapter 9 – Strings, printf, and Text

Text processing is what Vimscript was born for, and its string handling carries both the power and the quirks of that heritage. The single most important quirk — the one that surprises every newcomer — is that a string's length is measured in *bytes*, not characters. vimlrs reproduces this and the whole family of character-aware functions that exist to work around it, along with a `printf()` implementation faithful enough to handle positional arguments, alternate forms, and sign flags. This chapter is the byte-versus-character story and the formatting story, grounded in the `strings.vim` example, which is one of the densest test files in the crate.

Why length is measured in bytes

The byte-length rule — `len('héllo')` is 6, not 5 — is the quirk that surprises every newcomer, and it is worth explaining why Vimscript made this choice, because it is not an oversight or a failure to support Unicode but a deliberate consequence of what Vim is and when it was built. Vim is a text editor, and a text editor works with text as it sits in a file, which is a sequence of bytes. When Vim measures, indexes, or slices a string, it is most often working with text that corresponds to bytes in a buffer, and measuring in bytes keeps the string operations aligned with the buffer operations — a byte offset into a string is a byte offset into the file. Moreover, Vimscript predates the universal adoption of UTF-8; it grew up in a world of mixed encodings where “character” was not even a well-defined notion without knowing the encoding, while “byte” always was. So the language took bytes as its fundamental unit of string length, and bolted on a whole family of character-aware functions — `strchars`, `strcharlen`, `byteidx`, `charidx`, `strcharpart`, and the rest — for the cases where a script genuinely needs to count or index by character rather than byte. The byte-length default is the editor-aligned, encoding-agnostic choice, and the character-aware family is the layer for when codepoints matter. A from-scratch interpreter built today would almost certainly make `len` count characters, because that is the modern default and the less surprising one — and it would be wrong, because every existing Vimscript that relies on byte length would break under it.

This is a place where faithfulness costs the port something real, and it is worth being honest about the cost. The byte-length rule is, by modern lights, the *worse* design — character length is what most programmers want most of the time, and byte length is a footgun that produces off-by-several errors on any non-ASCII text. A reimplementer would be tempted to “fix” it, to make `len` count characters and spare users the surprise. vimlrs does not, and the refusal is the whole discipline in miniature: the goal is not to build a better Vimscript but to build *Vimscript*, and Vimscript's `len` counts bytes, so vimlrs's `len` counts bytes, footgun and all. The value of a faithful port is proportional to how much existing Vimscript runs unchanged, and existing Vimscript was written against byte-length `len` — much of it deliberately, by authors who knew the rule and used it. Changing the rule would make vimlrs subtly incompatible with exactly the code it exists to run, trading a more pleasant language for a less faithful one, which is the trade the project declines everywhere. So `len('héllo')` is 6 in vimlrs, not because the project thinks that is the better answer but because it is Vim's answer, and the example suite asserts it precisely to lock the faithful-but-surprising behaviour against any well-meaning attempt to improve it into incompatibility.

Bytes versus characters

`len()` of a String is its byte count. `strchars()` is its character (codepoint) count. For ASCII they agree; for multibyte text they do not:

```
call assert_equal(6, len('h     '))
call assert_equal(5, strchars('h     '))
```

'h     ' is five characters but six bytes, because the    is two bytes in UTF-8. This is not a vimlrs choice; it is Vim's, and a faithful port has to carry it along with the whole apparatus Vim provides for navigating between byte and character indices. That apparatus is large — the reference's "String manipulation" chapter has 56 functions — and the byte/char converters are its core: `strlen` (bytes), `strchars` and `strcharlen` (characters), `byteidx` and `byteidxcomp` (character index to byte index), `charidx` (byte index to character index), `strpart` (byte substring), `strcharpart` (character substring), `strgetchar`, `strwidth`, `strdisplaywidth`, `strutf16len`, and `utf16idx`.

The composing-character handling is where this gets genuinely intricate, and `strings.vim` tests it directly. A base letter plus a combining accent is one "character" for some purposes and two codepoints for others:

```
let s = 'e' . nr2char(0x301)
call assert_equal(2, strchars(s))
call assert_equal(1, strchars(s, 1))
call assert_equal(1, strcharlen(s))
```

`strchars(s)` counts both codepoints (2); `strchars(s, 1)` with the `skipcc` ("skip combining characters") flag folds the accent into its base (1); and `strcharlen` always folds. The byte-index functions split the same way: `byteidx` folds a composing char into its base, while `byteidxcomp` counts it separately. These distinctions are exactly the kind of thing that only a faithful port gets right, and the fact that vimlrs passes its own assertions on them is evidence the port reaches that depth.

Codepoint round-trips

A small cluster of functions treats a string as a sequence of codepoint numbers, and they round-trip cleanly, which is occasionally exactly what a transformation needs. `char2nr({string})` returns the codepoint of the first character and `nr2char({expr})` builds a one-character string from a codepoint — the pair that constructs a composing-character test as `'e' . nr2char(0x301)`. Scaled up, `str2list({string})` explodes a whole string into a List of codepoint numbers and `list2str({list})` reassembles it, so `list2str(str2list(s))` reconstructs `s`. This is the tool for character-level manipulation that does not fit the substring functions: rotate every codepoint, filter out a range, map each character through a numeric transform, then rebuild. `strgetchar({str}, {index})` reads a single codepoint by character index (returning -1 past the end), the indexed read that complements the whole-string explosion. Because all of these work in codepoints rather than bytes, they handle multibyte text correctly — `str2list('h     ')` has five elements, one per character, not six per byte — which is the same byte-versus-character care the measurement functions take, applied to the codepoint view of a string.

printf

`printf(fmt, ...)` is the formatting workhorse, and vimlrs's implementation is thorough. The basics are C's: `%d`, `%s`, `%x`, `%o`, `%b` (binary), `%c` (character from codepoint), `%e`, `%f`, `%g`, and width/precision/padding. From `strings.vim`:

```
call assert_equal('hex ff pad 00042 float 3.142',
    \ printf('hex %x pad %05d float %.3f', 255, 42, 3.14159))
call assert_equal('Hi!', printf('%c%c%c', 72, 105, 33))
call assert_equal('1.234568e+04', printf('%e', 12345.678))
```

Beyond the basics, vimlrs handles three families of advanced conversion that a naive implementation would skip. Sign flags: `%+d` forces a leading sign, `% d` pads positives with a space, and they compose with width and precision (`%+05d`, `%+.2f`). Positional arguments: `%N$` selects the Nth argument (1-based), which may be reused and reordered, while sequential `%`-specs are unaffected:

```
call assert_equal('b a', printf('%2$s %1$s', 'a', 'b'))
call assert_equal('x x', printf('%1$s %1$s', 'x'))
call assert_equal('00042', printf('%2$05d', 7, 42))
```

And the `#` alternate form: it prefixes `0x/0X/0b` on hex and binary and a leading `0` on octal, *suppresses* the prefix for a zero value, and places zero-padding after the prefix:

```
call assert_equal('0xff', printf('%#x', 255))
call assert_equal('0', printf('%#x', 0))
call assert_equal('0x0000ff', printf('%#08x', 255))
```

The `%g` conversion shows six significant digits and chooses `%e` or `%f` by exponent, matching the float rendering `string()` uses — which is why `string(0.1 + 0.2)` is `'0.3'` and not the `0.30000000000000004` you might fear.

tr, trim, split, and substitute

The rest of the string toolkit is the everyday text operations. `tr(src, from, to)` maps characters position-by-position and errors with `E475` if a source character has no destination. `trim(str [, mask [, dir]])` strips characters from the ends, honouring a direction argument — 0 both ends, 1 left, 2 right:

```
call assert_equal('x', trim(' x '))
call assert_equal('x ', trim(' x ', ' ', 1))
call assert_equal('hixx', trim('xxhixx', 'x', 1))
```

The `tr` and `trim` functions are worth pinning down because their argument conventions are specific. `tr(src, fromstr, tostr)` maps characters position by position: each character of `src` that appears in `fromstr` is replaced by the character at the *same index* in `tostr`, and characters not in `fromstr` pass through unchanged. So `tr('hello there', 'ht', 'HT')` is `'Hello THere'` — every `h` becomes `H` and every `t` becomes `T`. The strict part is that a character in `fromstr` with no corresponding character in `tostr` is an error, `E475`, rather than a silent drop — so `tr('abc', 'ab', 'x')` fails because `b` (index 1 in `fromstr`) has no character at index 1 in the one-character `tostr`. `trim(text, mask, dir)` strips characters from the ends: the `mask` is the set of characters to strip (default whitespace), and the `dir` selects which ends — 0 or omitted for both, 1 for left only, 2 for right only. So `trim(' x ')` is `'x'`, `trim(' x ', ' ', 1)` is `'x '` (left only), and `trim('xxhixx', 'x', 1)` is `'hixx'`. Both functions are small, both have these precise conventions, and both are right in vimlrs because they are ported — `tr`'s `E475` on a missing destination character is exactly the kind of strict edge a from-scratch implementation would smooth over and that the ported function preserves.

`split(str [, pat [, keepempty]])` tokenizes — with no pattern it splits on whitespace, with a pattern it splits on matches, and the `keepempty` flag controls leading/trailing empties. `join(list [, sep])` is its inverse. And `substitute(str, pat, rep, flags)` is the search-and-replace that bridges into the regex engine of the next chapter — including the powerful `\=` form, where the replacement is an *expression* evaluated per match with `submatch()` giving access to the captured groups:

```
call assert_equal('ABCABC',
  \ substitute('abcABC', '[a-z]', '\=toupper(submatch(0))', 'g'))
call assert_equal('price: 200',
  \ substitute('price: 100', '\d\+', '\=submatch(0)*2', ''))
```

The `\=` replacement is the moment a substitution stops being string surgery and becomes arbitrary computation: `'\=submatch(0)*2'` doubles every number in the string. vimlrs wires this through a hook — the regex engine calls back into the bytecode evaluator to run the replacement expression, with `submatch()` reading the current match's groups from thread-local state. It is the same kind of ported-zone-calls-synthesis-zone seam we saw with `map/filter`, and it is what makes `substitute`'s `\=` feel like a first-class feature rather than a special case. The case escapes `\u`, `\l`, `\U`, `\L` round out the replacement syntax, upcasing or downcasing the next character or the rest of the replacement.

The `split` function has its own subtleties worth pinning down, because the `keepempty` flag changes its behaviour at the edges. By default `split` keeps internal empty fields but drops leading and trailing ones, so `split('a,b,,c', ',')` is `['a', 'b', '', 'c']` (the internal empty is kept) while `split(',a,b,', ',')` is `['a', 'b']` (the leading and trailing empties are dropped). With a truthy `keepempty`, `split(',a,b,', ', ', 1)` keeps them all: `['', 'a', 'b', '']`. With no pattern at all, `split` tokenizes on whitespace, the common case for breaking a line into words. The zero-width-pattern case is the one that enables character splitting: `split('hello', '\zs')` returns `['h', 'e', 'l', 'l', 'o']`, because a zero-width-capable pattern makes `split` advance one character at a time (Chapter 10). These behaviours are exactly Vim's, and they matter because `split` is the workhorse for turning text into structured data — a misplaced empty field or a dropped one would corrupt every downstream computation, and `strings.vim` asserts each case.

The substitute `flags` argument carries more than the `g` global flag. The empty flag replaces only the first match per line, `g` replaces all, and the count and `i/I` case flags are also accepted. Combined with the `\=` expression replacement, the case escapes, the backreferences `\1..\9` in the replacement, and the whole magic/very-magic pattern dialect, `substitute` is the single most capable function in the string family — it spans literal replacement, case-folding, group reordering, and arbitrary per-match computation, all in one call, and the regex engine of Chapter 10 backs every bit of it. With those, `substitute` covers the full range from literal replacement to case-folding to arbitrary expression evaluation, and the `strings.vim` example asserts every one of them.

Stepping back, the string family is best understood as three layers stacked on the byte-string substrate. The bottom layer is *measurement and navigation* — the byte, character, and display-width counts and the index-mapping functions that let a script locate a position in multibyte text. The middle layer is *extraction and transformation* — the substrings, the case folding, the translation and trimming, the split and join that turn text into structured data and back. The top layer is *pattern operations* — the match family and `substitute`, which bring the regex engine to bear for search, extraction, and replacement. A real text-processing program uses all three: measure to find a position, extract or transform a piece, match a pattern to locate or replace. The reason there are fifty-six functions is that each layer needs both the byte view and the character view of text, plus the format and conversion functions that bridge to numbers, plus the specialized tail. It is a large family, but it is not a grab-bag — it is the complete toolkit for the one thing Vimscript was born to do, and vimlrs carries the whole of it faithfully because text processing is precisely the standalone use case the project exists to serve.

Indexing and slicing a string

A string can be subscripted like a list, and `string_index.vim` documents the two helpers behind the syntax: `char_from_string()` for `str[i]` and `string_slice()` for `str[a:b]`. Both work in *characters*, not bytes, and both follow Vim's out-of-range and negative-index rules:

```
call assert_equal('h', 'hello'[0])
call assert_equal('', 'hello'[9])
call assert_equal('o', 'hello'[-1])
call assert_equal('ell', 'hello'[1:3])
call assert_equal('llo', 'hello'[2:])
call assert_equal('', 'abc'[2:1])
```

A single subscript returns one character, with an out-of-range index yielding the empty string rather than an error, and a negative index counting from the end. A slice `str[a:b]` has an *inclusive* end — `'hello'[1:3]` is `'ell'`, three characters — which is the same inclusive-bound rule that lists use and the opposite of `slice()`'s exclusive end, so `slice('abcdef', 1, 3)` is `'bc'` while `'abcdef'[1:3]` is `'bcd'`. A reversed range like `'abc'[2:1]` is the empty string. The multibyte behaviour is the part that matters: because indices are characters, `'hello'[1]` is the accented `'é'`, one unit, even though it is two bytes. The character-substring functions `strcharpart` and `strgetchar` work the same way — `strgetchar('hello', 1)` is the codepoint of `'é'` — while the byte-oriented `strpart` works in bytes, so the two families let a script choose character or byte semantics deliberately.

Display width and setcellwidths

There is a third notion of “length” beyond bytes and characters: *display width*, the number of terminal columns a string occupies. `strwidth` reports it, and most characters are one column — but East Asian wide characters and emoji are two, and a tab expands to the next `'tabstop'`. `sysinfo.vim` shows that the width table is even overridable, via the ported `setcellwidths/getcellwidths` from Neovim’s `mbyte.c`:

```
call assert_equal(1, strwidth('A'))
call setcellwidths([[0x41, 0x41, 2]])
call assert_equal(2, strwidth('A'))
call setcellwidths([])
call assert_equal(1, strwidth('A'))
```

`setcellwidths([[0x41, 0x41, 2]])` declares the codepoint `0x41` (A) to be two columns wide, after which `strwidth('A')` is 2; clearing the table with an empty list restores it. A range entry like `[[0x2600, 0x26ff, 2]]` widens a whole block of codepoints at once. This is a genuinely obscure corner of the language — most scripts never touch it — and its presence, faithfully ported and asserted, is one more data point on how far the port reaches. The same example checks the honest emptiness of the editor-tied queries alongside it: `foldlevel(1)` is 0 because there are no folds, exactly the editor-absent contract of Chapter 18. A file that tests cell-width overrides and fold levels in the same breath is a good miniature of what vimlrs is — a faithful language core with an honestly-absent editor around it.

Interpolated strings

Vim’s interpolated strings embed expressions directly in string literals, and vimlrs supports both spellings in both dialects. An interpolated string is written with a `$` sigil before the quote — ``${}`` or ``${}`` — and each `{expr}` inside the body is evaluated, converted to text the way `:echo` would render it (strings verbatim, other types structurally: ``${v}=[1, 2]`` yields `v=[1, 2]`), and concatenated with the literal parts. The two forms follow their base string’s escape rules: ``${}`` honours `\t` and the double-quote escapes, ``${}`` keeps backslashes literal and treats `'` as one quote. To get a literal brace, double it (`{ { / }`); in ``${}`` a backslash before a brace works too. The braces nest correctly — a `}` inside a string literal or a balanced inner dict does not close the interpolation — and interpolations may themselves nest, so ``${out} {`${<x>}`} end`` is well defined. The result is always a String, whatever the embedded expression’s type, and a plain (unsigilled) string with braces is left completely alone. `interp_strings.vim` pins the whole surface, from the `eval.txt sqrt(9)` example to nested dict subscripts and `ENV` access inside a brace. The feature works in legacy Vimscript as well as vim9script, so it is not gated on the `:vim9script` marker.

Chapter 10 – Pattern Matching and the Regex Engine

Vim’s regular expressions are their own dialect — not POSIX, not PCRE, but a backslash-heavy notation with “magic” levels, zero-width match-boundary atoms, and an optional-sequence construct found almost nowhere else. Reproducing this faithfully is one of the harder things vimlrs does, and it is the one place where the project most clearly chose *behavioural* fidelity over *line-by-line* port. This chapter explains that choice, walks the dialect through the example suite, and shows where the regex engine plugs into the rest of the language.

A dialect reproduced, not a VM ported

Neovim matches patterns with two C engines — `regexp_bt.c`, a backtracking matcher, and `regexp_nfa.c`, an NFA matcher — both of which compile a pattern to a bytecode program and run it on a dedicated regex VM. vimlrs does not port those. Its `viml_regex.rs` is an “EXTENSION — implements Vim’s regex DIALECT, not a line-by-line port,” a “backtracking matcher over a parsed AST that reproduces Vim’s documented pattern behavior (`:help pattern`).” This is the deliberate exception to the faithful-port rule, and the reasoning is sound: the C regex engines are tangled into Vim’s buffer and option machinery in ways that do not transplant, and what a user actually depends on is the *pattern semantics*, not the matcher’s internal bytecode. So the regex engine sits in the synthesis zone alongside the lexer and the bytecode compiler — code with no `csrc/` counterpart — and its contract is the documented behaviour rather than a specific C function. It backs the operators and builtins that take patterns: `~=` and `!~`, `matchstr()`, `match()`, `substitute()`, `pattern split()`, and `:catch /pat/`.

How Vim’s dialect differs from the ones you know

To appreciate why reproducing Vim’s regex dialect is hard, it helps to place it against the flavours a programmer is more likely to know, because Vim’s is genuinely its own thing and the differences are not cosmetic. Most modern languages use a Perl-compatible dialect — PCRE or something close to it — where the metacharacters are bare: `+`, `?`, `(`, `)`, `|`, `{` are all operators by default, and you backslash them to make them literal. POSIX extended regular expressions are similar in spirit. Vim’s default “magic” mode inverts a large part of this: `.`, `*`, `[`, `]`, `^`, and `$` are special bare, but `+`, `?`, `(`, `)`, `|`, and `{` are *literal* bare and must be backslashed to become operators — so where PCRE writes `(foo|bar)+`, magic-mode Vim writes `\(foo\|bar\)+`. This is not a minor notational difference; it is the opposite convention for a whole class of metacharacters, and a regex engine built for PCRE syntax would mis-parse essentially every non-trivial Vim pattern. The `\v` “very magic” prefix exists precisely to let users opt into the PCRE-like convention where all punctuation is an operator, which is why the `regex_verymagic.vim` example reads so much more familiarly than the magic-mode patterns elsewhere — under `\v`, `(foo|baz)` is an alternation group with no backslashes, the way a PCRE user would expect. Supporting both conventions, switchable mid-pattern by a `\v` prefix, is the first thing Vim’s dialect demands that an off-the-shelf engine does not provide.

Beyond the magic convention, Vim’s dialect has atoms that exist in almost no other flavour, and these are the features a casual reimplementer would simply omit. The `\zs` and `\ze` atoms move the *reported* boundaries of a match without changing what the pattern consumes, so a pattern can require context before or after the match while returning only the part in between — there is no PCRE equivalent that does exactly this (lookahead and lookbehind are related but not the same, and Vim has those distinctions too). The `\%[...]`

optional-sequence atom matches the longest in-order prefix of its atoms, the idiom for abbreviatable command names, and it appears in essentially no other regex language. The word-boundary atoms `\<` and `\>` are asymmetric start-of-word and end-of-word assertions, where PCRE has only the symmetric `\b`. These atoms are not edge cases that a few power users touch; they are woven through real Vim patterns, and an engine that lacks them cannot run the patterns that use them. This is exactly why vimlrs's regex engine is a from-scratch matcher with a node type for each of these atoms rather than a thin wrapper over an existing regex library: there is no existing library whose dialect matches Vim's, so reproducing the behaviour meant building an engine whose AST has a `\zs`, a `\ze`, an `OptSeq`, and asymmetric word boundaries as first-class nodes. The dialect is the specification, and the dialect is unusual enough that meeting it required a purpose-built engine.

Why a fresh engine rather than a ported one

This is the one major place in the project where the synthesis zone reproduces *behaviour* rather than porting *code*, and it is worth being clear about why the usual porting discipline was set aside here, because the reasoning is principled rather than a shortcut. Neovim matches patterns with two C engines — a backtracking matcher in `regexp_bt.c` and an NFA matcher in `regexp_nfa.c` — and both compile a pattern to a bytecode program that runs on a dedicated regex VM. Porting those faithfully would mean porting two regex VMs, their bytecode formats, and their compilers, all of which are deeply entangled with Vim's buffer representation, its option machinery, and its multibyte handling in ways that do not transplant to a standalone interpreter. And the payoff for all that work would be nil for the user, because what a script depends on is not the matcher's internal bytecode or which of the two engines ran — those are invisible — but the *pattern semantics*: whether `\zs` moves the boundary, whether `\%[ead]` abbreviates, whether `\v` flips the convention. So the project made the one principled exception to its line-by-line discipline: it reproduces the documented pattern behaviour with a fresh backtracking matcher over a parsed AST, and it marks the file with the EXTENSION banner so no reader mistakes it for a port. The contract is the behaviour, the test is the example suite, and the `regex_zs.vim`, `regex_backref.vim`, and `regex_optseq.vim` examples are the specification made executable. This is the same divergence-by-result that the JSON decoder makes — a new mechanism producing an output indistinguishable from Neovim's — and it is principled for the same reason: fidelity is owed to the observable result, and where porting the mechanism is intractable and the result can be reproduced directly, reproducing the result is the faithful choice. The boundary stays legible because the banner says exactly which kind of fidelity this file provides.

Magic and very-magic

The default matching mode is “magic,” where some metacharacters are special bare (`.`, `*`, `[...]`, `^`, `$`) and others need a backslash (`\+`, `\?`, `\(`, `\)`, `\|`, `\{n,m}`). The regex header lists what magic mode supports: literals, `.`, anchors `^`/`$`, character sets `[...]`/`[^...]` with ranges, the class atoms `\d` `\D` `\w` `\W` `\s` `\S` `\a` `\l` `\u` `\x` and their negations, quantifiers `*` `\+` `\?` `\=` `\{n,m}` and the non-greedy `\{-n,m}`, groups `\(...\)` capturing and `\%(...\)` non-capturing with `\|` alternation, word boundaries `\<` `\>`, and case control `\c`/`\C` plus the caller's ignore-case flag. The `strings.vim` example uses magic mode throughout — `'\d+'` matches a run of digits, `'\(\w\)-\(\w\).'` captures two word characters around a dash.

The `\v` “very magic” prefix flips the backslash convention: after `\v`, every ASCII punctuation character is an operator without a backslash. The `regex_verymagic.vim` example shows the payoff in readability:

```
call assert_equal('123', matchstr('abc123', '\v\d+'))
call assert_equal('colour', matchstr('colour', '\vcolou?r'))
call assert_equal('foo', matchstr('foobar', '\v(foo|baz)'))
call assert_equal('cat', matchstr('a cat here', '\v<cat>'))
```

Under `\v`, `+`, `?`, `(`, `)`, `|`, and the word-boundary `</>` are all bare operators, so `\v(foo|baz)` is an alternation group with no backslashes at all. A backslash in very-magic mode makes such a character *literal* instead — `\v\b` matches a literal parenthesized `b`. Magic mode still works unchanged when `\v` is absent, so the two coexist.

Match boundaries, backreferences, and optional sequences

Three of Vim's more distinctive pattern atoms get their own example files, because they are exactly the features a casual reimplementation would omit. The `\zs` and `\ze` atoms move the *reported* match boundaries without changing what the pattern consumes: `\zs` sets where the matched text starts, `\ze` where it ends. From `regex_zs.vim`:

```
call assert_equal('l', matchstr('hello', 'l\zs.'))
call assert_equal('foo', matchstr('foobar', 'foo\zebar'))
call assert_equal(['h', 'e', 'l', 'l', 'o'], split('hello', '\zs'))
```

`'l\zs.'` requires an `l` before the match but returns only the character after it; `'foo\zebar'` requires `bar` after but returns only `foo`. The zero-width `\zs` is also the idiom for splitting a string into characters — `split('hello', '\zs')` — because a zero-width-capable pattern makes `split` advance one character at a time.

Backreferences `\1..\9` match the exact text a previous group captured, which is how you find a doubled character. The `regex_backref.vim` example tests them working — and this is a place to trust the example over a stale source comment, since the regex header still says “Backreferences (`\1`) are not yet handled” while the AST carries a `BackRef(usize)` node and the CI-run example asserts:

```
call assert_equal('ll', matchstr('hello', '\(l\)\'1'))
call assert_equal(1, 'mississippi' =~ '\(.\)\'1')
call assert_equal('heXo', substitute('hello', '\(l\)\'1', 'X', ''))
```

Because every example is a self-test run in CI, `regex_backref.vim` passing is the authoritative statement: backreferences work. (The header comment is simply out of date relative to the code — a useful reminder that in this codebase the tests, not the prose, are the specification.)

The `\%[atoms]` optional-sequence atom matches the longest in-order prefix of its atoms, each only if the previous one matched — the idiom for abbreviatable keywords. The AST node is `OptSeq(Vec<Node>)`, and `regex_optseq.vim` shows the classic use:

```
call assert_equal('read', matchstr('read', 'r\%[ead]'))
call assert_equal('rea', matchstr('rea', 'r\%[ead]'))
call assert_equal('r', matchstr('r', 'r\%[ead]'))
```

`r\%[ead]` matches `r`, `re`, `rea`, or `read` — exactly how Vim lets you abbreviate command names. This atom exists in almost no other regex flavour, and its presence in vimlrs is a sign of how far the dialect reproduction reaches.

Where matching plugs in

The regex engine is not a standalone library; it is wired into the comparison operators and the string builtins. The `=~` operator (and case-flagged `=~#` / `=~?`) runs a match and yields 0 or 1. `matchstr` returns the matched text, `match/matchend` the byte offsets, `matchlist` the whole match plus the nine `\1..\9` submatch slots as a list. `substitute` replaces matches, with the `\=` expression form calling back into the bytecode evaluator and `submatch()` reading the current groups. And `:catch /pat/` matches a thrown exception's text against a pattern — which is the bridge into the next chapter, because exception handling is built on the same regex engine. The `'ignorecase'` option is wired into all of it: the README notes `'ignorecase'` is “wired into regex,” so a bare `=~` honours the option while `=~#/=~?` override it per the case flag. One engine, many entry points, all sharing the same faithful dialect.

Inside the engine: the AST and the backtracker

The regex engine deserves a look under the hood, because it is the one synthesis-zone module whose contract is *behavioural* rather than a line-by-line port, and seeing its shape clarifies what “reproduces the dialect” means mechanically. The engine is two stages, like the language itself: a pattern is parsed to a small AST, and the AST is matched by a backtracker. The AST node type, `Node`, has a variant per pattern construct — `Lit(char)` for a literal, `Class(Class)` for a character set or class atom, `Group(Vec<Branch>, Option<usize>)` for a group with its alternation branches and optional capture index, `WordB(bool)` for the `\</>` word boundaries, `BackRef(usize)` for `\1..\9`, and `OptSeq(Vec<Node>)` for the `\%[...]` optional sequence. A character class is its own little structure — a `Class` with a `negated` flag and a list of `ClassItems`, each a single character, a range, or a class atom like `digit` or `word` — so `[^a-z\d]` parses to a negated class of one range and one atom.

The header is candid that this is a dialect reproduction, not a VM port:

```
EXTENSION - implements Vim's regex DIALECT, not a line-by-line port of
regexp_bt.c/regexp_nfa.c (which compile a pattern to a bytecode program
and match it with a backtracking / NFA VM). This is the vimlrs analogue of
the bytecode-compiler carve-out: a backtracking matcher over a parsed AST
that reproduces Vim's documented pattern behavior (:help pattern).
```

The matcher walks the AST and backtracks, which is the simplest faithful strategy for a dialect this irregular — `\zs/\ze` move the reported boundaries, `\%[...]` matches the longest in-order prefix, and the non-greedy `\{-n,m\}` exists — and none of those transplant cleanly from Neovim’s two C engines, which are tangled into the editor’s buffer machinery. So vimlrs reproduces the *behaviour* with a fresh matcher and lets the example suite be the specification, which is why Chapter 10 trusted `regex_backref.vim` over the header’s stale “backreferences not yet handled” comment: the `BackRef` node and the matcher that handles it both exist, and the CI-run example asserts they work.

The submatch bridge

The one place the regex engine reaches outside itself is the `\=` substitute replacement, and the mechanism is a thread-local hook exactly like the filter/map hooks of Chapter 22. The engine holds a `SUBST_EXPR_HOOK` — “evaluates a `\=`-prefixed substitute replacement *expression* to its string result” — which the bridge installs with a function that compiles and runs the expression. While that expression runs, the engine has stashed the current match’s groups in a thread-local `SUBMATCHES` list, which the ported `submatch()` reads through `current_submatch(n)`. So `substitute('price: 100', '\d\+', '\=submatch(0)*2', '')` works by: the engine finding the match, storing `'100'` as group 0, calling the hook to evaluate `submatch(0)*2` (which reads `'100'`, coerces, doubles to 200), and splicing the result back. The engine even tracks whether a `\=` context is live, via `has_submatch_context()`, so that `submatch(n, 1)` outside a substitution returns an empty list rather than a one-empty-line list — a small faithful detail that only matters at the edges, and is right because the engine models the context explicitly. It is the same ported-zone-calls-synthesis-zone seam the whole machine is built on, here letting a regex replacement run arbitrary Vimscript.

Classes, quantifiers, and the matcher

The matcher’s internal structures are worth a closer look, because they are where Vim’s dialect actually lives. A character class is a `Class` struct with a `negated` flag and a list of `ClassItems`, and the item variants are exactly the things that can appear inside `[...]` or as a backslash class atom: `Ch(char)` for a single character, `Range(char, char)` for `a-z`, and the named atoms `Digit`, `Word`, `Space`, `Alpha`, `Lower`, `Upper`, and `Hex` for `\d`, `\w`, `\s`, `\a`, `\l`, `\u`, and `\x`. A negated set `[^...]` sets the `negated` flag, and the class atoms have negated spellings (`\D`, `\W`, `\S`) that the parser turns into a negated single-item class. So `[a-z\d]` parses to a `Class` with a `Range('a', 'z')` and a `Digit` item, and `\W` parses to a negated `Class` with one `Word` item —

a uniform representation that lets the matcher test membership the same way regardless of how the class was spelled.

The quantifiers are handled by the matcher's backtracking. A `*`, `\+`, `\?/\=`, or `\{n,m\}` wraps an atom, and the matcher tries the greedy maximum first and backtracks to shorter matches on failure — the standard backtracking strategy, which is the right choice for a dialect with zero-width atoms and optional sequences that an NFA would handle awkwardly. The non-greedy `\{-n,m\}` reverses the order, trying the minimum first. The header's support list is the contract the matcher implements in magic mode: "literals, `.`, `^`, `$`, `[...]`/`[^...]` with ranges, the class atoms `\d \D \w \W \s \S \a \l \u \x` (+negations), quantifiers `*` `\+` `\?` `\=` `\{n,m\}` and the non-greedy `\{-n,m\}`, groups `\( ... \)` (capturing) and `\%( ... \)` (non-capturing) with `\|` alternation, word boundaries `\< \>`, and case control `\c/\C` plus the caller's ignore-case flag."

Groups, alternation, and the boundary atoms

A group is `Group(Vec<Branch>, Option<usize>)` — a list of alternation branches and an optional capture index. A `Branch` is one alternative of a `\|` alternation, itself a sequence of nodes, so `\(foo\|bar\)` is a `Group` with two branches and a capture index, while `\%(foo\|bar\)` is the same with `None` for the index (non-capturing). The capture index is what `\1..\9` and `submatch()` read: when the matcher enters a capturing group it records the span it consumed under that index, and a later `BackRef(usize)` node matches "the exact text a previous group captured." The word-boundary atoms are `WordB(bool)` — `\<` is the start-of-word boundary, `\>` the end — and they are zero-width, asserting a transition between a word and a non-word character without consuming anything. The match-boundary atoms `\zs` and `\ze` are the same kind of zero-width assertion that adjusts the *reported* span rather than the consumed span: `\zs` marks where the reported match starts, `\ze` where it ends, which is how `matchstr('hello', 'l\zs.')` returns only the character after the `l`. And the optional-sequence atom `OptSeq(Vec<Node>)` matches "the longest in-order prefix of the atoms, each only if the previous one matched" — the abbreviatable-keyword idiom `r\%[ead]`. These atoms — backreferences, word boundaries, match boundaries, optional sequences — are exactly the ones that exist in almost no other regex flavour, and the fact that the matcher has a node type for each is the measure of how completely it reproduces Vim's dialect rather than approximating it with a more conventional engine.

Case control and the ignore-case flag

Case sensitivity in the matcher comes from two places, and they compose the way Vim's do. The pattern can carry `\c` (force ignore-case) or `\C` (force match-case) atoms, which set a mode for the rest of the pattern, and the caller passes an ignore-case flag derived from the `=/~` operator's case suffix and the `'ignorecase'` option. The matcher resolves them in Vim's precedence: an explicit `\c/\C` in the pattern overrides everything, and absent that, the caller's flag decides — which is bare `=~` consulting `'ignorecase'`, or `=~#/=~?` overriding it per the suffix the lexer recorded (Chapter 19). So `'FOO' =~ '\cfoo'` matches regardless of `'ignorecase'` because the `\c` forces it, while `'FOO' =~ 'foo'` depends on the option. Threading the case decision from the lexer's `CaseFlag`, through the comparison operator, into the regex engine's match call, and resolving it against the pattern's own `\c/\C` is a small chain of faithful details that together make case matching behave exactly as Vim documents — one more place where the dialect reproduction reaches all the way down.

Chapter 11 — Control Flow and Exceptions

Vimscript's control flow is conventional on the surface — `if/elseif/else`, `while`, `for`, `break`, `continue` — and unconventional underneath, because its exception mechanism (`try/catch/finally/throw`) matches caught exceptions by *regular expression* and exposes the in-flight exception through a special variable. vimlrs implements the whole of it, and on top of the exception machinery the example suite builds the assertion framework that makes every example a self-test. This chapter covers the loops, the exceptions, and the assert framework that depends on them.

Conditionals and loops

The `:if/:elseif/:else/:endif` chain is represented in the AST as a list of `(condition, body)` arms plus an optional `else` body. Conditions are ordinary expressions, truthy when non-zero. `fizzbuzz.vim` is the canonical shape:

```
function! FizzBuzz(n) abort
  if a:n % 15 == 0
    return 'FizzBuzz'
  elseif a:n % 3 == 0
    return 'Fizz'
  elseif a:n % 5 == 0
    return 'Buzz'
  else
    return string(a:n)
  endif
endfunction
```

The `:while {cond} ... :endwhile` loop runs while its condition holds, and it is the JIT's favourite construct — a numeric `while` over provably-Number locals trace-compiles to native code, as `FibIter` from Chapter 4 demonstrated. The `:for {var} in {expr} ... :endfor` loop iterates a List (or any iterable), binding each item to the loop variable. The variable may be a single name or a `[a, b]` unpack, so `for [k, v] in items(d)` destructures key-value pairs. `:break` and `:continue` do the usual. One important optimization: `for i in range(N)` does *not* build a list; the README notes it lowers to a “native integer counter loop (no list built) that **trace-JIT-compiles**,” even when the bound is dynamic like `range(a:n)`. So the idiomatic counting loop is as fast as a hand-written `while`, which is covered fully in Chapter 14.

Reading fizzbuzz closely

The `fizzbuzz.vim` function is the smallest program that exercises the whole conditional chain, and it rewards reading not for what it computes — everyone knows FizzBuzz — but for what each line proves about how vimlrs handles control flow. The function tests `a:n % 15 == 0` first, then `% 3`, then `% 5`, with an `else` returning the number as a string, and the ordering is load-bearing: testing `% 15` before `% 3` and `% 5` is what makes a multiple of fifteen produce `'FizzBuzz'` rather than stopping at the first of `'Fizz'` or `'Buzz'`, because the `:elseif` chain takes the first arm that matches and the multiple-of-fifteen case has to be caught before the more general divisibility tests claim it. This is the ordinary semantics of an `if/elseif` chain — first match

wins — but seeing it here makes concrete that vimlrs's `If` statement, which the AST represents as a list of `(condition, body)` arms plus an optional `else`, evaluates the arms in order and takes the first whose condition is truthy, exactly as Vim does. The conditions are ordinary expressions, truthy when non-zero, so `a:n % 15 == 0` is a comparison that yields the Number 0 or 1, and the arm is taken when it yields 1.

The `% 15` itself is worth dwelling on, because it is one of the operations that traces. Modulo of two integers lowers to a native `Op:Mod`, since the ported `num_modulus` and fusevm's integer mod agree on the truncated-toward-zero sign rule, so the divisibility tests in a FizzBuzz loop are native operations, not builtin calls. The comparison `== 0` is a native `NumLt`-family op. So a loop that calls `FizzBuzz` across a range, or a hand-inlined FizzBuzz loop, runs its arithmetic natively — the modulo, the comparison, the counter increment all on the fast path — and only the string returns and the `:return` itself route through the slower path. This is the deopt boundary of Chapter 14 visible in the smallest possible program: the numeric core of FizzBuzz is fast, the string production is not, and the two coexist in one function with the numeric part tracing and the string part falling back, which is precisely the graceful boundary the architecture promises. The example's own header names what it demonstrates — “`:function/:return, :if/:elseif/:else`, integer `%` (native modulo), and the built-in assert framework” — and reading the function with that list in mind, every clause of the header is one feature the four-line conditional exercises.

What the FizzBuzz assertions prove

The assertions that follow the function are where FizzBuzz stops being a toy and becomes a test of vimlrs's conditional handling. They check `FizzBuzz(1)` is `'1'`, `FizzBuzz(3)` is `'Fizz'`, `FizzBuzz(5)` is `'Buzz'`, and `FizzBuzz(15)` is `'FizzBuzz'`, and the last one is the assertion that matters, because it is the one a subtly-wrong `if/elseif` implementation would fail. An implementation that evaluated the arms in the wrong order, or that fell through to multiple arms, or that mis-handled the `else`, would produce `'Fizz'` or `'Buzz'` for fifteen instead of `'FizzBuzz'`, and the assertion would catch it. So `assert_equal('FizzBuzz', FizzBuzz(15))`, trivial as it looks, is a precise test that the conditional chain takes exactly one arm and takes the right one, and its passing in CI is the proof that vimlrs's `:if/:elseif/:else` lowering is correct. This is the pattern the whole example suite follows and the reason this book trusts it: a FizzBuzz that asserts its own output is not a demonstration of what FizzBuzz might do but a test of what vimlrs's control flow does do, run every day, and the smallness of the program is a virtue, because a small program that asserts an exact result leaves nowhere for a bug to hide. The conditional chain either produces the four expected strings or it does not, and it does, which is the strongest possible statement that the `If` lowering is faithful.

break, continue, and the iteration protocol

The loop-control statements are conventional but worth pinning down. `:break` exits the innermost enclosing loop, and `:continue` skips to its next iteration — both parsed as their own `Stmt` variants and lowered to jumps out of or back to the loop's control flow. They respect nesting: a `:break` inside an `if` inside a `for` exits the `for`, not the `if`, because the compiler tracks which loop is innermost as it lowers. The `:for` loop's iteration protocol is worth naming because it is more general than “iterate a list.” A `:for x in expr` evaluates `expr` to a List (or, via the native fast path, a `range()` it never materializes) and binds each item to `x` in turn; the loop variable can be a single name or a `[a, b]` unpack, so `for [k, v] in items(d)` destructures each `[key, value]` pair. Over a List the binding is the element; the `oneline.vim` and `lists_dicts.vim` examples lean on both forms. The compiler's native-range lowering (Chapter 21) is the optimization that makes `for i in range(N)` a counter loop with no list, but semantically it is the same iteration protocol — bind, run the body, advance — just with the list elided. So the idiomatic counting loop, the list iteration, and the key-value destructuring are all one construct with one set of control statements, and `:break/:continue` work identically in all three.

Exceptions

The `:try/:catch/:finally/:endtry` construct is ported from Neovim's `ex_eval.c`, and it behaves exactly as Vim's does — which is to say, more powerfully than most languages' try/catch, because the `:catch` clause takes a *pattern*. A bare `:catch` catches everything; `:catch /regex/` catches only exceptions whose text matches. `:throw {expr}` raises a value, and inside a `:catch` the special variable `v:exception` holds the thrown text. The `exceptions.vim` example walks the whole behaviour:

```
try
  throw 'boom'
catch /boom/
  let g:caught = v:exception
endtry
call assert_equal('boom', g:caught)
```

The pattern match is what makes `:catch` expressive. A function can throw differently-prefixed messages and let the catch clauses route them:

```
function! Classify(n) abort
  try
    if a:n < 0
      throw 'E-negative: ' . a:n
    endif
    throw 'ok: ' . a:n
  catch /^E-/
    return 'error'
  catch /^ok:/
    return 'fine'
  endtry
endfunction
call assert_equal('error', Classify(-3))
call assert_equal('fine', Classify(7))
```

`Classify(-3)` throws `'E-negative: -3'`, which matches `/^E-/` and returns `'error'`; `Classify(7)` throws `'ok: 7'`, matching `/^ok:/`.

The pattern-matching `:catch` is what makes Vimscript's exceptions more expressive than most languages', and it rewards a closer look. In a typical language, `catch` dispatches on the exception's *type* — you catch an `IOException` or a `ValueError`. In Vimscript, the exception is a *string* (or any value, stringified), and `:catch /pat/` dispatches on whether that string *matches a regular expression*. So instead of a type hierarchy, you get a naming convention: throw `'E-negative: ...'` for one error class and `'ok: ...'` for another, and route them with `/^E-/` and `/^ok:/`. This is how Vim's own errors are structured — they carry numeric codes like `E117` and `E475` — so `:catch /E117:/` catches the unknown-function error specifically, the idiom `assert_fails('call nope()', 'E117:')` relies on. The matching uses the same regex engine as everything else (Chapter 10), so the full dialect is available in a catch pattern. An unmatched `:catch` lets the exception keep propagating to an outer handler, so a chain of `:catch /pat/` clauses acts like a routing table, each pattern claiming the exceptions it recognizes and passing the rest along. And a bare `:catch` with no pattern catches everything, the fallback. This string-and-regex model is unusual and powerful, and vimlrs reproduces it exactly because it ports the `ex_eval.c` matching logic rather than mapping `:catch` onto a host language's type-based catch — which would have lost the pattern dispatch entirely. An unmatched catch pattern lets the exception keep propagating — `exceptions.vim` verifies that a `/will-not-match/` catch does *not* swallow a `'specific'` throw, which then reaches an outer `/specific/` handler.

The `:finally` clause always runs, on both the normal and the exceptional path, and crucially it runs *while an exception is unwinding* — the example asserts the exact order:

```
call assert_equal(['inner-body', 'inner-finally', 'outer-catch:inner-fail'], g:order)
```

The inner `:finally` runs as the exception propagates out through it to the outer `:catch`, producing the

interleaving `inner-body`, then `inner-finally`, then `outer-catch`.

The `:finally` clause's guarantee is stronger than it first appears, and it is worth stating precisely: the clause runs on every exit path from the `:try` body — normal completion, a `:throw`, a `:return` from inside the body, even a `:break` or `:continue` that leaves the body. This is what makes `:finally` the right place for cleanup that must happen regardless of outcome: deleting a temporary file, restoring a changed option, releasing whatever the body acquired. The `wordfreq.vim` and `glob.vim` examples lean on this implicitly — they create temporary files and clean them up — and the pattern generalizes to any resource a script must release. Because the cleanup runs even while an exception is unwinding, a `:try:finally` with no `:catch` is a valid and useful construct: it lets an exception propagate to an outer handler while still guaranteeing the cleanup runs on the way out. vimlrs's ported `ex_eval.c` state machine tracks which exit path is in progress and runs the `:finally` before continuing that exit, which is why a `:return` inside a `:try` body still runs the `:finally` before the function actually returns — a detail that is easy to get wrong and that vimlrs gets right because the unwinding logic is ported rather than approximated. This is precisely Vim's semantics, ported from `ex_eval.c`'s abort/exception state machine, and getting the unwinding order right is the proof that the port reaches the control-flow layer and not just the expression layer.

The exception state machine, ported

The exception machinery is worth seeing as the ported state machine it is, because the unwinding order Vimscript guarantees is not something you get for free from a host language's try/catch — it has to be modelled deliberately. vimlrs ports `ex_eval.c`, which maintains an explicit notion of the *current exception* being handled and a stack of active `:try` conditionals, each tracking whether it is in its body, its `:catch` clauses, or its `:finally`. The bridge mirrors this with the `PENDING_EXC` thread-local that “models `ex_eval.c`'s `current_exception`” (Chapter 22), and the `v:exception` and `v:throwpoint` values are read off the in-flight exception. When a `:throw` fires, the engine sets the pending exception and begins unwinding: it runs any `:finally` clauses it passes through on the way out, and it tests each `:catch` pattern against the exception text using the regex engine of Chapter 10 — which is why `:catch /^E-/` can route exceptions by prefix.

The interleaving Chapter 11 tests — `inner-body`, then `inner-finally`, then `outer-catch` — falls directly out of this model: when an exception thrown in an inner `:try`'s body propagates outward, the inner `:finally` runs *as the exception passes through it*, before the outer `:catch` that ultimately handles it sees it. A host try/catch that simply mapped `:try` to its own construct would not reproduce this order, because the timing of the `:finally` relative to the propagation is specific to Vim's state machine. By porting `ex_eval.c` rather than approximating it, vimlrs gets the order right, and the `exceptions.vim` example asserts the exact sequence — the proof that the port reaches the control-flow layer, not just the expression layer. The abort behaviour is part of the same machine: the `abort` function attribute and the `did_emsg` flag are `ex_eval.c/message.c` state, so a function marked `abort` stops at the first error because the ported state machine records the error and halts, exactly as Vim's does.

Why the finally-ordering test is the hard one

The interleaving assertion that `exceptions.vim` checks — `['inner-body', 'inner-finally', 'outer-catch:inner-fail']` — looks like a small detail, but it is the single test most likely to expose a shallow exception implementation, and it is worth understanding why the ordering it pins is not the obvious one. The naive mental model of try/catch/finally is “run the body; if it throws, run the catch; either way, run the finally,” and in that model a `:finally` runs *after* the matching `:catch`. For a single try/catch/finally that model is even correct. But the example nests a `:try` inside another `:try`, and the inner `:try` has a `:finally` but no matching `:catch` — the exception thrown in the inner body is caught by the *outer* `:catch`. Now the ordering question is genuine: when the inner body throws, and the exception is going to propagate out to the outer catch, does the inner `:finally` run *before* or *after* the outer catch handles the exception? The naive “finally runs after catch” model would run the outer catch first and the inner finally never, or in the wrong order, because it does not distinguish “the catch that matches this try” from “a catch further out that will handle

this exception.” Vim’s answer, and the answer the example asserts, is that the inner `:finally` runs as *the exception passes through it on its way out*, before the outer catch that ultimately handles it — so the order is inner-body, then inner-finally, then outer-catch.

The reason this is the correct order, and the reason it is hard to get right, is that `:finally`’s real guarantee is not “run after the catch” but “run on the way out of this try block, whatever the way out is” — and when an exception is unwinding through a nested try, the way out of the inner try is *through* its finally, which therefore runs before control reaches anything outside the inner try, including the outer catch. This is the behaviour of a proper unwinding state machine, the kind `ex_eval.c` implements: it tracks a stack of active try-conditionals, and when an exception propagates, it runs the `:finally` of each try it unwinds through, in order, before the exception reaches a `:catch` that handles it. An implementation that mapped `:try` onto a host language’s try/catch and ran the finally as a host-language finally might happen to get the simple case right and the nested-unwind case wrong, because the timing of the finally relative to a non-matching outer catch is specific to Vim’s state machine and not guaranteed by a host construct. By porting the state machine rather than mapping onto a host one, vimlrs gets the nested-unwind order exactly, and `exceptions.vim` asserting the precise three-element sequence is the proof that the port reaches the control-flow layer’s subtlest behaviour — the place where a shallow implementation would have quietly diverged.

Pattern-routing exceptions, narrated

The `Classify` function in the exceptions story deserves a close reading too, because it shows Vimscript’s pattern-based catch doing something a type-based catch cannot. `Classify` throws one of two differently-prefixed messages — `'E-negative: ...'` for a negative input, `'ok: ...'` otherwise — and then catches them with two pattern clauses, `/^E-/` returning `'error'` and `/^ok:/` returning `'fine'`. Trace `Classify(-3)`: it throws `'E-negative: -3'`, the exception begins unwinding, the first `:catch` clause tests its pattern `/^E-/` against the exception text, the text matches, the clause runs and returns `'error'`. Trace `Classify(7)`: it throws `'ok: 7'`, the `/^E-/` clause’s pattern does *not* match, so that clause is skipped, control falls to the `/^ok:/` clause whose pattern matches, and it returns `'fine'`. The catch clauses act as a routing table, each claiming the exceptions whose text matches its pattern and passing the rest to the next clause — and the patterns are real regular expressions run through the engine of Chapter 10, so the full dialect is available for routing. This is structurally different from type-based exception handling: there is no exception hierarchy, no class to catch, only a string and a set of patterns that partition the string-space. It is more flexible in one direction — you can route on any textual property of the message, not just its declared type — and it is exactly how Vim’s own errors work, carrying numeric codes like `E117` that a `:catch /E117:/` can claim. vimlrs reproduces it because it ports the `ex_eval.c` matching, and `exceptions.vim` asserting that `Classify(-3)` returns `'error'` and `Classify(7)` returns `'fine'` is the proof that the pattern-routing reaches the right clause for each thrown prefix — the routing table working as Vim’s does.

The assert framework and v:errors

The exception machinery underpins the testing framework that every example uses. The assert builtins — `assert_equal`, `assert_notequal`, `assert_true`, `assert_false`, `assert_match`, `assert_notmatch`, `assert_report`, `assert_inrange`, `assert_exception`, and `assert_equalfile` — do *not* throw on failure. Instead each appends a descriptive message to the special list `v:errors`, with wording “per `eval.lua`” (the Neovim test-spec source). This is Vim’s actual testing convention, and it is why an example can run many assertions and report all the failures at once rather than dying at the first:

```
call assert_equal('1', FizzBuzz(1))
call assert_equal('FizzBuzz', FizzBuzz(15))
```

Two further testers cover the error path rather than a value. `assert_fails(cmd [, error])` runs a command and *requires* it to error, optionally matching an error code — `assert_fails('call no_such_function()', 'E117:')` asserts that calling an undefined function raises `E117`. `assert_exception(pat)`, used inside a `:catch`, asserts the thrown text matches. The `testing.vim` example combines them:

```
try
  call ParseKV('garbage')
  call assert_report('ParseKV should have thrown on bad input')
catch
  call assert_exception('E0:')
endtry
call assert_fails("call ParseKV('nope')", 'E0:')
call assert_fails('let x = [] + 1')
```

The epilogue that closes every example — inspect `v:errors`, throw if non-empty — turns the accumulated failures into a non-zero process exit, which CI catches. The whole self-testing discipline rests on three ported pieces working together: `v:errors` as an ordinary list the asserts append to, `:throw` to escalate a non-empty list into an aborting exception, and the non-zero exit the driver produces when `did_emsg` is set. It is a small, elegant tower, and it is the reason this book can trust every snippet it quotes.

Chapter 12 – Metaprogramming: `:execute`, `eval`, and `:source`

Vimscript treats code as data more readily than most languages, because so much of it was written to assemble commands dynamically — building a `:substitute` from computed parts, sourcing a configuration file, evaluating a string a plugin passed in. vimlrs supports the three core metaprogramming tools — `:execute`, `eval()/execute()`, and `:source` with `autoload` — and they are worth understanding together because they cross the command/expression seam from Chapter 3 in different directions.

`:execute` builds a command from expressions

The `:execute` command evaluates its arguments, concatenates them space-separated into a single string, and runs that string as an ex command line. It goes from the *expression* layer to the *command* layer: you compute a command as text and then execute it. This is how you build a command whose parts are not known until runtime — a variable buffer number, a computed pattern, a name assembled from pieces. The AST records it as `Execute(Vec<Expr>)`, mirroring the way `:echo` takes a vector of expressions, with the difference that the joined result is *run* rather than displayed.

`eval()` and `execute()` go the other way

Two builtins invert the two halves of the language. `eval(string)` takes a string, parses it as an *expression*, and returns its value — it is the inverse of `string()`, so `eval(string(x))` reconstructs `x`. This is the string-to-value direction: run-string metaprogramming where the string is an expression.

`execute(command)` takes a string (or a List of strings), runs it as command(s) with output *captured*, and returns the captured output as a string. This is the command-to-string direction, and its exact behaviour is subtle enough that `execute_capture.vim` pins it down:

```
call assert_equal("\n5", execute('echo 5'))
call assert_equal('5', execute('echon 5'))
call assert_equal("\n1\n2", execute(['echo 1', 'echo 2']))
```

The key detail is that `:echo` is *preceded* by a newline, so capturing a single `echo 5` yields `"\n5"`, not `"5"` — while `:echon`, which suppresses the newline, captures `"5"`. A List of commands runs in order, each `:echo` contributing its leading newline, and the result is an ordinary string you can `split()` back into lines. This is exactly Vim's `execute()` contract, and it is the standard way a script captures the output of a command it cannot otherwise get a value from. Together, `eval()` and `execute()` are the two run-string tools: one for expressions returning a value, one for commands returning their output.

Why Vimscript is so metaprogramming-heavy

It is worth asking why Vimscript leans on code-as-data so much more than most languages, because the answer explains why these tools exist and why a faithful port has to support them well rather than treating them as exotic corners. Most languages use string-eval sparingly, and many discourage it outright as a code smell — `eval` is the function you are told not to use. Vimscript uses it constantly, and the reason is structural:

Vimscript grew as a configuration and extension language where a great deal of what you do is *assemble commands from parts you do not know until runtime*. You want to substitute in a buffer whose number you computed, so you build the `:buffer N` command as a string and `:execute` it. You want to define a mapping whose right-hand side depends on a setting, so you assemble the `:map` command and run it. You want to set an option whose name is held in a variable, so you build `:set {name}={value}` and execute it. None of these can be written directly, because Ex command arguments are not expressions — you cannot write `:buffer n` and have `n` be a variable — so the only way to parameterize a command is to build it as text and execute the text. This is not a misuse of metaprogramming; it is the *primary* way you write dynamic Vimscript, and a configuration that does anything conditional or computed is full of `:execute` calls.

This is why vimlrs has to support the metaprogramming tools as first-class, well-behaved features rather than as afterthoughts, and why their exact contracts matter. `:execute` building a command from space-joined expression values, `eval()` reading an expression from a string, `execute()` capturing a command's output with its precise leading-newline behaviour, `:source` running a file in the current scope, autoload sourcing a file on demand — these are not advanced techniques a few power users reach for; they are the load-bearing mechanisms of ordinary Vim configuration, and a port that got their contracts wrong would fail to run a large fraction of real Vimscript. The four tools also have to cross the command/expression seam cleanly in both directions, because real code crosses it both ways: `:execute` and `execute()` go between expressions and commands, `eval()` and `string()` go between strings and values, and a script uses whichever direction its task needs. So vimlrs implements all four faithfully — `execute_capture.vim` pinning the exact `"\n5"` capture, `sourcing.vim` pinning the library pattern, `autoload.vim` pinning the lazy load — because in Vimscript, metaprogramming is not the exception but a substantial part of the rule, and a faithful port has to be as good at running assembled commands as at running written ones.

:source and the library pattern

The `:source {file}` command reads and runs another `.vim` file *in the current scope*, so the sourced file's functions and globals persist after it returns. This is Vimscript's module system: a library is a file you source, after which its functions are callable. The `sourcing.vim` example sources a math library and then uses it:

```
source examples/lib/mathlib.vim
call assert_equal(1, g:mathlib_loaded)
call assert_equal(16, Square(4))
call assert_equal([1, 4, 9], map([1, 2, 3], {i, v -> Square(v)}))
```

After the `source`, `Square` and `Cube` are ordinary callable functions and `g:mathlib_loaded` is set, and they compose with lambdas and the higher-order builtins exactly as if they had been defined inline. The AST's `Source(String)` statement holds the raw filename; at runtime vimlrs reads the file, compiles it, and runs it so that its top-level definitions land in the shared scope.

The `:source` command is more than a convenience; it is what gives a multi-file Vimscript project a structure, and the details matter. The sourced file runs in the *current* scope, so its top-level `:let g:` globals and its `:function!` definitions persist after it returns — a library is exactly a file you source for its definitions. But it also establishes a fresh *script* identity for the duration of the source: the sourced file's `s:` variables and `s:` functions are private to it (Chapter 4), and `expand('<sfile>')` inside it resolves to *that* file's path, restored to the sourcing file afterward (`sfile.vim`). vimlrs tracks this with a sourcing-name stack, pushing the file when the source begins and popping it when it ends, which is what lets nested sourcing work — a file that sources another, which sources a third, each with its own `<sfile>` and its own `s:` scope. The `sourcing.vim` example sources a math library and then uses its `Square` and `Cube` functions and its `g:mathlib_loaded` global, proving the definitions persist; the `s:` internals of the library, had it any, would not be visible to the caller. This is Vimscript's module system, and vimlrs implements it faithfully because the whole premise of the project is running real `.vim` code, which is rarely a single file.

Autoload (Chapter 6) is `:source` made lazy and automatic. A call to `pkg#func()` sources `autoload/pkg.vim` on demand the first time the name is needed, then dispatches. The difference is *when*: an explicit `:source` runs the file now; an autoload name runs the file the first moment a function from it is called, and never if it

never is. Both leave their definitions persistent afterward. Between explicit sourcing for libraries you always need and autoload for code you might not, vimlrs gives a standalone script the same modular structure a Vim configuration has — which matters, because the whole premise of the project is running real `.vim` code, and real `.vim` code is rarely a single file.

Building commands at runtime

`:execute` is the workhorse of dynamic Vimscript, and it is worth seeing how it composes with the rest of the metaprogramming surface, because real plugins lean on it heavily. The command takes a list of expressions, evaluates each, joins them space-separated, and runs the result as a command line. The `streamedit.vim` and `excommands.vim` examples use it to build Ex commands whose file or range is not known until runtime:

```
exe ':edit ' . s:in
exe ':w ' . s:out
exe ':%r ' . s:out
```

Each `exe` (the common abbreviation for `:execute`) concatenates a literal command prefix with a computed path and runs the whole string — `:edit ' . s:in` becomes `:edit /tmp/vimlrs_se_in.txt`, which loads that file into the buffer. This is the only way to write a command whose argument is a variable, because Ex command arguments are not expressions: `:edit s:in` would try to open a file literally named `s:in`, so you must build the command text and `:execute` it. The companion `expandcmd()` expands environment variables and wildcards *inside* a command string without running it — `expandcmd('cat $HOME/.vimrc')` returns `'cat /home/you/.vimrc'` — which is the tool for preparing a command string that you will then pass to `system()` or `:execute`.

The four metaprogramming tools now form a complete set, and it is worth naming how they pair across the command/expression seam of Chapter 3. `:execute` goes from expressions to a command (compute text, run it as a command); `execute()` goes from a command to a string (run a command, capture its output); `eval()` goes from a string to a value (parse an expression, return it); and `string()` goes from a value to a string (render a value reparsably). Two cross each way through the command layer, two through the expression layer, and together they let a script treat both its commands and its values as data — assemble a command from computed parts, capture what a command prints, turn a value into text and back. That symmetry is why Chapter 3 insisted the two layers were worth seeing separately: the metaprogramming tools are exactly the bridges between them, and you cannot understand `:execute` versus `eval()` without first seeing that they cross in opposite directions.

Deferred definition: functions registered when their line runs

There is one more piece of metaprogramming that hides in plain sight, so ordinary that most Vimscript authors use it without noticing it is metaprogramming at all: a `:function` definition is not necessarily processed at parse time. When a `:function ... endfunction` block sits inside a control-flow construct — an `:if`, a `:while`, a `:for`, a `:try` — or inside another function's body, the definition is registered when its line *executes*, not when the file is parsed. This is faithful to Neovim's `userfunc.c`, where those enclosing blocks only adjust the definition's indent tracking, and the inner `:function` actually defines its function when the enclosing code is reached and run. The `deferred_function.vim` example pins the whole behaviour, and it matters because it is the mechanism behind an idiom that appears in nearly every real `vimrc` and colour scheme: the idempotent guard.

```

if 1
  function! TrueGuard()
    return 42
  endfunction
endif
call assert_equal(1, exists('*TrueGuard'))

if 0
  function! FalseGuard()
    return 7
  endfunction
endif
call assert_equal(0, exists('*FalseGuard'))

```

A `:function` inside a *true* `:if` is defined; a `:function` inside a *false* `:if` is never defined, because the line that would define it never runs. This is not a parser optimization — it is a semantic fact, and it is precisely what makes `exists('*FalseGuard')` answer 0. The idiom built on it is the re-source guard: wrap a definition in `if !exists('*F')` so that the first time a file is sourced the function is defined, and every subsequent re-source sees the function already present and skips the redefinition, so the original body wins.

```

if !exists('*ScriptGuard')
  function! ScriptGuard()
    return 'first'
  endfunction
endif
if !exists('*ScriptGuard')
  function! ScriptGuard()
    return 'second'
  endfunction
endif
call assert_equal('first', ScriptGuard())

```

The second guard block runs its `!exists` test, finds `ScriptGuard` already defined, and does not redefine it — so the function returns `'first'`, the body from the block that ran while the function was still undefined. A colour scheme like `zpwrmaklar.vim` relies on exactly this to stay safe under repeated sourcing, and the same shape covers a function defined inside another function's body (registered when the *outer* function runs, not at parse time, and not an error), and a function defined inside a `:while` loop (registered on the first iteration that reaches its line). All of these are the deferred-definition rule seen from different angles, and reproducing it is what lets the idempotent-guard idiom — one of the most common patterns in real Vimscript — behave in standalone vimlrs exactly as it does in the editor.

Chapter 13 — What “Faithful Port” Actually Means

The phrase “faithful Rust port of the Vimscript interpreter, from the Neovim C eval engine” is the crate’s one-line description, and it is the most important sentence in this book. Everything else — the standalone binary, the JIT speed, the LSP and DAP — is downstream of a single methodological commitment: that vimlrs’s semantics are *ported*, mechanically and verifiably, from real Neovim C source, rather than re-imagined from documentation. This chapter is the anatomy of that commitment: the two-zone layout, the citation discipline, the drift gate that enforces it, and — just as important — the places where vimlrs deliberately does *not* port, and why.

The two zones

vimlrs’s source tree is split into two zones, and the split is the project’s central structural idea. The `lib.rs` documentation names them. The first is the `ported` zone: “strict 1:1 ports of the Neovim eval C source under `csrc/`. Exact C names, `// c:NNN` citations, no invented helpers/structs.” The second is the set of “crate-root carve-outs — net-new synthesis with no C counterpart.”

The reason there must be two zones rather than one is that Neovim’s eval engine does not have all the parts vimlrs needs. As `lib.rs` puts it, “Neovim’s eval is a string-walking interpreter with no AST/bytecode,” so there is nothing to port for the lexer, the AST, the parser, the bytecode compiler, or the fusevm bridge — those are genuinely new code that has to exist for VimL to run on fusevm at all. If those new modules lived next to the ports with no marker, the “faithful port” claim would erode: a reader could not tell which code mirrors Neovim and which is invention. So every synthesis file opens with a loud banner — you have seen it quoted throughout this book — declaring itself an “EXTENSION — NO `csrc/` COUNTERPART.” The banner is not decoration; it is a firewall. It guarantees that synthesis code can never masquerade as a port, and that the ported zone stays pure. The banner’s exact wording — “EXTENSION — NO `csrc/` COUNTERPART” — is chosen to be unmissable and unambiguous: a reader opening any source file knows within the first lines whether they are looking at a faithful port (which will carry `// c:` citations) or a synthesis-zone extension (which carries the banner). There is no third category, no file that is “sort of a port,” because every file is one or the other and says which. This binary clarity is what makes the faithfulness claim auditable: you do not have to trust that the porter was careful, you can open a file and read its first lines to know which discipline applies to it. The lexer, parser, AST, compiler, bridge, regex engine, and JSON decoder all carry the banner because none has a C counterpart to port — Neovim’s string-walking interpreter simply has no lexer or bytecode — and the ported value layer and builtins all carry citations because each mirrors a specific C function. The seam between the two zones is the most important structural idea in the codebase, and the banner is how the seam is made visible at the level of an individual file, so that the boundary between “faithful to Neovim” and “net-new synthesis” is never in doubt for any line of code.

The `ported` module mirrors the C file by file. `ported/eval.rs` and its `eval/` subtree port `eval.c`; `eval_h.rs` ports `eval.h`’s types like `exprtype_T` and the `OK/FAIL` constants; `ex_eval.rs` ports the abort and exception state from `ex_eval.c`; `charset.rs`, `message.rs`, `ops.rs`, `option.rs`, `path.rs`, `profile.rs`, `sha256.rs`, and `strings.rs` each port a named Neovim source file. The crate-wide lint `#![allow(non_camel_case_types)]` exists precisely so the ported code can keep Vim’s exact C identifiers — `ufunc_T`, `except_type_T`, `VAR_FLAVOUR_*`, `typval_T` — rather than renaming them to Rust convention. A name that differs from the C is a name a reader cannot trust to mean the same thing, so the ports keep the C names verbatim even where they look un-Rusty.

Why two zones rather than one

The two-zone split can look, at first, like an admission of failure — if the goal is a faithful port, why is there a whole zone of code that is *not* a port? — and answering that objection is the fastest way to understand why the split is not a compromise but a necessity that the project handles unusually honestly. The objection assumes that a faithful port could, in principle, be all port, that every line of vimlrs could mirror a line of Neovim C. But that is impossible, because Neovim's eval engine does not contain the things vimlrs needs. Neovim parses and evaluates Vimscript in a single pass over the source string — it scans characters and computes results inline, with no separate lexer, no abstract syntax tree, no bytecode, no compiler. So there is no lexer in Neovim to port, no AST to port, no bytecode compiler to port. vimlrs needs all of these, because it does not evaluate by scanning the source — it lowers to fusevm bytecode, which requires a tokenizer, a tree, and a compiler that Neovim simply does not have. You cannot port a thing that does not exist in the source, so the frontend must be net-new code, and net-new code that has no C counterpart is, by definition, not a port. The synthesis zone is not a place where the project gave up on fidelity; it is the code that has to exist for a bytecode-lowering interpreter that Neovim, being a source-scanning interpreter, never needed.

So the two zones are not “the faithful part and the lazy part”; they are “the part that mirrors Neovim and the part that has no Neovim to mirror,” and the distinction is forced by the architectural choice to lower to bytecode rather than scan the source. What makes the project's handling of this honest, rather than a quiet blurring, is the banner. A less disciplined project would let the synthesis code and the ported code sit together undifferentiated, and a reader — or a future maintainer — would have no way to tell which code mirrors Neovim and which is invention, so the “faithful port” claim would slowly erode into “mostly faithful, we think.” The banner forecloses that erosion: every synthesis file declares itself, every ported file cites its source, and the two zones stay legibly distinct. The reason this is the single most important structural idea in the codebase is that it is what lets “faithful port” mean something precise even though a large fraction of the code is not a port. The claim is not “all of vimlrs is ported from Neovim” — that would be false, because the frontend cannot be — but “the semantics are ported, the synthesis is marked, and you can always tell which is which.” That is a claim the two-zone split with its banner makes true and auditable, and it is why the split is the foundation everything else in this chapter rests on: without it, the faithfulness claim would have no clean boundary, and with it, the boundary is visible in the first lines of every file.

What mechanical porting actually feels like

It is worth slowing down on what “ported, mechanically and verifiably” means as a daily activity, because the phrase is easy to nod past and the reality is more specific and more demanding than it sounds. Porting a function from Neovim's C is not reading the C, forming a mental model of what it does, and then writing Rust that does the same thing — that is reimplementation, and it is exactly what the project refuses, because a mental model is a lossy compression of the source and the places it loses information are precisely the edge cases that distinguish a faithful interpreter from an approximate one. Mechanical porting is the opposite discipline: you open the C function, you translate it statement for statement into Rust, you keep its control flow, its variable names, its early returns, its error codes, and its comments, and you resist the urge to improve it, to make it more idiomatic, to collapse two branches that “obviously” do the same thing — because if you collapse them and you are wrong about the obviousness, you have introduced a divergence that no test will catch until some script in the wild depends on the distinction. The Rust you produce looks un-Rusty: it has goto-shaped control flow expressed as loops with breaks, it has C integer types, it has the exact E-numbered error messages, it has the original's odd-looking special cases intact. That un-Rustiness is not a failure to write good Rust; it is the visible evidence that the port is faithful, that the translator chose fidelity over elegance at every point where the two conflicted.

The reason this discipline is tractable at all — the reason a single person can port a C codebase of Neovim's eval engine's size without drowning — is that the C is there, in `csrc/`, as the specification, and the translation is a local operation. You do not have to hold the whole engine in your head; you hold one function, its C source on one side and its Rust port on the other, and you make them agree line by line. The citation makes the agreement checkable after the fact: a reviewer, or the porter themselves six months later, can open the cited

C line and read it against the Rust, and the question “is this faithful?” has a concrete answer rather than a vibe. This is why the citations are not documentation niceties but load-bearing infrastructure — they convert an unbounded question (“does vimlrs behave like Vim?”) into a bounded one (“does this function match its cited C?”), and a bounded question is one a person can actually answer. The drift gate then automates the part of the discipline that is easiest to let slip: it does not check that a port is correct — no test can fully do that — but it checks that every function in the ported zone at least claims to be a port of a real C function, which closes off the failure mode where synthesis code quietly accumulates in the zone that is supposed to be pure. Between the citations that make fidelity checkable and the gate that makes the zone boundary enforced, the porting discipline is not a heroic act of sustained care but a set of mechanical constraints that make the faithful path the path of least resistance.

The alternative that was rejected

To see why this matters, it helps to picture the interpreter vimlrs deliberately did not build, because that interpreter is the obvious one and it would have been far less work. You read `:help eval`, the user-facing documentation of Vimscript’s semantics, and you write a clean, idiomatic Rust interpreter that implements what the help says. It would have a tidy value enum, a readable evaluator, well-factored builtins, and it would pass a test suite you wrote from the help. It would run FizzBuzz, it would handle most configuration blocks, and it would feel, to its author, like a faithful Vimscript. And it would be wrong in a hundred small ways that the help does not mention and the author did not think to test, because the help is a description written for humans, not a specification written for machines, and the gap between the two is exactly the set of behaviours that are true of the implementation but were never written down. The integer division that truncates toward zero, the `%` that errors on a float operand with `E804`, the `str2nr` that ignores quote separators only with a truthy third argument, the negative `strpart` start that folds into the length, the composing character that `byteidx` swallows but `byteidxcomp` counts, the `v:null` that stays distinct from `v:none` through a list — none of these are in the help in a form precise enough to implement from, and a from-the-help interpreter gets every one of them subtly wrong.

The author of that interpreter would never know, because their tests were written from the same incomplete source as their implementation, so the tests and the code would agree with each other and disagree with Vim. This is the trap of reimplementing from documentation: the documentation is the shared blind spot of the implementation and its tests, and the bugs that live in that blind spot are invisible from inside it. You cannot test your way out of it, because the tests share the blind spot; you cannot review your way out of it, because the reviewer reads the same help; you can only escape by taking the specification from somewhere the blind spot does not reach — the actual source — which is exactly what porting from `csrc/` does. The faithful port is more work, and the work buys the one thing the easy interpreter cannot have: correctness in the places the documentation does not reach. That is the whole wager of the project compressed into a sentence, and it is why the README can say “the C source is the spec” rather than “the help is the spec” — the help is where the easy interpreter goes wrong, and the source is where the faithful one stays right.

Citations and the value layer first

Inside the ported zone, every function cites its origin. The convention is a `// c:NNN` comment giving the source file and line — you can see it in `typval.rs`, where `tv_list_find()` is annotated “Port of `tv_list_find()` from `Src/eval/typval.c` (c:1612)” and `tv_list_check_range_index_one()` cites `typval.c:633`. The function keeps its exact C name (`tv_list_find`, not `list_find` or `find_item`), and the citation lets anyone open the vendored C and check the port line for line. This is what “the C source is the spec” means in practice: not a vibe but a cross-reference.

The value-layer-first ordering is not just a sequencing convenience; it is a correctness requirement, and seeing why illuminates the whole port. You cannot port the comparison operator until you have a correct value type to compare, because `typval_compare` operates on `typval_T` — port the comparison first and you would have nothing to compare. You cannot port `string()` until you have the List, Dict, and Blob it renders, because the

encoder walks those structures. You cannot port `num_divide` until you have the Number type it divides. So the dependency order is forced: the `typval_T` value type and its containers come first, then the coercions that read values out of them (`tv_get_number` and friends), then `typval_compare` and the arithmetic that operate on them, then the encoders that render them, then the lexer and parser that produce expressions over them, then the bytecode lowering, then the builtin surface that manipulates them. Each layer rests on a faithful one beneath, and the README's status table reads top to bottom as exactly this dependency order made visible — value layer, coercions, rendering, frontend, lowering, JIT, builtins. The discipline is that you never build a layer on an approximate version of the one below it: the substrate is ported faithfully first, so everything above it inherits faithful behaviour rather than compounding an approximation. This is why “ported first” is the phrase the engineering report uses — not “ported early” but ported *first*, because the rest of the engine could not be faithful if it were not.

The order of porting was chosen deliberately. The engineering report says the “Vim-native runtime values were ported first so the rest of the engine has a faithful substrate to build on.” You cannot port the comparison operator correctly until you have a correct value type to compare; you cannot port `string()` until you have the List, Dict, and Blob it renders. So `typval`, `list`, the insertion-ordered `dict`, and `blob` came first, then the coercions and `typval_compare`, then `num_divide/num_modulus`, then the encoders `encode_tv2string/encode_tv2echo`, then the lexer/parser, then the bytecode lowering, then the builtin surface — each layer resting on a faithful one beneath. The README's status table reads, top to bottom, as that dependency order made visible.

It is worth distinguishing the project's two documentation sources, because this book draws on both and they speak with different voices. The README and its status table are the *engineering* record: a frank, granular accounting of exactly which components are ported, working, in progress, or planned, written for a contributor who needs to know the state of the code. The engineering report (the published [report.html](#)) is the *narrative* record: it explains the design decisions — why the value layer was ported first, why the dict preserves insertion order, what lowering to fusevm buys — in prose aimed at a reader who wants the shape of the thing. Where the two overlap they agree, and where they differ in emphasis this book has noted it — the report counts only “three-tier Cranelift JIT” without naming the tiers, while the README and source distinguish block-JIT from trace-JIT, so this book attributes the tier names to the README accordingly. Both are grounded in the same code, and the discipline that makes the code trustworthy — the citations, the drift gate, the self-testing examples — is what makes both documents trustworthy too: a claim in the status table is backed by a test, a claim in the report by the source it describes. This book is a third document in the same family, the one you read front to back, and it inherits the same grounding: every snippet copied from a real file, every claim about the machine traceable to a specific source.

The drift gate

The most striking thing about vimlrs's faithfulness is that it is *enforced by a test*, not by reviewer discipline alone. The test `ported_fn_names_match.c.rs` calls itself “the immune system against porting drift,” and its rule is exact: “Every `fn` defined under `src/ported/` must trace back to upstream Neovim C: its name must appear in `docs/nvim_c_functions.txt` (generated from the vendored `csrc/`) OR be a sanctioned exception in `tests/data/fake_fn_allowlist.txt`.”

Read that again, because it is a genuinely unusual guarantee. A contributor who adds a helper function to the ported zone with an invented name — `make_helper`, `parse_v2`, some bag-of-globals accessor that Neovim does not have — *fails the build*. The only escapes are trait-impl methods (`default`, `new`, `fmt`, `clone`, and the like, which legitimately have no C origin) and an allowlist whose entries are “a deliberate, reviewed act — never a silent way to pass.” The effect is that the ported zone cannot silently drift away from Neovim: every name in it either exists in the vendored C or was explicitly, visibly sanctioned. Most “port” projects rely on the porter's good intentions; vimlrs relies on a CI gate that fails when good intentions lapse.

This pairs with the stub mechanism. The full interpreter C surface is *scaffolded*: a script, `gen_port_stubs.sh`, generates one stub per not-yet-ported Neovim C function — “real name + `csrc/<file>:<line>` citation” — under `src/ported/stubs/`. The stub directory mirrors the C tree (`buffer.rs`, `decode.rs`, `encode.rs`, `eval.rs`, `funcs.rs`, `list.rs`, and so on). The remaining work is therefore *enumerated*: you can see exactly

which functions are not yet ported, because each is a stub citing the C it will become. As functions are faithfully ported they “drop out of the stub tree.” The drift gate covers the stubs too, so even the not-yet-done work carries real C names. Between the gate and the stubs, the codebase always knows precisely how faithful it is and how much is left.

The stub tree as a map of remaining work

The stub mechanism deserves more than the paragraph Chapter 13 first gave it, because it is the thing that makes the port’s incompleteness *legible* rather than hidden. The full interpreter C surface is scaffolded: `gen_port_stubs.sh` reads the vendored `csrc/` tree and generates, under `src/ported/stubs/`, one Rust stub per not-yet-ported Neovim C function — each carrying the real C name and a `csrc/<file>:<line>` citation. The stub directory mirrors the C file structure (`buffer.rs`, `decode.rs`, `encode.rs`, `eval.rs`, `funcs.rs`, `list.rs`, and so on), so the shape of the remaining work matches the shape of Neovim’s eval tree. A contributor who wants to know what is left to port does not have to guess; they read the stub tree, where each stub is a named, cited placeholder for a function that will become a real port. As functions are faithfully ported “they drop out of the stub tree” — the stub is deleted and the real implementation takes its place in the mirrored module. This is why the README can say the remaining work is “enumerated”: the codebase always knows exactly how much of Neovim’s eval surface it has brought across, because the un-ported part is a directory of cited stubs you can count.

The drift gate covers everything

The drift gate, `ported_fn_names_match_c.rs`, is what keeps both the ports and the stubs honest, and its rule applies to the whole `src/ported/` tree, stubs included. Every `fn` defined under `src/ported/` must trace to upstream Neovim C — its name must appear in `docs/nvim_c_functions.txt`, which is generated from the vendored `csrc/`, or be a sanctioned exception in `tests/data/fake_fn_allowlist.txt`. Because the stubs carry real C names, they pass the gate; because a hand-invented helper does not, it fails the build. The only sanctioned exceptions are the trait-implementation methods that legitimately have no C origin — `default`, `new`, `fmt`, `clone`, and the like — and the explicit allowlist, whose entries the test itself calls “a deliberate, reviewed act — never a silent way to pass.” The effect is a guarantee most port projects cannot make: the ported zone cannot accumulate invented helpers, because the moment a function name appears there that is not in the vendored C, CI goes red. The faithfulness is not a promise the contributors make; it is a property the build enforces, and the stub tree plus the gate together mean the codebase can be both incomplete and trustworthy — every name in it, ported or stubbed, traces to a real line of Neovim C.

What a citation buys you

The `// c:NNN` citation convention is small but it is the thing that makes “the C source is the spec” a checkable claim rather than a slogan. A ported function keeps its exact C name and carries a comment giving the source file and line, so `tv_list_find` is annotated as a port of `tv_list_find()` from `Src/eval/typval.c` at a specific line. The payoff is reviewability at the finest grain: a reader who doubts whether the comparison of two lists is faithful can open the cited `typval.c` line and read the C beside the Rust. This is what distinguishes a port from a reimplementation that happens to agree — a reimplementation says “this behaves like Vim,” a port says “this is `typval_compare`, here is the line, check it.” The citations also make the porting order visible: the value layer came first because the comparison and encoding functions cite it, and you can read the dependency order off the citations as much as off the README’s status table. Faithfulness, in vimlrs, is a chain of cross-references from each Rust function to the C line it mirrors, and the drift gate is the machine that refuses to let a link in that chain be forged.

Where it deliberately diverges

Faithfulness is not literalism, and vimlrs diverges from Neovim in two principled ways. The first is *architectural*: the synthesis zone exists because Neovim’s string-walking interpreter has no AST or bytecode, so vimlrs cannot port a frontend that does not exist and instead synthesizes one. The regex engine and the JSON parser are the clearest cases — `viml_regex.rs` reproduces Vim’s pattern *behaviour* with a fresh backtracking matcher rather than porting `regexp_bt.c`’s VM, and `viml_json.rs` is a recursive-descent parser that yields “the identical `typval_T` tree” as the ported `decode.c` would, with the encode side a faithful port in `encode.rs`. In both, the *output* matches Neovim even though the *mechanism* is new; fidelity is to the observable result.

The second divergence is *environmental*: there is no editor. Neovim’s `has()` reports features like `windows`, `syntax`, and `folding`; vimlrs’s `has()`, the example explains, “reports only what it genuinely is” — the real build platform plus the language features it actually implements — and returns 0 for editor subsystems it does not have, and even 0 for `nvim` itself, because vimlrs is neither Vim nor Nvim. This is the opposite of dishonesty: rather than fake the editor’s answers, vimlrs tells the truth about being a standalone interpreter. Chapter 18 is the full account of this “editor-absent” contract. The point for this chapter is that both divergences are *principled and marked* — the architectural one by the EXTENSION banners, the environmental one by honest return values — so that “faithful port” never quietly becomes “mostly like Vim.” The boundary is always legible.

It is worth naming a third, smaller category of divergence: the *provisional* simplifications, the places where vimlrs has not yet reached full fidelity and says so rather than pretending. `:const` is the clearest: vimlrs parses the keyword and binds the value but does not yet enforce the immutability lock, so a later rebind that Neovim would reject with E741 is currently accepted — and the example comment and this book both flag it as not-yet-enforced. The `garbagecollect` and `islocked` builtins are honest stubs, because there is no cyclic collector or lock-tracking in the current build. These differ from the principled divergences in kind: the architectural and environmental ones are *permanent and correct* (vimlrs will always synthesize a frontend, and will always be honest about lacking an editor), while the provisional ones are *temporary and acknowledged* (vimlrs intends to enforce `:const` eventually). What unites all three categories is the refusal to hide a gap. A permanent divergence is marked by a banner or an honest return value; a provisional one is marked by a flagged caveat in the example and the docs. The faithfulness discipline is not the claim that vimlrs is already a complete Vim — its own status badge says “early · in development” — but the claim that every place it falls short of Vim is visible, so a reader always knows exactly how faithful the thing in front of them is. That legibility, across the permanent and the provisional alike, is what “faithful port” actually means.

The ported modules, file by file

The *ported* zone is not a monolith; it is a set of Rust modules, each mirroring one Neovim C source file and each citing it. Reading the module list is the fastest way to see the *shape* of the port — which parts of Neovim’s eval tree have been brought across, and what each contributes. The `ported.rs` module header restates the discipline before listing them: “Every file here mirrors a `csrc/` file, uses the exact C names, and cites its source ... Net-new synthesis (lexer, parser, AST, bytecode compiler, fusevm bridge) lives in the crate-root carve-out modules instead, never here.”

The value substrate comes first, because everything rests on it. `typval` ports `eval/typval.c` — the `typval_T` value type and its list/dict/blob helpers, where a function like `tv_list_find` keeps its exact C name and a `// c:1612` citation. `list`, `dict`, and `blob` port the three container files, with `dict` being the insertion-ordered one backed by `indexmap` to match Vim’s observable iteration. `typval_defs_h` ports `typval_defs.h`, carrying the `VarType` enum and the `VAR_*` constants Chapter 5 enumerated. On top of the values sit the operations: `eval` ports `eval.c` and its `eval/` subtree (the comparison, arithmetic, and index logic the bridge’s handlers call), `eval_h` ports `eval.h` with its `exprtype_T` and the OK/FAIL constants, and `ex_eval` ports `ex_eval.c` — the abort-and-exception state machine whose unwinding order Chapter 11 tested.

The encoding and decoding modules are next. `encode` ports `eval/encode.c` — the `string()`, JSON, and MessagePack *encode* paths — and `decode` ports `decode.c`, though as Chapter 16 noted the JSON *decode* side is actually a synthesis-zone recursive-descent parser that yields the identical tree. `strings`

ports `strings.c` (the `f_string`, `f_strlen`, `f_byteidx`, `f_tr` family), `charset` ports `charset.c` (the `vim_str2nr` and character-class helpers behind the number literals and class atoms), and `message` ports `message.c` (the `emsg/did_emsg` error machinery the CLI reads for its exit code). The builtin table itself lives in `funcs`, mirroring `eval/funcs.c`, which is the surface the reference catalogues.

The remaining modules are the smaller, targeted ports — each a *subset* of its C file, brought across only as far as the builtins that need it. `option` ports a subset of `option.c` (the option table, `fopt`, and `:set`), `ops` a subset of `ops.c` (the yank-register store behind `getreg/setreg`), `path` a subset of `path.c` (the helpers behind `pathshorten`), `os` a subset of `os/time.c` (the `os_hrttime` behind `reltime`), `profile` the `proftime_T` helpers, and `sha256` the FIPS-180-2 hash. The honesty of the zone is that each of these declares exactly what it ports and cites where — and the not-yet-ported surface is itself enumerated in `stubs`, “one stub per vendored C function definition (real name plus `csrc/<file>:<line>` citation),” regenerated by `gen_port_stubs.sh` and covered by the drift gate. So the ported zone is legible top to bottom: the value layer at the base, the operations and encoders above it, the builtin table at the surface, and a stub for everything not yet reached — every module a mirror, every function a citation, and a CI gate that fails the build if any name in the zone cannot be traced back to the C it claims to port.

Chapter 14 – The Machine Underneath

`vimlrs` has no virtual machine of its own. That sentence has been repeated several times in this book because it is the load-bearing fact about how `vimlrs` runs, and this chapter is where it gets unpacked. The execution path from source text to running code passes through a lexer, a parser, an AST, a bytecode compiler, and then out of `vimlrs` entirely into `fusevm` — the shared bytecode VM and its three-tier Cranelift JIT. We will follow a script down that path and then look closely at the JIT tiers, because the speed claims in Chapter 1 live there.

The pipeline

The pipeline, from the engineering report, is five stages:

```
VimL source → lexer → parser → AST → lower to fusevm bytecode → fusevm VM +  
↳ Cranelift JIT
```

The first three stages are `vimlrs`'s synthesis zone. The lexer (`viml_lexer.rs`) turns source text into a token stream — numbers, floats, blobs, strings, sigil-prefixed names, operators with their case flags, the `#{` literal-dict opener, and so on. Its token type records each token's byte span so the parser can make adjacency decisions like `d.key` versus `a . b`. The parser (`viml_parser.rs`) builds the AST, following the `eval1...eval7` precedence ladder for expressions and recognizing the `ex-command` statements. The AST (`viml_ast.rs`) is the `Expr` and `Stmt` types from Chapter 3.

The fourth stage, `compile_viml.rs`, is where `vimlrs` hands off. It lowers the AST to a `fusevm::Chunk` — `fusevm`'s compiled-program type. The crucial discipline here, stated in the compiler's header, is that “faithful VimL semantics are never inlined here — every operator routes to a `VIML_*` builtin whose handler calls the canonical ports.” So the compiler does not reimplement what `+` means; it emits a call to a `VIML_*` builtin, and that builtin's handler (in `fusevm_bridge.rs`) pops the operands and calls the ported arithmetic. The bridge is “pure `fusevm` plumbing, no operator logic”: it converts between the ported `typval_T` and `fusevm::Value`, holds the per-run state, and registers the `VIML_*` handlers that delegate to `crate::ported::*`. This is the seam between the two zones at runtime — the compiled bytecode is synthesis, but every actual *operation* it performs lands in a faithful port.

The output of compilation is a `CompiledProgram`: a `main` chunk plus the user functions it defines, including the anonymous `<lambda>N` functions generated from lambdas. That whole unit is what runs on `fusevm`, what gets cached (Chapter 15), and what the disassembler prints.

The `CompiledProgram` being a single self-contained unit is what makes the cache work cleanly. It holds the `main` chunk, the user functions (each a `UserFuncDef` with its name, parameters, compiled default-value chunks, bang flag, and body chunk), and the generated lambda functions — everything needed to run the program, with no reference back to the source. So caching a script means serializing one `CompiledProgram` and restoring it later restores the functions and lambdas along with the main chunk; a cache hit does not have to re-derive the function table, because the function table is part of the cached unit. This is why the cache can skip lexing, parsing, *and* lowering on a warm run: the thing it stores is the complete output of all three stages, a runnable program rather than an intermediate form. The serialization uses `bincode` for the inner chunk blobs inside the `rkyv` archive (Chapter 15), and the format is versioned so a change to the chunk layout or the meaning of emitted builtin ids bumps the version and invalidates old entries. The `CompiledProgram` is, in effect, `vimlrs`'s compiled-artifact format — the `.vim` analogue of a `.pyc` or a `.beam` — and its self-containedness is what lets a script be compiled once and run many times, cached on disk between invocations, or baked whole into an AOT binary.

Tree-walking versus bytecode, in concrete terms

The decision to lower to bytecode rather than walk the AST is easy to state and easy to understate, so it is worth making concrete what the two approaches actually do differently on a loop, because the difference is the whole reason vimlrs can post the speed numbers it does. A tree-walking interpreter holds the parsed abstract syntax tree in memory and evaluates it by recursive descent: to run `while i < n`, it walks to the condition node, recursively evaluates the `<` node, which recursively evaluates the `i` node and the `n` node, each of which is a variable lookup that hashes a name into a dictionary, and then the comparison node compares the two results and returns a boolean, and the loop node inspects that boolean and decides whether to walk the body. Every iteration walks the same subtree again, dispatching on each node's type through a big match, resolving each variable by name through a hash lookup, and allocating the intermediate values the recursion produces. The per-iteration cost is dominated not by the arithmetic — which is trivial — but by the *overhead* of the walk: the type dispatch, the name resolution, the recursion, the allocation. For a loop that runs twenty million times, that overhead is paid twenty million times, and it swamps the actual work by an order of magnitude or more.

A bytecode VM pays the walk exactly once, at compile time, and flattens the tree into a linear sequence of instructions that a tight dispatch loop executes directly. The condition `i < n` becomes a `GetSlot` (read slot 0), a `GetSlot` (read slot 1), a `NumLt` (compare), and a conditional jump — four instructions, no recursion, no name resolution (the names became slot indices at compile time), no type dispatch on tree nodes (the instructions carry their operation in the opcode). The per-iteration cost collapses to the cost of executing four simple instructions, and that cost is small enough that the loop's speed approaches the speed of the arithmetic itself. This is the difference between re-deriving how to run the loop on every pass and deciding once how to run it and then just running it. And it is the precondition for the JIT, because a JIT compiles a *linear instruction stream* to native code — there is nothing for a JIT to compile in a tree-walk, because there is no stream, only a tree re-traversed. So lowering to bytecode is not merely faster than tree-walking by some constant factor; it is the architecture that makes native compilation possible at all, which is why the choice to lower is the load-bearing decision behind every speed claim in this book.

Why vimlrs lowers rather than walks, given Neovim does not

There is an irony worth naming here, because it cuts against the faithful-port discipline that governs everything else. Neovim's own eval engine is *not* a bytecode VM and not even a clean tree-walker — it parses and evaluates in a single pass over the source string, with no AST and no bytecode, scanning characters off the input while computing results inline. So if vimlrs's guiding principle is to port Neovim's C faithfully, why does it not port Neovim's parse-and-evaluate-in-one-pass approach? The answer is the seam between the two zones that Chapter 13 drew. The *semantics* — what `+` means, how two values compare, what `string()` renders — are ported faithfully, because those are the thing a script depends on and the thing the C specifies. But the *execution strategy* — how those semantics are reached, whether by scanning a string or walking a tree or running bytecode — is not something a script can observe, and it is the place where vimlrs is free to do something Neovim does not, because doing so changes no observable behaviour while buying enormous speed. Porting Neovim's single-pass scanner would have meant inheriting its speed too, which for a hot loop is the speed of re-parsing the source on every iteration — far worse even than tree-walking. So vimlrs ports the semantics and synthesizes a new execution strategy, lowering to fusevm, precisely because the execution strategy is the one part of the engine where fidelity is not owed: no script can tell whether its arithmetic was reached by a scanner, a tree-walk, or a bytecode VM, only whether the arithmetic was right. The synthesis zone exists for exactly this reason, and the lowering pipeline is its largest member — net-new code that has no C counterpart because the C does something the standalone interpreter has every reason to improve upon, in the one dimension where improvement is invisible to the user and faithful to the result.

Seeing the bytecode

The `--disasm` flag prints the fusevm bytecode listing before the program runs. `fusevm_disasm.rs` walks the `Chunk` and emits, for each instruction, its index, source line, and the opcode — plus the chunk’s source, its name table, its sub-entries, and recursively its sub-chunks (so function bodies print nested). Running `vimlrs --disasm script.vim` shows you exactly what the compiler produced: the `Op::Add`, `Op::GetSlot`, `Op::CallBuiltin`, and jump instructions that the VM will execute. It is the ground truth for understanding what lowered to what, and it is the first tool to reach for when a numeric loop is not tracing the way you expect.

The two JIT tiers that matter for VimL

fusevm has a three-tier Cranelift JIT — the engineering report states it as a bare fact, “three-tier Cranelift JIT,” and counts it as a headline stat. The *naming* of the tiers and the specifics of which VimL constructs reach them come from the README and the source rather than the report, so this section is grounded there. Two compilation behaviours matter for Vimscrip, and both are about getting hot numeric code to native speed.

The first is *block-JIT compilation*: an integer expression block-compiles to machine code. The second, and the one that does the heavy lifting, is *trace-JIT compilation* of loops. The README’s status table describes the canonical case in detail: “a function’s numeric `while` loop (provably-Number `l`: locals → `Op::GetSlot/SetSlot`, loop rotated so the condition is the backedge) **trace-JIT-compile**s to native code.” Unpack the preconditions, because they explain why some loops are fast and others are not. The locals must be *provably Number* — the compiler’s `slot_plan` assigns VM slots only to locals it can prove hold Numbers, and those become `Op::GetSlot/SetSlot` accesses the JIT can compile; a name aliased to a dynamic scope (`g`, `s`, `a`;) stays dict-backed and the loop will not trace. This is why Chapter 4 made such a point of `l`: being the bare local scope: it is the precondition for tracing.

The fast path is broad. The README enumerates what stays trace-eligible: integer `+/-/*` lower to native `Op::Add/Sub/Mul`; integer comparisons to `Op::NumLt` and friends; float arithmetic to native `fadd`, so an integer counter and a float accumulator can share one trace; compound loop conditions (`&&/||`) of numeric compares, short-circuit trace; `if` inside loops and nested loops trace; native integer `%` lets modulo loops trace; native negation, native bitwise `and/or/xor/invert`, numeric ternaries, value-position comparisons (`let s += i > 5`), and logical-not of an integer all stay on the fast path. The idiomatic `for i in range(N)` lowers to a native integer counter with no list built and traces too, even when the bound is dynamic — `range(a:n)` hoists a `tv_get_number`-coerced bound once in the prologue so the body still traces. Numeric loops trace at both function scope and script (top-level) scope.

And there is a deopt fallback. When an operation is not provably numeric, the bytecode keeps it as a `CallBuiltin` — “the deopt fallback,” in the README’s words — which routes to the ported builtin. So a loop does not have to be *all* numeric to benefit; the numeric parts trace and the dynamic parts fall back, and the per-loop slot scoping means a hot loop can trace even when the same function also calls helpers (callees cannot see the `l`: locals) or runs a sibling list-`for`. Integer division, recall from Chapter 5, deliberately stays a `CallBuiltin` because fusevm’s native divide is float and VimL’s `/` is integer — a place where correctness pins an operation to the slow path on purpose.

Watching and disabling the JIT

The README documents two environment variables for observing and controlling the JIT, and both are real CLI controls. `VI MLRS_JIT_STATS=1 vimlrs script.vim` “reports loop traces compiled” — it tells you how many loops the JIT actually turned into native code, which is the direct way to confirm a loop you expected to trace did. `VI MLRS_NO_JIT=1` “forces the interpreter baseline,” disabling the JIT entirely so you can measure the difference. And the difference is the payoff the whole architecture exists for: the README reports that “a 20M-iteration loop runs **~15–100× faster** with the JIT” than the interpreter baseline. The `fib` example invites

you to see it directly — its header says “Run with `VIMLRS_JIT_STATS=1` to see the loop trace get compiled.” That a Vimscript loop compiles to native machine code at all is the strangest and best consequence of the lower-to-fusevm decision, and these two flags are how you watch it happen.

The deopt boundary: what does not trace

The fast path is broad, but it has a precise edge, and knowing where the edge is explains every loop that runs slower than you hoped. The compiler’s rule, from the slot planner of Chapter 21, is that a loop traces only when the whole body is free of anything that could reach a variable dynamically by name. So the boundary is a list of bail conditions, and the README’s status table and the compiler’s comments together enumerate it. A function or method *call* in the loop bails the slot plan, because the callee might read a global the planner cannot see — which is why a hot loop that calls a helper on every iteration stays on the builtin path, even though the arithmetic around the call would otherwise trace. `:execute` and `:set` bail, because they reach variables through computed strings. A nested `:function`, a `:try`, and certain `:let` targets bail. And any variable touched through a scope sigil — `g:`, `s:`, `a:`, `b:`, `w:`, `t:`, `v:` — stays dict-backed, because only the bare local (and its `l:` alias) gets a slot.

Three things hold a non-traced loop on the slow path even when the rest of it looks numeric. Integer division is the deliberate one: `/` is always a `CallBuiltin`, never a native op, because fusevm’s native divide is float and VimL’s integer `/` truncates — correctness pins it to the slow path, as Chapter 5 detailed. A value that is not provably a Number is the structural one: if the planner cannot prove a local always holds a Number, it stays unboxed in the dict and its reads and writes are builtin calls. And a string operation is the obvious one: concatenation is `VIML_CONCAT`, not a native op, because there is no native string-add. The disassembler (Chapter 22) makes the boundary visible — a loop body full of `CallBuiltin` where you expected `Op::Add` is a loop that bailed, and the bail condition is almost always one of the above.

The crucial design point is that the boundary is *graceful*, not a cliff. A loop does not have to be entirely numeric to benefit: the numeric parts trace and the dynamic parts fall back, because the deopt fallback is just the same `VIML_*` builtin the compiler would have emitted anyway. So a loop that mostly adds integers but occasionally calls a builtin gets the integer additions as native ops and the builtin as a call, and the per-loop slot scoping means the hot loop can still trace even when the same function runs a sibling list-`for` or calls helpers elsewhere. The fast path and the slow path are not two compilers; they are one compiler that emits native ops where it can prove they are safe and builtin calls everywhere else, with the proof computed conservatively so that a wrong guess never produces a wrong answer — only a slower one.

The fast path, enumerated

The README’s status table reads, in part, as a checklist of exactly which constructs stay on the fast path, and it is worth collecting because the breadth is the point. Integer `+/` lower to native `Op::Add/Sub/Mul`; integer comparisons to `Op::NumLt` and its siblings; native integer `%` (so `if i % 2 == 0` traces, the modulo idiom); native negation (`-x` to `Op::Negate`); and the native bitwise builtins `and/or/xor/invert` (so bit-manipulation loops trace). Float arithmetic lowers to native `fadd`, so an integer counter and a float accumulator can share one trace. Compound loop conditions — `&&` and `||` of numeric compares, short-circuited — trace; an `if` inside a loop traces; nested loops trace. A numeric ternary `cond ? a : b` is itself a Number, so `s += cond ? x : 0` loops trace. A value-position comparison `let s += i > 5` reifies the native compare to VimL’s `0/1` with a branch rather than a `CallBuiltin`, so counting loops trace. The logical-not of an integer `!x` becomes a native `x == 0` reified to `0/1`, staying trace-eligible. And the idiomatic `for i in range(N)` lowers to a native counter with no list, tracing even when the bound is dynamic. Numeric loops trace at both function scope and script (top-level) scope. The list is long because the project worked through construct after construct, checking each against the ported semantics and lowering it natively where it could — the fast path is broad not by luck but by accumulated effort, one trace-eligible construct at a time.

Watching it happen

The two environment variables make all of this observable, and they are real CLI controls worth keeping in your fingers. `VIMLRS_JIT_STATS=1 vimlrs script.vim` reports how many loop traces the JIT compiled during the run — the direct way to confirm that a loop you expected to trace actually did, and to catch one that silently bailed because of a slot-plan condition. `VIMLRS_NO_JIT=1 vimlrs script.vim` forces the interpreter baseline, disabling the JIT entirely, so you can measure the difference; the README reports a 20-million-iteration loop running “~15–100× faster” with the JIT than without. Pairing the two — run once with stats to confirm the trace, once with no JIT to measure the speedup — is the standard way to verify and quantify a hot loop, and the `fib.vim` example invites you to do exactly that, its header noting “Run with `VIMLRS_JIT_STATS=1` to see the loop trace get compiled.” That a Vimscript loop compiles to native machine code at all is the strangest and best consequence of lowering to fusevm, and these two flags are how you watch it happen on a real script.

Why this is the right shape

It would have been simpler to write a tree-walking interpreter over the AST. It would also have been slow, and it would have meant maintaining vimlrs’s own execution engine forever. By lowering to fusevm instead, vimlrs gets a mature bytecode VM, a three-tier JIT, persistent native-code caching, and AOT compilation — none of which it had to write, all of which improve whenever fusevm improves, and all of which are shared with `zshrs`, `stryke`, and `awkrs`. The cost is the bridge: the `VIML_*` builtin layer and the `typval_T` ↔ `fusevm::Value` conversion that lets ported semantics run on borrowed bytecode. That bridge is intricate — `fusevm_bridge.rs` is the largest source file in the crate — but it is the right place to spend complexity, because it is written once and buys an entire machine. The terseness of the slogan “no local VM, no local JIT” hides a real engineering judgment: own the semantics, borrow the machine.

It is worth being precise about what the bridge costs, because the cost is the honest counterweight to all the things lowering to fusevm buys. The bridge is the largest source file in the crate, and it has to exist because the two value worlds — ported `typval_T` and `fusevm::Value` — are different, so every operation that crosses between them pays a marshalling cost: a `VIML_*` handler pops fusevm values, converts them to `typval_T`, calls the port, and converts the result back. The refpool keeps that cost small for containers (a handle, not a deep copy) and the scalars cross by value, so the per-operation overhead is a few conversions, not a structural copy — but it is not zero, and a workload dominated by builtin calls pays it on every call. The compensating design is the numeric fast path: the operations that would pay the most if they went through the bridge — the arithmetic and comparisons in a hot loop — are exactly the ones the compiler lowers to native ops that bypass the bridge entirely, so the loop runs without crossing the boundary at all. So the bridge cost falls on the dynamic operations, where it is acceptable because they are not the hot path, and the hot path avoids the bridge by going native. This is the right place to spend the complexity: the bridge is written once, intricate but contained, and it buys an entire mature machine — the VM, the three-tier JIT, the disk cache, the AOT path — none of which vimlrs had to write. Own the semantics, borrow the machine, and pay the marshalling cost only where it does not matter: that is the engineering judgment the slogan compresses, and the bridge is where it is paid.

Chapter 15 – The Script Cache, AOT, and Deployment

A scripting language is invoked, not launched, and it pays its compilation cost on every invocation unless something prevents it. `vimlrs` has two mechanisms that prevent it — a bytecode script cache that skips recompilation on the second run, and an ahead-of-time builder that bakes scripts into a self-contained executable — and together they shape how a `vimlrs` program is deployed. This chapter covers both, plus the versioning discipline that keeps a cached script from ever breaking.

The rkyv script cache

On the first run of a script, `vimlrs` lexes, parses, and compiles it. On the second and later runs, it should not have to. The script cache, `script_cache.rs`, makes the second run skip all of that: a cache hit is “`mmap` + zero-copy `ArchivedHashMap` lookup + a `bincode`-decode of the inner `fusevm::Chunk` blob.” The cache is a single `rkyv`-archived file at `~/.cache/vimlrs/scripts.rkyv`, mapping each script's canonical path to a compiled entry. The same architecture backs `awkrs`, `strykelang`, and `zshrs`; it is another piece of shared design.

The read path is built for speed and safety in equal measure. It lazily `mmaps` the shard, validates the archived root with `rkyv::check_archived_root`, checks the header's magic (“`VIML`”, the constant `SHARD_MAGIC`), format version, pointer width, and the writing binary's version, and then — per entry — guards on the source file's mtime and the `vimlrs` binary's mtime. Any rebuild of the binary or edit of the source invalidates the entry silently, so a stale cache never serves wrong bytecode. The write path takes an exclusive `flock`, mutates, serializes, `fsyncs` to a temporary file, and atomically renames it into place, so concurrent writers serialize and a reader never sees a half-written shard. This is the same atomic-rename durability trick filesystems have used forever: a reader sees either the whole old shard or the whole new one.

The header carries a `format_version` (currently 3), and the cache is “versioned from day one” with a stated compat-floor: “never break a cached `.vim` that worked.” When the schema or the meaning of emitted builtin ids changes, the version bumps and old entries are simply re-compiled rather than mis-read. The engineering report frames this as a longevity guarantee — “an old cached `.vim` upgrades or rebuilds silently rather than breaking.” You can wipe the cache with `vimlrs --clear-cache` if you ever want a clean slate, but you should never *need* to, which is the whole point of versioning it from the start. The dependencies that make this work — `rkyv` for the zero-copy archive, `memmap2` for the `mmap`, `bincode` for the inner chunk blob, `nix` for the flock, `dirs` for the cache directory — are all listed in the manifest with exactly these roles.

What the cache header guards

The cache's safety rests on a header that is checked on every read, and the set of things it guards is worth enumerating because each one is a way a stale cache could otherwise serve wrong bytecode. The shard begins with a magic number — the constant `SHARD_MAGIC`, the bytes “`VIML`” — so a file that is not a `vimlrs` cache is rejected outright. After the magic comes a `format_version` (currently 3), bumped whenever the schema or the meaning of emitted builtin ids changes, so that a cache written by an older binary with a different bytecode layout is recognized as incompatible and re-compiled rather than mis-read. The header also records the pointer width and the writing binary's own version. Then, per entry, the cache guards on

two mtimes: the source file's modification time and the vimlrs binary's modification time. Either one changing invalidates the entry — edit the `.vim` file and its cached bytecode is stale; rebuild vimlrs and every entry is stale, because a new binary might lower or JIT differently. The read path performs all of these checks after `mmap`-ing the shard and validating the archived root with `rkyv::check_archived_root`, so a corrupt, truncated, or version-mismatched shard fails the validation and the script is simply re-compiled. The result is that a cache hit is only ever taken when it is provably safe — same binary, same source, same format — and a miss costs only a recompile, never a wrong answer.

The write path and atomicity

The write path is built so that concurrent runs of vimlrs cannot corrupt the shard. A writer takes an exclusive `flock` on the cache, mutates the in-memory map, serializes it, writes it to a temporary file, `fsyncs`, and atomically renames the temporary over the real shard. The `flock` serializes concurrent writers — two vimlrs processes compiling the same script will not interleave their writes — and the write-temp-then- rename discipline means a reader either sees the entire old shard or the entire new one, never a half-written file, because rename is atomic on POSIX filesystems. This is the same durability trick the rkyv key-value store and countless databases use, and it is what lets the cache be safe to share across the many short-lived processes a scripting language spawns. The dependencies that implement it are named in the manifest with exactly these roles: `rkyv` for the zero-copy archive, `memmap2` for the `mmap`, `bincode` for the inner chunk blob, `nix` for the `flock`, and `dirs` for locating `~/.cache/vimlrs/`. A scripting language does not usually think this hard about its compilation cache; vimlrs does, because the whole point of caching is to be invisible, and an invisible cache is one that is never wrong and never the cause of a corrupted run.

AOT: baking scripts into a binary

The cache speeds up repeated runs of a script that lives on disk. The ahead-of-time builder, `aot.rs`, does something different: it bakes one or more scripts *into a copy of the vimlrs binary*, producing a self-contained executable that carries its program inside it. You invoke it with `vimlrs --build OUT file.vim ...`, and the result is a binary that, when run, detects the embedded scripts and runs them in order as one program.

The mechanism is a trailer appended to a copy of the vimlrs binary. The original executable bytes are unchanged — the copy “still runs as `vimlrs`” — followed by a zstd-compressed payload and a fixed 32-byte trailer ending in the 8-byte magic `VIMLRAOT`. The payload (format version 2) is an ordered file list: a count, then per file a name and its source. At startup vimlrs checks for the trailer; if present, it decompresses the payload and runs every embedded script in build order. This is the same design as `zshrs`'s `aot.rs`, and like it, it has no C counterpart — Vim's `.vim` scripts have no compile-into-binary form, so this is pure standalone-interpreter capability.

There is a second, stronger build mode. `vimlrs --build --native` AOT-compiles to *native machine code* — a Cranelift object linked standalone — rather than embedding the source. This is why the crate is built as both a `lib` and a `staticlib`: the `staticlib` produces `libvimlrs.a` so that `--build --native` can link AOT-compiled objects (via `fusevm::aot`) against the VimL runtime into a single executable. The `aot` feature on the `fusevm` dependency is what enables this path. The source-embedding build is the portable default; the native build is the option for when you want the program compiled all the way down.

The two build modes trade portability against ahead-of-time compilation, and the distinction is worth drawing because they produce genuinely different artifacts. The default `--build` embeds the *source* as a compressed trailer on a copy of the vimlrs binary, so the result still contains the full interpreter and lexes, parses, and compiles its embedded scripts at startup (though the script cache means the compile happens once). It is portable in the sense that it carries the same interpreter as any other vimlrs, just with its program baked in. The `--build --native` mode goes further: it AOT-compiles the scripts to native machine code through `fusevm::aot`, emitting a Cranelift object that is linked against the VimL runtime — which is why the crate builds a `staticlib` producing `libvimlrs.a`, so the AOT-compiled objects have a runtime to link against. The native build skips even the bytecode interpreter for the compiled paths, running native code directly, at

the cost of being tied to one target architecture. Both inherit the trailer-and-magic mechanism for detecting the embedded program, and both produce a single self-contained executable with no external interpreter to install. The choice mirrors the choice any compiled language offers — portable bytecode versus native code — except that vimlrs offers it for *Vimscript*, a language that until now had no compiled form at all, no way to be anything but interpreted text sourced inside an editor. That a `.vim` file can be compiled to a native object and linked into a standalone binary is among the strangest consequences of the whole project, and it falls out of having lowered to a target that already had an AOT path.

Two caches, two layers

It is worth being precise that there are *two* caches working together, because they sit at different layers and persist different things. The rkyv script cache above persists *bytecode*: on a warm run it skips lexing, parsing, and compiling, handing fusevm a ready `Chunk`. But fusevm itself has a second cache, enabled by the `jit-disk-cache` feature the manifest turns on, which persists *native code* — the machine code the Cranelift JIT produced for hot loops. The two stack: on a warm start of a script you have run before, the script cache supplies the bytecode without recompiling it, and the JIT disk cache supplies the native code for the loops that previously got hot, without re-JITting them. The first run pays for lexing, parsing, lowering, and JIT compilation; an identical later run pays for none of it.

This is a benefit vimlrs gets entirely for free from lowering to fusevm. The script cache is vimlrs's own code (`script_cache.rs`), but the JIT disk cache is fusevm's, shared with `zshrs`, `stryke`, and `awkrs` — vimlrs just turns the feature on. It is the clearest single illustration of Chapter 1's argument for the shared machine: a capability as sophisticated as cross-run native-code persistence, which vimlrs would never have written for itself, arrives because the engine it borrowed already had it. A scripting language whose loops compile to native machine code *and* cache that machine code across invocations is a strange thing for Vimscript to have become, and vimlrs has it without owning a single line of the JIT or its cache.

What deployment looks like

Put the two mechanisms together and vimlrs offers a deployment story unusual for a scripting language. For development and repeated local runs, the script cache means you pay compilation once and warm runs are fast — and because fusevm's `jit-disk-cache` also persists compiled native code across runs, a warm start skips both recompilation and re-JITting. For shipping, the AOT builder turns a collection of `.vim` files into one executable with no external interpreter to install: you build it once and copy the single binary to wherever it runs. A language that began life trapped inside an editor can now be cached like a compiled program and shipped like a static binary. That is a long way from `vimrc`, and it is the concrete sense in which vimlrs makes Vimscript an ordinary program: not just runnable outside Vim, but cacheable, buildable, and deployable the way any other compiled language is. Consider how strange that sentence would have read a few years ago. Vimscript was, by definition, the thing you could not deploy — it lived in your `vimrc`, it ran because Vim ran, and shipping it meant shipping Vim and a configuration. Now the same language has a compilation cache like a compiled program, a build command that produces a standalone executable, and an AOT mode that compiles it to native code — the full deployment vocabulary of a real language. The mechanism behind each is something vimlrs borrowed rather than built: the cache is rkyv and mmap, the build is a trailer-and-magic scheme, the native compile is `fusevm::aot` and Cranelift. But the *capability* is vimlrs's, and the capability is what changes Vimscript's relationship to the world: a `.vim` file can now be a deliverable artifact, not just editor configuration. That is the concrete end state of taking the language out of its host — not merely running it standalone, which Chapter 2 showed, but giving it the whole lifecycle a program has, from a fast cached development loop to a single-binary deployment.

A development loop and a deployment story

It is worth separating the two things the cache and the AOT builder give a vimlrs program, because they serve different phases of a program's life and together they cover the whole arc. The first is a fast *development loop*. While you are writing and running a script repeatedly — the edit-run-edit cycle of development — the script cache means you pay the compilation cost once and every subsequent run restores the compiled bytecode from disk, and the JIT disk cache means the native code for hot loops persists too, so a warm run skips both recompilation and re-JITting. The practical effect is that after the first run, repeated runs of a script start fast and stay fast, which matters because a scripting language's development loop is dominated by startup-and-compile cost, and eliminating the recompile on every iteration makes the loop tight. This is the phase Chapter 2's REPL and file modes serve, accelerated by the cache so that the iteration cost is the program's own work, not the compiler's.

The second is a *deployment story*, for when the script is done being developed and needs to run somewhere else. Here the AOT builder turns the script into a single self-contained executable — either by embedding the source as a compressed trailer on a copy of the vimlrs binary, or, with `--native`, by compiling it all the way to native machine code linked against the runtime. Either way, the artifact you ship is one binary with no external interpreter to install, which is exactly the deployment model a compiled language offers: copy one file to the target, run it. This is a sharp contrast with how Vimscript was “deployed” before — which is to say, it was not deployable at all, because shipping a Vimscript program meant shipping Vim and a configuration and asking the recipient to source it inside their editor. A standalone binary that runs a `.vim` program with nothing else installed is a different category of thing. Put the two phases together and vimlrs gives a Vimscript program the same lifecycle a Python or a Go program has: a fast cached development loop while you write it, and a single-binary deployment when you ship it. That a language born as editor configuration now has both ends of that lifecycle is the concrete payoff of lowering to a machine that already had a cache and an AOT path — vimlrs wrote neither the rkyv cache's zero-copy machinery nor Cranelift's object emitter, and it has the development loop and the deployment story that those mechanisms provide, because it lowered to the engine that owns them.

Chapter 16 – JSON, MessagePack, and the Outside World

A standalone interpreter has to talk to things that are not itself: configuration files, other programs, the process environment, serialization formats. Vimscript grew a respectable set of interop builtins inside Neovim — `json_encode`, `msgpackdump`, `system`, `environ` — and because these are “pure data transforms with no editor dependency,” as the msgpack example notes, they port cleanly and work identically standalone. This chapter covers the serialization codecs and the OS-interaction builtins, which together are how a vimlrs program exchanges data with the world.

The phrase “pure data transforms” is the key to why this whole family ports so cleanly, and it is worth making explicit. A function like `json_encode` takes a value and returns a string; `msgpackdump` takes a list and returns bytes; `system` takes a command and returns its output; `strftime` takes a time and returns a formatted string. None reads or writes editor state — no buffer, no window, no cursor, no event loop. They are functions in the mathematical sense: input to output, with the only side effect being, for `system`, an external process the OS provides standalone just as it does inside an editor. So porting them is porting pure logic, with no editor scaffolding to transplant, which is why they “work identically standalone” — there is nothing about them that *depends* on the editor. This is the inverse of the stub functions: a stub like `winheight` is a pure *query of editor state*, so with no editor there is nothing to query and it returns the empty value, while `json_encode` is a pure *transform of a value*, so with or without an editor it does exactly the same thing. The interop family is, in a sense, the easiest part of the port — no editor coupling to break — which is why it reaches the strictest fidelity (the byte-exact MessagePack) and why these are among the functions a standalone script can most rely on. The lesson generalizes: the more a function is a pure transform of values, the more cleanly it ports and the more faithfully it behaves standalone, and the interop family is the purest such transforms in the whole surface.

JSON

`json_encode(value)` serializes a Vimscript value to a JSON string and `json_decode(string)` parses one back. The encode side is a faithful port (`encode.rs`’s `encode_tv2json`); the decode side is a synthesis-zone recursive-descent parser (`viml_json.rs`) that yields “the identical `typval_T` tree” the ported `decode.c` would — another case where the mechanism is new but the result matches Neovim. The `json.vim` example round-trips a nested structure and asserts losslessness:

```
let config = {
  \ 'name': 'vimlrs',
  \ 'version': [0, 1, 0],
  \ 'features': ['jit', 'lsp', 'dap'],
  \ 'standalone': v:true,
  \ }
let encoded = json_encode(config)
let decoded = json_decode(encoded)
let expected = '{"name":"vimlrs","version":[0,1,0],\'
  \ . '"features":["jit","lsp","dap"],"standalone":true}'
call assert_equal(expected, encoded)
call assert_equal(config, decoded)
```

Two faithful details earn their keep here. The Bool `v:true` encodes to JSON `true` and decodes back to a Bool, not to 1 — the type survives the round trip. And the insertion-ordered dict (Chapter 5) is what makes the

encoded string *exactly* predictable, key order and all, so the example can assert the precise output rather than parsing it back to compare. The decoder is strict the way Vim's is: trailing non-whitespace after a valid document is malformed and yields `None`. Float rendering matches `string()`'s `%g`, so `json_encode({'x': 0.5})` is `'{"x":0.5}'`. JSON is the lingua franca for configuration and inter-process data, and vimlrs's implementation is faithful enough to interoperate with anything that speaks it. The round-trip property is the practical payoff: `json_decode(json_encode(x))` reconstructs `x`, so a script can read a JSON config file, manipulate it as ordinary Vim values, and write it back without losing structure — the Bool stays a Bool, the nested lists and dicts keep their shape, the numbers keep their type. This is what makes vimlrs a usable tool for configuration-processing tasks: read a `package.json`-style file with `json_decode(join(readfile(path), "\n"))`, transform the resulting dict, and write it with `writefile(split(json_encode(d), "\n"), path)` — a complete JSON-editing pipeline in three functions, all real, all faithful. That a language born inside a text editor can now act as a standalone JSON processor is one more concrete instance of the standalone premise paying off: the interop builtins that Neovim grew for talking to language servers and external tools become, outside the editor, exactly the toolkit a shell-adjacent script needs to exchange structured data with the world.

The decoder's strictness and float rendering

Two details of the JSON codec are worth drawing out because they are where a casual implementation diverges from Vim. The first is strictness: the decoder is strict the way Vim's is, so “trailing non-whitespace after a valid document is malformed and yields `None`.” A document like `{"a":1}garbage` does not parse the leading object and ignore the rest — it fails, because the whole input must be one well-formed JSON value (plus optional surrounding whitespace). This matters for interoperation: a script that relies on `json_decode` rejecting malformed input gets that rejection, rather than a permissive parse that silently accepts trailing junk. The second detail is float rendering: `json_encode` renders floats through the same `%g` six-significant-digit path that `string()` uses, so `json_encode({'x': 0.5})` is `'{"x":0.5}'` and a float that prints cleanly under `%g` prints cleanly in the JSON. Because the dict is insertion-ordered (Chapter 5), the encoded string's key order is deterministic, which is what lets `json.vim` assert the *exact* encoded string rather than parsing it back to compare — the encoder is a pure function of the value, key order and all, so its output is predictable byte for byte.

The synthesis decoder, the ported encoder

The JSON codec is a good miniature of the two-zone architecture, because its two halves live on opposite sides of the boundary. The *encode* side is a faithful port: `encode.rs`'s `encode_tv2json` mirrors Neovim's `encode.c`, walking the `typval_T` tree and emitting JSON text. The *decode* side is a synthesis-zone module: `viml_json.rs` is a fresh recursive-descent parser, because Neovim's `decode.c` is tangled into its own value-construction machinery in a way that did not transplant cleanly. But the decoder's contract is to produce “the identical `typval_T` tree” the ported `decode.c` would, so the *result* is faithful even though the *mechanism* is new — the same divergence-by-result the regex engine makes (Chapter 10). This is why Chapter 13 named JSON as one of its clearest cases of principled divergence: the encoder is ported and cited, the decoder is invented but produces a tree indistinguishable from the ported decoder's, and the round-trip `json.vim` asserts — `json_decode(json_encode(x)) == x` — holds because both halves agree on the tree shape regardless of which side was ported and which was synthesized. The boundary is legible even inside a single feature: one function carries a `// c:` citation, the other a `NO csrc/ COUNTERPART` banner, and the example proves they meet in the middle.

MessagePack

`msgpackdump` and `msgpackparse` are the binary serialization pair, and they are the most byte-exact part of vimlrs's interop story. The `msgpack.vim` example states the standard: “The byte output is identical to Neovim's

(msgpack-c minimal-width packing).” The Blob form is byte-exact, and the example asserts against known MessagePack encodings:

```
call assert_equal(0x80, msgpackdump([{}], 'B'))
call assert_equal(0x93010203, msgpackdump([[1, 2, 3]], 'B'))
call assert_equal(0x81A16101, msgpackdump([{'a': 1}], 'B'))
call assert_equal(0xC0C3C2, msgpackdump([v:null, v:true, v:false], 'B'))
call assert_equal(0x7F, msgpackdump([127], 'B'))
call assert_equal(0xCD0100, msgpackdump([256], 'B'))
```

The encoding follows MessagePack’s minimal-width rule precisely — 127 packs as a single positive fixint byte `0x7F`, 256 as a uint16 `0xCD0100`, -1 as a negative fixint `0xFF` — and a Float always packs as float64. `msgpackparse` inverts it for the self-describing subset, and a full round-trip through the Blob form preserves numbers and containers exactly. The example even documents the lossy edges honestly: a String dumps as a MessagePack BIN and parses back as a Blob, and a Funcref cannot be dumped at all (`E5004`). This is the most demanding kind of fidelity — matching another implementation byte for byte — and the fact that vimlrs hits it is a strong signal about the depth of the port.

The minimal-width rule is the heart of the byte-exactness, and it is worth understanding because it is where most reimplementations would diverge. MessagePack offers several encodings for an integer — a positive fixint, a uint8, a uint16, a uint32, a uint64 — and the spec leaves room to choose, but Neovim always picks the *smallest* that fits, and vimlrs matches that choice exactly. So 127 packs as a single positive-fixint byte `0x7F`, 256 as a uint16 `0xCD0100` (because it overflows a uint8), -1 as a negative-fixint byte `0xFF`, and a Float always as a float64 `0xCB...`. The example asserts each of these against the known encoding, so a regression that picked a wider encoding — packing 127 as a uint8, say — would fail CI immediately. The example is also honest about the lossy edges, which a careful port documents rather than hides: a String dumps as a MessagePack BIN and parses back as a Blob (because MessagePack’s string/binary distinction does not map cleanly onto Vim’s), and a Funcref cannot be dumped at all, raising `E5004`. These are not bugs but documented consequences of the format’s type system, and the example pins them so they are part of the contract. Matching another implementation’s exact byte output, minimal-width rule and all, is the strictest fidelity bar in the whole interop story, and it is the one MessagePack sets — which is why it is the clearest single proof of how deep the encode port reaches.

Two kinds of fidelity, weighed

Reading the JSON and MessagePack examples back to back teaches a distinction that runs deeper than serialization, and it is worth drawing out because it clarifies what “faithful” means when the target is interoperability rather than Vim itself. JSON and MessagePack ask for two different *kinds* of fidelity, and vimlrs clears both. JSON asks for *interoperable* fidelity: the output must be valid JSON that any JSON parser accepts and that round-trips back to an equal value, but there is latitude in the exact bytes — whitespace, key order in principle, the precise rendering of a float. Meeting interoperable fidelity means producing *correct* JSON, and the `json.vim` example can assert the exact encoded string only because vimlrs’s insertion-ordered dict makes key order deterministic, which is a stronger property than JSON requires but a convenient one to have. MessagePack asks for *byte-exact* fidelity: the output must match Neovim’s output byte for byte, because the whole point of the example is to prove vimlrs encodes identically to the reference, and there is no latitude — 127 must be the single byte `0x7F`, 256 must be `0xCD0100`, the minimal-width rule must be followed exactly. Meeting byte-exact fidelity is strictly harder than meeting interoperable fidelity, because it pins not just correctness but the exact encoding choice among several correct ones.

The reason both matter, and the reason the project pursues the harder one where it can, is that byte-exactness is the most *falsifiable* form of fidelity available, and falsifiability is the project’s whole methodology. A claim that vimlrs “encodes JSON correctly” is true but loose — there are many correct encodings, and “correct” is a property you argue for. A claim that vimlrs encodes a given value to the exact bytes `0x81A16101` is sharp — it either does or it does not, and a single assertion settles it. So MessagePack, by demanding byte-exactness, gives the project a fidelity target that can be checked with an equality assertion rather than argued about, and the fact that vimlrs hits it across the full range of types — fixints, uint16s, negative fixints, float64s, maps, arrays,

nil, true, false — is a stronger statement about the encode port than any number of “correct JSON” assertions could be. This is the same preference for falsifiable evidence that runs through the whole book: exact numbers over approximate behaviour, exact error codes over “it errors,” exact bytes over “it encodes correctly.” The two serialization formats together let the project demonstrate fidelity at both levels — interoperable where that is what the format demands, byte-exact where the format allows it — and the byte-exact MessagePack is the sharper of the two precisely because it leaves nowhere for an approximation to hide.

Shelling out

`system(cmd [, input])` runs an external command, captures its stdout, and returns it as a string; `systemlist` returns the output split into lines. After either, the special variable `v:shell_error` holds the exit status. The `system.vim` example exercises the whole interface using only POSIX tools so it runs the same on Linux and macOS:

```
let greeting = system('echo hello')
let lines = systemlist('printf "one\ntwo\nthree\n"')
let shout = system('tr a-z A-Z', 'quiet')
call system('exit 7')
let code = v:shell_error
call assert_equal("hello\n", greeting)
call assert_equal(['one', 'two', 'three'], lines)
call assert_equal('QUIET', shout)
call assert_equal(7, code)
```

`system('tr a-z A-Z', 'quiet')` passes 'quiet' on the command's stdin and gets back 'QUIET' — the second argument is input, fed to the process. `call system('exit 7')` runs a command for its exit status alone, which `v:shell_error` then reports as 7. This is the standard way a Vimscript program drives other tools, and outside an editor it is arguably *more* useful than inside one, because a standalone vimlrs script can take its place in a shell pipeline. The related `systemlist`, `shellescape` (quote an argument safely), and the language-eval bridges (`luaeval`, `py3eval`, and friends, many of which are standalone stubs) round out the System & Environment chapter of the reference, 44 functions in all.

Time, hashing, and encoding

Beyond serialization, a handful of pure-data builtins do real work standalone because they touch no editor. Time is one. `localtime()` returns the seconds since the Unix epoch, `strftime({format} [, {time}])` formats a time with a `strftime` format string, and `strptime({format}, {timestring})` parses one back to seconds. The high-resolution timer trio — `reltime()`, `reltimefloat()`, and `reltimestr()` — measures elapsed intervals: `reltime()` returns “a high-resolution time value, or an elapsed interval,” `reltimefloat` renders it “as a Float number of seconds,” and `reltimestr` as a `'seconds.microseconds'` string. These are ported from Neovim's `profile.c` (the `proftime_T` helpers) and `os/time.c`, so a standalone script can time itself exactly as a plugin would. Hashing is another: `sha256({string})` is a real “SHA-256 hex digest,” ported from `sha256.c` as a FIPS-180-2 implementation, not a stub — which is why Chapter 13 could cite it as one of the pure functions ported early. And `rand/srand` produce pseudo-random Numbers from a seed list, using the same xoshiro128** generator Neovim uses, so the output is bit-exact against the reference.

Encoding conversion is reachable through `iconv({string}, {from}, {to})`, which `sysinfo.vim` shows behaving as identity for same-or-UTF-8 encodings:

```
call assert_equal('hello', iconv('hello', 'utf-8', 'utf-8'))
call assert_equal('héllö', iconv('héllo', 'latin1', 'latin1'))
```

Together with `system/systemlist` for shelling out, `environ` for the environment, the `json_*` and `msgpack_*` codecs, and `sha256`, these time and encoding functions are the part of the System & Environment family that is real rather than stubbed — the 44-entry chapter of the reference is a mix, and these are the working half.

They are exactly the builtins a standalone text-processing script reaches for: stamp a log line with `strftime`, time a loop with `reltime`, hash a payload with `sha256`, all with no editor anywhere in sight.

The environment

The process environment is reachable three ways, and `env_vars.vim` shows them agree. The `$NAME` sigil reads an environment variable as a string; `:let $NAME = val` sets it for the process; `getenv()/setenv()` are the functional forms; and `expand('$NAME')` performs the same lookup, including the `${NAME}` brace form:

```
let $VIMLRS_DEMO = 'hello'
call assert_equal('hello', getenv('VIMLRS_DEMO'))
call assert_equal('hello', expand('$VIMLRS_DEMO'))
call setenv('VIMLRS_DEMO', v:null)
call assert_equal(v:null, getenv('VIMLRS_DEMO'))
```

One faithful distinction matters: an *unset* variable read through `getenv()` returns `v:null`, but read through the `$X` sigil it is the empty string `''`. The two surfaces disagree on purpose — `getenv()` can tell “unset” from “set to empty,” while `$X` flattens both to `''` — and vimlrs reproduces the difference. `setenv(name, v:null)` removes a variable entirely. The `environ()` builtin returns the whole environment as a Dict, which `system.vim` uses to check that `PATH` is present. Between `$`, `getenv/setenv`, `expand`, and `environ`, a vimlrs program has the same environment access any shell-adjacent tool needs, and it behaves exactly as it would inside Neovim.

The four environment surfaces are not redundant; each exists because it does something the others cannot. The `$NAME` sigil is the terse read, returning the empty string for an unset variable — convenient but unable to distinguish unset from empty. `getenv()` is the precise read, returning `v:null` for unset so a script *can* tell the difference, and `setenv(name, val)` the precise write, with `v:null` removing a variable entirely. `expand('$NAME')` performs the same lookup as part of its larger job of expanding wildcards and special tokens in a string, including the `${NAME}` brace form, which is why `expandcmd('cat $HOME/.vimrc')` can expand a variable embedded in a command. And `environ()` returns the whole environment at once as a Dict, the form `system.vim` uses to check that `PATH` is present. The `env_vars.vim` example pins the agreement and the one disagreement: a variable set via `:let $X` reads back identically through all three read surfaces, but an *unset* variable reads as `v:null` through `getenv()` and as `''` through `$X`, and vimlrs reproduces both. This is the kind of detail a from-scratch implementation would smooth over — most would make all four surfaces agree on `''` for unset — and vimlrs keeps the distinction because the ported `getenv/setenv` keep it, which is what lets an environment-sensitive script distinguish “the variable is not set” from “the variable is set to empty,” a difference that genuinely matters in shell-adjacent code.

Chapter 17 — Tooling: LSP and DAP

A language that wants to be used outside its original host needs the things every modern language has: editor integration and a debugger. vimlrs ships both as built-in modes of the same binary — `vimlrs --lsp` is a Language Server Protocol server and `vimlrs --dap` is a Debug Adapter Protocol server — and both are listed as *Implemented* in the engineering report. Neither has a Neovim C counterpart (they are synthesis-zone modules), but both reuse the same lexer, parser, and builtin corpus the interpreter does, so the editor sees exactly the language vimlrs runs. This chapter covers what each provides.

The language server

`vimlrs --lsp` serves the Language Server Protocol on stdio. Its module (`lsp.rs`) is “self-contained, reusing the synthesis lexer/parser,” and it advertises four capabilities: diagnostics, completion, hover, and document symbols. The README and report agree on the set; here is what each draws on.

Diagnostics come from parsing. The server runs `parse_stmt` per line and reports parse errors as LSP diagnostics, so a syntax error surfaces in the editor as you type. Because it is the *same* parser the interpreter uses, a line that the editor flags is a line that would fail to compile — there is no second, looser grammar that disagrees with the real one.

Completion and hover draw on the builtin corpus. The crucial thing here is the single source of truth: `builtin_docs.rs` holds the documentation, and both the LSP server *and* the published `reference.html` are generated from it. The reference page says so on its build line — “generated from `vimlrs/builtin_docs.rs`” — and describes itself as “the exact corpus that `vimlrs --lsp` serves.” So the hover text you get in your editor, the completion descriptions, and the reference manual are literally the same strings. When you hover a builtin, the server formats its entry as a fenced `vim` signature block followed by the one-line description, via `hover_markdown`. Completion offers the builtins, the ex-command words, and the predefined `v:` constants. The single-source discipline has a maintenance payoff worth naming: when a builtin is ported, its entry is added to `builtin_docs.rs` once, and that one addition makes it appear in the LSP hover, the LSP completion, and the published reference simultaneously. There is no second place to update, no documentation that can lag the implementation, because the documentation *is* the corpus the implementation’s tooling reads. This is the same single-source-of-truth pattern the project uses elsewhere — the regex engine as one source for `=~` and `matchstr` and `substitute`, the buffer as one source for `getline` and `:%s` and `:normal` — applied to documentation. The result is that the reference manual, the editor’s autocomplete, and the interpreter’s actual builtin surface are guaranteed to agree, not by discipline but by construction: they all read one slice. A reader who finds a builtin in the reference can trust it is implemented (or honestly labelled a stub), and a builtin the editor completes is one the interpreter can run — the documentation cannot drift, because there is only one of it.

The corpus is real and sized: the reference reports 463 entries across 11 chapters — 439 builtin functions grouped into eight categories (String manipulation, List, Dictionary, Variables & Type, Number & Float, Files/Buffers/Lines, System & Environment, and a large Functional & Misc catch-all), plus 6 ex-commands, 15 options, and the 3 documented `v:` constants.

Document symbols come from the parse tree: the server walks the parsed statements and reports the functions a file defines, so the editor’s outline view and symbol search work. Nothing reaches the terminal — the server speaks JSON-RPC on stdio only — and its structure, the module notes, is “ported from awkrs’s `lsp.rs`,” another instance of the shared-design pattern that runs through the whole project.

How hover and completion are formatted

The hover and completion features draw on `builtin_docs.rs`, and the exact formatting is worth seeing because it is the same in the editor and the published reference. When you hover a builtin, the server's `hover_markdown` formatter looks up the `(name, signature, doc)` tuple and renders it as a fenced `vim` code block containing the signature, followed by the one-line description as prose — the layout the `reference.html` mirrors per entry. So hovering `printf` shows the signature `printf({fmt} [, {expr1} ...])` in a code block and then “Formatted string from printf-style {fmt} and arguments,” exactly the strings the reference renders. The completion provider offers three kinds of item: the builtin function names (each with its doc as detail), the ex-command words the parser recognizes, and the predefined `v:` constants. Because all three are computed from the same corpus the interpreter implements, a completion the editor offers is always a real callable — there is no list of aspirational functions that do not exist, only the ones the ported evaluator actually has. The single-source discipline is what makes the tooling trustworthy: the completer cannot offer a function the interpreter lacks, and the hover cannot describe a behaviour the reference contradicts, because all of them read the one slice.

Diagnostics and document symbols, from the real parser

The diagnostics and document-symbol features draw on the parser rather than the corpus, and the key fact is that it is the *same* parser the interpreter uses. The server runs `parse_stmt` on each line and turns any parse error into an LSP diagnostic with the error's position, so a syntax error surfaces in the editor as a red squiggle as you type. Because there is no second, looser grammar — the LSP and the interpreter share `viml_parser.rs` — a line the editor flags is a line that would genuinely fail to compile, and a line the editor accepts is one the interpreter would accept. Document symbols come from walking the parsed statements and reporting the `Function` definitions a file contains, so the editor's outline view and symbol search list the functions a `.vim` file defines, with their positions. The whole server is “self-contained, reusing the synthesis lexer/parser,” and its structure is “ported from awkrs's `lsp.rs`” — another instance of the shared-design pattern, where the language server for one fusevm-hosted language is the template for the next. The server speaks JSON-RPC on stdio only; nothing reaches the terminal, which is what lets it run as an editor's language-server backend without corrupting the protocol channel.

The debugger

`vimlrs --dap` serves the Debug Adapter Protocol, and its design (`dap.rs`) is the more interesting of the two because of how it runs your program. It “mirrors zshrs's `dap.rs` architecture”: the script runs *in-process* on a dedicated executor thread, and the compiler emits a `SET_LINENO` marker before each statement whose handler calls `check_line`, which consults shared breakpoint state and `condvar`-waits to pause. The README lists the user-facing capabilities: breakpoints, stepping, variables, and evaluate.

The in-process design has a subtle payoff for the two introspection features. While paused at a breakpoint, the executor thread does not just sit idle — it services `variables` and `evaluate` requests itself, so that they read the executor's *own* `globvardict`. The `Pending` enum carries exactly these two request kinds. This means when you inspect a variable or evaluate an expression in the debugger, you are reading and running against the genuine, live state of the paused program, not a copy or a re-evaluation in a fresh context. Stepping is the `Next` resume mode (stop again at the next statement) versus `Continue` (run to the next breakpoint). Program `:echo` output is captured and streamed as DAP `output` events at each pause and at termination, so it never corrupts the JSON-RPC channel on stdout.

One binary, many modes

It is worth stepping back to see that the LSP and DAP servers are not separate programs but *modes of the same binary*, and that this is a deliberate and load-bearing design. `vimlrs` with a file runs a script; `vimlrs --lsp` is a language server; `vimlrs --dap` is a debug adapter; `vimlrs --disasm` prints the bytecode; `vimlrs --build` bakes an executable; `vimlrs --clear-cache` wipes the cache. One binary, one lexer, one parser, one compiler, one builtin corpus — six faces. The payoff is consistency that comes for free. Because the LSP's diagnostics run the *same* `parse_stmt` the interpreter runs, a line the editor flags red is a line that would actually fail to compile; there is no second, looser grammar that disagrees with the real one. Because the DAP runs the *same* compiled bytecode the normal run does — on a dedicated executor thread, with `SET_LINENO` markers the compiler emits — a breakpoint pauses the genuine program, and an inspected variable reads the executor's own live state, not a re-evaluation in a fresh context. Because hover, completion, and the published reference all draw on the *same* `builtin_docs.rs` corpus, the documentation can never drift from what the interpreter implements.

This single-binary shape is the same one Chapter 1 admired in the engine: just as `vimlrs` gets the JIT, the disk cache, and AOT by sharing `fusevm` rather than reimplementing them, the editor tooling gets its correctness by sharing the frontend rather than reimplementing it. A language server is only as trustworthy as its agreement with the compiler, and a debugger only as trustworthy as its agreement with the runtime; `vimlrs` guarantees both by making them literally the same code, reached through a different flag. For a language whose entire prior existence was as one application's configuration, becoming a single binary that is at once interpreter, server, debugger, disassembler, and builder is a striking inversion — and it falls out, again, of having built a real frontend and borrowed a real machine.

The Pending enum and the executor thread

The in-process design has a specific shape worth seeing, because it is what makes the debugger's introspection genuine rather than a re-evaluation. When `vimlrs --dap` runs a program, the script executes on a dedicated executor thread, while the main thread speaks the Debug Adapter Protocol to the editor. The compiler's debug variant emits a `VIML_SET_LINENO` marker before each statement (Chapter 21), whose handler calls `check_line`; `check_line` consults the shared breakpoint state and, if a breakpoint matches or the debugger is single-stepping, `condvar`-waits, parking the executor thread. While parked, though, the executor thread does not just sit idle — it services the introspection requests itself. The `Pending` enum carries exactly the two request kinds the DAP introspection needs, `variables` and `evaluate`, and when the main thread wants to inspect a variable or run an expression, it hands the request across as a `Pending` and wakes the executor to service it. The executor reads its *own* `globvardict` for a variable request, or compiles and runs the expression against its own live state for an evaluate request, then parks again. This is the payoff of the in-process design: the value you see in the debugger and the result of an expression you evaluate at a breakpoint are read from the genuine, paused program's state, not from a copy or a fresh context — because the same thread that ran the program up to the breakpoint is the one answering your questions about it.

Stepping, resuming, and output

The resume modes are two, and they map onto the protocol's step and continue. `Next` stops again at the next statement — single-stepping — and `Continue` runs to the next breakpoint. Both are implemented by the same `check_line` mechanism: stepping sets a flag that makes the next `VIML_SET_LINENO` marker pause unconditionally, while continuing clears it so the next pause waits for a breakpoint. Program output is handled so it never corrupts the JSON-RPC channel: the executor's `:echo` output is captured (the `ECHO_SINK` of Chapter 22 set to a buffer) and streamed to the editor as DAP `output` events at each pause and at termination, rather than written to `stdout` where it would collide with the protocol. The whole debugger is, like the language server, “self-contained, reusing the synthesis lexer/parser,” and its architecture “mirrors `zshrs`'s `dap.rs`” — one more shared-design inheritance. The result is a real debugger for a language that, until `vimlrs`, had no

standalone existence to debug: breakpoints that pause the genuine program, stepping that walks it statement by statement, variable inspection that reads its live state, and expression evaluation that runs against it.

Together the LSP and DAP servers make vimlrs a language an editor can treat as a first-class citizen: syntax-checked as you type, completed and documented from the same corpus the reference is built from, outlined by symbol, and debuggable with real breakpoints and live variable inspection. For a language that spent its entire prior existence *being* the editor's configuration, becoming a language the editor *tools for* is a notable inversion — and it falls out naturally from having a real lexer, parser, and compiler to reuse.

Why the inversion matters

The inversion is worth dwelling on, because it is more than a cute reversal and it speaks to what it means for vimlrs to treat Vimscript as a real language rather than as editor glue. Think about the asymmetry of Vimscript's prior existence. Inside Vim, Vimscript was the language you *wrote the editor's tooling in* — your statusline, your completion, your syntax tweaks were Vimscript — but it was never a language the editor *provided tooling for*. There was no Vimscript language server giving you completion as you wrote your vimrc, no debugger letting you set a breakpoint in a plugin and inspect its variables, no real diagnostics flagging a syntax error before you sourced the file. Vimscript was the tool, never the thing toolled. This is the mark of a language treated as configuration rather than as a first-class programming language: configuration languages get syntax highlighting if you are lucky, but they do not get the full apparatus of modern language support, because nobody thinks of them as languages worth supporting that way. vimlrs inverts this: it provides Vimscript with the apparatus a first-class language gets — a language server for diagnostics, completion, hover, and symbols; a debug adapter for breakpoints, stepping, and inspection. Vimscript becomes a language the editor tools *for*, not just a language the editor is configured *in*.

And the inversion falls out almost for free, which is the part that reveals why it was possible. Building a language server and a debugger from scratch is a large undertaking — you need a parser that produces good error locations, a way to run programs under control, a model of the program's state. vimlrs already had all of these, because it had built a real frontend and a real execution engine to run the language standalone. The LSP reuses the same lexer and parser the interpreter uses, so its diagnostics are the interpreter's parse errors and its completion is the interpreter's builtin corpus. The DAP reuses the same compiler and bridge, running the program on an executor thread with line markers the compiler already knew how to emit, so its breakpoints pause the genuine program and its inspection reads the genuine state. The tooling is not a separate construction bolted onto the language; it is the *same* lexer, parser, compiler, and corpus, reused through a different entry point. So the inversion — Vimscript becoming a tooled-for language — is the natural consequence of the deeper move, building a real standalone implementation: once you have a genuine frontend and engine, the language server and the debugger are mostly a matter of wiring them to a protocol, and the apparatus of a first-class language follows from having treated the language as first-class in the first place. The tooling is the visible proof that vimlrs treats Vimscript as a real language, and it falls out of the implementation that treats it that way.

Chapter 18 – The Standalone Contract: an Editor-Absent Vim

Here is the question that has hovered over this whole book: a great deal of Vimscript is *about the editor* — buffers, windows, folds, mappings, signs, the quickfix list, jobs, the event loop — and vimlrs has none of those. So what happens when a script calls `bufnr()`, `winheight()`, `getbufvar()`, or `jobstart()`? The answer is a deliberate, documented *contract* rather than a crash or a lie, and understanding it is the last piece of understanding what vimlrs is. This chapter is that contract: how vimlrs answers for the editor it does not have.

Honesty about features: `has()`

The foundation of the contract is `has()`, which reports what vimlrs genuinely is. The `has.vim` example states the principle: vimlrs “reports only what it genuinely is: the real build platform ... plus the language features it actually implements.” So the language and runtime features it does implement are true:

```
call assert_equal(1, has('eval'))
call assert_equal(1, has('float'))
call assert_equal(1, has('vimlrs'))
call assert_equal(1, has('lambda'))
call assert_equal(1, has('num64'))
```

And the editor features it does *not* have are false — `has('syntax')`, `has('windows')`, `has('folding')`, `has('gui_running')` all return 0. So does `has('nvim')`, because vimlrs is neither Vim nor Nvim and will not pretend otherwise, and so does any version/patch query like `has('patch-8.1.0')`. The platform answers are real: exactly one of `has('unix')` / `has('win32')` is true, and on macOS `has('mac')` implies `has('unix')`, `has('macunix')`, and `has('osx')`. Feature names match case-insensitively, as Vim's do. This is the keystone of the whole contract: a script can ask `has()` what is available and get a truthful answer, which means a well-written script can adapt rather than break. Honesty at the feature-detection layer is what makes everything below it safe.

The platform answers in full

The platform half of `has()` is genuinely informative, and `has.vim` and `sysinfo.vim` pin it down. Exactly one of `has('unix')` and `has('win32')` is true, reflecting the real build platform, and on macOS the unix answer composes: `has('mac')` implies `has('unix')`, `has('macunix')`, and `has('osx')`, because a Mac is a unix. The language-and-runtime features are true for what vimlrs actually implements: `has('eval')`, `has('float')`, `has('lambda')`, `has('num64')`, and `has('vimlrs')` are all 1, the last being vimlrs's own identifying feature. And the editor features are false: `has('syntax')`, `has('windows')`, `has('folding')`, `has('gui_running')` all return 0, as does `has('nvim')` — because vimlrs is neither Vim nor Nvim and declines to pretend — and so does any version or patch query like `has('patch-8.1.0')`, since vimlrs does not track a Vim version number. Feature names match case-insensitively, as Vim's do. The principle is the one the chapter opens with: report what you genuinely are. A script that guards an editor-dependent code path with `if has('nvim')` takes the false branch under vimlrs and runs its fallback, exactly as it would on a Vim that lacked the feature — which is the whole point of feature detection, and the reason the honest answers are more useful than flattering ones would be.

Well-defined answers where there is no editor

For builtins that query editor state, the contract is to return the *documented empty answer* — the value Vim itself returns when the queried thing is absent — rather than erroring. The `editor_absent.vim` and `editor_queries.vim` examples pin these down across a dozen subsystems. With no folds, every fold query reports “nothing here”: `foldclosed(1)` is -1, `foldtext()` is `''`. With no mappings, the mapping queries report nothing: `maparg('<F2>')` is `''`, `maplist()` is `[]`, `hasmapto('<C-x>')` is 0 — though `maparg('<F2>', 'n', 0, 1)` with a truthy dict argument returns an empty Dict `{}` rather than `''`, because the *type* of the documented return depends on the argument, and vimlrs honours that:

```
call assert_equal(-1, foldclosed(1))
call assert_equal('', maparg('<F2>'))
call assert_equal([], maplist())
call assert_equal({}, maparg('<F2>', 'n', 0, 1))
call assert_equal(0, highlight_exists('Normal'))
```

Some queries even have a *real* answer in the virtual context: `indent(1)` is 0 (the empty first line of the virtual buffer indents to zero) while `indent(99)` is -1 (a line past the end). The principle is consistency with Vim’s documented contract for the empty case, applied subsystem by subsystem.

The `editor_queries.vim` example pushes this further, into the *shape* of the dicts that inactive subsystems return, which is where the contract gets subtle. A `searchcount()` with no prior search returns a dict with every key present and set to its empty value — `current` and `total` 0, `exact_match` 0, `incomplete` 0 — but `maxcount` is 99, its documented default, not 0, because that key reports a setting rather than a count. A `complete_info()` returns `mode` empty, `pum_visible` 0, `items` an empty list, and `selected` -1. The reason this matters is that a plugin reads such a dict and branches on a *field* — `if complete_info().pum_visible` — so the dict must have every field the plugin might read, each set to the value that makes the branch take the inactive path. A dict missing a key would crash the plugin on the field access; a dict with the wrong default would send it down the wrong branch. So the editor-absent contract operates not just at the level of “what scalar does this function return” but at the level of “what is the exact shape of the dict it returns,” and `editor_queries.vim` pins those shapes key by key. Getting the dict shapes right is what lets a defensively- written plugin, full of these field accesses, run to completion under vimlrs taking the inactive path at each one.

Scoped-variable getters and the {def} fallback

The scoped-variable accessors are the clearest illustration of the contract, because Vim already documents what they do when the variable is absent, and vimlrs simply always takes that branch. The `scopedvars.vim` example shows it: a getter returns its `{def}` argument when the variable is missing, and the setters are accepted no-ops:

```
call assert_equal('DEF', getbufvar(1, 'missing', 'DEF'))
call assert_equal('', getbufvar(1, 'missing'))
call assert_equal(42, getwinvar(1, 'missing', 42))
call assert_equal(0, setbufvar(1, 'x', 1))
call assert_equal(0, setwinvar(1, 'x', 1))
```

`getbufvar(1, 'missing', 'DEF')` returns `'DEF'` — the documented fallback when the variable does not exist — and with no fallback argument it returns `''`. The setters return 0 (success) and do nothing observable, because there is no buffer to set a variable on. The example’s rationale is precise: “Pinning these contracts lets editor/async scripts load.” A plugin that sets a buffer variable on startup and reads it back with a default will *load and run* under vimlrs, taking the default path, rather than crashing on an unknown function. The contract is chosen to maximize the set of real scripts that can at least load.

Jobs, channels, and the event loop

The async builtins are the furthest from a standalone interpreter's reach, since there is no event loop, and the contract here is to fail cleanly with the documented failure value. `scopedvars.vim` enumerates them: `jobstart('ls')` returns -1 (the documented failure), `jobstop`, `jobpid`, `chanclose`, `chansend`, `feedkeys` return 0, `jobwait([1, 2])` returns [], `sockconnect` returns 0, `win_execute` returns '':

```
call assert_equal(-1, jobstart('ls'))
call assert_equal(0, jobstop(1))
call assert_equal([], jobwait([1, 2]))
call assert_equal(0, sockconnect('tcp', 'localhost:80'))
```

A job never starts, and the caller sees the failure value it would see if the job had failed to start in Vim — which is a state Vim scripts already handle, so they handle it here too.

The async family is the clearest case for why the documented-failure-value contract is better than the alternatives. These functions need an event loop — a scheduler that runs callbacks when a job produces output, a timer fires, or a socket becomes readable — and a standalone batch interpreter has no event loop, because there is nothing to drive it: the program runs top to bottom and exits. So `jobstart` cannot start a job, and the question is only what it should *return*. Omitting it would make any script that mentions a job fail to parse. Faking it — returning a positive job id for a job that will never run — would be a lie that strands the caller waiting on output that never comes. Returning the documented failure value (-1 for `jobstart`, [] for `jobwait`, 0 for the rest) is the one choice that lets the script continue sanely: a well-written plugin checks `jobstart`'s return for the failure value, and on seeing it takes the synchronous or degraded path it already has for “jobs are unavailable.” The same logic covers the timers (`timer_start` returns an id but the callback never fires), the channels, and the sockets. None of these can do real work without the event loop the standalone runtime lacks, and all of them return the value that lets a script detect that and adapt — which is, once more, the difference between an interpreter that can load a real plugin and a toy that cannot. The reference marks many of these “stub in standalone,” and that label is the contract made explicit: the function exists, takes its arguments, and returns the documented degenerate value, so that calling it neither crashes nor silently does the wrong thing.

stdpath and the XDG standard paths

One family of editor-adjacent builtins returns *real* answers standalone because what it computes is not editor state at all — it is the filesystem layout Neovim would use, which exists whether or not an editor is running. `stdpath()` resolves Nvim's standard directories from the XDG base-directory environment variables, and `stdpath.vim` asserts the exact paths:

```
call assert_equal($HOME . '/.cache/nvim', stdpath('cache'))
call assert_equal($HOME . '/.local/share/nvim', stdpath('data'))
call assert_equal($HOME . '/.config/nvim', stdpath('config'))
call assert_equal(type([]), type(stdpath('config_dirs')))
call assert_fails("call stdpath('bogus')", 'E6100')
```

The single-directory kinds (`cache`, `data`, `state`, `log`, `config`) return a string with the `nvim` app subdirectory appended; the `*_dirs` kinds return a List; and the app subdirectory honours `$NVIM_APPNAME`, defaulting to `nvim`, exactly as Neovim's `get_appname` does. An unknown kind raises the faithful `E6100`. This is a useful contrast with the genuinely-absent builtins around it in the same example: `luaeval`, `rubyeval`, `termopen`, and the GUI file dialogs all return their documented inactive values, because they need a runtime or an editor that is not there. The line is the same one this whole chapter has drawn — `stdpath` computes a path from the environment and so is real, while `termopen` needs a terminal and so is a stub — and `stdpath.vim` is a tidy demonstration of both halves in one file: the path resolver works exactly, and the provider bridges return emptiness, each honest about what it is.

Why the contract is the right design

There were three options for the editor-shaped builtins: omit them (so calling one is an undefined-function error), fake them (return plausible-looking editor state), or implement the documented empty contract. vimlrs chose the third, and it is the only one that serves the project's goal. Omitting them would break every script that so much as mentions a buffer. Faking them would be a lie that produces subtly wrong behaviour downstream — a `winheight()` that returns 80 when there is no window invites a script to do something nonsensical with the number. Implementing the documented empty contract is honest *and* useful: the script loads, the queries return the values Vim returns for absent things, and a script that checks those values — as well-written Vimscript already does — adapts correctly. Combined with `has()` telling the truth about features, this is what lets vimlrs be a real standalone interpreter rather than a toy that only runs editor-free scripts. The editor is absent, but its shape is respected, and that respect is exactly what makes the language portable out of its host. The contract, reduced to a single rule, is this: a function does real work if it operates on something the standalone interpreter genuinely has — the filesystem, the environment, the one virtual buffer, the cursor, a register, a mark, an in-memory table — and returns the documented empty value if it queries or drives something the interpreter lacks — a second buffer, a window, the screen, the event loop, a peer process. Everything in this chapter and the catalogue chapters is an application of that one rule, subsystem by subsystem. Learn the rule and you can predict any function's standalone behaviour without looking it up: ask “does this touch data the interpreter has, or activity it lacks?” and the answer tells you whether it is real or a stub. That predictability is the deepest sense in which the editor's shape is respected — not just that each function returns a sensible value, but that the *pattern* of which functions are real and which are stubs is itself coherent and learnable, so a programmer can reason about the whole surface from one principle rather than memorizing a list of special cases.

Honesty as an engineering strategy

There is a deeper principle under the editor-absent contract than “return sensible values,” and it is worth naming because it runs through the whole project: vimlrs's strategy everywhere it falls short of full Vim is *honesty*, and honesty turns out to be not merely an ethical posture but the engineering choice that maximizes how much real Vimscript can run. Consider the faking alternative more carefully, because it is tempting and it is wrong. A `winheight()` that returns 80 looks helpful — it gives the script a plausible number to work with. But the number is a lie, and the lie propagates: the script computes a layout from a window that does not exist, draws conclusions from a height that is fictional, and produces behaviour that is wrong in a way that is hard to trace, because nothing errored and nothing looked obviously off — the script just did something nonsensical with a number it had every reason to trust. Faking trades a visible failure (an error the script can detect) for an invisible one (wrong behaviour the script cannot detect), and invisible failures are the worst kind, because they surface far from their cause. The documented-empty-value contract refuses this trade: `winheight()` returns -1, the value Vim returns when the window cannot be measured, which is a value the script's author already had to handle, so the script takes its already-written “no window” path rather than computing from a fiction.

This is why honesty maximizes compatibility, which is the counterintuitive part. You might think that faking editor state would let *more* scripts run, by giving them the numbers they ask for. The opposite is true. A well-written Vim plugin is full of defensive checks — `if winheight(0) < 0`, `if has('nvim')`, `if empty(maparg('<F2>'))` — because Vim itself returns these absent values in states the plugin must handle, so the plugin already knows what to do when a query comes back empty. By returning exactly those absent values, vimlrs lets the plugin's existing defensive code do its job: the plugin detects the absence and adapts, loading and running on its fallback path. A faking interpreter, by giving plausible-looking values, would *defeat* the plugin's defensive checks — the checks would pass, the plugin would proceed as if the editor were present, and it would then fail deeper in, where it actually tried to use the fictional window. So honesty does not merely avoid lying; it actively cooperates with the defensive idioms that real Vimscript already contains, which is why a script written for Vim, full of these checks, can load and run under vimlrs taking the paths it already has for the absent case. The contract is honest, the honesty is systematic, and the systematic honesty is what makes vimlrs a real standalone interpreter rather than a toy — not despite being honest about what it lacks, but *because* of it.

The line between editor and language

Stepping all the way back, the editor-absent contract is really an answer to a question that had no answer before vimlrs, because the question never came up: where is the line between Vimscript the language and Vim the editor? For Vimscript's entire prior existence the question was moot, because the language only ran inside the editor, so every function had an editor to talk to and the line was invisible. Taking the language out of the editor forces the line into the open: some functions are *language* — arithmetic, string manipulation, list operations, control flow, the pure data transforms — and some are *editor* — buffers, windows, folds, the event loop. vimlrs's contribution, beyond running Vimscript standalone, is to draw that line precisely and principledly, and the line it draws is “data the interpreter can have versus activity that needs the editor.” The filesystem, the environment, a buffer of text, a cursor, registers, marks, mappings, signs — these are data, and a standalone interpreter can keep them faithfully, so they are on the language side of the line and they do real work. The window layout, the redraw, the syntax engine, the job scheduler, the typing loop — these are activity, an editor doing things over time, and a batch interpreter has no occasion for them, so they are on the editor side and they return the documented empty value. That the line is this clean — data on one side, activity on the other — is not obvious in advance, and discovering that so much of “the editor” is really just data structures (a mark table, an autocommand registry, a quickfix list) that transplant cleanly is one of the genuine findings of the project. The editor-absent contract is the precise articulation of that finding: it is the map of which half of Vimscript is the language and which half is the editor, drawn by the one project that ever had to draw it.

Chapter 19 – The Lexer: From Text to Tokens

Chapter 14 named the five stages of the pipeline and then moved on to the JIT. The next four chapters slow down and walk the first four stages one at a time — the lexer, the parser and AST, the bytecode compiler, and the bridge — because they are the synthesis zone, the half of vimlrs that has no Neovim C to port and therefore had to be invented. This chapter is the lexer: the first thing that touches your source text, and the smallest of the four. Its job is narrow and its boundaries are sharp, and seeing exactly where those boundaries lie clears up a confusion this book has so far left slightly blurred.

What the lexer is, and what it is not

The lexer lives in `viml_lexer.rs`, and like every synthesis-zone file it opens with the firewall banner: “EXTENSION — NO `csrc/` COUNTERPART.” Its header is unusually candid about *why* it exists, and the explanation is worth quoting in full because it is the cleanest single statement of the whole synthesis-zone rationale:

```
Neovim's eval.c has no separate lexer – eval1...eval7 scan characters
off the source string inline while evaluating. The bytecode frontend needs a
real token stream, so this is net-new code, NOT a port. It is bound by the
"no fake C names" rule only in the negative sense: nothing here may claim to
be a port or carry a // c: citation it doesn't have. The token set is still
dictated by what eval.c recognizes (operator spellings, literal forms,
sigil-prefixed names).
```

Read that last sentence carefully, because it draws the exact line the lexer walks. The *code* is new — Neovim has no lexer to copy — but the *token set* is not free invention: every literal form, every operator spelling, every sigil is one that `eval.c` recognizes, so the lexer reproduces Vim's surface vocabulary even though it reproduces none of Vim's lexing code. This is the synthesis-zone discipline in miniature: invent the mechanism, inherit the contract.

The other boundary is the one this book has so far glossed. The lexer tokenizes *one expression*. It does not split commands on the bar `|`, it does not strip `"`-comments, it does not join line-continuation backslashes, and it does not recognize the heredoc `=<<`. All four of those are *command-line* concerns, and they are handled one layer up, in the parser's source splitter (Chapter 20). When earlier chapters described the bar splitting as “string-/operator-/comment- aware,” that awareness lives in the parser's `split_commands`, not here. The lexer's world is a single expression string, already separated from its neighbours, and its only output is a flat list of tokens ending in `Tok::Eof`. Keeping that boundary clean is what lets the same lexer serve the expression parser, the `-e` evaluator, and the LSP without any of them having to agree on command-line rules.

The token set

The token type is `Tok`, a plain enum, and each variant is one lexical atom. The literal-bearing variants carry their already-decoded value: `Number(i64)` holds an integer that is “already parsed,” `Float(f64)` a float, `Blob(Vec<u8>)` “the decoded bytes” of a `0z...` literal, and `Str(String)` a string that is “already unescaped.” This is a deliberate choice — the lexer does the work of parsing a number's base or unescaping a string's backslashes *once*, at lex time, so the parser and everything downstream see a clean value and never re-parse

the surface syntax.

The name-like tokens carry their text: `Ident(String)` is “a bare identifier or scoped name (`x`, `g:foo`, `v:true`),” `Option(String)` an option reference `&name`, `Env(String)` an environment variable `$NAME`, and `Register(char)` a register `@x`. The operators are mostly one variant each — `Plus`, `Minus`, `Star`, `Slash`, `Percent`, `Bang`, `Dot`, `DotDot`, `Arrow` (`->`), the brackets and braces, `Comma`, `Assign`, `Colon`, `Question`, `QuestionQuestion` (`??`), `OrOr` (`||`), and `AndAnd` (`&&`). Two structural tokens are worth singling out: `HashBrace` opens a literal-key Dict (`{a: 1}`), and `Eof` marks the end. And one token is richer than the rest: `Cmp(CmpOp, CaseFlag)`, “a comparison operator, with its case-sensitivity flag,” which is where Vimscript’s strangest operator family is captured.

Comparison operators carry a case flag

Chapter 3’s precedence ladder noted that `==#`, `==?`, `==~#`, and `==~?` are all distinct operators. The lexer is where that distinction is born. The `CmpOp` enum names the ten relational families — `Equal` (`==`), `NotEqual` (`!=`), `Match` (`==~`), `NoMatch` (`!=~`), `Greater`, `GreaterEqual`, `Less`, `LessEqual`, `Is`, and `IsNot` — and its own comment ties it straight to the source: these are “the relational families recognized in `eval4` (`eval.c`),” and they “map onto the ported `exprtype_T` in the bridge.” So the lexer’s comparison vocabulary is not an approximation; each `CmpOp` variant corresponds to a value the ported engine ultimately compares with.

The case flag is a separate enum, `CaseFlag`, with three variants and a comment that states the rule exactly: a comparison may carry a “case-sensitivity suffix (`==#` match-case, `==?` ignore-case, bare `==` follows ‘ignorecase’).” So `Default` means “no suffix — follows ‘ignorecase’,” `MatchCase` is the `#` suffix, and `IgnoreCase` is the `?` suffix. The lexer applies it uniformly: after it consumes any comparison operator, it peeks one more byte — a `#` becomes `MatchCase`, a `?` becomes `IgnoreCase`, anything else stays `Default`. Because the rule is applied to every comparison spelling at once, not just `==`, the forms `!=#`, `>=?`, `==~#`, and the rest all fall out for free. The lexer’s own tests pin it: `==#` lexes as `Cmp(CmpOp::Equal, CaseFlag::MatchCase)`. The flag then rides through the AST’s `Compare` node (Chapter 20) and into the bytecode, so the three-way case choice survives all the way to the comparison the ported `typval_compare` performs.

Sigils, scope colons, and the autoload hash

A name in Vimscript can begin with a scope sigil, and the lexer absorbs the colon that follows a single-letter scope prefix directly into the identifier token. Its comment spells out both the rule and the trap it avoids:

```
A leading single-letter scope prefix (a:/b:/g:/l:/s:/t:/v:/w/) absorbs its
: and the name after it. Only the real scope letters do this – otherwise z:1
(a no-space ternary ?z:1) or a literal-Dict key #{z:1} would wrongly merge.
```

The guard is precise: the colon is absorbed only when exactly one character has been consumed so far *and* that character is one of the eight scope letters `a b g l s t v w`. This is why `g:foo` lexes as a single `Ident("g:foo")` while `z:1` does not merge — `z` is not a scope letter — and why a ternary written tightly as `cond?z:1` still lexes correctly. After the identifier, the lexer also absorbs autoload `#name` segments, so `pkg#func` is one identifier token: it loops while the next character is `#` and the one after it is an identifier character. The option and environment sigils have their own small wrinkle — `gl:opt` and `$VAR` are read by a dedicated `lex_sigil_name` that special-cases a `l:/g:` scope prefix on the option before consuming the name — and the `&&/&` distinction is resolved at the very top of dispatch: `&&` is the logical-AND operator `OrOr`’s sibling `AndAnd`, while “only a lone `&` is an option sigil.”

Numbers, bases, and the float that needs a dot

Integer literals understand four bases, and the lexer dispatches on the character after a leading `0`: `x/X` goes to a radix-16 reader, `b/B` to radix-2, `o/O` to radix-8, and `z/Z` to the blob reader. Bare octal — a leading `0` followed

by only octal digits — is handled by its own rule, and the comment cites the C it mirrors:

```
Vim octal literal: a leading 0 followed only by octal digits (010 == 8). A
8/9 anywhere (08, 0129) keeps it decimal, matching vim_str2nr's STR2NR_OCT
detection in eval_number() (Src/eval.c).
```

So `010` is 8, `0777` is 511, but `08` is 8 (decimal, because the `8` disqualifies octal) and `0129` is 129. These are exactly the results `numbers.vim` asserts, and they are right because the lexer reproduces `vim_str2nr`'s octal detection rather than guessing.

The float rule is the subtle one, and it is a place vimlrs is faithful to a specific Neovim quirk. A float literal must have a dot with digits on both sides; an exponent alone is not enough. The lexer's comment is explicit:

```
An exponent is only part of the literal after a .{digits} fraction:
Vim/Neovim's float grammar is [0-9]+\.[0-9]+([eE][+-]?[0-9]+)?, so a
dotless 1e100 is the Number 1 followed by the name e100 (an error
at parse time), never a float.
```

This is why `float_literals.vim` asserts that `1.0e10` is a Float but `1e3` raises an error: `1.5e3` has the mandatory fraction and parses as `1500.0`, while `1e3` lexes as the number `1` followed by the bareword `e3`, which the parser then rejects. A lexer that treated `1e3` as a float would be more permissive than Vim, and vimlrs declines to be.

Strings and blobs

The two string flavours are lexed by two functions with two different escape rules, matching Chapter 2's account. A single-quoted string is almost literal: the only escape is `'` for an embedded quote, so `'a'b'` decodes to the three-character `a'b`, and an unterminated one raises `E115: Missing quote`. A double-quoted string honours the full backslash vocabulary — `\n`, `\t`, `\r`, `\e` (escape, `\x1b`), `\b` (backspace, `\x08`), `\\`, `\"`, octal `\0–\7` up to three digits, and hex `\x/\X` up to two — and an unterminated one raises `E114`. Both decode at lex time, which is why the `Str` token holds text that is “already unescaped.”

The blob reader handles the `0z` form, and its comment cites the source: it is a port-in-spirit of “the `0z` branch of `eval_number()` (Src/eval.c).” It reads hex digit pairs, skips a `.` used to group pairs for readability, stops on an odd trailing nibble (Vim requires pairs), and decodes each pair to a byte. So `0z00112233` becomes the four bytes `[0, 17, 34, 51]`, `0z00.11.22` is the same as `0z001122`, and a bare `0z` is the empty blob — exactly what `blobs.vim` checks.

Decoding at lex time rather than later is a small architectural choice with a real payoff: every stage above the lexer sees a clean value and never has to think about escapes or bases again. The parser builds a `Str(String)` node holding the actual characters and a `Number(i64)` holding the actual integer, the compiler emits them as constants, and the runtime manipulates the actual bytes — none of them re-process `\t` or `'` or a hex prefix, because the lexer already turned `"\t"` into a tab byte, `'a'b'` into `a'b`, and `0xff` into 255. This is also why the single-quote-versus- double-quote distinction matters so much for regular expressions (Chapter 2): a pattern in double quotes would have its backslashes consumed by the lexer's escape decoding before it ever reached the regex engine, so `"\d\+"` would arrive mangled, whereas `'\d\+'` in single quotes passes through almost verbatim — the only single-quote escape being `'` — and reaches the regex engine intact. Every regex example in the crate uses single quotes for exactly this reason, and the lexer's two-function escape handling is the mechanism that makes the choice consequential: the double-quote reader applies the full C-style escape vocabulary, the single-quote reader applies almost none, and which runs is decided by the opening quote.

The dot, decided later

One thing the lexer deliberately does *not* resolve is the meaning of the dot. A `.` always tokenizes as `Tok::Dot` (or `Tok::DotDot` for `..`); the lexer never decides whether a given dot is member access (`d.key`) or concate-

nation (`a . b`). What it does instead is record, on every token, a `span` and an `end` — the byte offset where the token starts and the byte offset just past where it ends. The `end` field exists, its comment says, “for adjacency checks like `d.key` member access vs `a . b` concatenation.” The parser reads those offsets to decide: a dot that abuts its operands on both sides is member access, a dot with spaces is concatenation. The lexer’s job is only to preserve the information; the decision is Chapter 20’s. This separation is why the lexer can stay a simple, linear pass — it never needs lookbehind or context, because it defers every context-sensitive question to the parser by handing it precise byte spans.

The dispatch and the lookahead budget

The lexer’s main loop is a single dispatch on the next byte, and its lookahead budget is deliberately tiny — a `peek` of the current byte and a `peek2` of the one after, both returning `0` past the end of input so the loop never reads out of bounds. That two-byte window is all the lexer needs, because every token’s identity is decided by its first one or two characters: a digit starts a number, a quote starts a string, a `0` followed by `x/b/o/z` selects a base, a scope letter followed by `:` starts a scoped name, an operator character maps to its token, and the comparison operators read one trailing `#/?` for the case flag. The handful of places that need more — distinguishing `&&` from `&`, `||` from a lone bar that is an error, `..` from `.`, `~/=/<<` from `=` — are resolved by peeking the second byte and consuming both. There is no backtracking: once the loop commits to a token kind from the leading byte, it reads that token to completion and never rewinds. This is what keeps the lexer linear and fast, and it is possible only because Vimscript’s lexical grammar is genuinely prefix-decidable at the expression level — the context-sensitive questions (member dot versus concat, lambda versus dict, the bar as separator versus operator) are all deferred to the parser, which is exactly why the lexer can afford so small a window.

Errors and multibyte

The lexer raises faithful error codes for the malformed inputs it can detect locally. An unterminated single-quoted string is `E115: Missing quote`, an unterminated double-quoted string is `E114`, and a character that cannot begin any expression token — a lone `|` outside a string, say — is `E15: Invalid expression`. These are Vim’s own error numbers, so a script that checks for them, or a test that asserts them with `assert_fails`, gets the right code. Multibyte text is handled by advancing a whole UTF-8 character at a time when the lexer is inside a string or scanning an identifier, so a name or string containing `é` or an emoji lexes correctly, and the byte spans the lexer records account for the multibyte width — which is what lets the downstream byte/character index functions of Chapter 9 compute correct offsets. The lexer does not, however, try to *interpret* multibyte content beyond passing it through; the codepoint-level work (counting characters, folding composing marks) is the ported string functions’ job, and the lexer’s responsibility ends at producing a faithful token stream with accurate spans. Small window, linear scan, faithful error codes, multibyte-aware spans, and resolutely expression-scoped — the lexer’s virtues are all virtues of restraint, and the restraint is what makes the parser’s harder job tractable.

The shape of a lex

Put together, the lexer is a single left-to-right scan. It skips leading whitespace, records the start offset, reads one token, records the end offset, and repeats until the input is exhausted, appending a final `Eof`. There is no backtracking and no lookahead beyond a byte or two (`peek` and `peek2` return `0` past the end of input). Multibyte text is handled by advancing a whole UTF-8 character at a time. A character it cannot make sense of — a bare `|` outside a string, say, which is meaningful to the command splitter but not to a single expression — raises `E15: Invalid expression`. That error is itself a sign of the boundary: the lexer sees expressions, and a lone bar is not part of one. Small, linear, value-decoding, span-recording, and resolutely expression-scoped — that is the whole of the lexer, and its smallness is exactly what makes the parser’s job in the next chapter tractable.

Chapter 20 — The Parser and the eval1...eval7 Ladder

The parser turns the lexer's flat token stream into the tree the compiler lowers. It is the largest of the synthesis-zone frontend files, and it does two quite different jobs that are worth separating in your mind: it parses *expressions* with a recursive-descent precedence ladder transcribed straight from Neovim's `eval.c`, and it parses *statements* — the command layer — with a line-oriented dispatcher that also handles all the command-line concerns the lexer declined. This chapter takes the expression side first, because it is the more elegant, then the statement side, because it is where the real Vimscript fiddliness lives.

The AST, shaped by the ladder

Before the parser, the target. The AST (`viml_ast.rs`) is two enums, `Expr` and `Stmt`, and its banner states the design constraint that gives it its shape:

```
Neovim's eval.c parses and evaluates in one pass over the source string;  
there is no AST. This tree is net-new, its shape dictated by the eval1...eval7  
precedence ladder so the compiler can lower it to fusevm bytecode.
```

So the AST exists only because vimlrs compiles rather than tree-walks, and its structure is chosen to mirror the precedence ladder so the lowering in Chapter 21 is a straightforward recursion. The `Expr` enum has the variants Chapter 3 listed — `Number`, `Float`, `Str`, `List`, `Dict`, `Lambda`, `Var`, `Option`, `Env`, `Register`, `Unary`, `Arith`, `Compare`, `And`, `Or`, `Ternary`, `Coalesce`, `Index`, `Slice`, `Member`, `Call`, `CallExpr`, and `Method` — and each carries a comment naming the precedence level it belongs to. `Ternary` and `Coalesce` are tagged `eval1`, `Or` is `eval2`, `And` is `eval3`, `Compare` is `eval4` (and “carries the case flag and `is/isnot`”), `Arith` is `eval5/eval6`, and `Unary` is the `eval7` leader. The operator enums are small and exact: `UnaryOp` is `Neg`, `Plus`, `Not`; `ArithOp` is `Add`, `Sub`, `Mul`, `Div`, `Mod`, and `Concat` (which covers both `.` and `..`); and the comparison's `CmpOp/CaseFlag` are imported from the lexer, so the same types flow from token to tree.

Two `Expr` variants distinguish call shapes that look similar but mean different things. `Call { name, args }` is an ordinary named call, `name(args)`. `CallExpr { callee, args }` is the “direct call of a funcref-valued expression,” the thing `funcref_call.vim` exercises — `function('toupper')('hi')` or `fns[0]('Hi')` — where the callee is itself an expression that evaluates to a `Funcref`. And `Method { base, name, args }` is the `->` arrow, which records the receiver separately rather than rewriting itself into a `Call`, because (as Chapter 3 noted) the receiver is evaluated once and the spelling carries intent. One small honesty: the AST declares a `Member { base, key }` variant for `d.key`, but the parser actually builds member reads as an `Index` whose index is a string literal — so `Member` exists in the type but the parser emits `Index { index: Str(key) }`. It is the kind of vestigial detail that only matters if you are reading the lowering code, and it is mentioned here so that you are not surprised by it there.

The precedence ladder, function for function

The parser's expression half is a textbook recursive descent, and its author kept Neovim's function names rather than renaming them to parser convention. The header draws the grammar as a box:

```

eval1  expr2 ? expr1 : expr1  |  expr2 ?? expr1
eval2  expr3 || expr3
eval3  expr4 && expr4
eval4  expr5 (== != ==- !=- > >= < <= is isnot) expr5    (no assoc)
eval5  expr6 (+ - . ..) expr6
eval6  expr7 (* / %) expr7
eval7  (! - +)* primary subscripts

```

Each line is a method. `eval1` parses the ternary and the falsy-coalesce, and it is right-recursive — the branches of `cond ? a : b` are themselves full `eval1` expressions, which is why ternaries nest to the right. `eval2` and `eval3` are left-associative loops over `||` and `&&`, building `Or` and `And` nodes that the compiler will short-circuit. `eval4` is deliberately *non-associative* — it parses exactly one comparison, with no loop, because `a < b < c` is not meaningful in Vimscript — and it is where the `is/isnot` identity operators are folded in alongside the lexer's `Cmp` tokens, each tagged with `CaseFlag::Default`. `eval5` and `eval6` are the arithmetic loops, `eval5` handling `+`, `-`, and concatenation `./..`, `eval6` handling `*`, `/`, `%`. `eval7` collects the unary leaders `!`, `-`, `+` and then descends to the primary atom and its postfix tail.

The leaders are applied in reverse, which is the right thing: a source `- - x` collects two `Neg` leaders and wraps the atom in them outermost-last, so the tree is `Neg(Neg(x))`. Below `eval7`, `primary` reads the leaf atoms — literals, a parenthesized expression, a `[` list, a `{` that is either a lambda or a dict, a `#` { literal-dict, and an identifier that may begin a call — and `postfix` reads the tail: `[index]` and `[from:to]` subscripts, `.name` member reads, `expr(args)` funcname calls, and `->method()` arrows. The whole ladder is entered through `eval1`, and a parse of `1 + len(x) * 2 > 5 ? 'big' : 'small'` walks down to `eval7` for each atom and back up through the arithmetic, comparison, and ternary levels exactly as the precedence demands.

Why the parser mirrors the C function names

There is a choice embedded in the parser that is worth drawing out, because it is the porting discipline applied somewhere you might not expect it — the synthesis zone. The parser is net-new code; Neovim has no separate parser to port, because it parses and evaluates in one pass. So the parser is free, in principle, to be structured however its author likes — it could use a generic operator-precedence algorithm, a Pratt parser, a table-driven scheme, any of the standard parsing techniques. Instead it is structured as a ladder of functions named `eval1` through `eval7`, the exact names of the precedence-level functions in Neovim's `eval.c`. This is a deliberate choice, and it is a kind of faithfulness even though the parser is not a port. The point is that Vimscript's operator precedence — which operators bind tighter than which — is defined by the structure of Neovim's `eval1...eval7` cascade: `eval1` handles the loosest operators and defers to `eval2`, which defers to `eval3`, down to `eval7` which handles the tightest. The precedence is not written down as a table somewhere; it is the call structure of those seven functions. So the most faithful way to reproduce Vimscript's precedence is to reproduce that call structure, function for function, name for name, even in a parser that is otherwise new code.

This is the synthesis zone borrowing the porting discipline where it helps. The parser does not cite `eval.c` lines the way the ported zone does — it carries the EXTENSION banner, because it is not a port — but it takes its *shape* from the C, because the C's shape is where the precedence lives. A generic precedence-climbing parser with a table of binding powers would produce the same parse trees if the table were right, but the table would be a second place the precedence was encoded, a second chance to get it wrong, and a structure a maintainer would have to check against the C by reading both. The ladder-of-named-functions structure makes the correspondence direct: `eval4` in vimlrs handles exactly what `eval4` in `eval.c` handles, so a maintainer wondering whether comparison binds looser than addition reads the two `eval4/eval5` functions side by side and sees the same structure. The parser is net-new, but its skeleton is Neovim's, because the skeleton is the specification of precedence, and reproducing the skeleton is the faithful way to reproduce the precedence even in code that has no line-by-line C to mirror. It is a small thing — a naming convention and a call structure — but it is the porting instinct reaching into the synthesis zone wherever the C's structure encodes a behaviour worth matching.

Lambda or dict: a one-token lookahead

A `{` can open either a lambda `{x -> x + 1}` or a dict `{'k': v}`, and the parser decides with a small lookahead past the opening brace. Its rule, from the `at_lambda` comment, is exact:

```
a lambda is { -> ... } or { ident (, ident)* -> ... } — a top-level -> reached
through only bare names and commas. Anything else (a : key, a string key, })
is a dict.
```

So it scans forward over bare identifiers and commas; if it reaches a `->` it is a lambda, and if it hits a `:` key, a string, or a closing `}` first it is a dict. This is why `{}` is the empty dict and `{-> 42}` is a no-argument lambda, and why the disambiguation never needs to backtrack — one linear scan settles it. The literal-dict `#{` has its own parser, and it carries the wrinkle Chapter 7 described: because the lexer absorbs a scope-letter's colon, a key like `a` in `#{a: 1}` arrives already glued to its colon, so the literal-dict parser splits it back apart and reads `a` as the key and `1` as the value, rather than seeing the `a:` argument scope. The blob literal is desugared right here in `primary` too: `0z...` becomes a call to the ported `list2blob` builtin over the decoded byte list, “reusing the ported `list2blob` builtin to build the Blob value.”

The member dot, adjacency-decided

Here is where the byte spans the lexer recorded earn their keep. The parser's `at_member_dot` predicate decides whether a `Tok::Dot` is a member read or a concatenation, and the rule is pure adjacency:

```
True when the Tok::Dot at the current position is a dict member access
d.key rather than the ..-style concat operator: it must directly abut
the base (no space before) and be immediately followed by a bare name (no
space after). a . b (spaced) stays concatenation.
```

There is a second condition that catches a real Vimscript subtlety: `.name(` is *concatenation with a function call*, not a member call, because legacy Vimscript has no `dict.key(args)` call syntax — that spelling is Vim9 script. So `f(x).g(y)` concatenates two call results rather than reading a member, and the parser only treats `.name` as a member read when the name is not immediately followed by `(`. The `concat_dot.vim` example is the proof that the parser gets all of this right: `'a'.'b'` concatenates, `toupper('a').toupper('b')` is `'AB'`, `d.k` reads a member, and `nested.a.b` chains two member reads. The same byte-span machinery decides funcref calls: an abutting `(` after a postfix result is a `CallExpr` (`function('x')(a)`), while a spaced `echo F (1)` stays two arguments — the `lparen_abuts_prev` check is what tells them apart.

The statement dispatcher

The other half of the parser is the command layer, and it is line-oriented in a way the expression grammar is not. The entry point is `parse_stmt`, which takes one logical line and returns one `Stmt`. It isolates the leading command word — the run of ASCII letters at the start — and matches on it: `echo/echon/ echomsg`, `let`, `const`, `call`, `eval`, `execute/exe`, `set` and its `setlocal/setglobal` siblings, `source/so`, `unlet`, `return`, `throw`, `break`, `continue`, the `command/delcommand/autocmd/augroup` family, and the block openers. A line starting with `:` or `%` becomes an `ExCmd` (an Ex command with an optional line range, the subject of Chapter 24); a word beginning with an uppercase letter becomes a `UserCmd` invocation; and anything that matches none of these is parsed as a bare expression statement.

The dispatcher carries several comments that document genuine ambiguities. The `:map` family collides with the `map()` builtin, so the parser only treats `map` as a mapping command when it is *not* immediately followed by `(` — `map(l, e)` is the builtin call, `map <F2> :foo` is the mapping. The same guard distinguishes a user command `Foo args` from a funcref call `Foo(x)`. And the parser is honest about `:const`: its comment notes that “Vim also locks the variable (reassigning is E741); that immutability is not yet enforced here — `:const` parses and assigns as `:let`,” which is the source of the provisional caveat Chapter 4 flagged. The `Stmt` enum

that backs all this is broad — beyond the constructs of earlier chapters it has `Map`, `CommandDef`, `CommandDel`, `UserCmd`, `Autocmd`, `Augroup`, `Doautocmd`, and `ExCmd`, each holding the raw command text to be re-parsed by its runtime — which is why Appendix E can list so much more than the six ex-commands the reference documents with hover text.

The complete expression vocabulary

It is worth enumerating the full `Expr` type once, because the list of variants is the list of everything a Vimscript expression can be, and the parser is exactly the machine that builds one variant per construct. The literal atoms are `Number(i64)`, `Float(f64)`, and `Str(String)` — the values the lexer already decoded — plus the container literals `List(Vec<Expr>)` and `Dict(Vec<Expr, Expr>)`, where a dict's keys are themselves expressions (which is why the ordinary `{}` form evaluates its keys, the distinction `literal_dict.vim` draws against `#{}).` The reference atoms are `Var(String)` for a variable or scoped name, `Option(String)` for `&opt`, `Env(String)` for `$VAR`, and `Register(char)` for `@r` — the four lvalue-capable rvalues that read an option, an environment variable, or a register as easily as a variable.

The operator nodes follow the precedence ladder one rung at a time. `Ternary { cond, then, else }` and `Coalesce { expr, alt }` are the `eval1` pair; `Or(Vec<Expr>)` and `And(Vec<Expr>)` the short-circuiting `eval2/eval3` (built as vectors so a chain of `||` is one node); `Compare { lhs, op, case, rhs }` the non-associative `eval4` that carries the `CmpOp` and `CaseFlag`; `Arith { lhs, op, rhs }` the `eval5/eval6` arithmetic with its `ArithOp`; and `Unary { op, expr }` the `eval7` leader with `UnaryOp::Neg, Plus, or Not.` The postfix nodes are `Index { base, index }` for `base[i]` and member reads, `Slice { base, from, to }` for `base[a:b]` with `Option`-typed bounds, `Call { name, args }` for a named call, `CallExpr { callee, args }` for a func-ref-valued-expression call, `Method { base, name, args }` for the `->` arrow, and `Lambda { params, body }` for `{args -> body}`. That is the whole expression language: twenty-odd variants, each a construct, each a rung on the ladder or a leaf at the bottom, and the parser's recursive descent is just the machine that decides, token by token, which variant to build.

The complete statement vocabulary

The `Stmt` type is the command language, and it is broader because there are more commands than operators. The output statements are `Echo(Vec<Expr>)`, `Echon`, and `Echomsg`. The binding statements are `Let { target, op, value }` (whose `op` carries the compound operator before the parser desugars it), `Const`, and `Unlet { args, bang }` with its `UnletArg::Name/Item` variants. The calling-and-evaluating statements are `Call`, `Eval(Expr)`, and `Execute(Vec<Expr>)`. The control-flow statements are `If { arms, else }` (a vector of `(Expr, Vec<Stmt>)` arms), `While { cond, body }`, `For { vars, expr, body }` with its `ForVars::One/List` unpacking, `Break`, `Continue`, `Try { body, catches, finally }` (each catch a `(Option<pattern>, Vec<Stmt>)`), `Throw(Expr)`, and `Return(Option<Expr>)`. The definition statements are `Function { name, params, defaults, body, bang }` and `Source(String)`. The option statement is `Set` carrying the raw option arguments.

Beyond these, the statement vocabulary reaches the editor-facing commands the parser recognizes and the runtime resolves: `Map` for the `:map` family, `CommandDef` and `CommandDel` for `:command/:delcommand`, `UserCmd` for an invocation of a user-defined command, `Autocmd`, `Augroup`, and `Doautocmd` for the autocommand subsystem, and `ExCmd` for a `:/%-prefixed Ex command line with its optional range — the gateway to the :s/:g/:d/:sort/:normal machinery of Chapter 24. Each of these holds the raw command text to be re-parsed by its runtime, which is why the parser can recognize a much larger surface than the six ex-commands the reference documents with hover text. Reading the two enumerations together — the expression variants and the statement variants — is reading the grammar of the language as the parser sees it: two flat lists of constructs, the expression ones nested by precedence, the statement ones run in sequence, meeting wherever a statement holds an expression.`

The source splitter: where the command-line concerns live

Above `parse_stmt` sits a small cursor type, `Lines`, that does everything the lexer refused to. When it is constructed it runs three passes over the raw source. Pass zero collapses heredoc assignments: a `let x =<< [trim] [eval] MARKER` and the verbatim body lines that follow it are folded into a single synthesized `let x = [...]` list-literal line, with the body taken “VERBATIM from the raw source — before continuation-folding or bar-splitting — exactly as Vim’s `ea_getline` feeds `heredoc_get()`.” This is why a heredoc body can contain bars and quotes freely: it is captured before any of the splitting happens. Pass one joins line-continuation backslashes into the previous line. Pass two splits `|`-separated commands so that a block opener appearing anywhere on a line becomes its own logical line.

The bar splitter, `split_commands`, is the “string-/operator-/comment-aware” machinery earlier chapters credited to the lexer but which actually lives here. Its comment names the four cases where a `|` is *not* a command separator, matching Vim’s `do_one_cmd`: when it is inside a string, when it is part of the `||` operator, when it is backslash-escaped, and when it follows a `"` line comment. It tracks single-quote and double-quote state as it scans so that a bar inside `'a | b'` is left alone, and it stops splitting at a `"` that begins a comment. This is the piece that makes `let l = [1] | echo l` two statements and `if 1 | let g:msg = 'yes' | endif` a complete one-line block — and putting it in the parser rather than the lexer is what lets the lexer stay a pure expression tokenizer.

Block parsing

The block openers — `if`, `while`, `for`, `try`, `function` — are parsed by dedicated routines that read statements until they hit the matching terminator. `parse_if` loops over `(condition, body)` arms terminated by `elseif`, `else`, or `endif`, raising `E171: Missing :endif` if the file ends first. `parse_while`, `parse_for`, and `parse_try` follow the same shape with their own terminators and their own error codes (`E170`, `E690`, and so on), each faithfully numbered. `parse_function` is the fussiest: it must find the `)` that matches the parameter list’s `(` by tracking nesting depth, because a default value may itself contain parentheses (`a = abs(-7)`) or brackets, and it must separate each parameter from its default at the top-level `=` while not mistaking a `==` or `==~` inside a default expression for the separator. The result is the `Function` statement with its name, parameters, `(index, default)` pairs, body, and bang flag — everything Chapter 6 needed.

parse_stmt and the LSP

The parser exposes a per-line entry point, `parse_stmt`, that the LSP server calls (Chapter 17), and its design reflects the two uses it serves. When the interpreter parses a file, a parse error is fatal — the script cannot run if it does not parse. But when the LSP parses a file as you type, an error on one line should not blind it to the rest: the editor wants a diagnostic on the broken line *and* the symbols and completions from the good lines. So `parse_stmt` parses one logical line to one `Stmt` and reports an error confined to that line, letting the caller continue with the next. This is what lets the LSP run `parse_stmt` per line and surface a diagnostic at each broken line’s position while still walking the rest of the file for document symbols. Crucially, it is the *same* `parse_stmt` the interpreter uses — there is no separate, more-lenient editor grammar — so a line the LSP flags is a line that would genuinely fail to compile, and a line it accepts is one the interpreter accepts. The single-parser discipline is the same one Chapter 17 praised for the builtin corpus, applied to the grammar: one parser, two consumers, no drift between what the editor reports and what the interpreter runs. The per-line granularity is the only accommodation the parser makes for the editor, and it is an accommodation of *reporting*, not of *grammar* — the rules are identical, only the error containment differs. The whole statement layer, splitter and block parsers together, is what reduces a messy multi-line, bar-laden, continuation-folded `.vim` file to the clean `Vec<Stmt>` the compiler consumes next.

Chapter 21 – Lowering: From AST to fusevm Bytecode

The compiler, `compile_viml.rs`, is the stage where `vimlrs` stops being a language frontend and starts being a `fusevm` program. It walks the AST and emits a `fusevm::Chunk` — `fusevm`’s compiled-program type — and it is the single most consequential file for the speed claims in Chapter 1, because every decision about what becomes a native op and what becomes a builtin call is made here. This chapter is the anatomy of that walk: the division of labour that keeps semantics out of the compiler, the slot planner that decides which locals go fast, and the specific lowerings — loops, ranges, lambdas, operators — that let hot numeric code reach the JIT.

The division of labour

The compiler’s banner states a discipline that is the hinge of the whole architecture, and it is worth quoting exactly:

```
Lowens the synthesis AST to a fusevm::Chunk. Neovim has no bytecode compiler;
this is the net-new piece that makes VimL run on fusevm. Each expression
compiles to a sequence leaving one value on the stack; faithful VimL
semantics are never inlined here – every operator routes to a VIML_* builtin
whose handler calls the canonical ports.
```

“Faithful VimL semantics are never inlined here” is the rule. The compiler does not know what `+` means. When it lowers `a + b` it does not add — it emits a call to the `VIML_ADD` builtin, whose handler (in the bridge, Chapter 22) pops the two operands, converts them to ported `typval_T`, and calls the faithful arithmetic. The compiler’s job is “purely structural: evaluation order, control flow, scope/slot management, and the calling convention.” This is what keeps the two-zone boundary intact at compile time: the bytecode is synthesis, but every operation it names lands in a port. The output is a `CompiledProgram` — “the top-level `main` chunk plus the user functions it defines,” serialized as a unit into the script cache so a cache hit restores functions and all — where each `UserFuncDef` carries its name, parameters, compiled default-value chunks, bang flag, and body chunk.

The discipline that the compiler never inlines semantics is more important and more subtle than it first appears, because the tempting thing — the thing a less disciplined compiler would do — is exactly to inline the simple operations, and the temptation has to be resisted on principle even where yielding to it would seem harmless. Consider `a + b`. The compiler knows it is an addition; it would be easy, and faster, to just emit an add instruction and be done. But “addition” in Vimscript is not a single operation: `+` over two numbers is arithmetic, `+` over two lists is concatenation, `+` over a number and a list is an error, and the exact rules for which is which and what each produces live in the ported `eval5` logic. If the compiler inlined “addition,” it would have to encode those rules itself — it would have to know that lists concatenate and numbers add and the mixed case errors — and the moment it encoded them, the semantics of `+` would live in *two* places: the ported arithmetic and the compiler’s inlined version. Two copies of a semantic rule is two chances to disagree, and the whole faithfulness guarantee rests on there being exactly one place each rule lives, the ported one. So the compiler refuses to inline even something as simple as addition, emitting a `VIML_ADD` call instead, so that the meaning of `+` exists only in the ported zone and the compiler holds none of it. The compiler is deliberately ignorant of semantics, and its ignorance is what keeps the semantics single-sourced.

The numeric fast path is the one place this discipline bends, and the way it bends is instructive because it shows the discipline being relaxed *safely* rather than abandoned. The fast path emits a native `Op::Add` for

integer addition instead of a `VIML_ADD` call — which looks like exactly the inlining the compiler just refused. The difference is that the fast path is *guarded* and *fallible*: it emits the native op only when it can prove both operands are integers, and even then the native op is paired with a deopt fallback to the builtin, so if a value turns out not to be the integer the proof assumed, execution falls back to the faithful `VIML_ADD`. So the fast path does not duplicate the semantics of `+`; it duplicates only the *integer-addition* case of `+`, the case where the compiler can prove native and faithful coincide, and it keeps the builtin as the authority for every other case. This is inlining the part you can prove safe while keeping the port as the authority for the whole — a much narrower and more careful thing than inlining the operator. The rule “never inline semantics” and the exception “fast-path the provably-numeric cases” are not in tension; the exception is the rule applied with a proof obligation attached, optimizing only the slice where optimization cannot change the result. The compiler’s deliberate ignorance and its one careful exception are two views of the same discipline: hold no semantics, except the slice you can prove is identical to the port’s, and even there keep the port as the fallback.

The one exception: the numeric fast path

There is exactly one place the compiler breaks its own rule, and it breaks it on purpose. A few numeric operations have a *fast path* that emits native fusevm arithmetic instead of a builtin call, “guarded by a runtime number check, so the JIT can trace hot integer loops.” The banner is careful to justify why this is still faithful: integer `+/-/*` lower to native `Op::Add/Sub/Mul` because “`Value::Int` $\leftrightarrow$ Number typval is transparent at the VM-stack boundary” and “i64 wrap matches VimL’s `varnumber_T` arithmetic.” The slow path is always still there — non-integer operands “keep the dynamic builtin (`b_add` etc.) which is also the JIT deopt fallback” — so a fast-pathed operation that meets an unexpected type at runtime falls back to the faithful builtin rather than producing a wrong answer. This is the same principle Chapter 5 saw with division: optimize only where the optimization is provably semantics-preserving, and decline it otherwise. Integer `/` is exactly the decline — it “always stays on the builtin because fusevm `Op::Div` is float division,” while `%` does go native to `Op::Mod` “because fusevm’s integer mod matches the `num_modulus` port’s truncated-toward-zero sign rule.”

The slot planner

Whether a local can be fast-pathed at all depends on whether it lives in a fusevm *slot* or in the variable dict, and that is decided before any code is emitted, by `slot_plan`. Its doc comment is the clearest statement of the rule that makes Chapter 14’s “provably-Number locals” concrete:

```
Decide which bare function-local variables can live in fusevm slots.
Sound & conservative: returns empty (so nothing is slotted and behaviour is
unchanged) unless the whole body is free of anything that could reach a
variable by name dynamically – function/method calls (the callee may read a
global), :execute/:set, nested :function, :try, :for, or any :let target
other than a bare name. A name is slotted only if every assignment to it
provably evaluates to a Number (fixed-point over the set, so let s = s + i
keeps s a slot only while i is one too).
```

Two ideas are doing the work here. The first is the *bail*: if the body contains anything that could reach a variable dynamically by name — a call whose callee might read a global, an `:execute` that builds a command from a string, a `:try`, even a sibling `:for` at the wrong scope — the planner gives up and slots nothing, because it can no longer prove the slotted locals are invisible to that code. The second is the *fixed point*: a name is slotted only if every assignment to it provably yields a Number, and “provably” is computed to convergence, so `let s = s + i` keeps `s` numeric only as long as `i` is numeric too. The planner returns two sets — the slots, and the subset `int_slots` “proven always-Integer (the rest may hold Float)” — and the integer subset is what keeps `range()` bounds integer while the broader numeric set lets native `+/-/*` and comparisons accept either, because “fusevm’s `arith_int_fast` promotes int $\leftrightarrow$ float exactly like VimL.”

The slot key carries the load-bearing detail from Chapter 4. Inside a function, `l:name` is bare `name` — “legacy VimL has no closures, so both share a slot” — while every other prefix (`g:`, `s:`, `a:`, `b:`, `w:`, `t:`, `v:`) is “a distinct

dict-backed store and can't be slotted.” This is the precise mechanical reason a numeric loop over bare locals traces and one over `g`: aliases does not: the bare local gets a slot, the alias stays in the dict, and only slot reads and writes lower to the native `Op::GetSlot/SetSlot` the JIT can compile. Slotting is also disabled when exceptions add per-statement unwinds, since the unwind path needs the dict-backed names.

Loops, rotated for the tracer

A `while` loop is lowered in a shape chosen specifically for fusevm's tracing JIT. The compiler's comment explains the rotation:

```
Loop rotation: enter at the test, put the body first, and make the
condition the CONDITIONAL BACKEDGE (JumpIfTrue back to the body). This is
semantically identical to a top-tested while (the initial jump checks the
condition before the first iteration), but the only backward branch is the
test itself – the shape fusevm's tracing JIT records.
```

So the emitted layout jumps first to the condition at the bottom, falls through the body from the top, evaluates the condition, and uses a single `JumpIfTrue` back to the body as the only backward branch. A top-tested loop with a forward-branch exit in the middle would give the trace recorder a mid-body side-exit that aborts the trace; the rotated shape gives it one clean backedge. When the condition is provably numeric it emits a native compare so the whole loop is one traceable region; otherwise the condition routes through `VIML_truthy`, the deopt path. The payoff is the `FibIter` loop of Chapter 4 compiling to machine code — and it compiles because the compiler shaped it for the tracer, not because the tracer is clever about arbitrary loops.

range(N) without a list

The idiomatic `for i in range(N)` gets a special lowering that builds no list at all. The compiler recognizes a `for` whose iterable is a `range()` call of one to three arguments and emits “a native integer counter loop (rotated for the tracing JIT).” Its comment notes that `range()` is “evaluated once: the bound is hoisted into a hidden slot, as Vim materializes the list a single time,” so even a dynamic bound like `range(a:n)` is coerced once with the `VIML_TONUMBER` builtin in the loop prologue, leaving the body itself free of builtin calls. The counter uses a hidden slot — named with a control character so it can never collide with a user variable — and emits `SetSlot/GetSlot`, a `LoadInt` of the step, an `Op::Add`, and the loop test as `Op::NumLt` for the one-argument form (`i < n`) or `Op::NumLe` for the two-and-three-argument forms, with the `JumpIfTrue` backedge. Any iterable that is *not* a `range()` call falls back to the generic list-iteration path. This is why Chapter 11 could promise that the counting loop is as fast as a hand-written `while`: they lower to the same rotated, slot-based, builtin-free shape.

Lambdas, desugared and captured

A lambda is compiled to a hidden named function and a partial that binds its captures. The compiler generates a unique `<lambda>N` name, collects the lambda's free variables, and desugars the body. Its comment lays out the transform:

```
Desugar to an anonymous function <lambda>N(captures..., params...)
whose body binds each into the local scope (so each is referenced
by bare name, as lambdas allow) and returns the body expression.
Free variables of the body are captured BY VALUE here: the lambda
value is a Partial that pre-binds their current values.
```

The captured names become leading parameters of the generated function; the body is a sequence of `let` statements binding each capture from its argument, followed by a `return` of the body expression. The value the compiler emits at the lambda's source position is a call to the `function` builtin — `function('<lambda>N')` for a capture-free lambda, or `function('<lambda>N', [cap0, cap1, ...])`, a capturing Partial, when there

are captures. Which names are captured is decided by `collect_free_vars`, whose rule is exactly the one Chapter 8 tested: a name is capturable when it has no scope colon, or begins `a:`, or begins `l:` — the function-tied scopes — while the dynamic scopes “resolve globally when the lambda runs, so they are not captured.” Capture is by value at creation time, which is why `Const(1)` and `Const(2)` yield independent closures and why a captured `g:` read sees the current global, not the one in place at creation.

Compound assignment, desugared upstream

One lowering happens before the compiler ever sees the statement. Compound assignment — `+=`, `-=`, and the rest — is desugared in the *parser*, not here. The parser reads the operator character just before the `=` and rewrites `target op= rhs` into `target = target op rhs`, “reusing the same store path so it stays JIT-eligible.” By the time the compiler reaches a `Let`, it is always a plain store, and the slot fast path handles the numeric accumulator case with no special casing. This is the mechanical reason Chapter 4 could claim that `let i += 1` trace-compiles: it is literally `let i = i + 1` by the time it reaches the slot planner, indistinguishable from the spelled-out form.

The slot planner’s two sets and the fixpoint

The slot planner is worth a second, closer look, because its mechanism is the single thing that determines whether a loop is fast. It returns two sets, not one: the `slots` map from a local name to a dense slot index, and an `int_slots` set naming the subset “proven always-Integer (the rest may hold Float).” The two-set design is what lets an integer counter and a float accumulator share one trace — the counter is in `int_slots` so the `range()` bound and the loop test stay integer comparisons, while the accumulator is merely in `slots` so its arithmetic can promote to native `fadd`. The promotion is safe because “fusevm’s `arith_int_fast` promotes `int` $\leftrightarrow$ `float` exactly like VimL,” so a slot that holds an integer on one iteration and a float on the next still computes correctly on the native path.

The set is grown to a fixpoint, which is the subtle part. A name is slotted only if every assignment to it provably yields a Number, and “provably” is computed relative to the names already known numeric — so the planner sweeps the body repeatedly, adding a name when all its right-hand sides are numeric given the current set, until a sweep adds nothing. This is why `let s = s + i` keeps `s` a slot only while `i` is one too: if `i` ever takes a non-numeric value, it drops out of the set, which drops its consumers, which can cascade. The classifier that decides whether a right-hand side is numeric recurses over the expression — a number literal, an arithmetic node over numeric operands, a comparison (which is a 0/1 Number), a unary negation or logical-not of a numeric operand, a numeric ternary, and a small allowlist of numeric builtins (`len`, `str2nr`, `float2nr`, `abs`, `char2nr`, and the like) all count as provably numeric. Anything else — a string concatenation, a function call to a user function, a dict index — does not, and taints its target.

The bail conditions, precisely

The planner is “sound and conservative,” and its conservatism is a list of bail conditions that return the empty set so that “nothing is slotted and behaviour is unchanged.” The body must be free of anything that could reach a variable by name at runtime, because a slotted local is invisible to such a reach and the planner cannot prove the reach does not touch it. So a function or method call bails, because the callee might read or write a global. An `:execute` bails, because it builds a command from a string the planner cannot analyze. A `:set` bails. A nested `:function` definition bails. A `:try` bails, because the exception unwind path needs the dict-backed names. A `:for` whose lowering is not the native-range form bails. And any `:let` whose target is not a bare name — an index assignment, a list-unpack, an option, an environment variable, a register — bails, because those reach storage the slot machinery does not model. There is also a class of *introspection* bail: a body that calls `exists`, `eval`, `execute`, or `call` cannot be slotted, because those functions can name a variable dynamically. The effect of the whole list is that a loop traces only when its body is provably self-contained — pure arithmetic and control flow over bare numeric locals — and the moment it reaches outside that, the

planner declines and the loop runs correct-but-slow on the builtin path. This is the deopt boundary of Chapter 14 stated as the compiler sees it: a precise, conservative list of “if the body does X, give up slotting,” chosen so that a wrong guess is impossible and only a missed optimization is at stake.

The native range loop in detail

The `range()` fast path is the most elaborate single lowering, because it has to turn a list-building call into a counter loop with no list. The compiler recognizes a `for` whose iterable is a one-, two-, or three-argument `range()` and emits a counter loop “rotated for the tracing JIT,” exactly like the `while` of the previous section. The bound is “hoisted into a hidden slot” so that even a dynamic bound such as `range(a:n)` is coerced once, in the loop prologue, with the `VIML_TONUMBER` builtin — after which the body itself contains no builtin call and is fully traceable. The hidden counter slot is named with a leading control character (a `\u{1}` prefix) so it can never collide with a user variable. The loop test is emitted as `Op::NumLt` for the one-argument form (`for i in range(n)` counts `i < n`) and `Op::NumLe` for the two- and three-argument forms (which have an explicit inclusive-or-stepped bound), with the step added by an `Op::Add` of a `LoadInt` and the whole thing closed by the `JumpIfTrue` backedge. Any iterable that is not a `range()` call falls through to the generic list-iteration path, which materializes the list and iterates it. The reason this matters is the one Chapter 11 gave: `for i in range(N)` is the idiomatic counting loop, and lowering it to a builtin-free native counter is what makes the idiom as fast as a hand-written `while` rather than a list allocation followed by an iteration.

Lambda generation as a function plus a partial

The lambda lowering bears one more detail worth seeing, because it explains the runtime value a lambda becomes. The compiler generates a hidden function named `<lambda>N` whose parameter list is the captured free variables followed by the lambda’s own parameters, and whose body binds each capture into the local scope and returns the body expression. The *value* emitted at the lambda’s source position is then a call to the `function` builtin: `function('<lambda>N')` for a capture-free lambda, or `function('<lambda>N', [cap0, cap1, ...])` — a `Partial` — when there are captures. This is why a closure is a `VAR_PARTIAL` at runtime (Chapter 8): a lambda that closes over `a:n` becomes `function('<lambda>3', [a:n])`, a partial that pins the captured value as the first bound argument, and calling it later supplies the remaining arguments. The capture values are evaluated at the creation site, which is the mechanical reason capture is by value at creation time — the `[cap0, cap1, ...]` list is built then, from whatever the captured names hold at that moment, and frozen into the partial. A naive implementation that captured by reference would build the list lazily and read the names at call time; `vimlrs` builds it eagerly, which is exactly Vim’s semantics and exactly what `closures.vim` asserts.

Entry points and the debug variant

The compiler exposes a few entry points. `compile_program` lowers a whole script to a `CompiledProgram`; `compile_expr_only` lowers a single expression for the `-e` path; and `compile_program_debug` lowers a program that has been paired with source line numbers, emitting a `VIML_SET_LINENO` builtin call before each statement so the debugger (Chapter 22) can pause at breakpoints. An uncaught exception at top level is handled by emitting a `VIML_REPORT_UNCAUGHT` call, which sets the error flag and produces the non-zero exit Chapter 2 described. The result of any of these is a chunk — or a chunk plus functions — ready to hand across the last seam in the pipeline, into the bridge and onto `fusevm`, which is where the next chapter goes.

The three entry points exist because the compiler serves three callers with three needs, while sharing one core. `compile_program` is the file path: it lowers a whole script to a `CompiledProgram` — the `main` chunk plus the user functions and lambdas — which is what gets cached and run. `compile_expr_only` is the `-e` path: it lowers a single expression to a chunk that leaves one value on the stack, which the driver runs and prints, the form behind `vimlrs -e '1 + 1'`. And `compile_program_debug` is the debugger path: it lowers a program paired with source line numbers, emitting a `VIML_SET_LINENO` marker before each statement so

the DAP's `check_line` can pause at breakpoints. The three differ only in what they wrap around the same core lowering — the expression form wraps nothing, the program form wraps the function table, the debug form wraps the line markers — so the slot planner, the loop rotation, and the operator lowering are identical across all three. A debugged run and a normal run compile to the *same* bytecode except for the interleaved line markers, which is exactly why the debugger inspects the genuine program (Chapter 17): it is the genuine program, with markers added, not a separate interpretation. The entry points are thin wrappers; the compiler underneath them is one machine, and that singularity is what keeps the `-e` evaluator, the file runner, and the debugger all running the same language.

Chapter 22 – The Bridge, the Disassembler, and `VIML_*` Builtins

The compiler emits bytecode that *names* operations; something has to make those names do the right thing on the actual fusevm stack. That something is the bridge, `fusevm_bridge.rs`, the largest file in the crate and the place where the two zones finally meet at runtime. This chapter is the bridge — how it marshals values between fusevm and the ported engine, how it registers the `VIML_*` handlers that delegate to the ports, and the re-entry hooks that let ported code call back into the VM — plus the small disassembler that lets you watch the whole machine work.

Pure plumbing, no semantics

The bridge's banner makes the same promise the compiler's did, from the other side:

```
The vimlrs analogue of zshrs's fusevm_bridge.rs: pure fusevm plumbing, no
operator logic. It only 1. converts between typval_T and fusevm::Value, 2.
holds the per-run bridge state, 3. registers the VIML_* builtin handlers,
each of which pops operands and calls the CANONICAL PORTS in
crate::ported::* (never reimplementing VimL semantics here).
```

So the compiler holds no semantics and the bridge holds no semantics; the semantics live entirely in `ported/`. The bridge is the connective tissue, and its three jobs — convert, hold state, register handlers — are exactly the three things you need to run ported `typval_T` logic on a borrowed `fusevm::Value` stack. Where a handler has to reconstruct logic the bytecode replaced — the `eval5/eval6` arithmetic dispatch, the `eval7` unary application, the index subscripts — it cites the `eval.c` lines it mirrors, because those particular tree-walkers were not ported (the bytecode replaced them) and the citation keeps even the bridge honest about its origins.

The seam where the two zones meet at runtime

The bridge is the place where the book's two-zone architecture stops being a static layout of files and becomes a dynamic interaction at runtime, and it is worth dwelling on what actually happens at that seam when a program runs, because it is the mechanism that lets faithful semantics execute on borrowed bytecode. Picture a single operation, `a + b`, as it runs. The compiler, in the synthesis zone, has already lowered it — to a native `Op::Add` if both operands were provably integers, or to a `CallBuiltin` for the `VIML_ADD` builtin if not. Take the builtin case, the faithful one. The VM, executing borrowed fusevm bytecode, reaches the `CallBuiltin` and invokes the handler the bridge registered for `VIML_ADD`. That handler — `b_add` — is the seam: it is bridge code, synthesis-zone plumbing, but its only job is to pop the two operands off the VM stack, convert them across the value boundary from `fusevm::Value` to ported `typval_T`, and then call into the ported zone, `crate::ported::eval`'s arithmetic, which decides whether this is a list concatenation or a numeric add and computes the faithful result. The result comes back as a `typval_T`, the handler converts it to a `fusevm::Value`, pushes it onto the stack, and returns to the VM. So the operation crossed the seam twice — synthesis-zone VM to ported-zone arithmetic and back — and the semantics were computed entirely in the ported zone, while the dispatch, the stack management, and the value marshalling were entirely in the synthesis zone. The two zones met, did their respective jobs, and parted, for one `+`.

This is the architecture's central claim made operational: the bytecode is synthesis (no C counterpart), but

every *operation* it performs lands in a faithful port. The compiler never inlined what `+` means; it emitted a call to a builtin whose handler delegates to the port. The bridge holds no operator logic of its own; it is the connective tissue that lets the VM reach the port. And the port computes the meaning, faithfully, with its exact C name and citation. Multiply this single operation by the thousands an ordinary script performs, and you have the runtime picture: a stream of bytecode flowing through the borrowed VM, with each operation that has a faithful meaning dipping across the seam into the ported zone to compute that meaning, and each operation the JIT could prove numeric staying on the native fast path that bypasses the seam entirely. The seam is busy — it is crossed on every dynamic operation — which is why the refpool and the conversion functions are written for speed, and why the JIT's fast path matters so much, because the fast path is precisely the set of operations that do *not* need to cross the seam. The bridge is the largest file in the crate because it is where the two halves of the project, the faithful semantics and the borrowed machine, are stitched together operation by operation, and the stitching is the thing that makes “own the semantics, borrow the machine” run rather than merely describe.

The value boundary and the refpool

The single place the two value worlds meet is a pair of conversion functions, `tv_to_value` and `value_to_tv`. Scalars convert directly: a `VAR_NUMBER` typval becomes a `Value::Int`, a `VAR_FLOAT` a `Value::Float`, a `VAR_BOOL` a `Value::Bool`, a string a `Value::str`. Compound values — Lists, Dicts, Blobs, Funcrefs — are handled by what the bridge calls the “refpool smuggle”: rather than deep-copying a whole list onto the VM stack, the bridge stashes the `typval_T` in a thread-local pool and puts a small reference handle on the stack in its place. A `Value::Ref` carrying the pool index is what travels through the bytecode, and `value_to_tv` looks the real value back up when a handler needs it. This is what lets a Vimscript List flow through fusevm — which knows nothing about Vim's value types — without fusevm ever having to understand it: from the VM's point of view it is just an opaque reference, and only the `VIML_*` handlers, on the ported side of the boundary, ever unwrap it.

The refpool design has a performance payoff worth naming. A naive bridge would convert a whole List to some fusevm-native representation every time it crossed the boundary, deep-copying the structure onto the VM stack and back — and since a script touches its lists constantly, that copying would dominate. The refpool avoids it entirely: a compound value is stashed once and travels as a small integer handle, so passing a thousand-element list to a builtin costs a handle copy, not a thousand-element copy. The handle is stable for the value's lifetime, which is also what makes Vim's reference semantics work — `let b = a` for two lists makes `b` and `a` share the same pooled value, so `a is b` is true and a mutation through one is visible through the other, the identity behaviour `compare.vim` asserts. The scalars, by contrast, are cheap to copy and so cross the boundary by value: a `VAR_NUMBER` becomes a `Value::Int`, a `VAR_FLOAT` a `Value::Float`, a `VAR_BOOL` a `Value::Bool`, with no pooling needed. So the boundary treats scalars and containers differently for exactly the right reason — scalars are small and value-like, containers are large and reference-like — and the refpool is what lets the reference-like ones stay reference-like across a VM that has no notion of them. It is the single cleverest piece of the bridge, and it is why a language with rich heap values can run on a VM designed without them.

Registering the handlers

Everything is wired up in one function, `install`, called once per run before execution. It enables fusevm's tracing JIT (unless `VIMLR5_NO_JIT` is set, the flag Chapter 14 documented), seeds the `v:` variables, installs the re-entry hooks, and registers every `VIML_*` handler against the integer id the compiler emits. The handlers come in two styles. The operator handlers are hand-written and cite their C origin: `b_add`, the handler for `VIML_ADD`, pops two operands, checks whether it is a List-or-Blob concatenation or a numeric add (citing `eval5`, “+ is List+List / Blob+Blob concat, else numeric”), and calls the appropriate port. The builtin-function handlers share a generic wrapper, `call_func`, that “bridges builtin-function ids straight to ported `f_*` functions”: it pops the argument count, reverses them into argument order, pre-initializes the return value to the Number 0 exactly as Neovim's `call_func` does, calls the ported `f_*`, and converts the result back. So `submatch()` is

registered as `call_func` over the ported `f_submatch`, `len()` over `f_len`, and so on — hundreds of builtins reach their faithful implementation through one small wrapper. This is the seam Chapter 14 called “own the semantics, borrow the machine,” made concrete: the machine is fusevm’s register-and-dispatch, the semantics are `crate::ported::*`, and `call_func` is the ten-line bridge between them.

The `install` function is the single setup routine that wires all of this up before a run, and the order of its work is worth seeing because it is the run’s prologue. It enables fusevm’s tracing JIT — unless `VIMLR5_NO_JIT` is set, the flag Chapter 14 documented, in which case it leaves the JIT off so the interpreter baseline runs. It seeds the `v:` variables, so `v:true`, `v:false`, `v:null`, and the engine-maintained ones like `v:errors` exist before any code reads them. It installs the re-entry hooks — the filter/map hooks, the substitute `\=` hook, the sort-comparator hook — by filling the thread-local function pointers the ported zone calls back through. And it registers every `VIML_*` handler against its integer id, the operator handlers by hand and the builtin-function handlers through `call_func`. After `install` returns, the VM is a fully-wired vimlrs engine: JIT armed, specials seeded, hooks live, handlers registered. The whole thing is called once per run, which is why the bridge can be stateless between runs and thread-local within one — `install` builds a fresh engine for each invocation, and the per-run state lives in the thread-locals it borrows. This per-run setup is also what makes the debugger’s executor thread work: it runs `install` on its own thread, so it has its own fully-wired engine with its own thread-locals, which the DAP thread then inspects across the boundary. One setup routine, run once, turns a bare fusevm VM into a vimlrs interpreter, and everything this chapter described — the marshalling, the handlers, the state, the hooks — is what it installs.

The per-run state

The bridge holds the standalone analogue of Vim’s global eval state in a set of thread-locals borrowed by every handler. The refpool is one. The others include the user-function registry (`FUNCTIONS`), the echo sink (which is `Some(buffer)` when output is being captured for `execute()` or a test, and `None` when it goes to stdout), the pending-exception cell (which “models `ex_eval.c`’s `current_exception`”), the `v:exception` value, and the `v:val/v:key` cells that `map/filter` set per element. There is also the in-memory buffer and cursor that Chapter 24 is about, the registers shared with `:yank/put`, and the `did_emsg` flag the CLI reads to decide its exit code. All of it is per-run and thread-local, which is what lets the debugger run a script on a dedicated executor thread (Chapter 17) while the DAP thread inspects it: the state belongs to the executor, and the inspector reads it across the boundary.

Re-entry: when ported code calls back into the VM

The most intricate part of the bridge is the set of hooks that let *ported* code — which has no VM handle — call *back* into the compiled VM. Chapter 6 and Chapter 9 mentioned these in passing; here is the mechanism. The ported `f_map/f_filter` are faithful, but evaluating a per-element body (`'v:val * 2'`, a funcref, a lambda) requires the VM, so the bridge installs `FILTER_MAP_EVAL_HOOK` — a thread-local the ported list code calls, which sets `v:val/v:key` and evaluates the expression or invokes the funcref. A companion `FILTER_MAP_CMD_HOOK` handles the command-string form. The `substitute \=` replacement works the same way: the regex engine holds a `SUBST_EXPR_HOOK` that the bridge fills with a function that compiles and runs the replacement expression once per match, while `submatch()` reads the current capture groups from thread-local match state. There are several more in the same family — a sort-comparator hook for funcref comparators, a function-call hook, an eval-string hook, a function-existence hook — all installed together in `install`. Each is the same shape: a thread-local function pointer that the ported zone can call without ever depending on the synthesis zone directly. It is a deliberately loose coupling — the ports do not know fusevm exists, they only know there might be a hook — and it is what lets the standard higher-order functions feel native while keeping their implementations in the faithful zone.

The debug line hook

One handler exists only for the debugger. `VIML_SET_LINENO`, emitted before every statement by `compile_program_debug`, is “a debug line marker: pop a line number, notify the DAP `check_line` hook.” Its handler reads the line number off the stack and calls `crate::dap::check_line`, which (under `--dap`) consults the breakpoint set and condvar-waits to pause; with no debugger attached it is a cheap no-op. This is the whole mechanism behind Chapter 17’s breakpoints: the compiler sprinkles line markers, the bridge turns each into a `check_line` call, and the DAP layer decides whether to stop. Because the markers are ordinary builtin calls, they cost nothing structurally — they are just one more `VIML_*` id — and they are only emitted in the debug compile path, so a normal run never pays for them.

The operator handlers, citing their C

The operator handlers are the part of the bridge that does the most reconstruction, because they re-implement the dispatch logic the bytecode replaced — and because that logic was *not* ported (there was no tree-walker to port), each handler cites the `eval.c` lines it mirrors so the bridge stays as honest as the ported zone. `b_add`, the handler for `VIML_ADD`, is the clearest example: it pops two operands and decides, citing `eval5`, whether this is a List-or-Blob concatenation (“+ is List+List / Blob+Blob concat”) or a numeric add, dispatching to the ported `typval` arithmetic in the numeric case and to the list or blob concatenation in the other. The sibling handlers `b_sub`, `b_mul`, `b_div`, and `b_mod` cite `eval6` and route to `num_divide/num_modulus`; `b_concat` cites the `./.` rule and joins as strings; `b_compare` cites `eval4` and calls the ported `typval_compare` with the operator and case flag the compiler baked in; and the unary handlers cite `eval7`. The index and slice handlers reconstruct the subscript logic of `eval7`’s postfix tail. None of these inline a *new* meaning for the operator — each is a thin dispatcher that chooses among ported operations — but because they encode evaluation-order and type-dispatch decisions that the original interpreter made inline, the citations keep them anchored to the C, so even the bridge’s reconstructed dispatch can be checked line for line against Neovim.

The index and slice handlers are worth singling out, because subscripting is one of the operations whose meaning depends most on the operand’s type, and the handler has to dispatch on it. `VIML_INDEX` (the handler for `base[i]`) must do different things for a List (return the element at the integer index, with negative indices counting from the end), a Dict (return the value for the key), a String (return the character at the index), and a Blob (return the byte value) — four behaviours behind one operator, each citing the `eval7` postfix logic it mirrors. The slice handler `VIML_SLICE` (for `base[a:b]`) is similarly polymorphic, with the inclusive-end semantics for the index form that contrasts with `slice()`’s exclusive end. These handlers are where the type system meets the operator syntax: the same `[i]` in the source compiles to the same `VIML_INDEX` call, and the handler decides at runtime what `[i]` means based on what `base` evaluated to. This is why the compiler does not need to know the operand type — it emits one `VIML_INDEX` call and lets the handler, on the ported side, dispatch on the actual `typval_T`. The polymorphism lives in the handler, the syntax in the compiler, and the boundary between them is one builtin id — which is the same division-of-labour the whole bridge embodies, here applied to the most type-dependent operator in the language.

call_func: one wrapper for the builtin table

The hundreds of builtin functions are not hand-written handlers; they share one generic wrapper, `call_func`, which “bridges builtin-function ids straight to ported `f_*` functions.” Its discipline mirrors Neovim’s own `call_func`: it pops the argument count off the stack, reverses the popped values back into argument order, pre-initializes the return value to the Number `0` exactly as Neovim does (so a builtin that does not set a result returns 0, the Vim default), invokes the ported `f_*` function with the argument array and the return slot, and converts the result back across the value boundary. So `f_len`, `f_type`, `f_substitute`, `f_sort`, `f_matchfuzzy`, and every other catalogued builtin reach their faithful implementation through this one ten-line wrapper, registered against the builtin id the compiler emits. This is the structural reason the builtin surface can grow function by function with no per-function bridge code: porting a new builtin means writing its `f_*` in the

ported zone and adding one `call_func` registration, and the arity check of Chapter 6 — validated against the same `funcs[]` metadata — happens in the wrapper before the ported function runs. The bridge’s smallness here is the payoff of the discipline: the semantics live in `f_*`, the marshalling lives in `call_func`, and the two meet at exactly one place.

The thread-local state in detail

The per-run state the handlers borrow is a set of thread-locals, and naming them shows how much of “the engine” is really just this borrowed context. The `refpool` (`REFPOOL`) holds the compound `typval_T` values the VM stack references by handle. `FUNCTIONS` is the user-function registry. The echo sink (`ECHO_SINK`) is `Some(buffer)` when output is being captured — for `execute()`, for a test, for the DAP’s output events — and `None` when output goes straight to stdout, which is how the same `:echo` handler serves both the normal run and the captured run. `PENDING_EXC` is the cell that “models `ex_eval.c`’s `current_exception`,” the in-flight exception an unwinding `:try` threads through `:finally` to `:catch`; the companion `v:exception` value is read inside a handler. `V_VAL` and `V_KEY` are the cells `map/filter` set per element so the body expression can read `v:val` and `v:key`. And the `did_emsg` flag is the one the CLI reads to choose its exit code. All of it is thread-local and per-run, which is what lets the debugger’s executor thread own a complete engine state while the DAP thread inspects it across the boundary — the inspection reads the executor’s own thread-locals, not a copy, which is why an evaluated expression at a breakpoint sees the genuine live program.

Seeing it: the disassembler

The disassembler, `fusevm_disasm.rs`, is the window into all of this. It is “a thin pretty-printer over fusevm’s `Chunk` for `vimlrs --disasm`,” printing a listing to stdout before the VM runs. For each chunk it prints a context header, the chunk’s source, its name table (the slot and variable names), its sub-entries (the functions and lambdas it defines, each with the instruction offset where it begins), and then one line per instruction — a four-digit index, a right-aligned source line number, and the opcode’s debug rendering — before recursing into the nested sub-chunks so function and lambda bodies print indented beneath their parent. Running `vimlrs --disasm fib.vim` shows you the rotated `while` loop as its `Op::GetSlot`, `Op::Add`, `Op::NumLt`, and `Op::JumpIfTrue` instructions, the `VIML_* CallBuiltin`s for everything dynamic, and the `<lambda>N` sub-chunks laid out in full. It is the ground truth for everything the last two chapters described: when a loop you expected to trace is full of `CallBuiltin` where you wanted `Op::Add`, the disassembler is where you see why, and the slot planner’s bail conditions are usually the answer.

Reading a listing

A listing has a stable shape, and learning to read it turns the abstract pipeline of the last chapters into something concrete. Each chunk begins with a context header, then prints its source line, its name table (the slot and variable names the chunk uses), its sub-entries (the functions and lambdas it defines, each with the offset where its body begins), and then one instruction per line — a zero-padded index, a right-aligned source line number, and the opcode’s debug rendering — before recursing into the nested sub-chunks. Reading the listing for a traced numeric `while` loop, the shape is unmistakable: a `Jump` to the condition near the bottom, then the body’s `GetSlot/SetSlot` accesses interleaved with `Add`, `Sub`, or `Mul` for the integer arithmetic, then the condition’s `NumLt` (or `NumLe/NumGt/...`) followed by a `JumpIfTrue` that branches back to the top of the body — the rotated, single-backedge shape the tracing JIT records. Where the body does something dynamic, you see a `CallBuiltin` naming the `VIML_*` id instead: a `CallBuiltin VIML_CONCAT` for a string concatenation, a `CallBuiltin VIML_DIV` for an integer division (which never goes native), a `CallBuiltin` for a user-function call. The contrast between a run of native ops and a `CallBuiltin` is exactly the contrast between the fast path and the deopt path, and the listing makes it visible at a glance. A function body that block-compile ends in a native `ReturnValue`; one that does not ends in a `CallBuiltin` to the return handler. So the disassembler is not just a debugging aid but a teaching tool: run `vimlrs --disasm` on any of the example files and the

whole architecture — the slots, the rotated loops, the native ops, the builtin fallbacks, the nested function sub-chunks — is laid out as text you can read top to bottom.

Chapter 23 – Numbers, Floats, and the Arithmetic Port

Chapter 5 covered the type system and the two coercions most likely to bite — integer division and string-to-number. This chapter goes deeper into the numeric tower, because numbers are where vimlrs’s faithfulness is most testable: a number either comes out bit-for-bit right or it does not, and the example suite asserts the exact results across literals, rendering, the float math library, and the arithmetic edge cases that diverge from intuition. Everything here is grounded in four example files — `numbers.vim`, `float_literals.vim`, `floatmath.vim`, and `numeric_edge.vim` — each a CI-checked self-test.

Integer literals and their bases

Chapter 20 showed how the lexer reads integer bases; here is what the language guarantees as a result. A leading `0` with only octal digits is octal, `0x` is hex, `0b` is binary, and `0o` is explicit octal. The `numbers.vim` example asserts the whole set at once:

```
call assert_equal(8, 010)
call assert_equal(511, 0777)
call assert_equal(129, 0129)
call assert_equal(255, 0xff)
call assert_equal(5, 0b101)
call assert_equal(15, 0o17)
```

The interesting member is `0129`, which is `129` and not an error: the `9` is not an octal digit, so the leading-zero octal rule does not apply and the literal is read as decimal. This is precisely `vim_str2nr`’s octal detection — a leading `0` makes a number octal *only if every following digit is octal* — and vimlrs reproduces it because it ports the detector rather than approximating it. A Number is 64-bit (`has('num64')` is true), which is why `FibIter(50)` in Chapter 19 can return `12586269025` exactly rather than overflowing a 32-bit slot.

The float literal needs its dot

A Float literal is stricter than a casual reader expects, and `float_literals.vim` documents the rule in its header: Neovim’s float grammar is digits, a mandatory dot, more digits, and an optional exponent, so “the dot is mandatory, so `1.0e10` is a Float but `1e10` is the Number 1 followed by a name.” The example asserts both the acceptances and the rejection:

```
call assert_equal(1500.0, 1.5e3)
call assert_equal(0.002, 2.0e-3)
call assert_equal(1, 1.0e10 == 10000000000.0)
call assert_fails('echo string(1e3)', 'E15')
```

`1.5e3` is `1500.0` because it has the mandatory fraction; `1e3` raises `E15` because it lexes as the number `1` followed by the bareword `e3`, which is not a valid expression. A more permissive interpreter would quietly accept `1e3` as a float and be subtly incompatible with every script that relies on the dot rule; vimlrs declines, and the example proves it declines.

Float rendering: %g, and the negative zero

A Float renders through C's `printf("%g")` — six significant digits — and vimlrs matches Neovim's printer, which differs in one respect from Vim 9.x's: it appends `.0` when the value is integral and has no dot or exponent in its `%g` form. The `numbers.vim` and `float_literals.vim` examples assert the rendering exactly:

```
call assert_equal('0.3', string(0.1 + 0.2))
call assert_equal('1.0', string(1.0))
call assert_equal('0.333333', string(1.0 / 3.0))
call assert_equal('1e+06', string(1000000.0))
call assert_equal('1.23457e+08', string(123456789.0))
call assert_equal('0.0001', string(0.0001))
```

Two details earn their keep. First, `string(0.1 + 0.2)` is `'0.3'`, not the `0.30000000000000004` that haunts naive float printing, because `%g`'s six significant digits round it cleanly — and because `json_encode` uses the same renderer, `json_encode({'x': 0.5})` is `'{"x":0.5}'`. Second, `%g` switches to exponent form by magnitude: `1000000.0` prints as `1e+06` and `123456789.0` as `1.23457e+08`, while `0.0001` stays decimal. And the IEEE negative zero keeps its sign through `%g`: `float_literals.vim` asserts `string(-0.0)` is `'-0.0'` and `string(0.0 / -1.0)` is `'-0.0'`, while positive `string(0.0)` is `'0.0'`. These are the kind of distinctions that only fall out of using the real `%g` rather than a hand-rolled formatter.

The float math library

The `floatmath.vim` example exercises the transcendental library — the ported `funcs.c` float functions — and its testing technique is itself instructive. Exact float identities are asserted directly:

```
call assert_equal(3.0, floor(3.7))
call assert_equal(-4.0, floor(-3.1))
call assert_equal(4.0, ceil(3.2))
call assert_equal(3.0, round(2.5))
call assert_equal(-3.0, round(-2.5))
call assert_equal(3.0, trunc(3.9))
call assert_equal(1.0, cos(0.0))
call assert_equal(4.0, sqrt(16.0))
```

The four directed-rounding functions each round toward a different target — `floor` down, `ceil` up, `round` to nearest (with `.5` going away from zero, so `round(2.5)` is `3.0` and `round(-2.5)` is `-3.0`), and `trunc` toward zero — and each returns a Float, while `float2nr` truncates toward zero into an actual Number. For the transcendentals, where the last bit of a result depends on the platform's libm, the example compares fixed-point integers instead of floats so the assertions stay exact regardless of the final ULP:

```
call assert_equal(1570796, float2nr(round(asin(1.0) * 1000000)))
call assert_equal(785398, float2nr(round(atan(1.0) * 1000000)))
call assert_equal(2718, float2nr(round(exp(1.0) * 1000)))
```

`asin(1.0)` is $\pi/2$, scaled by a million and rounded to `1570796`; `atan(1.0)` is $\pi/4$, `785398`; `exp(1.0)` is e , `2718` at three places. This is a careful way to test a math library faithfully without pinning the assertions to one platform's rounding, and it is worth noting because it shows the example suite is engineered, not merely written.

Mixed promotion: when integers meet floats

Arithmetic that mixes integers and floats follows Vim's promotion rules, and vimlrs gets them right because the ported arithmetic and the native fast path agree on them. When one operand of `+`, `-`, or `*` is a Float, the result is a Float — `3 + 2.5` is `5.5`, an integer promoted to float for the operation. The native fast path preserves

this: the README notes that “fusevm’s `arith_int_fast` promotes `int` ↔ `float` exactly like VimL,” so a slot that holds an integer on one iteration and a float on the next still computes correctly, which is what lets an integer counter and a float accumulator share one JIT trace (Chapter 14). Division is the exception that proves the rule: integer `/` integer is *integer* division (`7 / 2` is 3), but if either operand is a Float the division is floating-point (`7 / 2.0` is 3.5) — and because the native fast path cannot tell which at compile time and fusevm’s native divide is always float, integer `/` stays on the builtin to preserve the truncating semantics, while a float divide could safely go native. `float2nr({expr})` is the explicit demotion, truncating a Float toward zero to a Number, and `abs({expr})` preserves the type — “Float in -> Float out.” These promotion rules are small but they are everywhere arithmetic appears, and a from-scratch implementation that promoted differently — say, making `7 / 2` a float — would be subtly incompatible with every script that relies on integer division, which is exactly the trap Chapter 5 opened on and exactly why vimlrs ported the arithmetic rather than borrowing fusevm’s.

Non-finite values

The float predicates handle infinities and NaN exactly, and `floatmath.vim` checks the sign-bearing `isinf`:

```
call assert_equal(1, isnan(0.0 / 0.0))
call assert_equal(1, isinf(1.0 / 0.0))
call assert_equal(-1, isinf(-1.0 / 0.0))
call assert_equal(0, isinf(1.0))
```

`isnan` is a plain predicate, but `isinf` is three-valued: `1` for positive infinity, `-1` for negative, `0` otherwise. The reference’s one-line doc says exactly that — “1 if {expr} is +inf, -1 if -inf, else 0” — and the example asserts both signs. A divide producing `0.0 / 0.0` is a NaN and `1.0 / 0.0` an infinity, both of which arise here because *float* division does produce IEEE results, in contrast to integer division’s truncation.

The arithmetic edges that diverge

`numeric_edge.vim` collects the cases where Vimscript’s arithmetic diverges from what a C or Python programmer would assume, and they are all ported behaviours. The sharpest is that `%` is integer-only:

```
call assert_equal(1, 7 % 3)
call assert_equal(-1, -7 % 3)
call assert_fails('echo 1.0 % 2.0', 'E804')
```

Integer modulo works and follows the truncated-toward-zero sign rule, so `-7 % 3` is `-1`, not `2` — and this is the same sign rule fusevm’s native `Op:Mod` uses, which is why Chapter 21 could fast-path `%` to a native op while keeping `/` on the builtin. But a *float* operand to `%` is an error, `E804`, even though `*` and `/` happily accept floats. That asymmetry is pure Vim: modulo is defined only on integers, and vimlrs raises the faithful `E804` rather than silently computing a float remainder. The example also pins `str2float`’s `strtod` semantics, including that it parses hex the way C’s `strtod` does:

```
call assert_equal(31.0, str2float('0x1f'))
call assert_equal(-16.0, str2float('-0x10'))
call assert_equal(1500.0, str2float('1.5e3'))
```

And it checks that `charidx` maps a byte index to a character index without crashing on a byte that falls *inside* a multibyte character — `charidx('héllø', 2)` is `1`, landing on the same character as byte 1, because byte 2 is the second byte of the two-byte `é`. These are exactly the corners a from-scratch interpreter gets wrong, and they are right in vimlrs because the arithmetic, the conversions, and the indexing are all ported functions with documented behaviour, asserted by a test that runs every day.

Why numbers are the strongest test

It is no accident that this book keeps returning to numeric examples when it wants to demonstrate fidelity. Numbers are the strongest test a port can face, because their answers are not approximate. A string function can be “mostly right” and pass casual use; a number is either `12586269025` or it is not, `'0.3'` or it is not, `-1` for `-7 % 3` or it is not. There is no rounding-down of a wrong answer to make it look acceptable, and the example suite exploits this by asserting exact values everywhere — exact integers, exact `%g` renderings, exact fixed-point transcendentals, exact error codes like `E804` and `E805`. When `float_literals.vim` asserts `string(-0.0)` is `'-0.0'` and `string(0.0)` is `'0.0'`, it is checking a single sign bit, and a port that got the negative zero wrong would fail loudly. The numeric examples are, in effect, the highest-resolution measurement of how faithful the port is — and the fact that they pass, down to the last sign bit and the last error code, is the most precise evidence this book can offer that “ported from the Neovim C eval engine” means exactly what it says.

Numbers and the JIT, together

The numeric tower and the JIT meet in the loops that compute over numbers, and it is worth closing the chapter by seeing how the two reinforce each other. The faithfulness constraint shapes the JIT: integer `+/-/*` go native because the ported arithmetic and fusevm’s integer arithmetic agree (both wrap a 64-bit integer the same way), `%` goes native because the sign rules match, but `/` stays on the builtin because they disagree (fusevm divides as float, VimL truncates). So the very faithfulness this chapter is about is what decides which numeric operations the JIT may accelerate — an optimization is taken only where it provably preserves the ported semantics. The payoff runs the other way: because the numeric operations that *can* be accelerated are, a loop full of integer arithmetic over bare locals — `FibIter`’s accumulator, a counting loop, a numeric reduction — compiles to native machine code, and the 64-bit integers it computes over are exact, so `FibIter(50)` returns `12586269025` without overflow. The numeric tower gives the JIT correct operations to accelerate, and the JIT gives the numeric tower the speed that makes a faithful arithmetic worth having in a tight loop. Neither would be as good alone: a fast loop over wrong arithmetic would be useless, and a correct arithmetic that could never be accelerated would leave the language slow. vimlrs has both, and the seam between them — the per-operator decision to go native or stay on the builtin — is where the whole design philosophy of “own the semantics, borrow the machine” is visible at the finest grain.

Chapter 24 – The Virtual Buffer and the Ex-Command Layer

Chapter 18 made the case for the editor-absent contract: vimlrs answers honestly for the editor it does not have, returning the documented empty value for buffer, window, and job queries. That chapter told the truth, but not the whole truth. There is one piece of the editor vimlrs *does* implement — a single in-memory buffer — and it is enough to make a large and surprising slice of Vimscript’s line-editing vocabulary work standalone: the Ex commands with line ranges, the `:substitute` family, `:sort`, the shift and shell-filter commands, a bounded `:normal`, marks, registers, search, and user-defined commands. This chapter is that layer, because it is the part of vimlrs most likely to surprise a reader who took Chapter 18 to mean “no editor at all.”

One buffer, really there

The bridge’s per-run state (Chapter 22) holds “the in-memory buffer lines (so `setline/getline/s/:g/:normal` work)” and a cursor into them. This is a real data structure, not a stub: `setline` writes lines into it, `getline` reads them back, and the line-range Ex commands operate on it. The `options.vim` example shows the buffer responding to a query that Chapter 18 might have led you to expect would be empty:

```
call setline(1, ['x3', 'y10', 'z7'])
let bi = getbufinfo()[0]
call assert_equal(1, bi.bufnr)
call assert_equal(3, bi.linecount)
call assert_equal(1, bi.loaded)
```

`getbufinfo()` describes the single virtual buffer — buffer number 1, three lines, loaded — because there genuinely is a buffer with three lines in it. The distinction with Chapter 18 is precise and worth stating: the *multi-buffer*, *window*, and *tab* introspection (`bufnr('%')`, `winnr`, `winheight`) reports absence, because there is no editor managing many buffers in windows; but the *single current buffer’s content* is real, because line-editing operations need somewhere to edit. vimlrs is not an editor, but it has a buffer.

The single buffer’s identity is consistent across the functions that report on it, which is what makes the contract coherent rather than a patchwork. `getbufinfo()` returns a one-element list describing buffer number 1 with its line count and a `loaded` flag of 1; `line('$')` reports the same line count; `getbufline(1, 1, '$')` returns the same lines `getline(1, '$')` does; and the buffer-numbered content functions, asked about buffer 1, operate on the same lines. So a script that mixes `getbufinfo()[0].linecount`, `line('$')`, and `getline` sees one coherent buffer, not three independent answers. The buffer always has at least one line — a fresh buffer reads as one empty line, which `windows.vim` checks — because Vim’s buffer model has no zero-line state, and `deletebufline` will not delete the buffer’s last line out of existence. This consistency is the difference between a virtual buffer that is a real data structure and a collection of functions that each fake an answer: there is one buffer, with one line count and one set of lines, and every function that touches it touches the same one. That is why the editing examples can mix `setline`, `:%s`, `:normal`, `search`, and `getbufinfo` freely and get coherent results — they are all views of a single, genuinely-present buffer.

Ex commands with line ranges

The `excommands.vim` example is the clearest demonstration, and its header states the scope: these are “buffer-editing Ex commands with line ranges (ported from Neovim’s `ex_cmds.c` / `ex_docmd.c`): `:\[range\]s/g/v/d/m/t/j/y/pu`. They run on the in-memory buffer, so they work fully standalone.” A `:`-prefixed or `%`-prefixed line is an Ex command; the substitute, global, delete, move, copy, join, yank, and put commands all resolve their ranges against the buffer:

```
call setline(1, ['foo a foo', 'bar', 'foo c'])
:%s/foo/X/
call assert_equal(['X a foo', 'bar', 'X c'], getline(1, '$'))
```

`:%s/foo/X/` substitutes the first match on every line by default, and the `g` flag makes it every match. The deletes and the structural commands work the same way: `:2,4d` deletes a line range, `:g/DROP/d` deletes every matching line while `:v/yes/d` keeps only matching lines, `:1m$` moves line 1 to the end, `:1t2` copies line 1 to after line 2, and `:1,2j` joins lines. Ranges can be relative, too — after `cursor(2, 1)`, `.,+1d` deletes the cursor line and the one below, because `.` is the cursor and `+1` an offset. And the fallback is graceful: “an unrecognized `:` command falls back to running as a statement,” so `:let g:x = 42` on an Ex-prefixed line simply runs the `:let`. This fallback is what makes the Ex-prefixed line syntax safe to use freely: a `:`-prefixed line is first tested against the range-and-Ex-command grammar (the `:s/:g/:d/:sort` family), and if it does not match one of those, it is parsed as an ordinary statement and run. So a script can prefix any line with `:` — as documentation often does to mark a command — without changing its meaning, because a `:let`, a `:call`, or a `:echo` with a leading colon falls through to the normal statement path. The two grammars coexist: the Ex-command grammar claims the range-and-pattern commands, and everything else is a statement. This is the same graceful-fallback principle the bar splitter and the `:map`-versus-`map()` disambiguation use — try the special interpretation, fall back to the general one — and it is what lets the buffer-editing Ex commands share the `:` prefix with the ordinary statement language without either claiming a line that belongs to the other. This is a substantial reproduction of Vim’s `do_one_cmd` range machinery, and it runs with no editor because the only thing the range needs is the buffer, which is present.

A stream editor

`streamedit.vim` pushes the buffer into the role of a sed-or-awk replacement, and its header says so: “vimlrs as a stream editor: file I/O (`:e/:r/:w`), `:sort`, the `:>/:<` shift, and the `!/:%! shell filter ... These turn the in-memory buffer into a sed/awk-style pipeline.” The example loads a file, sorts it, and filters it through a shell command:`

```
call writefile(['cherry', 'apple', 'banana'], s:in)
exe ':edit ' . s:in
:sort
call assert_equal(['apple', 'banana', 'cherry'], getline(1, '$'))
:%!sort
```

`:edit` loads a file into the buffer, `:sort` sorts the whole buffer (with `:sort!` reversing, `:sort n` sorting numerically, and `:sort u` removing duplicates), `:w` writes it back out, `:r` reads a file in after a line, the `:>/:<` commands indent and dedent by `'shiftwidth'` while leaving blank lines alone, and `!/:%!cmd` filters the entire buffer through an external command — here `:%!sort` pipes the buffer through the system `sort`. The `:r !cmd` form inserts a command’s output after a line. This is a genuine stream-editing toolkit, and the point Chapter 1 made about vimlrs taking awk’s niche is most literally true right here: a standalone `.vim` script can load, transform, filter, and write a file with the same commands you would type at the `:` prompt inside Vim.

The `:sort` command deserves its own note, because its flags make it a small sorting toolkit. Bare `:sort` orders the buffer (or a range) lexically; `:sort!` reverses; `:sort n` sorts numerically, comparing the numeric value of each line rather than its text, so `['30', '4', '100', '20']` sorts to `['4', '20', '30', '100']` rather than the lexical `['100', '20', '30', '4']`; and `:sort u` removes duplicate lines after sorting, collapsing `['a', 'a', 'b', 'a', 'b']` to `['a', 'b']`. This is the `sort(1)` command’s vocabulary brought into

the editor's command line, operating on the buffer, and `streamedit.vim` asserts each flag. Combined with `:g/v` for line filtering and `:%!cmd` for shell filtering, the buffer becomes a genuine sed-and-sort pipeline: load with `:edit`, filter with `:g/pat/d`, sort with `:sort n`, transform with `:%!tr a-z A-Z`, write with `:w` — each step a ported Ex command on the one virtual buffer, and the whole sequence a text-processing pipeline that needs no editor and no separate tool.

The range grammar

Before the individual Ex commands, the *range* that prefixes them deserves its own treatment, because it is shared by all of them and it is more expressive than a pair of line numbers. A range is one or two addresses separated by a comma, and each address can be an absolute line number, `.` for the cursor line, `$` for the last line, a `'m` mark reference, a `/pat/` or `?pat?` search, or any of these with a `+N` or `-N` offset. So `:2,4d` deletes lines 2 through 4, `:$d` deletes from the cursor to the end, `:.+1d` deletes the cursor line and the one below (the relative form `excommands.vim` checks after `cursor(2, 1)`), and `:'a,'bd` deletes the lines between two marks (the form `marks.vim` uses). The `%` shorthand is the whole buffer, so `:%s/foo/X/` substitutes over every line. This range resolution is ported from Neovim's address-parsing in `ex_docmd.c`, and it works standalone because resolving an address needs only the buffer, the cursor, the marks, and — for a `/pat/` address — the regex engine, all of which vimlrs has. The same range grammar prefixes the substitute, delete, move, copy, join, yank, put, sort, shift, normal, and filter commands, which is why learning it once unlocks all of them.

The editing commands, one by one

With the range understood, the individual commands are each a small, faithful operation on the buffer. `:[range]s/pat/sub/flags` is the substitute, replacing the first match per line by default and every match with the `g` flag — and its replacement honours the `\=` expression form and the `\u/l` case escapes, so `:%s/\d\+/\=submatch(0) + 1/` increments every number in the buffer (`options.vim`). `:[range]d` deletes the range; `:[range]m{addr}` moves the range to after `{addr}` (`:1m$` moves line 1 to the end); `:[range]t{addr}` (or `:co`) copies it; `:[range]j` joins the range's lines into one, dropping the joined lines' leading whitespace. `:[range]y {reg}` yanks to a register and `:[reg]pu` puts it after a line, the pair `registers.vim` shows sharing state with the `@r` syntax. The global commands `:g/pat/cmd` and `:v/pat/cmd` run a command on every line that matches (or, for `:v`, does not match) a pattern — `:g/DROP/d` deletes every matching line, the classic filter idiom. `:[range]sort` sorts the range, with `!` to reverse, `n` for numeric, `u` to remove duplicates; `:[range]>` and `:[range]<` shift by 'shiftwidth'; and `:[range]!cmd` and `:%!cmd` filter the range through an external command. Each is a ported operation from `ex_cmds.c` or `ops.c`, each works on the virtual buffer, and together they are a substantial reproduction of Vim's line-editing command set — enough that a `.vim` script can do the work you would otherwise reach for `sed`, `awk`, or `sort` to do.

A bounded normal mode

Even normal-mode editing has a faithful subset. `normal.vim`'s header is honest about the boundary — it is “a bounded subset: motions, delete/yank/put, and simple edits. No insert mode, so `i/a/o` are not supported” — but within that subset the `:normal` command runs real normal-mode keys against the buffer and cursor:

```
call setline(1, ['hello', 'world', 'third'])
call cursor(1, 1)
:normal x
call assert_equal('ello', getline(1))
:normal dd
call assert_equal(['world', 'third'], getline(1, '$'))
```

`x` deletes the character under the cursor, `dd` the line, `3x` three characters (counts work), the motions `$`, `0`, `|`, `G`, `gg`, and `w` move the cursor, `dw` deletes a word, `yy/p` copy and paste a line, `r` replaces a character, `~`

toggles case, `D` deletes to end of line, and `J` joins lines. A `:%normal x` runs the keys on every line in a range. Insert mode is out of scope because it needs an interactive input loop, but the editing operators that transform existing text — the ones a script would actually script — are ported from Neovim's `normal.c` and `ops.c` and work on the virtual buffer.

The `:normal` command's design reuses the buffer, cursor, and register state the rest of the chapter built. The keys it runs operate on the same virtual buffer that `setline/getline` populate, the same cursor that `cursor()` moves, and the same registers that `@r` and `:yank` share — so a `:normal yy` followed by `:normal p` copies and pastes a line through the same register store an expression would use, and a `:normal dd` deletes from the same buffer `getline` reads. A count prefix repeats a command (`:normal 3x` deletes three characters), the motions move the shared cursor (`$`, `0`, `G`, `gg`, `w`), and the operators combine with motions (`dw` deletes a word, `D` deletes to end of line). The `:[range]normal` form runs the keys on every line in a range — `:%normal x` deletes the first character of every line — which is the batch-editing idiom that makes `:normal` useful in a script. Because it runs on the same shared state as the Ex commands and the line functions, a script can mix `:normal`, `:%s`, `setline`, and `search` freely, all touching one coherent buffer — which is what makes the virtual buffer a genuine editing substrate rather than a collection of independent functions.

Marks, registers, search, and the shared state

The buffer brings its companions. Marks are buffer data, so `marks.vim` sets them with `:mark/:k`, resolves them in positions and ranges (`getpos("a")`, `:'a,'bd`), and lists them with `getmarklist()`. Registers are shared between the `@r` expression syntax and the `:yank/put` commands, which `registers.vim` verifies — `:2y c` yanks line 2 into register `c`, after which `@c` reads `"two\n"`. Search builds on the buffer, the cursor, and the regex engine together: `search.vim` moves the cursor to a pattern with `search()/searchpos()`, honours the `c/n/b/e` flags, finds matching nested pairs with `searchpair()`, and counts matches with `searchcount()`. The search flags are worth enumerating because each changes the operation meaningfully: by default `search` moves forward to the next match and skips a match at the cursor, the `c` flag accepts a match at the cursor, `n` finds without moving the cursor (a peek), `b` searches backward, and `e` moves to the *end* of the match rather than its start. So `search('foo', 'n')` answers “is there a `foo` ahead?” without disturbing the cursor, while `search('foo', 'b')` finds the previous one — the same flag vocabulary Vim's interactive search uses, here driving the cursor over the virtual buffer. `searchpair('\<if>', '', '\<endif>')` is the structural search that finds the matching `endif` of a nested `if`, the operation an editor uses for %-style bracket matching, and it works standalone because it needs only the buffer, the cursor, and the regex engine — all of which vimlrs has. That a function as editor-flavoured as `searchpair` runs faithfully with no editor is one more demonstration that “the editor” is largely buffer-and-regex operations that transplant cleanly once the buffer and regex engine are present. All of this is the same buffer state seen through different builtins, kept consistent because there is exactly one buffer and one cursor underneath. Even the command line itself is modelled: `cmdline.vim` shows `setcmdline()/getcmdline()` operating a settable in-memory command line ported from `ex_getln.c`.

Marks deserve a closer look because they thread through several of the functions and commands at once, which is what makes them a good test of the buffer's coherence. A mark is set with `:mark a` or `:k a` at the cursor's line, or programmatically with `setpos("a", [0, lnum, col, 0])`, and read with `getpos("a")`, `line("a")`, or `col("a")`. An unset mark reads as a zero position, distinguishable from a real one. The marks then participate in two other systems: they are line addresses, so `:'a,'bd` deletes the range between two marks (the range grammar of this chapter consuming a mark reference), and they are listed by `getmarklist()`, which returns each as a `{mark, pos}` dict. `:delmarks` removes named marks and `:delmarks!` clears them all. So a single mark is set by a command, read by three functions, used as an address by the range grammar, listed by a function, and removed by a command — six surfaces, all touching one mark table, all consistent because the table is one genuine data structure ported from `mark.c`. This is the coherence that distinguishes the virtual editor from a bag of independent functions: a mark you set with `:mark` is the same mark `getpos` reads, the same mark `:'a,'bd` addresses, and the same mark `getmarklist` reports — one table, many views, which is exactly how marks behave inside Vim.

User-defined commands

The last piece of the Ex layer is user commands, ported from Neovim's `usercmd.c`. `:command` defines one and `:Name` invokes it, and `user_commands.vim` exercises the whole attribute vocabulary:

```
command -nargs=1 PutB let g:b = <args>
PutB 1 + 2
call assert_equal(3, g:b)
command -nargs=* FArgs let g:f = Collect(<f-args>)
FArgs 1 2 3
call assert_equal('1+2+3', g:f)
```

A command is “just a stored replacement, expanded and run on invocation.” The argument escapes all work: `<args>` substitutes the verbatim argument text (so `<args>` of `1 + 2` evaluates to `3`), `<q-args>` quotes the arguments as a single string, `<f-args>` splits them into comma-separated quoted values for passing into a function call, `<bang>` expands to `!` when the command is invoked with one, and `<lt>` is a literal `<`. A unique prefix invokes a command (`Set` invokes `SetA` when it is the only match), redefining replaces, and `:delcommand` removes. None of this needs an editor — a user command is a text-substitution-and-run mechanism — so it works standalone, and it is one more way a real `.vim` configuration's machinery loads and runs under vimlrs.

The argument-substitution escapes are the heart of what makes user commands useful, and each is worth naming. `<args>` substitutes the verbatim argument text, so a command defined `command -nargs=1 PutB let g:b = <args>` invoked as `PutB 1 + 2` evaluates `1 + 2` to `3` — the arguments spliced in as code. `<q-args>` substitutes the arguments as a single quoted string, so `QArgs hello there` yields `'hello there'`. `<f-args>` splits the arguments into comma-separated quoted values for passing into a function call, so `FArgs 1 2 3` with `Collect(<f-args>)` calls `Collect('1', '2', '3')`. `<bang>` expands to `!` when the command is invoked with a bang and to empty otherwise, so a `-bang` command can branch on whether the user wrote `Foo!` or `Foo`. And `<lt>` is a literal `<`. The substitution-and-run model is what makes all of this work without an editor: defining a command stores its replacement template, and invoking it expands the escapes against the invocation and runs the result as an ex command line — the full user-command lifecycle, ported from `usercmd.c`, working standalone because a user command is data and text substitution, not editor state.

Data subsystems: mappings, autocommands, and signs

The buffer is not the only piece of the editor vimlrs implements for real. Several of Vim's subsystems are, at bottom, just *data tables* — a mapping is a key plus a replacement, a sign is a name plus attributes, an autocommand is an event plus a handler — and a data table needs no event loop, no window, and no buffer to exist. So vimlrs implements them as live in-memory tables, ported from Neovim's `mapping.c`, `sign.c`, and `autocmd.c`, and the only thing absent is the editor activity that would *fire* them automatically. This refines Chapter 18 in an important way: the mapping queries return empty not because mappings are stubbed but because nothing is mapped yet — and the moment you map something, the table is real.

`mappings.vim` shows it. `mapset()` creates a mapping and `maparg()` reads it back, mode-scoped and attribute-complete:

```
call mapset({'lhs': '<C-a>', 'rhs': ':wall<CR>', 'mode': 'n', 'noremap': 1, 'silent': 1})
call mapset({'lhs': 'jj', 'rhs': '<Esc>', 'mode': 'i'})
call assert_equal(':wall<CR>', maparg('<C-a>', 'n'))
call assert_equal('', maparg('jj', 'n'))
let m = maparg('<C-a>', 'n', 0, 1)
call assert_equal(1, m.silent)
```

The mapping is a real record: `maparg` with a truthy dict argument returns the full mapping Dict — `lhs`, `rhs`, `mode`, `noremap`, `silent`, `expr` — and mappings are mode-scoped, so the insert-mode `jj` is invisible to a normal-mode query. `hasmapto` checks the right-hand side, `mapcheck` prefix-matches the left, `maplist`

returns every mapping, and re-mapping an `lhs` replaces it in place. This is a genuine key-mapping table that a configuration script can build and query exactly as it would inside Vim; it simply never *triggers*, because there are no keystrokes.

Autocommands are the same story with an event-loop-shaped twist. `autocommands.vim` registers handlers with `:autocmd`, and because nothing auto-fires standalone, it fires them by hand with `:doautocmd`:

```
autocmd User Foo call add(g:log, 'foo1')
autocmd User Foo call add(g:log, 'foo2')
call assert_equal(1, exists('#User#Foo'))
doautocmd User Foo
call assert_equal(['foo1', 'foo2'], g:log)
```

Multiple handlers for an event fire in registration order, `exists('#event#pat')` queries the table, file-glob patterns match the `:doautocmd` target (so a `BufRead *.txt` handler fires for `notes.txt` but not `script.vim`), comma-separated events register one handler each, `:augroup` groups them, and `:autocmd!` removes them. The whole subsystem is testable precisely because `:doautocmd` provides the manual trigger an editor would provide automatically — the *table* is real, only the *firing* is manual. Signs round out the trio: `sign_getdefined.vim` shows `sign_define/sign_getdefined/sign_undefine` maintaining “an in-memory name → attributes map (no buffers to place signs in),” where defining, listing, redefining, and undefining all behave exactly as Vim documents, with only the placement-into-a-buffer step absent because there is no buffer column to draw in.

The lesson across all three is the one Chapter 18 set up and this chapter has been sharpening: vimlrs implements the *data* of the editor and stubs the *activity*. A mapping table, an autocommand registry, a sign definition map, a buffer of lines, a register store, a set of marks — these are data, and vimlrs keeps them faithfully. The keystroke loop, the window layout, the redraw, the job scheduler — these are activity, and vimlrs has no use for them standalone. Knowing which side of that line a given builtin falls on tells you immediately whether it does real work or returns the documented empty value, and almost every surprising “this actually works standalone?” turns out to be a data subsystem that simply did not need the editor.

Command-name and glob translation

Two small builtins do the string translations the Ex layer relies on, and both work standalone because they are pure functions over text. `fullcommand()` expands an abbreviated Ex command name to its full form, ported from `ex_docmd.c`'s command resolver, and `fullcommand.vim` walks the rules:

```
call assert_equal('edit', fullcommand('e'))
call assert_equal('substitute', fullcommand('s'))
call assert_equal('split', fullcommand('sp'))
call assert_equal('write', fullcommand(':w'))
call assert_equal('quit', fullcommand('q!'))
call assert_equal('', fullcommand('zzz'))
```

A single letter expands to the command it abbreviates (`e` to `edit`, `s` to `substitute`), a longer prefix selects a more specific command (`sp` to `split`, `vs` to `vsplit`), a leading `:` and a trailing `!` are stripped before matching, and an unknown command yields the empty string. This is the same abbreviation logic the statement dispatcher of Chapter 20 uses to recognize `exe` as `:execute` or `norm` as `:normal`, exposed as a builtin a script can call directly.

`glob2regpat()` converts a shell-style glob to a Vim regex, and `glob2regpat.vim` checks the translation by matching against the result:

```
call assert_equal(1, 'a.txt' =~ glob2regpat('*.txt'))
call assert_equal(0, 'a.txtx' =~ glob2regpat('*.txt'))
call assert_equal(1, 'axb' =~ glob2regpat('a?b'))
call assert_equal(1, 'a.c' =~ glob2regpat('a.c'))
```

In the glob, `*` becomes `.`, `?` becomes `.`, a literal `.` is escaped, a `[class]` passes through, and the whole pattern is anchored to match the entire string — so `'*.txt'` matches `a.txt` but not `a.txtx`, and the literal-dot rule means `glob2regpat('a.c')` matches `a.c` but not `axc`. The function is the bridge between the two pattern languages the project implements — the file globs of Chapter 24 and the regex dialect of Chapter 10 — and because it is pure string translation it needs no editor at all.

The digraph table is one more pure-data subsystem in the same spirit: `digraphs.vim` shows `digraph_get/digraph_set/digraph_getlist` maintaining a two-character-trigger to character map ported from `digraph.c`, where the built-in RFC-1345 subset resolves `a:` to `ä` and `->` to `→`, a user digraph overrides the built-in, and `digraph_getlist` lists the user entries sorted by trigger. Like the mappings, autocommands, and signs above it, a digraph table is data, so vimlrs keeps it faithfully — the only thing absent is the insert-mode keystroke sequence that would *use* a digraph, which is editor activity vimlrs has no occasion for.

Why this is consistent with Chapter 18

There is no contradiction between this chapter and the editor-absent contract. Chapter 18's rule was: return the documented empty value for editor state that is genuinely absent. The buffer is not absent — vimlrs has one — so its queries return real answers. What is absent is the *plurality* of buffers, the windows that display them, the tabs that group them, and the event loop that drives jobs and timers, and those still report emptiness exactly as Chapter 18 described. The line between the two is the line between “content of the one buffer I am editing,” which a stream editor needs and vimlrs provides, and “the editor's window-management state,” which a standalone interpreter has no use for and vimlrs honestly reports as missing. Seen together, the two chapters describe a coherent design: implement the buffer because line-editing scripts need it, stub the editor because nothing standalone does, and tell the truth about which is which.

Chapter 25 – The Builtin Catalogue, Family by Family

The reference manual is the place to look up exactly what `byteidxcomp` returns; this book has mostly sent you there rather than reproducing it. But the *shape* of the builtin surface — how many functions there are, how they group, and what the catalogue’s structure tells you about the project — is a chapter’s worth of material, because the catalogue is generated, single-sourced, and honest in ways worth seeing. This chapter is a guided read of the reference, family by family, grounded in `builtin_docs.rs` and the `reference.html` it generates.

One corpus, three consumers

The reference page states its own provenance on its build line: “vimlrs v0.1.2 · 463 entries · 11 chapters · generated from `vimlrs/builtin_docs.rs`,” and its subtitle calls itself “the exact corpus that `vimlrs --lsp` serves.” That single sourcing is the most important fact about the catalogue. `builtin_docs.rs` holds one slice of `(name, signature, doc)` tuples — “every implemented Vim builtin (each `f_<name>` in the ported evaluator) paired with an accurate one-line description and a signature derived from Neovim’s `funcs[]` arg-count metadata” — and three things read it: the LSP hover provider, the `gen-docs` binary that renders the static `reference.html`, and the LSP completion list. So the hover text in your editor, the descriptions in the published reference, and the corpus the completer offers are *literally the same strings*. The hover formatter, `hover_markdown`, wraps each into a fenced `vim` signature block followed by the one-line description, which is exactly the layout the HTML mirrors per entry. There is no second, drifting copy of the documentation; there is one slice, sorted by name, and everything points at it.

The catalogue’s honesty mechanism is a flag. Editor-integration builtins that are no-ops standalone carry the literal suffix “(stub in standalone)” in their description, and the module header says why: “so the LSP stays honest.” When you hover `jobstart` or `timer_start` in an editor, the doc tells you it is a stub here, rather than pretending it drives a job or a timer. This is Chapter 18’s contract surfaced at the documentation layer — the function exists, is documented, and admits in its own description that it returns the degenerate value.

The eleven chapters

The reference groups its 463 entries into eleven chapters. Eight are function families — String manipulation (56), List manipulation (22), Dictionary manipulation (9), Variables & Type (8), Number & Float (13), Files, Buffers & Lines (77), System & Environment (44), and the Functional & Misc catch-all (210) — which sum to 439 functions. The remaining three are Ex Commands (6), Options (15), and Special v: Variables (3), for the full 463. The catch-all is the largest by far, and its size is itself informative: the category table maps each name to a chapter, and “any uncategorized implemented builtin lands in Functional & Misc and is never dropped,” so the 210 there are everything that did not fit one of the seven sharp families — the assert framework, the higher-order functions, and the long tail of editor-state queries that are mostly stubs. The point of the catch-all is that nothing falls off the edge: an implemented builtin always appears somewhere.

String manipulation

The 56 string functions are the largest sharp family, fitting for a language born for text. The everyday workhorses head the list with their exact signatures: `printf({fmt} [, {expr1} ...])` is “Formatted string from printf-style {fmt} and arguments,” `substitute({string}, {pat}, {sub}, {flags})` replaces matches with “{flags} ‘g’ replaces all,” `split({string} [, {pat} [, {keepempty}]])` is “List of {string} split on {pat} (default whitespace),” and `string({expr})` is the reparsable renderer, “quoting embedded strings (inverse of eval()).” The family reaches well past the basics, though. `matchfuzzy({list}, {str} [, {dict}])` keeps “Items of {list} fuzzy-matching {str}, best first,” and its sibling `matchfuzzypos` adds “match positions and scores” — both ported from Neovim’s `search.c` fuzzy matcher, as `fuzzy.vim` proves. `sha256({string})` is a real “SHA-256 hex digest” (FIPS-180-2, ported from `sha256.c`), `soundfold({word})` gives a phonetic form, `strtrans` renders unprintables as `^X`, and `charclass` reports a character’s class. The byte/character navigation functions Chapter 9 detailed — `byteidx`, `byteidxcomp`, `charidx`, `strchars`, `strcharpart`, and the rest — all live here too. It is a family deep enough that a faithful port of it alone would be a substantial project.

List, Dictionary, and Type

The 22 List functions are the in-place-and-return standard library Chapter 7 toured: `sort({list} [, {how} [, {dict}]])` “Sort {list} in place; {how} sets comparison; returns {list},” `uniq` removing “adjacent duplicate items from sorted {list} in place,” `reduce({object}, {func} [, {initial}])` folding “to a single value,” `deepcopy` for “a recursive (deep) copy,” and `slice({expr}, {start} [, {end}])` for the exclusive-end “Sub-list/Blob/String.” The non-mutating variants sit beside their mutating cousins — `mapnew` “Like map() but returns a new container, leaving {expr1} unchanged” — so a caller can choose in-place or fresh. The 9 Dictionary functions are the complete set Chapter 7 named, including the watcher pair `dictwatcheradd/dictwatcherdel` that fire “when keys matching {pattern} in {dict} change,” and `extend/extendnew` for the merge in-place and merge-fresh forms. The 8 Variables & Type functions are the introspection core: `type({expr})` returning “Numeric type code (0 Number, 1 String, 2 Func, 3 List, 4 Dict, ...),” `empty`, `exists`, `len`, the Funcref constructors `function` and `funcref` (which “ignores later redefinition”), and `islocked/garbagecollect`, both flagged stubs.

Number & Float, and the bitwise builtins

The 13 Number & Float functions include the directed-rounding and transcendental library Chapter 23 toured, plus the bitwise quartet that matters for the JIT: `and`, `or`, `xor`, and `invert`. The reference describes them plainly — `and` “Bitwise AND of {expr1} and {expr2},” `invert` “Bitwise complement (NOT)” — but Chapter 14 noted their significance: their integer forms lower to native fusevm ops (`Op::BitAnd` and friends), so bit-manipulation loops trace-JIT. The family also has `abs` (“Float in -> Float out”), the three-valued `isinf`, the `isnan` predicate, `pow`, `fmod`, `float2nr`, `atan2`, and the random pair `rand/srand`, where `rand` is a “Pseudo-random 32-bit Number” — and, as the README notes, a bit-exact one, using xoshiro128** to match Neovim’s output.

Files, System, and the honest stubs

The two largest families after the catch-all are where the standalone contract is most visible. Files, Buffers & Lines has 77 entries, and the split Chapter 18 described runs right through it: the filesystem functions are real — `readfile/writefile/readblob` for I/O, `glob/globpath/glob2regpat` for wildcards, `mkdir`, `fnamemodify`, `getperm`, `getftype`, `simplify`, `resolve` — while the buffer-numbered functions like `bufnr` are “stub in standalone.” The `glob.vim` example shows the real ones working: `glob({expr} [, {nosuf} [, {list} ...]])` “Expand file wildcards {expr}; String (or List) of matching paths,” with the list form returning sorted matches. System & Environment has 44, and the same split: `system/systemlist` shell out for real and

set `v:shell_error`, the `json_*` and `msgpack*` codecs are faithful, `envron/getenv/setenv` reach the real environment, `reltime/strftime/strptime` are ported time functions, and `sha256` is real — while the `jobstart`, `rpc*`, and `server*` functions are stubs, each labelled. The reference does not hide the split; it labels every stub, so a reader of the catalogue can tell at a glance which functions do real work standalone and which return the documented empty value.

The Functional & Misc catch-all

The 210-entry catch-all is the long tail, and a few of its members are the most real functions in the whole catalogue. The assert family lives here — `assert_equal({expected}, {actual} [, {msg}])` “Add an error to v:errors unless {expected} == {actual},” and its `assert_true/assert_match/assert_fails/assert_exception` siblings — and it is the family every example depends on. So do the higher-order `map` and `filter` (“in place”), `call`, `eval`, and `execute`; `has` for feature detection; and the argument-list functions `argc/argidx/argv` that `arglist.vim` checks. The rest of the catch-all is the editor-state surface — `search` and `searchpair` (real, on the virtual buffer), `wordcount` (real), `getpos/setpos` and the mark functions (real), and a long list of window, tab, sign, fold, syntax, timer, and completion queries that are mostly stubs. The catch-all is large precisely because it is where everything that is not a clean data-structure operation ends up, and reading it top to bottom is the fastest way to feel the breadth of what a faithful Vimscript port has to cover — and how much of that breadth is editor machinery that a standalone interpreter answers for with a documented empty value rather than a real subsystem.

Options, Ex Commands, and constants

The three non-function chapters round out the reference. The 15 Options are the ones Chapter 24 set and read — `ignorecase`, `smartcase`, `magic`, `expandtab`, `number`, `relativenumber`, `wrap`, `hlsearch`, `incsearch`, `autoindent`, `tabstop`, `shiftwidth`, `softtabstop`, `textwidth`, `scrolloff` — each rendered with its abbreviation, type, and default, so `shiftwidth` is “Number, default 8” and `ignorecase` is “Bool, default 0.” The 6 Ex Commands are the ones with hover text — `echo`, `echon`, `echomsg`, `let`, `call`, `eval` — documented by their descriptions alone, with no signature block, because a command is not a function. And the 3 Special v: Variables are the documented constants `v:true`, `v:false`, and `v:null`, each with its numeric and string values. Appendix B and Appendix C reproduce these lists for quick reference; the point of reading them here is to see that the catalogue is not a flat dump but a structured, generated, honestly- labelled corpus — and that its single source means the documentation can never drift from the language the LSP and the interpreter actually implement.

How to read the catalogue chapters

The eight chapters that follow are a narrative reference, and they reward a particular way of reading. Each chapter takes one family, opens by naming what the family is for and how big it is, walks its functions grouped by sub-family with each function’s semantics and edge cases, folds in a real example where the suite provides one, and closes with a verbatim signature reference and a collection of the family’s faithful quirks. The single most useful thing to carry into them is the real-versus-stub distinction: every editor-touching function carries a “stub in standalone” label in the reference, and the chapters mark which side of the line each function falls on. A function is real if it operates on data the standalone interpreter genuinely has — the filesystem, the environment, the one virtual buffer, an in-memory table — and a stub if it addresses the editor’s display, event loop, window system, or a peer process. Knowing which is which is most of what you need to predict a function’s behaviour, and the chapters teach it family by family. Read straight through, the eight chapters are the longest stretch of the book; read a family at a time, when you need to know what some function actually does and how it fits its neighbours, they are the companion reference this book set out to be — the guided read the lookup table cannot be. We begin where Vimscript itself begins, with the strings.

From overview to detail

This chapter has been the map; the next eight are the territory. Where this one gave counts and a representative function or two per family, the chapters that follow walk the families in full — every implemented String function, every List and Dict and numeric function, the seventy-seven file-and-buffer functions, the forty-four system functions, the sprawling two-hundred-entry Functional & Misc surface broken into three thematic chapters, and finally the ex-commands, options, and special variables. Each function is given with its exact signature from the corpus, a precise statement of what it does, the edge cases worth knowing, and — where the example suite provides one — a real worked snippet. The result is a narrative reference: not a replacement for the [reference.html](#) lookup table, but the thing the lookup table cannot be, a guided read of the whole surface that tells you not just what each function returns but how it fits the family around it and which side of the standalone line it falls on. Read straight through it is long; read a family at a time it is the companion to the reference this book set out to be. We begin where Vimscript itself begins, with the strings.

Chapter 26 – The String Functions

The String manipulation family is the largest sharp category in the reference — fifty-six functions — and it is the right place to start a detailed catalogue, because text is what Vimscript was built for and because these functions exercise every coercion and every byte-versus-character subtlety the earlier chapters established. This chapter walks all fifty-six, grouped by what they do, with each function's signature and semantics and, where the example suite supplies one, a worked line. The signatures are reproduced exactly as the reference renders them, including the `{required}` and `[optional]` argument conventions.

Byte and character measurement

Six functions measure a string, and the whole point of having six is that “length” is ambiguous in a byte-string language with multibyte text. `strlen({string})` is the byte count, `strchars({string} [, {skipcc}])` is the character count “counting composing chars unless {skipcc},” and `strcharlen({string})` is the character count “ignoring composing characters” — always folding a combining mark into its base. `strdisplaywidth({string} [, {col}])` is the “display cell width of {string}, honoring ‘tabstop’,” while `strwidth({string})` is the same display width with “no tab expansion.” And `strutf16len({string} [, {countcc}])` is the “number of UTF-16 code units,” which matters for interoperating with editors and protocols that count in UTF-16. Chapter 9 showed the six diverging on 'héllo': `strlen` is 6, `strchars` is 5, and the composing-character cases split `strchars` from `strcharlen`.

The index-mapping functions translate between the three coordinate systems. `byteidx({string}, {nr} [, {utf16}])` is the “byte index of the {nr}’th character,” and `byteidxcomp` is “like byteidx() but counting composing characters separately” — the pair Chapter 9 used to show a composing accent folding under one and counting under the other. `charidx({string}, {idx} [, {countcc} [, {utf16}]]])` goes the other way, the “character index of byte {idx},” and is multibyte-safe on a byte that lands inside a character (Chapter 23). `utf16idx({string}, {idx} [, {countcc} [, {charidx}]]])` is the “UTF-16 code-unit index of byte {idx}.” These five are the machinery a script reaches for whenever it must navigate multibyte text by position rather than by content.

Extracting substrings and characters

A cluster of functions pull pieces out of a string. `strpart({src}, {start} [, {len} [, {chars}]]])` is the “byte (or char) substring of {src} from {start} for {len},” the byte-oriented workhorse, while `strcharpart({src}, {start} [, {len} [, {skipcc}]]])` is its character-oriented twin, the “character substring of {src} from char {start} for {len} chars.” A single character comes from `strgetchar({str}, {index})`, the “codepoint at char {index} of {str}, or -1” past the end — and `string_index.vim` showed `strgetchar('héllo', 1)` returning the codepoint of é. The search functions `stridx({haystack}, {needle} [, {start}])` and `strridx` return the “byte index of first” and “byte index of last” needle, respectively, or -1, with `stridx` scanning forward from {start} and `strridx` backward from it. The slice and subscript syntax of Chapter 9 — `str[i]` and `str[a:b]` — sits on top of these same helpers.

Case, translation, and trimming

The transformation functions change a string's content. `tolower({string})` and `toupper({string})` case-fold all letters. `tr({src}, {fromstr}, {tostr})` does character-by-character translation — “each char in {fromstr} replaced by the matching char in {tostr}” — and errors [E475](#) when a source character has no destination, the strict behaviour Chapter 9 noted. `trim({text} [, {mask} [, {dir}]])` strips “leading/trailing {mask} chars (default whitespace),” with the direction argument selecting both ends, left only, or right only:

```
call assert_equal('x', trim(' x '))
call assert_equal('hixx', trim('xxhixx', 'x', 1))
```

`escape({string}, {chars})` backslash-prefixes each character in {chars}, the tool for making a string safe to splice into a pattern or command; `fnameescape` escapes a string “for use as a Vim command file-name argument”; and `shellescape({string} [, {special}])` quotes a string “for safe use as a shell command argument,” the companion to `system()`. `keytrans({string})` renders key codes readably (turning a raw `<C-A>` byte into the text `<C-A>`), and `strtrans({string})` shows “unprintable characters as `^X /`” — `strtrans("a\tb")` is `'a^Ib'`.

Building strings from numbers and back

Four functions cross between strings and codepoint numbers. `char2nr({string} [, {utf8}])` is the “codepoint of the first character,” and `nr2char({expr} [, {utf8}])` its inverse, the “character whose codepoint is {expr}” — the pair that builds a composing-character test string as `'e' . nr2char(0x301)`. `str2list({string} [, {utf8}])` explodes a string to “a List of codepoint numbers for each character,” and `list2str({list} [, {utf8}])` reassembles “a String built from the List of codepoint numbers.” Together they let a script manipulate text as a list of numbers and rebuild it, which is occasionally exactly what a transformation needs.

The string/number coercions Chapter 5 detailed live here too: `str2nr({string} [, {base}])` reads the “leading integer parsed from {string} in {base},” and `str2float({string} [, {quoted}])` the “Float parsed from {string}” with `strtod` semantics including hex. And `string({expr})` is the reparsable renderer — the “String representation of {expr}, quoting embedded strings (inverse of `eval()`)” — the backbone of the round-trip property `eval(string(x)) == x`.

Splitting, joining, and formatting

`split({string} [, {pat} [, {keepempty}]])` tokenizes into “a List of {string} split on {pat} (default whitespace),” and `join({list} [, {sep}])` is its inverse, “joined by {sep} (default space).” `repeat({expr}, {count})` repeats a string (or list) {count} times, with the three-type behaviour `repeat.vim` pins down. And `printf({fmt} [, {expr1} ...])` is the formatting workhorse, the “formatted string from printf-style {fmt} and arguments” whose positional, alternate-form, and sign-flag conversions Chapter 9 explored in depth. The matching and replacing functions — `match`, `matchend`, `matchstr`, `matchstrpos`, `matchlist`, `matchstrlist`, `matchbufline`, `matchfuzzy`, `matchfuzzypos`, and `substitute` — bridge into the regex engine of Chapter 10; their signatures appear in the catalogue but their behaviour is that chapter's subject. `substitute({string}, {pat}, {sub}, {flags})` in particular, with its `\=` expression replacement and `submatch()` access, is the most powerful of them.

Digraphs, encoding, and the rest

The family's tail is the specialized functions. The digraph quartet — `digraph_get({chars})`, `digraph_set({chars}, {digraph})`, `digraph_getlist`, `digraph_setlist` — manages the two-character-trigger table Chapter 24 described, `digraph_get('->')` returning `→`. `iconv({string}, {from}, {to})` converts between encodings, identity for same-or-UTF-8. `sha256({string})` is the real FIPS-180-2 “SHA-256 hex digest.” `charclass({string})` reports the “character class (0 white, 1 punctuation, 2 word, ...)” of the first character, as `width.vim` checks — the same classification the regex word-boundary atoms `\</>` use to decide where a word begins and ends, exposed as a builtin a script can call directly. `soundfold({word})` gives a phonetic form, `gettext({text} [, {package}])` a translated string (identity with no catalog), and `expandcmd({string} [, {options}])` expands “command-line special items (‘%’, ‘,’, ...)” inside a string — the function `expand.vim` uses to expand `$HOME` inside a command. Fifty-six functions, and every one of them is either a faithful port of its `funcs.c` counterpart or, for the regex-backed ones, a faithful reproduction of its documented behaviour; the family is the densest single demonstration that vimlrs took Vimscript’s text heritage seriously.

strings.vim end to end

The `strings.vim` example is the densest single test in the crate, and walking it in full is the best way to see the string surface working together rather than function by function. It opens with the everyday operations on a sample line:

```
let s = 'The Quick Brown Fox'
call assert_equal(['The', 'Quick', 'Brown', 'Fox'], split(s))
call assert_equal('The-Quick-Brown-Fox', join(split(s), '-'))
call assert_equal('Fox Brown Quick The', join(reverse(split(s)), ' '))
call assert_equal(19, strlen(s))
```

`split` with no pattern tokenizes on whitespace, `join` reassembles with a separator, and `reverse(split(s))` then `join` reverses the word order — a one-line idiom for reversing words that composes three list-and-string functions. `strlen` is the byte length, 19 for this ASCII string.

The `printf` section is the most thorough demonstration of the formatter anywhere in the suite. Beyond the basics, it pins the octal/binary/char/scientific conversions (`printf('%x %o %b', 255, 8, 5)` is `'ff 10 101'`, `printf('%08b', 5)` is `'00000101'`), the sign flags (`%+d` forces a sign, `% d` space-pads positives, `%+05d` combines them), the positional `%N$s` arguments that may be reused and reordered (`printf('%2$s %1$s', 'a', 'b')` is `'b a'`, `printf('%1$s %1$s', 'x')` is `'x x'`), and the `#` alternate form whose prefix is suppressed for a zero value:

```
call assert_equal('0xff', printf('%#x', 255))
call assert_equal('0', printf('%#x', 0))
call assert_equal('0x0000ff', printf('%#08x', 255))
call assert_equal('0.1 1e+06 0.0001', printf('%g %g %g', 0.1, 1000000.0, 0.0001))
```

The `%g` line shows the magnitude-driven choice between fixed and exponential form, the same six-significant-digit renderer `string()` uses. Each of these is a distinct faithful behaviour that a naive `printf` would skip.

The multibyte section is the most intricate, using a string built from a base letter plus a combining acute accent so the byte-versus-character distinctions all fire:

```

let cc = 'e' . nr2char(0x301) . 'x'
call assert_equal([0, 3, 4], [byteidx(cc, 0), byteidx(cc, 1), byteidx(cc, 2)])
call assert_equal([0, 1, 3, 4],
    \ [byteidxcomp(cc, 0), byteidxcomp(cc, 1), byteidxcomp(cc, 2), byteidxcomp(cc, 3)])
call assert_equal(1, charidx(cc, 3))
call assert_equal(2, charidx(cc, 3, 1))

```

The string `cc` is `e` + combining-acute + `x`, occupying bytes 1 + 2 + 1. `byteidx` folds the combining character into its base, so it sees two characters and `byteidx(cc, 1)` is 3 (past the folded `é`); `byteidxcomp` counts the accent separately, so it sees three characters and the offsets shift. `charidx` inverts the mapping, and its `{countcc}` argument flips between folding and counting the composing char — exactly the pair of behaviours Chapter 9 introduced, here asserted at every offset.

The substring functions reveal a faithful quirk in `strpart`'s negative-start handling:

```

call assert_equal('h', strpart('hello', -2, 3))
call assert_equal('', strpart('hello', -5, 3))

```

A negative `{start}` “clamps to 0 AND folds its offset into the length,” so `strpart('hello', -2, 3)` keeps only `'h'` — the start clamps to 0 but the length 3 is reduced by the two-character underflow, leaving one character. This is the kind of behaviour nobody would reproduce by intuition, and it is right because `strpart` is ported. The `trim` direction argument (0 both, 1 left, 2 right), the `tr` character mapping with its E475 error on a missing destination, and the `substitute \=` expression replacement with `submatch()` and the `\u/\l/\U/\L` case escapes all follow, each asserted across several cases:

```

call assert_equal('ba-c', substitute('a-b-c', '\(\w\)-\(\w\)',
    \ '\=submatch(2).submatch(1)', 'g'))
call assert_equal('Hello World', substitute('hello world', '\w\+', '\u\0', 'g'))

```

The `\=` form runs an expression per match — here swapping two captured groups with `submatch(2).submatch(1)` — and the case escapes upcase or downcase the next character or the rest of the replacement. Finally the `str2nr` section pins the base argument and the `{quote}` flag that ignores embedded `'` digit separators:

```

call assert_equal(1000000, str2nr("1'000'000", 10, 1))
call assert_equal(1, str2nr("1'000", 10, 0))
call assert_equal(65535, str2nr("ff'ff", 16, 1))

```

One example file, and it has asserted the byte/character index family, the full `printf` conversion set, the substring quirks, the translation and trimming functions, the regex-backed match and substitute operations, and the number parser — every one against an exact expected value. It is the single best demonstration in the repository that the string family is not approximately right but exactly right, and it is the reason this book could quote string behaviour so freely: `strings.vim` runs in CI every day and asserts all of it.

Signature reference: the string functions

The measurement and indexing functions: `strlen({string})`, `strchars({string} [, {skipcc}])`, `strcharlen({string})`, `strdisplaywidth({string} [, {col}])`, `strwidth({string})`, `strutf16len({string} [, {countcc}])`, `byteidx({string}, {nr} [, {utf16}])`, `byteidxcomp({string}, {nr} [, {utf16}])`, `charidx({string}, {idx} [, {countcc} [, {utf16}]]])`, and `utf16idx({string}, {idx} [, {countcc} [, {charidx}]]])`. The substring and search functions: `strpart({src}, {start} [, {len} [, {chars}]]])`, `strcharpart({src}, {start} [, {len} [, {skipcc}]]])`, `strgetchar({str}, {index})`, `stridx({haystack}, {needle} [, {start}])`, and `strridx({haystack}, {needle} [, {start}])`. The case and translation functions: `tolower({string})`, `toupper({string})`, `tr({src}, {fromstr}, {tostr})`, `trim({text} [, {mask} [, {dir}]]])`, `escape({string}, {chars})`, `fnameescape({string})`, `shellescape({string} [, {special}])`, `keytrans({string})`, and `strtrans({string})`.

The number-and-string converters: `char2nr({string} [, {utf8}])`, `nr2char({expr} [, {utf8}])`, `str2list({string} [, {utf8}])`, `list2str({list} [, {utf8}])`, `str2nr({string} [, {base}])`, `str2float({string} [, {quoted}])`, and `string({expr})`. The splitting, joining, and formatting functions: `split({string} [, {pat} [, {keepempty}]])`, `join({list} [, {sep}])`, `repeat({expr}, {count})`, and `printf({fmt} [, {expr1} ...])`. The regex-backed match functions: `match({expr}, {pat} [, {start} [, {count}]])`, `matchend(...)`, `matchstr(...)`, `matchstrpos(...)`, `matchlist(...)`, `matchstrlist({list}, {pat} [, {dict}])`, `matchbufline({buf}, {pat}, {lnum}, {end} [, {dict}])`, `matchfuzzy({list}, {str} [, {dict}])`, `matchfuzzypos({list}, {str} [, {dict}])`, `submatch({nr} [, {list}])`, and `substitute({string}, {pat}, {sub}, {flags})`. And the specialized tail: `digraph_get({chars})`, `digraph_set({chars}, {digraph})`, `digraph_getlist([ {listall} ])`, `digraph_setlist({list})`, `iconv({string}, {from}, {to})`, `sha256({string})`, `charclass({string})`, `soundfold({word})`, `gettext({text} [, {package}])`, and `expandcmd({string} [, {options}])`. Fifty-six functions, the full string surface with exact arities.

Edge cases worth knowing

The string family carries a handful of faithful quirks that catch the unwary, and collecting them is useful because each is a place a from-scratch implementation would diverge. `strpart` with a negative `{start}` clamps the start to 0 *and* folds the underflow into the length, so `strpart('hello', -2, 3)` is 'h', not 'hel'. `tr` errors with [E475](#) when a source character has no corresponding destination, rather than silently dropping it — `tr('abc', 'ab', 'x')` fails. `str2nr` with a truthy third argument ignores embedded ' digit separators (`str2nr("1'000'000", 10, 1)` is a million) but without it stops at the quote (`str2nr("1'000")` is 1). A `len()` of a String is its *byte* length, so `len('héllo')` is 6 while `strchars` is 5. `printf`'s # alternate form suppresses its `0x0b` prefix for a zero value, so `printf('%#x', 0)` is '0', not '0x0'. `byteidx` folds a composing character into its base while `byteidxcomp` counts it separately. And `string()` of a float uses `%g`, so `string(0.1 + 0.2)` is '0.3', not the seventeen-digit value floating-point would suggest. Every one of these is asserted by `strings.vim` or its siblings, and every one is right because the function was ported from the C that defines the quirk.

Chapter 27 – Containers and Numbers

This chapter covers three families that together form the data-and-arithmetic core of the standard library: the twenty-two List functions, the nine Dictionary functions, the eight Variables & Type functions, and the thirteen Number & Float functions. They are grouped together because they are the functions you reach for once you have data in hand — building it, transforming it, measuring it, and computing over it — and because, having been among the first things ported (the value layer was the “faithful substrate,” Chapter 13), their behaviour is Vim’s down to the edges.

Building and copying lists

The mutating list builders all follow the in-place-and-return convention Chapter 7 established. `add({object}, {expr})` appends to “List or Blob {object}” and returns it; `insert({object}, {item} [, {idx}])` inserts “before index {idx} (default 0),” so a bare `insert` prepends; and `remove({object}, {idx} [, {end}])` removes and returns “item {idx} (or range {idx}..{end}).” `extend({expr1}, {expr2} [, {expr3}])` appends or merges one container into another in place — over lists it concatenates, over dicts it merges — while `extendnew` is the non-mutating version “leaving {expr1} unchanged.” The copy pair is the safety mechanism the convention requires: `copy({expr})` is a “shallow copy” and `deepcopy({expr} [, {noref}])` a “recursive (deep) copy,” which is why idiomatic code writes `sort(copy(nums), 'n')` to avoid mutating the original.

Searching, counting, and ordering lists

`index({object}, {expr} [, {start} [, {ic}]])` returns the “index of first item equal to {expr},” and `indexof({object}, {expr} [, {opts}])` the “index of first item for which {expr} is true” — the predicate form `indexof.vim` exercised, with its type-dependent `startidx` edges. `count({comp}, {expr} [, {ic} [, {start}]])` counts occurrences across “String/List/Dict {comp}.” The ordering functions are `sort({list} [, {how} [, {dict}]])`, which sorts “in place; {how} sets comparison” (the string ‘n’ for numeric, a funcref, or a lambda), and `uniq({list} [, {how} [, {dict}]])`, which removes “adjacent duplicate items from sorted {list}” — the two composing as `uniq(sort(copy(words))` to deduplicate. `reverse({object})` reverses “List/Blob {object} in place,” and `max({expr})/min({expr})` return the extreme “among the items of List/Dict {expr} (0 if empty)” — note the dict form ranges over *values*, as `numbers.vim` checks, and the empty case is `0` rather than an error.

Transforming and folding

The higher-order trio is the heart of functional Vimscript. `map({expr1}, {expr2})` replaces “each item of List/Dict/Blob/String {expr1} with {expr2}; in place,” and its non-mutating twin `mapnew` “returns a new container, leaving {expr1} unchanged.” `filter({expr1}, {expr2})` keeps “items of {expr1} for which {expr2} is non-zero; in place.” `reduce({object}, {func} [, {initial}])` folds “to a single value via {func}, starting at {initial},” and `foreach({expr1}, {expr2})` applies a body “for its side effects; returns {expr1}.” All accept the three callable forms — string expression, funcref, lambda — and all work over both Lists and Dicts, with the index argument being the key over a Dict. The flattening pair handles nested lists: `flatten({list} [, {maxdepth}])` flattens “in place up to {maxdepth},” and `flattennew` returns a flattened copy. `range({expr} [, {max} [, {stride}]])` builds “a List of numbers,” and `slice({expr}, {start} [, {end}])` takes

the “Sub-list/Blob/ String of {expr} from {start} up to (not including) {end}” — the exclusive-end slice that contrasts with the inclusive `[a:b]` index (Chapter 9).

Blobs and the converters

The blob converters bridge the byte buffer to the list world. `blob2list({blob})` produces “a List of byte-value Numbers for each byte,” and `list2blob({list})` rebuilds “a Blob from the List of byte-value Numbers” — the round-trip `blobs.vim` asserts, with `list2blob` erroring on a value outside the byte range. The mutating list functions `add`, `insert`, and `remove` work on a Blob’s bytes too, and `slice` and `get` index it by byte. Blobs are the least-used container, but their full operation set — literal `0z...`, byte indexing, byte slicing, list conversion, and the mutators — is present and faithful.

Dictionaries

The nine Dictionary functions are the complete set. The three views are `keys({dict})`, `values({dict})`, and `items({dict})` — the last “a List of [key, value] pairs,” the form `:for [k, v] in items(d)` destructures. `has_key({dict}, {key})` tests membership, returning “1 if {dict} has an entry {key}, else 0,” and `get({collection}, {key} [, {default}])` is the safe read across “Dict/List/Blob,” returning the “{default} if absent” — the heart of the `get(freq, w, 0)` frequency idiom. `extend/extendnew` merge (shared with the list family). And the watcher pair, `dictwatcheradd({dict}, {pattern}, {callback})` and `dictwatcherdel`, register and remove a callback that fires “when keys matching {pattern} in {dict} change” — the machinery Chapter 4 noted is wired through the element-set path, so a dict assignment a watcher is registered for genuinely fires it. The watcher mechanism is a small but telling demonstration of how deep the port goes: it is not the kind of feature a reimplementer would add unprompted, because it is rarely used, but it is in Neovim, so it is in vimlrs. A watcher registered with a pattern fires its callback whenever a key matching that pattern is added, changed, or removed, receiving the key, the old value, and the new value — the reactive-dict mechanism plugins use to observe configuration changes. Wiring it through the element-set path (the `let d[key] = value` lvalue of Chapter 4) means the watcher fires on the genuine mutation, not on a separate notification call, so a script that mutates a watched dict through ordinary assignment triggers the watcher exactly as it would in Neovim. That a feature this peripheral is ported and wired through the assignment path is the kind of completeness that distinguishes a port aiming at fidelity from one aiming at the common cases — the common cases would skip dict watchers, and vimlrs keeps them because the C has them.

Variables and type introspection

The eight Variables & Type functions are the reflection core. `type({expr})` returns the “numeric type code of {expr} (0 Number, 1 String, 2 Func, 3 List, 4 Dict, ...)” that the whole type system keys off. `len({expr})` is the universal “length of String/List/Dict/Blob,” and `empty({expr})` tests emptiness — “1 if {expr} is empty (0, '', [], {}, v:null).” `exists({expr})` reports whether a “variable/option/function/command {expr} exists,” the feature-detection sibling of `has`. The Funcref constructors are `function({name} [, {arglist}] [, {dict}])`, which builds a “Funcref for builtin/user function {name}, optionally bound to args/dict,” and `funcref({name} ...)`, which binds “by reference (ignores later redefinition)” — the distinction being whether a later `:function!` redefinition is seen. `garbagecollect` and `islocked` round out the family, both honestly flagged “stub in standalone”: there is no cyclic collector to run and no lock to query in the current build.

Numbers, floats, and bits

The thirteen Number & Float functions are the arithmetic library Chapter 23 toured. `abs({expr})` is the absolute value (“Float in -> Float out”), `float2nr({expr})` truncates “toward zero to a Number,” and `fmod({expr1},`

`{expr2}`) is the “floating-point remainder.” `pow({x}, {y})` and `atan2({expr1}, {expr2})` are the exponential and the two-argument arctangent. The non-finite predicates are `isnan({expr})`, “1 if {expr} is a NaN Float,” and `isinf({expr})`, the three-valued “1 if +inf, -1 if -inf, else 0.” The random pair is `rand([ {expr} ])`, a “pseudo-random 32-bit Number, optionally from seed-list {expr},” and `srand([ {expr} ])`, which initializes “and returns a seed List” — bit-exact against Neovim’s xoshiro128**. And the bitwise quartet — `and({expr1}, {expr2})`, `or`, `xor`, and `invert({expr})` (“bitwise complement (NOT)”) — are the ones whose integer forms lower to native fusevm ops, so the bit-manipulation loops of Chapter 14 trace-JIT. The transcendental trig, hyperbolic, exponential, and rounding functions (`sin`, `cos`, `sqrt`, `floor`, `ceil`, `round`, `trunc`, and the rest) are catalogued under the float math the `floatmath.vim` example exercises; the thirteen here are the arithmetic and bitwise core that the reference groups under Number & Float.

Signature reference: the container and numeric functions

The list functions with their signatures: `add({object}, {expr})`, `insert({object}, {item} [, {idx}])`, `remove({object}, {idx} [, {end}])`, `extend({expr1}, {expr2} [, {expr3}])`, `extendnew({expr1}, {expr2} [, {expr3}])`, `copy({expr})`, `deepcopy({expr} [, {noref}])`, `index({object}, {expr} [, {start} [, {ic} ]])`, `indexof({object}, {expr} [, {opts}])`, `count({comp}, {expr} [, {ic} [, {start} ]])`, `sort({list} [, {how} [, {dict} ]])`, `uniq({list} [, {how} [, {dict} ]])`, `reverse({object})`, `max({expr})`, `min({expr})`, `map({expr1}, {expr2})`, `mapnew({expr1}, {expr2})`, `filter({expr1}, {expr2})`, `reduce({object}, {func} [, {initial}])`, `foreach({expr1}, {expr2})`, `flatten({list} [, {maxdepth}])`, `flattennew({list} [, {maxdepth}])`, `range({expr} [, {max} [, {stride} ]])`, `slice({expr}, {start} [, {end}])`, `blob2list({blob})`, and `list2blob({list})`. The dictionary functions: `keys({dict})`, `values({dict})`, `items({dict})`, `has_key({dict}, {key})`, `get({collection}, {key} [, {default}])`, `dictwatcheradd({dict}, {pattern}, {callback})`, and `dictwatcherdel({dict}, {pattern}, {callback})`.

The variables-and-type functions: `type({expr})`, `len({expr})`, `empty({expr})`, `exists({expr})`, `function({name} [, {arglist} [, {dict}])`, `funcref({name} [, {arglist} [, {dict}])`, `garbagecollect([ {atexit}])`, and `islocked({expr})`. The number-and-float functions: `abs({expr})`, `float2nr({expr})`, `fmod({expr1}, {expr2})`, `pow({x}, {y})`, `atan2({expr1}, {expr2})`, `isnan({expr})`, `isinf({expr})`, `rand([ {expr} ])`, `srand([ {expr} ])`, `and({expr1}, {expr2})`, `or({expr1}, {expr2})`, `xor({expr1}, {expr2})`, and `invert({expr})`. Fifty-two functions across the four families — the building, transforming, measuring, and computing core of the standard library, every one with its exact arity and every one ported from the value layer that was the project’s first faithful substrate.

Edge cases worth knowing

The container and numeric families have their own set of faithful quirks. `get()` on a String — as opposed to a List, Dict, or Blob — is an error (E1531), not a silent default, because a string is not a keyed collection (`index_get.vim`). `max([])` and `min([])` return 0 rather than erroring on the empty case, and over a Dict they range over the *values*, not the keys (`numbers.vim`). `indexof`’s out-of-range `startidx` differs by type: a List yields -1, a Blob clamps a very-negative start to 0 (`indexof.vim`). `list2blob` errors if any element is outside the byte range 0–255, and `insert(0z..., 300)` fails with E475. The mutating functions `sort`, `reverse`, `map`, `filter`, `add`, and `extend` all mutate in place and return the same container, which is why idiomatic code copies first. On the numeric side, % with a Float operand is E804 even though * and / accept floats; integer / truncates toward zero, so `7 / 2` is 3; `isinf` is three-valued (1/-1/0); and a leading-zero integer is octal only if every following digit is octal, so `0129` is decimal 129. The bitwise builtins `and/or/xor/invert` take integers and their results lower to native ops. Each quirk is asserted by the family’s example file, and each is a place the port’s fidelity is measurable to the bit.

The data layer in one program

Seeing these families combine is more instructive than seeing them apart, and `lists_dicts.vim` is the program that does it — a single file exercising lists, dicts, the higher-order functions, the numeric comparisons, and both dict literals together. The four transformations near its top each draw on a different family member while protecting the source with `copy`:

```
let nums = range(1, 10)
let evens = filter(copy(nums), 'v:val % 2 == 0')
let squared = map(copy(nums), 'v:val * v:val')
let total = reduce(nums, function('Add'), 0)
let desc = reverse(sort(copy(nums), 'n'))
```

`range` builds the list; `filter` keeps the evens via a string-expression predicate; `map` squares each via a string body; `reduce` folds it with a named-function Funcref; `sort` orders numerically and `reverse` flips. Five lines, and the three callable forms (string, funcref, lambda) all appear, all interchangeable. The frequency-dict idiom that follows combines `get` and index-assignment — `let freq[w] = get(freq, w, 0) + 1` — and the assertions check the lot: `evens` is `[2, 4, 6, 8, 10]`, `total` is 55, the slice `nums[2:4]` is the three-element `[3, 4, 5]`, and `#{a:1, b:2}` equals `{'a': 1, 'b': 2}`. One program, and you have seen the building, transforming, measuring, folding, and literal-construction functions of all four families working together — which is exactly how they appear in real code, never one family at a time but composed into a computation over data.

Chapter 28 – Files, Buffers, and Lines

The Files, Buffers & Lines family is the second largest in the reference at seventy-seven functions, and it is the family where the standalone split of Chapter 18 runs right down the middle: the filesystem functions do real work, and the buffer-numbered functions are mostly stubs because there is no editor managing many buffers. This chapter walks the whole family, marking which side of the line each function falls on, because that mark is the single most useful thing to know about any function here.

Reading and writing files

The file-I/O functions are real, and they are what make vimlrs a usable text-processing tool. `readfile({fname} [, {type} [, {max}]]]` returns “a List of lines read from file {fname},” and `writefile({object}, {fname} [, {flags}])` writes “a List of lines or Blob {object} to file {fname}; 0 on success” — the round-trip `wordfreq.vim` builds on. `readblob({fname} [, {offset} [, {size}]]]` reads “a Blob of the (partial) contents,” the binary counterpart. `readdir({directory} [, {expr}])` lists “file names in {directory}, optionally filtered by {expr}.” And `delete({fname} [, {flags}])` deletes a “file (or, with ‘d’/‘rf’, directory),” `rename({from}, {to})` moves a file, `filecopy({from}, {to})` copies one, and `mkdir({name} [, {flags} [, {prot}]]]` creates a directory, “‘p’ makes parents.” All operate on the real filesystem, faithfully ported, exactly as they would inside Neovim.

Querying the filesystem

A large cluster inspects files and paths without modifying them, and these are real too. `filereadable({file})`, `filewritable({file})` (“1 if writable, 2 if a writable directory”), `isdirectory({directory})`, and `isabsolutepath({path})` are the predicates. `getfsize({fname})` returns the “size in bytes (-1 if absent, -2 if too large),” `getftime({fname})` the modification time, `getftype({fname})` the type (“file”, “dir”, “link”, “fifo”, etc.), and `getfperm/setfperm` read and write the “9-character ‘rwxrwxrwx’” permission string. The path manipulators are `fnamemodify({fname}, {mods})`, which applies “the :p:/h:/t:/r:/e {mods}”; `simplify({filename})`, which resolves “‘.’, ‘..’ and doubled separators (no disk access)”; `resolve({filename})`, which follows symbolic links; and `pathshorten({path} [, {len}])`, which abbreviates directory components. `glob({expr} ...)` and `globpath({path}, {expr} ...)` expand wildcards (the real functions `glob.vim` tests), `glob2regpat({string})` converts a glob to a regex (Chapter 24), and `findfile/findidir` search a path. `executable({expr})` and `exepath({expr})` find programs in `$PATH`, `getcwd` and `chdir` read and change the working directory, and `tempname()` returns “a unique, not-yet-existing temporary file.” Every one of these is genuinely implemented, which is what lets a vimlrs script do real filesystem work.

The virtual buffer’s line functions

The line-level functions operate on the in-memory buffer of Chapter 24, and they are real for the single current buffer. `getline({lnum} [, {end}])` returns “the text of line {lnum} (or List of lines {lnum}..{end}),” and `setline({lnum}, {text})` sets “line {lnum} to {text} (String or List of lines).” `append({lnum}, {text})` appends “below line {lnum},” and `deletebufline({buf}, {first} [, {last}])` deletes a range — the trio that, with the Ex commands, makes the buffer editable. The position functions `line({expr} [, {winid}])`, `col({expr} [, {winid}])`, `charcol`, `virtcol`, and `virtcol2col` resolve buffer positions,

with `virtcol` expanding tabs to ‘tabstop’ as `registers.vim` checks. `indent({lnum})` returns the “indent (in spaces) of line {lnum},” `nextnonblank/prevnonblank` find non-blank lines, and `byte2line/line2byte` convert between line numbers and byte offsets. `cindent`, `lispindent` compute language-aware indents. `getbufinfo([{buf}])` describes the buffer (Chapter 24), and `wordcount` (catalogued under Functional & Misc) counts its bytes, characters, and words.

The buffer-numbered stubs

The other half of the family is the multi-buffer surface, and here the stub label dominates. `bufnr([{buf} [, {create}]]]`, `bufname`, `bufexists`, `buflisted`, `bufload`, `bufloaded`, `bufadd`, `bufwinid`, `bufwinnr`, `winbufnr`, `last_buffer_nr`, and `changenr` are all “stub in standalone,” because they address a plurality of buffers that does not exist — there is one virtual buffer, not a numbered list of them. The buffer-content functions that take a buffer number — `getbufline`, `getbufoneline`, `appendbufline`, `setbufline` — operate on the single virtual buffer when asked about it and otherwise return emptiness. The buffer-variable accessors `getbufvar/setbufvar` follow the {def}-fallback contract of Chapter 18, returning the default for a missing variable and accepting the setter as a no-op. And the editor-only file functions — `browse`, `browsedir` (GUI file dialogs), `getfontname`, `swapname`, `undofile`, `undotree`, `swapinfo`, `swapfilelist` — are stubs or compute the path Vim *would* use without an editor behind them. The pattern is the one Chapter 28 opened with: a function here is real if it touches the filesystem or the one current buffer, and a stub if it addresses the editor’s buffer-list, window, or GUI — and the reference’s “stub in standalone” label tells you which at a glance.

Signature reference: the file and buffer functions

For quick lookup, here are the family’s functions with their exact signatures and glosses, grouped as above. The file-writing and -reading functions: `readfile({fname} [, {type} [, {max}]]]` reads “a List of lines”; `writefile({object}, {fname} [, {flags}])` writes “a List of lines or Blob ... 0 on success”; `readblob({fname} [, {offset} [, {size}]]]` reads “a Blob of the (partial) contents”; `readdir({directory} [, {expr}])` lists “file names ... optionally filtered by {expr}”; `delete({fname} [, {flags}])` deletes “a file (or, with ‘d’/‘rf’, directory)”; `rename({from}, {to})` renames; `filecopy({from}, {to})` copies “v:true on success”; `mkdir({name} [, {flags} [, {prot}]]]` creates “directory {name} (‘p’ makes parents)”; and `tempname()` returns “a unique, not-yet-existing temporary file.”

The filesystem queries: `filereadable({file})`, `filewritable({file})` (“1 if writable, 2 if a writable directory”), `isdirectory({directory})`, `isabsolutepath({path})`, `getsize({fname})` (“-1 if absent, -2 if too large”), `getftime({fname})`, `getftype({fname})` (“‘file’, ‘dir’, ‘link’, ‘fifo’, etc.”), `getfperm({fname})` / `setfperm({fname}, {mode})` (the “9-character ‘rwxrwxrwx’”), `executable({expr})`, `exepath({expr})`, `getcwd([{winnr} [, {tabnr}]]]`, `chdir({dir})` (returns “the previous directory”), and `haslocaldir([{winnr} [, {tabnr}]]]`. The path manipulators: `fnamemodify({fname}, {mods})` (the “p:h:t:r:e {mods}”), `simplify({filename})` (“no disk access”), `resolve({filename})`, `pathshorten({path} [, {len}])`, `glob({expr} [, {nosuf} [, {list} [, {alllinks}]]])`, `globpath({path}, {expr} ...)`, `glob2regpat({string})`, `findfile({name} [, {path} [, {count}]]]`, and `finddir({name} [, {path} [, {count}]]]`.

The buffer-line functions: `getline({lnum} [, {end}])`, `setline({lnum}, {text})`, `append({lnum}, {text})` (“below line {lnum}; 0 on success”), `deletebufline({buf}, {first} [, {last}])`, `getbufline({buf}, {lnum} [, {end}])`, `getbufoneline({buf}, {lnum})`, `appendbufline({buf}, {lnum}, {text})`, `setbufline({buf}, {lnum}, {text})`, and `getbufinfo([{buf}])`. The positions and indents: `line({expr} [, {winid}])`, `col({expr} [, {winid}])`, `charcol({expr} [, {winid}])`, `virtcol({expr} [, {list} [, {winid}]]]`, `virtcol2col({winid}, {lnum}, {col})`, `indent({lnum})`, `cindent({lnum})`, `lispindent({lnum})`, `nextnonblank({lnum})`, `prevnonblank({lnum})`, `byte2line({byte})`, and `line2byte({lnum})`. The buffer-list stubs: `bufnr([{buf} [, {create}]]]`, `bufname([{buf}])`, `bufexists({buf})`, `buflisted({buf})`, `bufload({buf})`, `bufloaded({buf})`, `bufadd({name})`,

`bufwinid({buf})`, `bufwinnr({buf})`, `winbufnr({nr})`, `last_buffer_nr()`, `changenr()`, `getbufvar({buf}, {varname} [, {def}])`, and `setbufvar({buf}, {varname}, {val})`. And the editor-file functions: `browse`, `browsedir`, `getfontname`, `getchangelist({buf})`, `swapname({buf})`, `swapinfo({fname})`, `swapfilelist()`, `undofile({name})`, and `undotree([{buf}])`. Seventy-seven entries, every one with its exact arity, half doing real work and half honestly stubbed — the file family in full.

Edge cases worth knowing

The file family has its own faithful quirks. `getfsize` returns `-1` for a file that does not exist and `-2` for one too large to measure, distinguishing the two failure modes rather than collapsing them. `delete` deletes a file by default but takes the flag `'d'` to remove an empty directory and `'rf'` to remove a directory recursively — so the same function deletes a file or a tree depending on the flag, and `glob.vim`'s cleanup uses `delete(dir, 'd')`. `mkdir` with the `'p'` flag creates parent directories as needed, the analogue of `mkdir -p`. `glob`'s list form returns *sorted* matches, which is why `glob.vim` can assert the exact list, and a no-match yields the empty list (or empty string in the string form), never an error. `simplify` resolves `.` and `..` purely textually with “no disk access,” so it works on a path that does not exist, while `resolve` follows real symbolic links and needs the file. `getline` past the end of the buffer returns the empty string, and `getline(1, '$')` returns all lines as a list. `undofile` computes the undo-file path Vim would use even with no undo history (`undo.vim`). Each quirk is asserted by the family's example, and each is a place the “-1 here, -2 there, sorted list, this flag means that” specificity of a real port shows.

A file pipeline in practice

The file family's real members combine into the text-processing role that most justifies the standalone premise, and `wordfreq.vim` is the program that shows it — read a file, tokenize it, count, rank, report. The file-I/O round-trip is the spine:

```
let tmp = tempname()
call writefile(['the quick brown fox', 'the lazy dog', 'the fox jumps'], tmp)
let lines = readfile(tmp)
call delete(tmp)
```

`tempname` gives a unique path, `writefile` writes a list of lines to it, `readfile` reads them back, and `delete` cleans up — four file functions, all real, all operating on the actual filesystem. The rest of the program leaves the file family for the container and string families (tokenize with `split`, count into a `freq` dict, sort the pairs with a comparator), but the file functions are what make it a *file*-processing program rather than a computation over hard-coded data. This is the family at work: not `readfile` in isolation but `tempname`, `writefile`, `readfile`, and `delete` composing into a genuine file round-trip, the same operations a script would use to read a real input file, process it, and write a real output — exactly the awk-or-quick-Python task Chapter 1 said vimlrs is for, done in faithfully-ported Vimscript running as a standalone program.

Chapter 29 — System and Environment

The System & Environment family is forty-four functions that connect a script to the world outside the interpreter: the shell, the environment, the clock, serialization formats, and the inter-process and language-bridge mechanisms that have no standalone backing. As with the file family, the split between real and stub is the organizing fact, and this chapter walks both halves.

Shelling out and the environment

The process-interaction functions are real and are what let a vimlrs script take its place in a pipeline. `system({cmd} [, {input}])` runs “shell {cmd}, returning its output as a String,” and `systemlist` returns it “as a List of lines,” both setting `v:shell_error` and both accepting optional stdin input — the interface `system.vim` exercises. `environ()` returns “a Dict of all environment variables,” `getenv({name})` the “value of environment variable {name}, or v:null if unset,” and `setenv({name}, {val})` sets one (“v:null unsets it”) — the functional surface whose `v:null`-versus-`' '` distinction `env_vars.vim` pins down. `getpid()` is the “process ID,” `hostname()` the “name of the host,” and `stdpath({what})` resolves “standard config/data/cache/state path(s),” the XDG resolver `stdpath.vim` checks. `localtime()` is the “current time as seconds since the Unix epoch.”

Time and serialization

The high-resolution timer trio is real, ported from the profile sources: `reltime([start] [, {end}])` returns “a high-resolution time value, or an elapsed interval,” `reltimefloat({time})` renders it “as a Float number of seconds,” and `reltimestr({time})` “as a ‘seconds.microseconds’ String.” `strftime({format} [, {time}])` formats a time and `strptime({format}, {timestring})` parses one back — the pair for timestamping and reading timestamps. The serialization codecs are the faithful interop of Chapter 16: `json_encode({expr})` and `json_decode({string})` are the JSON pair (inverse encoders), and `msgpackdump({list} [, {type}])` and `msgpackparse({data})` the byte-exact MessagePack pair whose minimal-width packing `msgpack.vim` asserts. These, with `sha256` from the string family, are the pure data transforms that port cleanly precisely because they touch no editor.

The timer functions deserve a closer look because they are how a standalone script measures itself, the way a profiler or a benchmark would. `reltime()` with no arguments returns an opaque high-resolution time value; `reltime(start)` returns the interval since `start`; and `reltime(start, end)` the interval between two such values. The opaque value is then rendered by `reltimefloat`, which gives “a Float number of seconds” suitable for arithmetic, or `reltimestr`, which gives “a ‘seconds.microseconds’ String” suitable for display. So the idiom is: capture `let t = reltime()`, do work, then `let elapsed = reltimefloat(reltime(t))` — start, work, measure. These are ported from the `proftime_T` helpers in Neovim’s profile and time sources, using the OS’s monotonic high-resolution clock, so the measurements are real wall-clock intervals, not approximations. `strftime` and `strptime` are the calendar-time pair — format a timestamp for a log line, parse one back from a file — and `localtime` gives the current epoch seconds. Together they let a standalone script timestamp its output, measure its own performance, and parse timestamps from data it reads, all without an editor and all with genuine OS-backed time, which is exactly the toolkit a script that wants to profile a hot loop or stamp a log reaches for.

The bridges that are stubs

The remaining System functions are the ones that need a runtime, an event loop, or a peer process that a standalone interpreter does not host, and they are honestly stubbed. The job functions — `jobstart({cmd} [, {opts}])` (returns “a job id (or <=0 on failure)”), `jobstop`, `jobpid`, `jobwait`, `jobresize` — fail cleanly because there is no event loop to run a job in. The channel and socket functions — `chanclose`, `chansend`, `sockconnect`, `stdioopen` — are stubs for the same reason. The RPC and server functions — `rpcnotify`, `rpcrequest`, `rpcstart`, `rpcstop`, `serverstart`, `serverstop`, `serverlist` — have no server to talk to. The language-evaluation bridges — `luaeval`, `py3eval`, `pyeval`, `pyxeval`, `perleval`, `rubyeval` — would each embed another interpreter vimlrs does not carry, and return their inactive values. And `libcall/libcallnr` (dynamic-library calls), `termopen` (a terminal), and `windowsversion` (a Windows version string) round out the stubs. The contract is Chapter 18’s exactly: each function exists, takes its arguments, and returns the documented failure value, so a script that calls `jobstart` and checks for the <=0 failure adapts rather than crashing. The real half of this family — shell, environment, time, serialization — is the half a text-processing script actually needs; the stub half is the editor-and-runtime machinery it does not.

Signature reference: the system and environment functions

The real functions with their signatures: `system({cmd} [, {input}])`, `systemlist({cmd} [, {input}])`, `environ()`, `getenv({name})` (“or v:null if unset”), `setenv({name}, {val})` (“v:null unsets it”), `getpid()`, `hostname()`, `stdpath({what})`, `localtime()`, `reltime([start] [, {end}])`, `reltimefloat({time})`, `reltimestr({time})`, `strftime({format} [, {time}])`, `strptime({format}, {timestring})`, `json_encode({expr})`, `json_decode({string})`, `msgpackdump({list} [, {type}])`, and `msgpackparse({data})`. The stub functions: the jobs `jobstart({cmd} [, {opts}])`, `jobstop({id})`, `jobpid({id})`, `jobwait({ids} [, {timeout}])`, `jobresize({id}, {width}, {height})`; the channels `chanclose({id} [, {stream}])`, `chansend({id}, {data})`, `sockconnect({mode}, {address} [, {opts}])`, `stdioopen({opts})`; the RPC and server functions `rpcnotify({channel}, {event} [, {args} ...])`, `rpcrequest({channel}, {method} [, {args} ...])`, `rpcstart({prog} [, {argv}])`, `rpcstop({channel})`, `serverstart([address])`, `serverstop({address})`, `serverlist()`; the language bridges `luaeval({expr} [, {expr}])`, `py3eval({expr})`, `pyeval({expr})`, `pyxeval({expr})`, `perleval({expr})`, `rubyeval({expr})`; and `libcall({lib}, {func}, {arg})`, `libcallnr({lib}, {func}, {arg})`, `termopen({cmd} [, {opts}])`, and `windowsversion()`. Forty-four entries: the eighteen real ones are the shell, environment, clock, and serialization functions a script reaches for, and the twenty-six stubs are the inter-process, RPC, and language-bridge machinery a standalone interpreter has no peer to talk to.

Edge cases worth knowing

The system family’s quirks are mostly about the failure-value contract. `system` sets `v:shell_error` to the command’s exit status, so a script reads the output from `system` and the status from `v:shell_error` — two separate reads, as `system.vim` shows with `call system('exit 7')` followed by `let code = v:shell_error`. `system`’s optional second argument is `input`, fed to the command’s stdin, so `system('tr a-z A-Z', 'quiet')` returns `'QUIET'`. `getenv` returns `v:null` for an unset variable (distinguishable from the empty string the `$X` sigil returns), and `setenv(name, v:null)` removes a variable. `stdpath` raises `E6100` for an unknown kind rather than returning an empty string, and its `*_dirs` kinds return a List while the singular kinds return a String. The job functions return their documented failure values — `jobstart` returns `-1` (or any value <= 0), `jobwait` returns `[]` — so a script can detect the absence of an event loop. And the language bridges (`py3eval`, `perleval`, and the rest) return `v:null`, which a script can test with `isnot v:null` to detect whether a provider is available, the guard `providers.vim` demonstrates. Each is the value Vim returns

for the corresponding absent or failed case, chosen so a defensive script adapts.

A shell pipeline in practice

The system family's real members combine into the shell-citizen role Chapter 1 promised, and `system.vim` is the program that shows it — a vimlrs script doing what you would otherwise open a shell for:

```
let greeting = system('echo hello')
let lines = systemlist('printf "one\ntwo\nthree\n"')
let shout = system('tr a-z A-Z', 'quiet')
call system('exit 7')
let code = v:shell_error
let env = environ()
```

`system` captures stdout including the trailing newline; `systemlist` splits output into a list of lines; `system('tr a-z A-Z', 'quiet')` feeds 'quiet' on stdin and gets back 'QUIET'; `system('exit 7')` runs a command for its status alone, which `v:shell_error` then reports as 7; and `environ()` returns the process environment as a dict in which `PATH` is present. Six lines, and the script has shelled out, captured output two ways, fed input, read an exit status, and inspected the environment — every one a genuine OS interaction, ported faithfully, working with no editor in sight. This is the family at work as a unit: not `system` in isolation but `system`, `systemlist`, `v:shell_error`, and `environ` composing into a script that takes its place in a pipeline, the role awk and Perl one-liners have always played and that vimlrs can now play in Vimscript.

Chapter 30 – The Functional Surface I: Testing, Iteration, and Introspection

The reference’s Functional & Misc chapter holds two hundred and ten entries — the catch-all where everything that is not a clean data-structure operation lands — and it is large enough to need three chapters of its own. This first one covers the parts that are most real: the testing framework every example depends on, the higher-order functions and metaprogramming bridges, the argument list, and the introspection functions that report on the running program.

The assert framework

The assert family carries a handful of edge behaviours worth naming, because they are what make the self-test pattern work. The asserts do not throw — each *appends* to `v:errors` on failure and returns a value (0 for pass, 1 for fail), so a script runs all its checks and accumulates failures rather than dying at the first. This is why `equalfile.vim` and `beeps.vim` can test the *failure* path by calling an assert, checking that it recorded one error, and then clearing `v:errors` with `let v:errors = []` so the expected failure does not fail the script. The error-path testers behave specially: `assert_fails` and `assert_beeps/assert_nobeep` run a command and require it to error (or “beep”), and crucially they *capture* the errors the command produces rather than letting them reach `v:errors` — so an `assert_fails('call nope()', 'E117')` that succeeds leaves `v:errors` clean. The message wording follows `eval.lua`, Neovim’s test-spec source, so a failure message reads the same as it would in a Neovim test. And the whole self-test epilogue — inspect `v:errors`, `:throw` if non-empty — converts the accumulated failures into the non-zero exit the CLI driver reports, which CI catches. The asserts are the foundation everything else stands on, and their non-throwing, error-capturing, `eval.lua`-worded behaviour is exactly what makes the examples both documentation and a regression suite.

The assert family is the most-used set of functions in the entire codebase, because every example is a self-test built on it. None of them throw; each “adds an error to `v:errors`” on failure, so a script runs all its checks and reports them together. `assert_equal({expected}, {actual} [, {msg}])` and `assert_notequal` are the value checks; `assert_true({actual} [, {msg}])` and `assert_false` the boolean checks; `assert_match({pat}, {actual} [, {msg}])` and `assert_notmatch` the pattern checks; and `assert_inrange({lower}, {upper}, {actual} [, {msg}])` the range check (“unless {lower} <= {actual} <= {upper}”). The error-path testers are `assert_exception({error} [, {msg}])`, used in a `:catch` to check that “v:exception contains {error},” and `assert_report({msg})`, which adds a message “unconditionally” — the loud-failure tool when a throw that should have happened did not. `assert_equalfile({fname1}, {fname2} [, {msg}])` compares two files’ contents. Together with `assert_fails` (which runs a command and requires it to error, catalogued among the test helpers) they are the whole testing vocabulary `testing.vim` demonstrates, and the message wording follows `eval.lua` so it matches Neovim’s own test output.

Higher-order and metaprogramming

The higher-order functions `map`, `filter`, and `reduce` are catalogued with the list family, but their dispatch partners live here. `call({func}, {arglist} [, {dict}])` invokes “a function with the List {arglist} as its arguments,” reaching builtins and user functions alike — the form `call('printf', ['%d-%d', 3, 4])` uses. `eval({string})` evaluates “a Vim expression (inverse of `string()`),” and `execute({command} [, {silent}])` runs “Ex command(s) and returns their captured output” — the run-string pair of Chapter 12, whose

`execute('echo 5')` captures `"\n5"` with its leading newline. (The reference flags `call/eval/execute` “stub in standalone” in their doc strings, a conservatism left over from before they were wired; the `execute_capture.vim` and `sourcing.vim` examples assert they work, and in this codebase the CI-run example is the authority over a stale doc flag.) `foldtext()` and `foldtextresult({lnum})` compute fold display text, mostly empty standalone.

The three callable forms, side by side

The higher-order functions accept a callable in three interchangeable forms, and it is worth seeing them together because the interchangeability is a real Vimscript feature that vimlrs’s filter/map machinery (the `FILTER_MAP_EVAL_HOOK` of Chapter 22) makes work. The first is a *string expression*, evaluated against each element with `v:val` (and, over a Dict, `v:key`) bound — `filter(copy(nums), 'v:val % 2 == 0')` keeps the even values, the predicate being a string the builtin evaluates per element. The second is a *Funcref*, named or builtin — `reduce(nums, function('Add'), 0)` folds with the named `Add` function. The third is a *lambda* — `map([1,2,3,4], {i, v -> v * v})` squares each element with an inline anonymous function. All three appear in `lists_dicts.vim` within a few lines, and they are genuinely interchangeable: anywhere one is accepted, the others are too. The mechanism is the re-entry hook — the ported `f_map/f_filter` cannot evaluate a body on their own, so they call back into the VM through a thread-local hook that handles whichever form the caller supplied, binding `v:val/v:key` for the string form, invoking the funcref for the funcref form, calling the generated `<lambda>N` for the lambda form. This is the seam that makes the standard higher-order functions feel native rather than bolted on: a faithful ported list function reaching back into compiled VimL to run a user-supplied body, in any of the three forms Vimscript allows.

The argument list

The argument-list functions report on the files named on the command line, which in standalone vimlrs is the script-file arglist of `arglist.vim`. `argc([winid])` is the “number of files in the argument list,” `argidx()` the “current index,” `argv([nr] [, {winid}])` the “argument-list entry {nr} (or the whole List),” and `arglistid([winnr] [, {tabnr}])` the “identifier of the argument list” — `0` for the global, unnamed list a standalone run uses. They are real: `argv(0)` is the script’s own path, an out-of-range index is the empty string rather than an error, and `argc()` counts the files passed on the command line.

Introspecting the running program

A cluster of functions reports on vimlrs itself. `has({feature} [, {check}])` is the feature detector — “1 if Vim supports {feature}” — whose honest answers (true for the language features it implements, false for editor subsystems and even for `nvim`) are the keystone of Chapter 18’s contract. `exists({expr})` tests whether a name is defined. `id({expr})` returns “a unique String identifier for the container or value {expr},” the identity primitive underneath `is`. `mode([{expr}])` reports “the current mode as a short String,” `'n'` standalone since there is no interactive mode. `api_info()` describes “the Nvim API (version, functions, types).” `getscriptinfo([{opts}])` lists “the sourced Vim scripts,” and `getstacktrace()` describes “the current Vimscript call stack” — both real reflections of the running program’s state. The remaining introspection functions — `eventhandler`, `did_filetype`, `interrupt`, `debugbreak`, `state` — are stubs, because they report on or control editor activity (an event handler, a filetype autocommand, a Ctrl-C interrupt) that a batch run does not have. The real members of this group are exactly the ones a script uses to ask “what am I running on, and what can I do?” — and they answer truthfully, which is the whole basis of a script adapting to the standalone environment rather than assuming an editor.

Signature reference: testing, iteration, args, and introspection

The assert functions with their signatures: `assert_equal({expected}, {actual} [, {msg}])`, `assert_notequal({expected}, {actual} [, {msg}])`, `assert_true({actual} [, {msg}])`, `assert_false({actual} [, {msg}])`, `assert_match({pat}, {actual} [, {msg}])`, `assert_notmatch({pat}, {actual} [, {msg}])`, `assert_inrange({lower}, {upper}, {actual} [, {msg}])`, `assert_exception({error} [, {msg}])`, `assert_report({msg})`, and `assert_equalfile({fname1}, {fname2} [, {msg}])` — each “adds an error to `v:errors` unless” its condition holds. (The error-path testers `assert_fails`, `assert_beeps`, and `assert_nobeep`, which `beeps.vim` exercises, run a command and require it to error or to “beep,” capturing the error rather than leaking it to `v:errors`.) The higher-order and metaprogramming functions: `call({func}, {arglist} [, {dict}])`, `eval({string})`, `execute({command} [, {silent}])`, `foldtext()`, and `foldtextresult({lnum})`; the in-place `map({expr1}, {expr2})` and `filter({expr1}, {expr2})` are catalogued with the list family.

The argument-list functions: `argc([winid])`, `argidx()`, `argv([nr] [, winid])`, and `arglistid([winnr] [, tabnr])`. The introspection functions: `has({feature} [, {check}])`, `exists({expr})`, `id({expr})`, `mode([{expr}])`, `state([{what}])`, `api_info()`, `getscriptinfo([opts])`, `getstacktrace()`, `eventhandler()`, `did_filetype()`, `interrupt()`, and `debugbreak({pid})`. The first set is the most real surface in the whole Functional & Misc family — the asserts every example depends on, the higher-order dispatch, the argument list, and the honest feature-detection of `has` — and it is no accident that the most-used functions are also the most faithfully real: they are the ones a standalone script genuinely needs, and the project ported them first.

The testing vocabulary in practice

The assert family is best seen as a unit, and `testing.vim` is the program that uses the whole vocabulary on one small function under test — parse `KEY=VALUE`, throwing on malformed input. The happy-path checks are ordinary value asserts, but the error-path testers are the point:

```
try
  call ParseKV('garbage')
  call assert_report('ParseKV should have thrown on bad input')
catch
  call assert_exception('E0:')
endtry
call assert_fails("call ParseKV('nope')", 'E0:')
call assert_fails('call no_such_function()', 'E117:')
call assert_inrange(1, 10, 7)
call assert_match('\d\+', 'abc123')
```

The first block tests the throw two ways: it calls `ParseKV('garbage')` expecting a throw, and if the throw does *not* happen, control reaches `assert_report`, which records a failure unconditionally — the test asserts the throw by failing loudly if it is absent. If the throw does happen, the bare `:catch` catches it and `assert_exception('E0:')` checks the thrown text. The more compact `assert_fails` packages the same idea: run a command, require it to error, optionally match the code — `no_such_function()` must raise `E117` (Vim’s faithfully-numbered unknown-function error). And `assert_inrange`, `assert_match`, `assert_notmatch` cover range and pattern. One file, and you have seen the whole testing vocabulary — value equality, range, pattern, and the exception path two ways — which is the family at work as a unit, and which is exactly the toolkit vimlrs’s own examples use at scale.

Chapter 31 – The Functional Surface II: Cursor, Registers, Search, and the Command Line

This second Functional & Misc chapter covers the functions that operate on the virtual buffer’s editing state — the cursor and positions, the registers, marks, search, the highlight-match list, and the command line. Many of these are real, because they touch the in-memory buffer and cursor of Chapter 24 rather than the absent window system, and the chapter notes which are which.

The cursor and positions

The position functions read and write places in the buffer. `cursor({lnum}, {col} [, {off}])` moves “the cursor to line {lnum}, column {col},” and `setcursorcharpos` does the same with a character column. `getpos({expr})` returns the “position [bufnum, lnum, col, off] of {expr}” — a mark, `'.'` for the cursor, `'$'` for the end — and `setpos({expr}, {list})` writes one, the pair `marks.vim` and `search.vim` use. `getcurpos([winid])` adds the cursor’s “curswant,” and the `char`-suffixed variants `getcharpos`, `getcursorcharpos`, `setcharpos` work in character columns instead of byte columns. `getcharsearch/setcharsearch` carry the `f/t` character- search state. These are real on the virtual buffer: `cursor(2, 5)` genuinely moves the cursor, and `getpos('.')` reads it back, which is what makes the search and editing examples work.

Registers

The register functions are real and shared with the `:yank/put` commands (Chapter 24). `getreg([regname] [, {list} [, {opts}]]])` reads “the contents of register {regname},” `setreg({regname}, {value} [, {options}])` writes one “with optional type {options},” and `getregtype/getreginfo` report a register’s type (“v”, “V”, or blockwise”) and full info. The `@r` expression syntax and these functions touch the same store, which `registers.vim` verifies — `:2y c` and `@c` see the same value. `getregion({pos1}, {pos2} [, {opts}])` returns “the text lines for the region between {pos1} and {pos2},” charwise or linewise, and `getregionpos` the position pairs — the functions that extract a visual-style selection from the buffer by coordinates. The recording-state functions `reg_executing`, `reg_recorded`, `reg_recording` are stubs, because there is no interactive macro recording in a batch run.

Search and matches

The search functions build on the buffer, the cursor, and the regex engine, and they are real. `search({pattern} [, {flags} ...])` searches “for {pattern}; returns the matching line number, else 0,” moving the cursor unless the `n` flag is given; `searchpos` returns “the [line, col]” instead. `searchpair({start}, {middle}, {end} [, {flags} ...])` finds “the matching {end} of a nested {start}/{end} pair,” and `searchpairpos` its position — the functions that match a nested `if/endif`. `searchcount([options])` returns “a Dict of search match counts,” the position among matches `search.vim` checks. (`searchdecl` is a stub.) The highlight-match functions are a separate, mostly-stub

group: `matchadd({group}, {pattern} ...)` and `matchaddpos` “highlight {pattern} with {group}”; return the match id,” `matchdelete` removes one, `clearmatches` clears all, and `getmatches/setmatches` save and restore them. They maintain a match list but have no screen to highlight on, so they are data-real but visually inert standalone.

The command line

The command-line functions operate the settable in-memory command line of `cmdline.vim`, ported from `ex_getln.c`. `getcmdline()` reads “the current command-line contents,” `setcmdline({str} [, {pos}])` sets them “with the cursor at {pos},” and `getcmdpos/setcmdpos` read and move the cursor position. `getcmdtype()` reports “the type of the current command line (‘:’, ‘/’, ‘?’, ‘@’, ‘-’, ‘=),” and the completion-context functions `getcmdcomplat`, `getcmdcomptype`, `getcmdprompt`, `getcmdscreenpos`, `cmdcomplete_info` describe an in-progress completion. `getcmdwintype` and the wildmenu functions `wildmenu`/`wildtrigger` are stubs. The completion functions proper — `getcompletion({pat}, {type} [, {filtered}])` (“a List of command-line completion candidates”), `getcompletiontype`, `complete`, `complete_add`, `complete_check`, `complete_info`, `pum_getpos`, `pumvisible`, `preinserted` — are mostly stubs, because Insert-mode completion needs an interactive editor, though `getcompletion` computes real candidates for the contexts it supports. The input functions — `input`, `inputsecret`, `inputlist`, `inputdialog`, `confirm`, `getchar`, `getcharstr`, `getcharmod`, `feedkeys`, `inputsave`, `inputrestore` — prompt the user and so are stubs or read from stdin in a batch run. The pattern, as ever, is that the command-line *data* is real and the interactive *editing* of it is not.

Signature reference: positions, registers, search, and the command line

The position functions with their signatures: `cursor({lnum}, {col} [, {off}])`, `setcursorcharpos({lnum}, {col} [, {off}])`, `getpos({expr})` (“[bufnum, lnum, col, off]”), `setpos({expr}, {list})`, `getcurpos([winid])`, `getcharpos({expr})`, `getcursorcharpos([winid])`, `setcharpos({expr}, {list})`, `getcharsearch()`, and `setcharsearch({dict})`. The register functions: `getreg([regname] [, {list} [, {opts}]]])`, `setreg({regname}, {value} [, {options}])`, `getregtype([regname])` (“v”, “V”, or blockwise”), `getreginfo([regname])`, `getregion({pos1}, {pos2} [, {opts}])`, `getregionpos({pos1}, {pos2} [, {opts}])`, and the recording-state stubs `reg_executing()`, `reg_recorded()`, and `reg_recording()`.

The search functions: `search({pattern} [, {flags} [, {stopline} [, {timeout} [, {skip}]]]])`, `searchpos(...)` (same arguments, returns “the [line, col]”), `searchpair({start}, {middle}, {end} [, {flags} ...])`, `searchpairpos(...)`, `searchcount([options])`, and `searchdecl({name} [, {global} [, {thisblock}]]])`. The highlight-match functions: `matchadd({group}, {pattern} [, {priority} [, {id} [, {dict}]]])`, `matchaddpos({group}, {pos} [, {priority} [, {id} [, {dict}]]])`, `matchdelete({id} [, {win}])`, `clearmatches([win])`, `getmatches([win])`, `setmatches({list} [, {win}])`, and `matcharg({nr})` (“[pattern’, ‘group]”).

The command-line functions: `getcmdline()`, `setcmdline({str} [, {pos}])`, `getcmdpos()`, `setcmdpos({pos})`, `getcmdtype()` (“:”, “/”, “?”, “@”, “-”, “=”), `getcmdprompt()`, `getcmdcomplat()`, `getcmdcomptype()`, `getcmdscreenpos()`, `getcmdwintype()`, `cmdcomplete_info([what])`, `wildmenu`, and `wildtrigger`. The completion functions: `getcompletion({pat}, {type} [, {filtered}])`, `getcompletiontype({pat})`, `complete({startcol}, {matches})`, `complete_add({expr})`, `complete_check()`, `complete_info([what])`, `pum_getpos()`, `pumvisible()`, and `preinserted()`. And the input functions: `input({prompt} [, {text} [, {completion}]]])`, `inputsecret({prompt} [, {text}])`, `inputlist({textlist})`, `inputdialog({prompt} [,`

```
{text} [, {cancelreturn}]]), confirm({msg} [, {choices} [, {default} [, {type}]]]),  
getchar([ {expr} [, {opts}]]), getcharstr([ {expr} [, {opts}]]), getcharmod(),  
feedkeys({string} [, {mode}]), inputsave(), and inputrestore().
```

Edge cases worth knowing

The position-and-register functions carry a set of faithful return shapes worth knowing. `getpos` returns a four-element list `[bufnum, lnum, col, off]`, and `getcurpos` a five-element list that adds `curswant` — so code that reads a position must expect the right arity. An unset mark reads as `[0, 0, 0, 0]` from `getpos`, and `line("'z")` for an unset mark is 0. `matchadd` returns the match id it assigned (the explicit id if given, a positive auto-id otherwise), and its default priority is 10 when omitted; `matchdelete` returns 0 on success and -1 *quietly* for a missing id, rather than erroring (`matches.vim`). `search` returns the matching line number or 0, and skips a match at the cursor unless the `c` flag is given — a subtlety that catches scripts expecting it to find the match under the cursor. `searchpos` returns `[0, 0]` for no match. `getreg` reads a register's contents and `getregtype` its type (`'v'/'V'`/blockwise), with the register store shared with `:yank/put`. And the command-line `getcmdtype` returns one of a fixed set of characters or empty when no command line is active. Each shape is Vim's, asserted by the family's example, and each is a place a script that reads the wrong number of elements or expects an error instead of a quiet -1 would break.

Chapter 32 – The Functional Surface

III: Windows, Folds, Signs, and the Editor Tail

This third and final Functional & Misc chapter covers the largest and most stub-heavy region of the surface: the window and tab system, folds and diffs, syntax and highlighting, signs, timers, history, the quickfix and location lists, tags, and the remaining editor machinery. Most of these are stubs, because they address the window layout, the redraw, the event loop, or the syntax engine — the editor activity vimlrs has no occasion for. The value of walking them is seeing the full shape of the surface a faithful port must present, and which corners of it are live data versus honest emptiness.

Windows and tabs

The window functions are the densest stub cluster, because windows are exactly the display machinery a standalone interpreter lacks. `winnr([ {arg} ])` reports “the number of the current (or related) window” (1, the implicit window), `winheight({nr})` and `winwidth({nr})` return “-1” (unmeasurable with no display), and `winline/wicol`, `win_getid`, `win_gotoid`, `win_id2win`, `win_splitmove`, `win_move_separator`, `win_move_statusline`, `winrestcmd`, `winrestview` are all stubs. The descriptive functions `getwininfo`, `winlayout`, `winsaveview`, `win_screenpos`, `win_findbuf`, `win_id2tabwin`, `win_gettype`, `getwinpos` return the documented empty or single-window value — `winsaveview` returns a real dict for the one virtual window, as `expand.vim` checks. The window-variable accessors `getwinvar/setwinvar` follow the `{def}`-fallback contract. The tab functions mirror them: `tabpagenr` is 1, `tabpagewinnr` and `tabpagebuflist` describe the single implicit tab, and `gettabinfo`, `gettabvar/settabvar`, `gettabwinvar/settabwinvar` follow the same empty-or-default contract. `win_execute({id}, {command} ...)` runs a command “as if in window {id}” and is a stub.

Folds, diffs, syntax, and highlighting

The fold functions report no folds, the documented empty answer of Chapter 18. `foldclosed({lnum})` and `foldclosedend` return “-1,” `foldlevel({lnum})` returns 0 (every line at level 0, as `sysinfo.vim` checks), and `foldtext/foldtextresult` return empty. The diff functions `diff_filler` and `diff_hlID` report no diff. The syntax and highlight functions are stubs because there is no syntax engine running: `synID`, `synIDattr`, `synIDtrans`, `synconcealed`, `synstack` all return the absent value, and the highlight-group functions `hlID({name})` (“0 if undefined”), `hlexists`, `highlight_exists` report no defined groups. These are the subsystems most thoroughly tied to a live editor, and the contract returns the value Vim returns when the thing queried is absent.

Signs, marks, and the lists

The sign functions are a real in-memory data table (Chapter 24), even though there is no buffer column to draw in. `sign_define({name} [, {dict}])` defines a sign, `sign_getdefined([ {name} ])` lists the defined signs, and `sign_undefine` removes them — the table `signs.vim` exercises. The placement

functions `sign_place`, `sign_placelist`, `sign_getplaced`, `sign_unplace`, `sign_unplacelist`, `sign_jump` manage placements, mostly stubs since there is no buffer display. `getmarklist([{{buf}}])` lists the marks `marks.vim` sets — real buffer data. The quickfix and location lists — `getqflist/setqflist` and `getloclist/setloclist` — maintain real list data even without a window to show it in, and the tag functions `taglist({expr} [, {filename}])`, `tagfiles`, `gettagstack/settagstack`, `getjumplist`, `getchangelist` report tag and jump state, mostly empty standalone.

Timers, history, screen, and the remainder

The timer functions need an event loop and so are stubs: `timer_start({time}, {callback} [, {options}])` “returns its id” but never fires, and `timer_stop`, `timer_stopall`, `timer_pause`, `timer_info` manage the absent timers. The history functions are real data: `histadd({history}, {item})`, `histdel`, `histget`, `histnr` maintain “the {history} list (cmd/search/expr/input).” The screen-inspection functions — `screenattr`, `screenchar`, `screenchars`, `screencol`, `screenrow`, `screenpos`, `screenstring` — are stubs, because there is no rendered screen to read cells from. The remaining tail is editor machinery: the prompt-buffer functions `prompt_*` (six stubs), the context-stack functions `ctx*` (push/pop the editor context, stubs), the menu functions `menu_get/menu_info`, the spell functions `spellbadword/spellsuggest`, the mouse `getmousepos`, `visualmode`, `foreground`, `wait`, and the test helpers `test_garbagecollect_now/test_write_list_log`. The function `shiftwidth([{{col}}])` is real — the “effective ‘shiftwidth’ value (resolving 0 to ‘tabstop’)” the indent commands use — and `wordcount()` is real, the “Dict of byte/char/word counts” `registers.vim` checks. The honest accounting this chapter performs is the point: the editor’s display, event, and rendering subsystems are absent, their functions present and stubbed, and a script that consults them gets the documented empty value rather than a crash or a lie — which is exactly what lets a real Vim plugin, full of these calls, at least load and run under vimlrs.

Signature reference: windows and tabs

The window functions with their signatures: `winnr([{{arg}}])` (“number of the current ... window”), `winheight({nr})` and `winwidth({nr})` (“-1”), `winline()` and `wincol()` (cursor screen row/column within its window), `win_getid([{{win}} [, {tab}])` (“window-ID”), `win_gotoid({id})`, `win_id2win({id})`, `win_id2tabwin({id})` (“[tabnr, winnr]”), `win_findbuf({bufnr})` (“List of window-IDs”), `win_screenpos({nr})` (“[row, col]”), `win_gettype([{{nr}}])` (“‘popup’, ‘command’, ‘autocmd’, ...”), `win_splitmove({nr}, {target} [, {options}])`, `win_move_separator({nr}, {offset})`, `win_move_statusline({nr}, {offset})`, `win_execute({id}, {command} [, {silent}])`, `getwininfo([{{winid}}])`, `getwinpos([{{timeout}}])`, `getwinposx()`, `getwinposy()`, `getwinvar({nr}, {varname} [, {def}])`, `setwinvar({nr}, {varname}, {val})`, `winlayout([{{tabnr}}])`, `winrestcmd()`, `winsaveview()`, and `winrestview({dict})`. The tab functions: `tabpagenr([{{arg}}])`, `tabpagewinnr({tabnr} [, {arg}])`, `tabpagebuflist([{{arg}}])`, `gettabinfo([{{tabnr}}])`, `gettabvar({tabnr}, {varname} [, {def}])`, `settabvar({tabnr}, {varname}, {val})`, `gettabwinvar({tabnr}, {winnr}, {varname} [, {def}])`, and `settabwinvar({tabnr}, {winnr}, {varname}, {val})`.

Signature reference: folds, diffs, syntax, and highlighting

The fold and diff functions: `foldclosed({lnum})` and `foldclosedend({lnum})` (“-1”), `foldlevel({lnum})`, `foldtext()`, `foldtextresult({lnum})`, `diff_filler({lnum})`, and `diff_hlID({lnum}, {col})`. The syntax functions: `synID({lnum}, {col}, {trans})`, `synIDattr({synID}, {what} [, {mode}])`, `synIDtrans({synID})`, `synconcealed({lnum}, {col})`, and `synstack({lnum}, {col})`. The highlight-group functions: `hlID({name})` (“0 if undefined”), `hlexists({name})`, and `highlight_exists({name})` (the “alias of `hlexists()`”). All of these return the

documented absent value, because there is no syntax engine, no diff, and no loaded highlight table — the most thoroughly editor-bound corner of the surface.

Signature reference: signs, marks, lists, and tags

The sign functions: `sign_define({name} [, {dict}])`, `sign_getdefined([{{name}}])`, `sign_undefine([{{name}}])`, `sign_place({id}, {group}, {name}, {buf} [, {dict}])`, `sign_placelist({list})`, `sign_getplaced([{{buf}} [, {dict}])`, `sign_unplace({group} [, {dict}])`, `sign_unplacelist({list})`, and `sign_jump({id}, {group}, {buf})`. The mark and list functions: `getmarklist([{{buf}}])`, `getqflist([{{what}}])`, `setqflist({list} [, {action} [, {what}])`, `getloclist({nr} [, {what}])`, and `setloclist({nr}, {list} [, {action} [, {what}])` — the quickfix and location lists that are real data. The tag and jump functions: `taglist({expr} [, {filename}])`, `tagfiles()`, `gettagstack([{{winnr}}])`, `settagstack({winnr}, {dict} [, {action}])`, `getjumplist([{{winnr}} [, {tabnr}])`, and `getchangelist([{{buf}}])`.

Signature reference: timers, history, screen, and the remainder

The timer functions (all stubs): `timer_start({time}, {callback} [, {options}])`, `timer_stop({timer})`, `timer_stopall()`, `timer_pause({timer}, {paused})`, and `timer_info([{{id}}])`. The history functions (real data): `histadd({history}, {item})`, `histdel({history} [, {item}])`, `histget({history} [, {index}])`, and `histnr({history})`. The screen-inspection functions (stubs): `screenattr({row}, {col})`, `screenchar({row}, {col})`, `screenchars({row}, {col})`, `screencol()`, `screenrow()`, `screenpos({winid}, {lnum}, {col})`, and `screenstring({row}, {col})`. The prompt-buffer functions (stubs): `prompt_appendbuf`, `prompt_getinput`, `prompt_getprompt`, `prompt_setcallback`, `prompt_setinterrupt`, and `prompt_setprompt`. The context-stack functions (stubs): `ctxget([{{index}}])`, `ctxset({context} [, {index}])`, `ctxpush([{{types}}])`, `ctxpop()`, and `ctxsize()`. And the remainder: `menu_get({path} [, {modes}])`, `menu_info({name} [, {mode}])`, `spellbadword([{{sentence}}])`, `spellsuggest({word} [, {max} [, {capital}])`, `getmousepos()`, `visualmode([{{expr}}])`, `foreground()`, `wait({timeout}, {condition} [, {interval}])`, `shiftwidth([{{col}}])` (real — “resolving 0 to ‘tabstop’”), `wordcount()` (real), and the test helpers `test_garbagecollect_now()` and `test_write_list_log({fname})`. With these, the two-hundred-and-ten-entry Functional & Misc family is fully accounted for across Chapters 30, 31, and 32 — every function named, signed, and placed on the real or stub side of the standalone line.

What is real in the tail

It is worth collecting the *real* functions out of this stub-heavy chapter, because they are easy to lose among the editor machinery and they are the ones a standalone script can actually use. The history rings are real: `histadd`, `histdel`, `histget`, and `histnr` maintain genuine in-memory rings (`history.vim`). The quickfix and location lists are real: `getqflist/setqflist` and `getloclist/setloclist` keep real error-list data with the full normalized schema (`quickfix.vim`). The sign tables are real: `sign_define/sign_getdefined` and the placement functions maintain genuine maps (`signs.vim`, `sign_place.vim`). `getmarklist` reports real marks. `shiftwidth()` is real, computing the effective shift width by resolving a 0 ‘shiftwidth’ to ‘tabstop’, which the indent commands consult. And `wordcount()` is real, returning a dict of byte, character, and word counts for the buffer and up to the cursor (`registers.vim`). The common thread is the one this whole part of the book has drawn: these are *data* subsystems — rings, lists, tables, counts — that a standalone interpreter can keep faithfully, distinct from the *display and event* subsystems (windows, folds, syntax, timers, screen) that need a live editor and so return the documented empty value. Knowing which functions in the tail are real

is knowing which of them you can build a standalone script on, and the answer is: the ones that manipulate data, not the ones that drive a screen.

Chapter 33 – Ex Commands, Options, and Special Variables

The reference's last three chapters are the non-function surface: the six ex-commands it documents with hover text, the fifteen options it implements, and the three documented `v:` constants. They are short lists, but they anchor the language — the commands are how you do anything, the options how you tune matching and formatting, and the constants the truth values everything else coerces toward. This chapter walks all three, and folds in the larger statement and option surface the parser implements beyond what the reference documents with hover text.

The documented ex-commands

The reference documents six ex-commands, the ones the LSP serves hover text for. `:echo {expr}` evaluates “and displays {expr} (space-separated)”; `:echon {expr}` is “like :echo but without a trailing newline”; and `:echomsg {expr}` displays “and saves it in the message history.” `:let {var} = {expr}` assigns “the value of {expr} to {var},” the binding command whose many target forms Appendix A enumerates. `:call {func}({args})` calls “a function and discards its return value,” the way to invoke a function for its side effects. And `:eval {expr}` evaluates “{expr} for its side effects,” the command that crosses from the command layer into the expression layer purely to run an expression. The reason the reference documents only six ex-commands with hover text, while the parser recognizes dozens, is worth explaining, because it is a deliberate scoping of the *documentation* rather than a limit on the *language*. The six — `:echo`, `:echon`, `:echomsg`, `:let`, `:call`, `:eval` — are the commands with a single, unambiguous, one-line meaning that a hover tooltip can usefully state. A command like `:if` or `:try` or `:function` does not fit a one-line hover, because its meaning is bound up with a block structure and a matching terminator; a command like `:s` or `:g` carries a whole range and pattern grammar; a command like `:set` spans fifteen options. So the LSP-doc corpus documents the six that hover cleanly, and the parser — which has to *run* the language, not just describe a tooltip — recognizes the full vocabulary. This is the same documented-versus-working split the `v:` variables have: the documented surface is a curated subset chosen for what a reference can usefully say, and the working surface is everything the engine actually implements. A reader who wants the full command vocabulary reads Appendix E and the parser's statement dispatcher, not the six-entry hover list.

These six are the documented core; the parser recognizes the full statement vocabulary of Appendix E — `:if`, `:while`, `:for`, `:function`, `:try`, `:execute`, `:source`, `:runtime`, `:unlet`, `:delfunction`, `:const`, `:set`, the `:map` family, `:command`, `:autocmd`, the vim9script `def/enddef` and `var/const` forms, the no-op editor commands (`:nohlsearch`, the `:fold` family, the `:python/``:ruby/`... interface commands), and the Ex-command-with-range syntax of Chapter 24 — but the hover-doc corpus covers the six above because they are the ones with a single unambiguous one-line meaning.

The options

The fifteen options divide into the boolean toggles and the numeric settings, each rendered in the reference with its abbreviation, type, and default. The boolean options are `ignorecase` (`ic`, default 0, “ignore case in search patterns”), `smartcase` (`scs`, “override 'ignorecase' when the pattern has uppercase letters”), `magic` (default 1, “use 'magic' regexp special-character meanings”), `expandtab` (`et`, “insert spaces instead of a ”), `number` (`nu`) and `relativenumber` (`rlnu`) for line numbering, `wrap` (default 1, “wrap long lines”), `hlsearch` (`hls`)

and `incsearch` (`is`) for search display, and `autoindent` (`ai`, “copy the indent of the current line to a new line”). The numeric options are `tabstop` (`ts`, default 8, “number of spaces a counts for”), `shiftwidth` (`sw`, default 8, “number of spaces used for each step of (auto)indent”), `softtabstop` (`sts`, default 0), `textwidth` (`tw`, default 0, “maximum text width; 0 = off”), and `scrolloff` (`so`, default 0, “minimum lines kept above and below the cursor”). Each is settable through `:set`, `:setlocal`, or the `fopt` sigil, all reaching the same value (Chapter 24), and `'ignorecase'` and `'magic'` are wired into the regex engine so a bare `=~` honours them — `options.vim` shows `&shiftwidth` round-tripping and `:>` honouring it, while `excommands.vim` and `streamedit.vim` show `shiftwidth` driving the `:>/:<` shift. The options that affect formatting (`tabstop`, `shiftwidth`, `expandtab`) are the ones the buffer’s editing commands actually consult; the display options (`number`, `wrap`, `hlsearch`) are settable and readable but have no screen to affect.

The options divide, like the builtins, into the ones that do real work and the ones that are settable-but-inert standalone, and the dividing line is the same. `'ignorecase'` and `'magic'` are wired into the regex engine, so setting them genuinely changes how a bare `=~` or a `:%s` matches — `options.vim` shows `let &ignorecase = 1` taking effect. `'shiftwidth'`, `'tabstop'`, `'expandtab'`, and `'softtabstop'` drive the indent and shift commands, so setting `'shiftwidth'` changes what `:>` and the `:normal` indent operators do, and `registers.vim` and `streamedit.vim` show the shift honouring it. These are the options a text-processing script actually uses, and they are real. The display options — `'number'`, `'relativenumber'`, `'wrap'`, `'hlsearch'`, `'incsearch'`, `'cursorline'`-adjacent settings — are settable and readable (a script can `let &number = 1` and read it back) but have no screen to affect, so they are inert: the value is stored faithfully, but nothing draws line numbers or highlights a search. This is the same real-versus-inert split the builtins have, applied to options: an option is real if something standalone consults it (the regex engine, the indent commands) and inert if only a display would. A script that sets a display option does not error — the option exists and stores its value — it simply has no visible effect, which is exactly the right behaviour for a configuration that sets display options it does not strictly need.

The special variables

The reference documents three `v:` constants, the truth-and-null values everything else coerces toward. `v:true` is “Boolean true,” with “numeric value 1, string value ‘v:true,’” and `v:false` its counterpart with numeric value 0 — genuine Bool values, not aliases for 1 and 0, that render as their own names and survive a JSON round trip as `true/false`. `v:null` is the “Null value,” which “behaves like an empty string in most contexts” but keeps its own identity and rendering. These three are the documented constants; Appendix B catalogues the fuller working set the engine maintains beyond them — `v:none`, `v:val`, `v:key`, `v:errors`, `v:exception`, `v:throwpoint`, `v:shell_error`, `v:colnames`, and the rest — which the examples read even though the reference’s hover-doc list documents only the three constants. The distinction matters for accuracy: the three are the *documented* surface, the wider set is the engine’s *working* surface, and Appendix B keeps the two clearly apart.

The engine-maintained `v:` variables are worth naming as a group, because they are how the engine communicates with a running script. Some are set by the engine for the script to *read*: `v:errors` is the list the asserts append to, `v:exception` and `v:throwpoint` hold the in-flight exception’s text and origin inside a `:catch`, `v:shell_error` holds the last `system()`’s exit status, and `v:val/v:key` hold the current element and key inside a `map/filter` body. Others are constants the engine predefines: `v:true`, `v:false`, `v:null`, and `v:none`. The script does not assign most of these — the engine does, and the script reads them — which is the inverse of an ordinary variable, where the script writes and the engine merely stores. This is why `v:` is its own scope: it is the channel through which the engine hands a script the results of operations that have no return value (the asserts’ failures, the shell’s exit status) or the per-element context of a higher-order call (`v:val`). vimlrs maintains each of these in the bridge’s per-run state (Chapter 22), so reading a `v:` variable reads genuine engine state, not a faked value — `v:errors` is an ordinary list the asserts mutate, `v:exception` is the pending-exception cell’s text, `v:shell_error` is set after each `system`. The three documented constants are the tip of it; the engine-maintained set is the working channel, and a script that reads `v:errors` in its epilogue or `v:exception` in a `:catch` is using that channel exactly as Neovim’s would.

With the constants, the options, and the commands, the non-function half of the reference is complete — and

so, with the eight catalogue chapters behind it, is the full guided read of every implemented builtin, command, option, and constant the reference contains.

Chapter 34 — Autoload, Runtimepath, and the Sourcing Model

A Vimscript program is almost never a single file. The convention that grew up inside the editor — split the code across many small `.vim` files and load them lazily, by name, when something first needs them — is not a plugin-manager invention layered on top of the language; it is built into the language, into three mechanisms that the eval engine and the ex-command layer implement directly: `:source`, `:runtime`, and the autoload `#` name convention. In the editor these mechanisms are how a `vimrc` pulls in a hundred plugins without paying for the ones it never touches. In a standalone interpreter they are the module system, and the fact that they survive the removal of the editor intact is one of the quieter proofs of the book's thesis: sourcing, runtimepath search, and autoload were always language features that happened to live inside an editor, not editor features that happened to be scriptable. This chapter walks all three, in the order a real program grows into them.

`:source` — reading and running another file

The base case is `:source {file}`, which reads a file and executes it in the *current* scope, exactly as if its lines had been typed where the `:source` sits. There is no separate module namespace, no import list, no return value: whatever the sourced file defines — global functions, `g:` variables, user commands — simply exists afterward, because the sourced lines ran in the same interpreter state. The `sourcing.vim` example is the whole mechanism in miniature: it sources a library and then calls straight into it.

```
source examples/lib/mathlib.vim

call assert_equal(1, g:mathlib_loaded)
call assert_equal(16, Square(4))
call assert_equal(125, Cube(5))
call assert_equal([1, 4, 9], map([1, 2, 3], {i, v -> Square(v)}))
```

`mathlib.vim` defines `Square`, `Cube`, and sets `g:mathlib_loaded`; after the `source`, all three are live in the sourcing script, and the sourced functions compose with a lambda exactly as if they had been written inline. This is deliberately the least clever module system imaginable — it is textual inclusion with execution — and its faithfulness matters precisely because it is unclever: a real `vimrc` is a tree of `source` calls, and the behaviour a plugin depends on is that sourcing runs the file *now*, in *this* scope, with side effects that persist. vimlrs implements `:source` in its ex-command layer as a port of the same `do_source` path Vim uses, so a script that sources another script, which sources a third, sees the same left-to-right, depth-first execution order the editor would give it.

The one part of sourcing that has real subtlety is script identity. A sourced file is not anonymous: while its lines run, it *is* a script, with an `s:` scope of its own and a `<sfile>/<script>` path that names it. The `s:` variables and `s:`-prefixed functions a file declares are private to that file — visible to its own functions through the script scope, invisible to the file that sourced it — and this is what makes `:source` a module system rather than a glorified `#include`. Two libraries can each define `s:helper` without colliding, because each `s:helper` belongs to its own script identity. The `script_scope_isolation.vim` and `scriptlocal_funcname.vim` examples pin this: a `s:`-local function is reachable from inside its own file by its bare `s:Name`, resolves through the mangled script-local funcname the same way Vim's `<SID>` mechanism resolves it, and does not leak into the global function table that a sourcing file sees. In an editor the point of script-local scope is to keep a plugin's internals from polluting the global namespace shared by every other plugin; in standalone vimlrs the point is

identical, because a program built from sourced modules needs exactly that isolation to be buildable at all.

<sfile>, <script>, and knowing where you are

A sourced file frequently needs to know its own path — to find a sibling data file, to source a helper relative to itself, to compute a package root. Vim answers this with the special `expand()` tokens `<sfile>` and `<script>`, which resolve to the path of the script currently being sourced, and which accept the same trailing `:` filename-modifiers as `fnamemodify()`. vimlrs ports both, backed by a sourcing-name stack — the same structure Vim keeps so that `<sfile>` always names the innermost file — so the tokens resolve correctly even when one sourced file sources another. The `sfile.vim` example exercises the modifier grammar directly:

```
call assert_match('sfile\.\vim$', expand('<sfile>'))
call assert_equal('sfile.vim', expand('<sfile>:t'))
call assert_equal('vim',      expand('<sfile>:e'))
call assert_equal('sfile',    expand('<sfile>:t:r'))
call assert_match('examples$', expand('<sfile>:p:h'))
```

The modifiers compose the way they do in `fnamemodify()`: `:t` takes the tail, `:e` the extension, `:t:r` the tail with the extension rooted off, `:p:h` the head of the fully-qualified path — the containing directory. What makes `<sfile>` more than a convenience is that it is *dynamic with respect to the sourcing stack*: inside a `:source'd` helper, `<sfile>` is that helper's path, and when the helper returns, the outer file's `<sfile>` is restored. `sfile.vim` pins exactly this by sourcing a helper computed from its own path and checking that the token tracks the stack:

```
execute 'source ' . expand('<sfile>:p:h') . '/lib/sfile_helper.vim'
call assert_equal('sfile_helper.vim', g:sfile_helper_seen)
call assert_equal('sfile.vim', expand('<sfile>:t'))
```

The helper, while it runs, sees its own name; afterward the caller's is back. This is the sourcing stack made observable, and it is the mechanism a self-locating package uses to find its resources without hard-coded paths — the idiom `source expand('<sfile>:p:h') . '/other.vim'`, which appears throughout real plugins, works in standalone vimlrs because the token and the modifiers and the stack are all ported together.

The autoload # convention

Sourcing is eager: the file runs the moment the `:source` line does. The autoload convention is the lazy counterpart, and it is the one most Vimscript in the world actually relies on. The rule is a naming convention with a load hook behind it: a function named with `#`-separated components, such as `demo#double`, lives in an `autoload/demo.vim` file, and the *first* call to `demo#double` triggers vimlrs to search for `autoload/demo.vim`, source it, and then run the call. Subsequent calls skip the load — the function is already defined — so the cost of a large library is paid only for the parts a program actually touches, and only once. The `autoload.vim` example builds a throwaway package, calls into it, and checks both the result and the load:

```
call mkdir('autoload', 'p')
call writefile([
    \ 'function! demo#double(n) abort',
    \   return a:n * 2',
    \ 'endfunction',
    \ 'let g:demo_autoloaded = 1',
    \ ], 'autoload/demo.vim')

call assert_equal(10, demo#double(5))
call assert_equal(1, g:demo_autoloaded)
call assert_equal([2, 4, 6], map([1, 2, 3], {i, v -> demo#double(v)}))
```

The first `demo#double(5)` finds and sources `autoload/demo.vim`, which defines the function and sets

`g:demo_autoloaded`; the assertion that the global is set proves the file was sourced as a side effect of the call, not ahead of it. The `map()` over the list then reuses the already-loaded function without re-sourcing. The mechanism has two halves in the engine: the lexer, which treats `#` as a legal name character so that `demo#double` parses as a single identifier rather than a name followed by a comment, and the call machinery, which recognizes a `#`-bearing name that is not yet defined, translates the package prefix into a relative path (`demo` becomes `autoload/demo.vim`, `foo#bar#baz` becomes `autoload/foo/bar.vim`), and sources it before dispatching. The autoload hash is thus a lexer fact and a loader fact working together, and it is described where the lexer's name-absorption rules are, in the discussion of sigils and the autoload hash earlier in the book.

The path resolution deserves one clarification, because it is where the editor's absence shows. In the editor, `autoload/demo.vim` is searched along `'runtimepath'`, so the file may live in any plugin directory on the runtime tree. `autoload.vim` writes its package under `autoload/` relative to the current directory, which is where standalone vimlrs resolves them by default, and the example's own comment names this: the file is placed "relative to the current directory, where vimlrs resolves them." The convention is faithful; the search root defaults to the working directory rather than a populated runtime tree, because a standalone interpreter has no editor to have assembled one. What the program can always do is put its own tree on the runtimepath explicitly, which is the third mechanism.

:runtime and the runtimepath search

`:runtime[!]` {file} is the general form of "source this file from wherever it lives on the runtime tree." It searches the `'runtimepath'` option — an ordered, comma-separated list of directories — and sources the first matching file; with the `!` bang it sources every match, in runtimepath order, and the final path component may glob. This is how the editor loads its own plugins, filetype logic, and colour schemes, and vimlrs ports the `source_runtime` path that implements it, honouring a live `set runtimepath+=DIR`. The `runtime_path.vim` example is a complete, oracle-verified demonstration — it builds a runtime tree, adds it to the option, and checks reachability before, during, and after:

```
call mkdir('rtdemo/plugin', 'p')
call writefile(['let g:rt_plugin = 1'], 'rtdemo/plugin/rtdemo.vim')

" before adding rtdemo to <g>rtp, :runtime finds nothing
runtime plugin/rtdemo.vim
call assert_equal(-1, get(g:, 'rt_plugin', -1))

" add the tree; Vim honours this live
set runtimepath+=rtdemo
runtime plugin/rtdemo.vim
call assert_equal(1, get(g:, 'rt_plugin', -1))

" :runtime! sources EVERY match; the final component may glob
runtime! ftplugin/aa?.vim
call assert_equal(1, get(g:, 'rt_ftplugin_a', -1))
call assert_equal(1, get(g:, 'rt_ftplugin_b', -1))
```

Three behaviours are pinned here, and each is a faithful reproduction of the editor's. First, `'runtimepath'` is consulted *at the moment of the* `:runtime` call, so a `set rtp+=` between two identical `:runtime` lines changes what the second one finds — the option is live, not snapshotted. Second, `:runtime` without a bang stops at the first match, while `:runtime!` sources all of them, which is why the wildcard `aa?.vim` loads both `aaa.vim` and `aab.vim` under the bang. Third, `set rtp-=DIR` removes a directory, and a file reachable only through that directory becomes unreachable again — the example checks that a `set rtp-=rtdemo` makes a `plugin/after.vim` that lives only in `rtdemo` fail to load. The example's comment is careful about the one thing that is *not* faithful, and the carefulness is itself instructive: it asserts on behaviour, not on the `<g>rtp` string, because "Vim's default rtp is non-empty while vimlrs' stored rtp starts empty, so the string values differ." A standalone interpreter has no `$VIMRUNTIME`, no user config directory, no site directories to seed the option

with, so its runtimepath begins empty and the program fills it. The mechanism is identical; the initial contents differ, and the book's honesty about that difference is the same honesty it applies everywhere the editor's absence changes a default rather than a behaviour.

Three mechanisms, one module system

Put together, `:source`, `<sfile>`, autoload, and `:runtime` are not four unrelated features; they are one module system seen from four angles. `:source` is eager inclusion with script-scope isolation. `<sfile>` lets an included file locate itself and its siblings. Autoload is lazy inclusion keyed on a naming convention, so a large library costs only what a program uses. `:runtime` generalizes inclusion over a searchable tree of directories that the program controls through a live option. The `sourcing.vim`, `sfile.vim`, `autoload.vim`, and `runtime_path.vim` examples are the four corners, each self-testing and, for the two that touch runtimepath and the sourcing stack, oracle-verified against real Vim 9.2. That a program assembled from many files, loaded lazily by name along a runtime tree it configures itself, runs the same way outside the editor as inside it is not an accident of the port — it is the direct consequence of porting the sourcing and runtimepath machinery from the C rather than approximating a module system from scratch. The captive language turns out to have shipped its own module system all along, and taking the editor away leaves it working.

Chapter 35 – The Grammar of Failure: Error Codes and Aborting Semantics

Every Vim user has seen an error like `E684: list index out of range` scroll past, and most have never thought about it as a language feature. It is one. Vim's errors are numbered — the E-codes, a stable vocabulary of hundreds of specific failures, each with a fixed number and a canonical message — and those numbers are part of the observable contract of the language, because scripts match on them. `:catch /E684/, assert_fails(cmd, 'E716')`, the whole idiom of testing that a piece of code fails *in the expected way*, depends on the error number being what Vim says it is. A faithful port therefore cannot invent its own error text; it has to reproduce the E-codes, and just as importantly it has to reproduce the *aborting semantics* that decide what happens after an error is raised — whether the command dies cleanly, whether a fallback value leaks out, whether execution continues or unwinds. This chapter is about the grammar of failure: the numbered vocabulary, the abort rules, and the machinery vimlrs uses to make its failures indistinguishable from the editor's.

Errors are numbered, and the numbers are contract

An E-code is a promise. `E684` is always “list index out of range,” `E716` is always “key not present in Dictionary,” `E805` is always “using Float as a Number,” and a script written for one version of Vim can catch or assert on these numbers with confidence because they do not drift. This stability is what turns errors from noise into a testable interface, and it is why the `error_output.vim` example is written entirely in terms of the numbers rather than the messages:

```
call assert_fails('echo [1,2,3][10]', 'E684')
call assert_fails("echo printf('%d', 3.7)", 'E805')
call assert_fails("echo {'a': 1}.b", 'E716')
```

`assert_fails` runs the command, expects it to raise, and checks that the raised error *matches* the given pattern — here, that it contains the right E-code. For these assertions to pass, vimlrs must raise `E684` when a list index runs off the end, `E805` when `printf('%d', ...)` is handed a Float where the conversion demands an integer, and `E716` when a missing dictionary key is dereferenced with the dot form. It does, because the error strings are ported alongside the code that raises them: the index bounds check, the printf format-versus-argument check, and the dict lookup each carry the same numbered message the corresponding C path carries. The port's discipline here is exactly the discipline it applies to return values — the observable output of a failure is as much a specified result as the observable output of a success, and reproducing `[1,2,3][10]` faithfully means reproducing that it raises `E684`, not merely that it does not return a fifth element.

Aborting: an error kills the command, cleanly

The subtler half of the grammar is what happens *after* the raise. In many languages an expression that fails still yields something — a `null`, a `NaN`, an exception object that a lenient caller ignores — and a naive interpreter might, on hitting an out-of-range index inside `echo [1,2,3][10]`, produce an error *and then* echo a fallback like `v:null`, printing both. Vim does not. A runtime error in an expression *aborts the command*: the command that was running stops, produces no output of its own, and the error is the only thing that surfaces. The `error_output.vim` example's whole point is this cleanliness, as its header states — at the CLI, `echo [1,2,3][10]`

“prints only the E684 message — never a trailing v:null,” and `echo printf('%d', 3.7)` “prints only E805, never a trailing -1.” The example proves the negative by proving the positive around it: error-free `echo` still produces its normal output, captured and checked so that the aborting behaviour is pinned as *specific to the failing command*, not a blanket suppression.

```
call assert_equal('42', trim(execute('echo 42')))  
call assert_equal('hi there', trim(execute('echo "hi" "there"')))  
call assert_equal('2', trim(execute('echo [1,2,3][1]')))
```

The last line is the control: `[1,2,3][1]` is a valid index, so the `echo` runs to completion and emits `2`. The failing sibling `[1,2,3][10]` runs to the point of the error and then stops producing anything. This is the port of `eval.c/ex_docmd.c` command-abort semantics: an error sets the interpreter’s error state, the current command sees that state and unwinds without emitting its result, and the error message is delivered through the same channel it would use in the editor. The reason this matters beyond tidiness is that scripts depend on it. A pattern like `echo mightfail()` in a script that also has error handling relies on the failure *not* leaving a half-formed value behind to be mistaken for a real result, and code that inspects command output relies on a failed command producing no output rather than a plausible-looking wrong one. Getting the abort right is getting the failure to be legible.

Catch by number: errors become exceptions

Between “raise and abort the command” and “let the program die” sits the exception machinery, and it too is keyed on the numbered vocabulary. Inside a `:try` block, a raised error becomes a catchable exception whose message is the E-coded string, so `:catch /E716/` catches exactly the missing-key failures and lets everything else propagate. This is the same `:try/:catch/:finally/:throw` machine the control-flow chapter covers, viewed from the error side: the things it catches are not only user `:throws` but the engine’s own numbered errors, matched by the regex the `:catch` carries. Because the match is a full Vim pattern, a script can catch a family of errors by a prefix, a specific one by its exact number, or anything by an empty pattern, and the `v:exception` and `v:throwpoint` variables report the caught message and its origin. The faithfulness requirement compounds here: for `:catch /E684/` to work, not only must the out-of-range index raise, and not only must it abort its command, but the exception it becomes must carry the string `E684` in the place the pattern looks — so the numbered vocabulary has to be right at three layers at once. That the same number threads through the raise, the abort, the exception message, and `assert_fails` is what makes error handling in ported Vimscript behave like error handling in the editor rather than merely resembling it.

assert_fails and v:errors: failure as a tested result

The reason every claim in this chapter is checkable is the assertion framework, and it treats failure as a first-class result to be pinned. `assert_fails(cmd, pat)` is the negative-space counterpart to `assert_equal`: instead of asserting that an expression *equals* a value, it asserts that a command *fails*, and fails matching `pat`. When the expectation holds, nothing is recorded; when it does not — the command succeeds, or fails with the wrong error — a description is appended to `v:errors`, the running list of assertion failures that every example checks at its epilogue. This is why `assert_fails('echo [1,2,3][10]', 'E684')` can appear in a script that is supposed to *pass*: the out-of-range access is expected, `assert_fails` catches it, keeping it off stderr, and the assertion succeeds precisely because the failure happened as specified. The inversion is worth dwelling on, because it is the whole reason the error grammar can be tested at all. A test that an operation succeeds is easy; a test that an operation fails *in a particular way* requires the failure to be reproducible down to its number, catchable without escaping the test, and distinguishable from a differently-numbered failure. `assert_fails` provides that, and the fact that dozens of examples use it — pinning `E684`, `E716`, `E805`, and many more — is the error vocabulary being held to the same executable-specification standard the return values are. Failure, in vimlrs, is not the absence of a result; it is a result with a number, and the number is tested.

Chapter 36 – Characters, Bytes, and Cells: the Multibyte Story

Ask how long a string is and Vimscript answers with a question of its own: how long in *what*? A string of six visible characters may be six bytes, or eight, or twenty; it may occupy six terminal cells, or seven, or ten. Vim, born in a world of terminals and byte-oriented buffers and grown into a Unicode one, distinguishes three different lengths — bytes, characters, and display cells — and provides a different function for each, because in an editor all three are real quantities a script legitimately needs. A faithful port cannot collapse them; it has to reproduce the whole distinction, and it has to do so on top of Rust’s own UTF-8 strings without letting Rust’s notion of length leak through where Vim’s should be. This chapter is about the three lengths, the conversions between characters and codepoints, the base-aware number parsing that sits nearby, and the digraph table — the whole story of how vimlrs counts and converts text.

Three answers to “how long”

The distinction is exact and the `strwidth_funcs.vim` example pins all three against the same strings:

```
call assert_equal(6, strlen('héllo'))      " BYTES: é is 2 bytes
call assert_equal(5, strlen('a␣b'))        " BYTES: ␣ is 3, a and b 1 each
call assert_equal(5, strchars('héllo'))    " CHARACTERS: 5 codepoints
call assert_equal(3, strchars('a␣b'))      " CHARACTERS: a + ␣ + b
call assert_equal(5, strwidth('héllo'))    " CELLS: plain text == chars here
```

`strlen()` counts bytes: `héllo` is six because `é` encodes as two UTF-8 bytes, and `a␣b` is five because the snowman `␣` encodes as three while each ASCII letter is one — an astral-plane codepoint such as U+1F600 would encode as four. `strchars()` counts characters — codepoints — so `héllo` is five and `a␣b` is three regardless of how many bytes each takes. `strwidth()` and `strdisplaywidth()` count display cells, the number of columns the text would occupy in a fixed-width terminal, which for plain Latin text equals the character count but diverges the moment a wide character (a CJK ideograph, occupying two cells) or a zero-width combining mark enters. These are not three names for one idea; they are three genuinely different measurements, and a script that computes a column position needs cells, a script that indexes into content needs characters or bytes, and a script that allocates a buffer needs bytes. Vim gives each its own function because conflating them produces wrong answers in the editor, and vimlrs keeps them separate because the same wrong answers would follow from conflating them standalone.

The implementation challenge is that Rust hands the port a UTF-8 `String` whose most natural length — `.len()` — is the byte count, whose `.chars().count()` is the character count, and whose display width is neither, requiring an East-Asian-width and combining-class table to compute. So `strlen` maps to the byte length, `strchars` to the codepoint count, and `strwidth/strdisplaywidth` to a width calculation over the codepoints — the same three-way split Vim’s `mbyte.c` and `charset.c` implement, reproduced over Rust strings rather than over Vim’s own byte buffers. The functions that index into strings inherit the same care: a byte-index function and a character-index function are different functions in Vim (`byteidx` versus `byteidxcomp` and the character-aware slicers), and they stay different in the port, because a plugin that computes a byte offset to pass to a byte-oriented operation must get bytes, while one that counts characters for a display calculation must get characters.

Codepoints in and out: `char2nr` and `nr2char`

Beneath the length functions sits the character/codepoint conversion pair, `char2nr()` and `nr2char()`, and they are exact inverses over the whole Unicode range. `char2nr()` takes a string and returns the codepoint number of its *first* character; `nr2char()` takes a number and returns the one-character string at that codepoint. The `str_numconv.vim` example round-trips them across ASCII, Latin-1, and a BMP multibyte character:

```
call assert_equal(65, char2nr('A'))
call assert_equal(233, char2nr('é'))
call assert_equal(9731, char2nr('⌘'))
call assert_equal('é', nr2char(char2nr('é')))
call assert_equal('⌘', nr2char(char2nr('⌘')))
```

The round-trip `nr2char(char2nr(c)) == c` holding for `é` and `⌘` is the proof that the conversion goes through the full codepoint, not a truncated byte: `é` is codepoint 233 and `⌘` is 9731, well past the byte range, and both survive the trip. The pair reaches the astral plane too — `char2nr()` of an emoji returns codepoints such as 128512 (U+1F600), beyond the BMP — and `char2nr('')` of the empty string is 0, the documented sentinel. This pair is the primitive under a great deal of text manipulation — building a string from computed codepoints, testing character classes numerically, implementing a cipher — and its faithfulness is what lets numeric character code from the editor run unchanged.

Parsing numbers out of text, base and all

Adjacent to the character conversions is the string-to-number machinery, and it carries a Vim-specific richness that a naive `parse::<i64>()` would miss entirely. `str2nr()` parses an integer, but it takes an optional *base*, and in each base it tolerates the matching prefix, a leading sign, and leading whitespace. The `str_numconv.vim` walks the range:

```
call assert_equal(-42, str2nr('-42'))
call assert_equal(17, str2nr(' 17'))
call assert_equal(255, str2nr('ff', 16))
call assert_equal(26, str2nr('0x1A', 16))
call assert_equal(-255, str2nr('-ff', 16))
call assert_equal(511, str2nr('777', 8))
call assert_equal(5, str2nr('0b101', 2))
call assert_equal(0, str2nr('z', 16))
```

Base 10 is the default; base 16 accepts an optional `0x` prefix and bare hex digits; base 8 accepts octal; base 2 accepts an optional `0b` prefix. A sign binds outside the prefix (`-ff` in base 16 is `-255`), leading spaces are skipped, and input that has no valid digits in the requested base (`z` in base 16, or the empty string) yields 0 rather than raising. This is not general-purpose number parsing; it is Vim's `str2nr`, prefix conventions and tolerances and all, and reproducing it means reproducing the whole permissive grammar rather than deferring to a strict library parser. Its float sibling `str2float()` is similarly Vim-shaped: it parses decimal and scientific notation, accepts integer-looking input as a float, and always yields a value of type `v:t_float` even for `str2float('1')`, so the type is determined by the function, not by whether the text happened to contain a dot.

The digraph table

The last piece of the multibyte story is the one most tied to the editor's input model, and it survives the port intact: digraphs. A digraph is a two-character mnemonic for a character that is awkward to type directly — `a:` for `ä`, `Co` for `©`, `->` for `→` — and Vim ships a large table of them, ported into vimlrs from Neovim's `digraph.c`, reachable through `digraph_get()`, `digraph_set()`, `digraph_getlist()`, and `digraph_setlist()`. The `digraphs.vim` example shows the built-in table resolving, user digraphs registering and

overriding, and the list forms round-tripping:

```
call assert_equal('ä', digraph_get('a:'))
call assert_equal('ð', digraph_get('o'))
call assert_equal('→', digraph_get('->'))
call assert_equal('', digraph_get('zz'))          " unknown trigger → empty

call assert_equal(v:true, digraph_set('c', 'X'))
call assert_equal('X', digraph_get('c'))
call assert_equal(v:true, digraph_set('a:', 'A'))
call assert_equal('A', digraph_get('a:'))        " user override wins
```

An unknown trigger yields the empty string rather than an error; a user digraph registered with `digraph_set()` returns `v:true` and thereafter resolves; a user digraph for a trigger that already has a built-in overrides it, so `a:` can be redefined from `ä` to `A`. `digraph_setlist()` registers several at once and `digraph_getlist()` returns the user digraphs as `[chars, digraph]` pairs sorted by trigger. In the editor these functions feed the insert-mode digraph input; in standalone vimlrs there is no insert mode, but the table and its query and mutation functions are a real, self-contained piece of the language surface, and a script that consults or extends the digraph table runs unchanged. It is a small feature, but it is a fair emblem of the whole multibyte chapter: a piece of the editor's text machinery that turns out to be a library once the editor is taken away, ported faithfully — the RFC-1345 subset and all — because a program that used it inside the editor has every right to use it outside.

Chapter 37 — Four Languages, One Engine

`vimlrs` does not run on a machine of its own. It lexes and parses Vimscript, lowers the result to `fusevm` bytecode, and hands that bytecode to the shared `fusevm` engine to execute — the same engine that runs `zshrs`, `stryke`, and `awkrs`. This is stated in the first pages of the book and threaded through the machine chapters, but it deserves a chapter of its own, because the decision to share an engine across four languages is one of the most consequential architectural choices the project makes, and its consequences run in both directions: what `vimlrs` gains from the arrangement, and what the arrangement demands of `vimlrs` in return. This chapter looks at the shared engine as a thing in itself — what lives in it, what lives in each language, and why putting four unrelated languages on one VM is a sound idea rather than a compromise.

The line between a language and its machine

Every interpreted language has two halves that are easy to conflate: the *language* — its grammar, its semantics, its standard library, the things that make Vimscript Vimscript and `awk` `awk` — and the *machine* — the thing that actually executes instructions, manages memory, and, if it is ambitious, compiles hot code to native form. These are separable, and the whole `fusevm` arrangement is built on taking the separation seriously. The machine is a bytecode virtual machine with a three-tier Cranelift JIT, a value representation, a calling convention, a garbage or reference-counting discipline, and a native-compilation path. None of that is specific to Vimscript; a bytecode `ADD` instruction, a call frame, a heap handle, a compiled loop are the same shapes whether the source was a `.vim` file or a `.sh` file. So `fusevm` implements them once, and each language implements only its own half: its frontend (lexer, parser) and its lowering (the translation from its AST to `fusevm` bytecode) and its standard library (the builtins that give it its character). `vimlrs` is, in this framing, a frontend and a lowering and a library that target a machine it did not write — “the fourth language hosted on that engine,” carrying “no VM and no JIT of its own.”

The reason this is the right cut, rather than a merely convenient one, is that the machine is the part that is genuinely hard and genuinely general, while the language-specific part is the part that carries the meaning. A JIT with three tiers, register allocation, deoptimization, and a native code cache is an enormous amount of careful engineering, and essentially none of it cares what language it is compiling; the difficulty is in the compilation, not in the source dialect. The parts that *do* care about the source dialect — how Vimscript scopes a variable, how it coerces a string to a number, what `E684` means — are exactly the parts `vimlrs` implements itself, because they are the language. Sharing the machine therefore shares the hard, general work and keeps the language-specific work where it belongs. A new language on the engine writes a frontend and a lowering and inherits a production JIT for free, which is why the engine has four languages on it rather than one.

What the engine provides, and what `vimlrs` keeps

The division of labour is clean enough to enumerate. On the engine side: the bytecode instruction set and its interpreter loop; the value representation those instructions manipulate; the call frame and stack machinery; the three JIT tiers and the Cranelift backend that lower hot bytecode to native code; the deoptimization path that falls back to the interpreter when a JIT assumption breaks; and the memory discipline for heap values. On the `vimlrs` side: everything that makes the bytecode *Vimscript*. The lexer that turns `.vim` text into tokens, absorbing the scope sigils and the autoload hash and the magic in Vim’s syntax. The parser that builds the

AST through the `eval1...eval7` precedence ladder. The lowering that walks that AST and emits `fusevm` bytecode, mapping a Vimscript `for` loop to the engine's loop instructions, a Vimscript function call to its call instruction, a Vimscript list to its container operations. And the standard library — the hundreds of builtins, the ex-command layer, the regex engine, the option and special-variable machinery — which is Vimscript's semantics made executable and has no counterpart in the engine at all.

The value representation is the most interesting shared boundary, because it is where the language's types meet the machine's storage. Vimscript's types — Number, Float, String, List, Dict, Funcref, Blob, Bool, Special — must be expressible in whatever value representation `fusevm` provides, and the lowering and the builtins together are responsible for maintaining Vimscript's semantics on top of that representation: that a List is reference-shared and mutable, that a String coerces to a Number by its leading digits, that `v:true` is a distinct Bool and not just the integer 1. The engine stores the values; vimlrs decides what they mean. This is the same arrangement each sibling language has — `awkrs` maps awk's scalar-and-array model onto the same storage, `zshrs` maps zsh's string-and-array model onto it — and the fact that four different type systems coexist on one value representation is evidence that the representation was designed to be a substrate rather than any one language's home.

What sharing demands: the faithfulness constraint meets the general machine

The arrangement is not free, and the cost is the interesting part. A shared, general machine will happily do things a specific language must *not* do, and it is vimlrs's job to keep the machine's generality from leaking into Vimscript's semantics. The clearest case is the JIT. The engine's JIT wants to compile a hot numeric loop to native arithmetic, and native arithmetic is fast precisely because it does not stop to ask questions — it adds two machine integers and moves on. But Vimscript arithmetic has semantics the machine's raw `ADD` does not: coercions, overflow behaviour, the Number/Float distinction, the error that `printf('%d', 3.7)` must raise. If the JIT compiled Vimscript `+` to a bare machine add, it would be fast and *wrong*. So the lowering and the JIT integration carry a per-operator checking discipline — the loop is compiled to native code, but the operators retain the checks that keep their behaviour faithful, so speed does not buy infidelity. This is the tax of running a specific, faithful language on a general, fast machine: the machine offers a fast path that skips the language's guarantees, and the language must decline the offer at exactly the points where the guarantees matter, keeping the parts of the fast path that are safe and reinstating the checks that are not.

The other demand is subtler and runs the other way: sharing an engine means vimlrs cannot fix the engine for its own convenience without considering three other languages, and cannot depend on engine behaviour that the engine does not promise all four of them. The book notes this elsewhere as a real coupling — a vimlrs feature can be blocked on a needed `fusevm` capability, and the machine chapters mention specific engine versions vimlrs requires. This is the ordinary cost of a shared dependency, and it is worth naming honestly: the engine is a leverage multiplier and a coordination constraint at once. Four languages share the enormous work of a production JIT; in exchange, each of the four lives with an engine it does not solely control. The project judges the trade worth it, and the evidence is that vimlrs has a three-tier native-compiling JIT it could not plausibly have written for itself as a one-person port — the machine is borrowed, and the borrowing is what makes the fast, faithful standalone Vimscript feasible at all.

Chapter 38 – The Shape of a Hot Loop

The claim that vimlrs is *fast* — one of the four claims of the opening chapter — comes down, in the end, to a small number of shapes of code that the machine underneath can compile to native instructions, and a larger number of shapes it cannot. This is not a hedge; it is how JITs work. A tiered, tracing or method JIT does not make all code fast uniformly. It makes *hot* code — code that runs enough times to be worth compiling — fast, and among hot code it makes the *regular* kind, the kind whose types and control flow are stable, faster than the irregular kind. Knowing which shapes those are is the difference between a benchmark that flatters the machine and an honest account of what it buys. This chapter takes the canonical hot loop — a numeric recursion or iteration — and traces what has to be true for the JIT to compile it, what the compilation gains, and where the gains stop.

Why a loop, and why numeric

The example the whole book returns to is `fib.vim`, Fibonacci, and it is not chosen for charm. It is chosen because it is the cleanest possible instance of the shape a JIT is built to accelerate: a small body, run an enormous number of times, over values of a single stable type, with control flow that depends only on those values. Fibonacci computed by recursion is a function called exponentially often; Fibonacci computed by iteration is a loop that runs its body once per step over an accumulator that is always a Number. Either way, the interpreter would execute the same handful of bytecode instructions millions of times, dispatching each one through the interpreter loop — decode the opcode, jump to its handler, do the work, advance — and that per-instruction dispatch overhead, multiplied by millions, is precisely the cost a JIT exists to remove. When the body is compiled to native code, the dispatch disappears: the instructions become a straight run of machine operations with no decode step between them, and the loop runs at something close to the speed the same arithmetic would run at if it had been written in a compiled language to begin with.

The “numeric” part is as important as the “loop” part. The JIT can compile the loop cheaply because the accumulator is always a Number — the type does not change from iteration to iteration, so the compiled code can assume a Number, use native integer arithmetic, and skip the type-dispatch that the interpreter would do on every operation. This assumption is the engine of the speedup and also its fragility: it holds only as long as the type is stable, and a JIT that assumes a type must have a way to cope when the assumption breaks. That is what the third tier and the deoptimization path are for — if a value that was always a Number suddenly is not, the compiled code bails back to the interpreter rather than computing a wrong answer on a wrong-typed value. A hot numeric loop is the ideal case precisely because the assumption never breaks: the type is stable by construction, so the fast path stays on the fast path.

The preconditions the loop must meet

For the loop over a local accumulator to be trace- or method-JIT-compilable, several things have to line up, and the book names the load-bearing one where it discusses scopes: the accumulator must be a *bare function-local*. A bare name inside a function is `l:`, function-local, and that locality is “the precondition for trace-JIT-compiling a numeric loop over it.” The reason is about what the compiler can prove. A function-local variable is reachable only from inside the function; nothing outside can observe or mutate it mid-loop, so the compiler can keep it in a machine register for the duration of the loop and know that register is the whole truth about the variable. A global (`g:`) accumulator has no such guarantee — another function, an autocommand, any code the loop calls could read or write the global — so the compiler cannot cache it in a register without risking a stale or

divergent value, and the loop over a global cannot be compiled as tightly, if at all. This is why the shape of the *variable*, not just the shape of the loop, decides whether the JIT engages: locality is what makes the register caching sound, and register caching is what makes the compiled loop fast.

The second precondition is type stability, already discussed, and the third is that the body stays inside the compilable subset — arithmetic, comparisons, local reads and writes, simple control flow — rather than reaching for operations that the compiled tier cannot express and must defer to the interpreter for. A loop whose body is pure numeric arithmetic over a local is entirely inside the subset. A loop whose body calls a builtin that manipulates a Dict, matches a regex, or sources a file is not — those operations run through the full interpreter and standard-library machinery, and the JIT's contribution is limited to the parts of the loop around them. This is the honest boundary of the speedup: it is largest for tight numeric kernels and shrinks as the body does more of the irregular, type-shifting, library-calling work that makes up most real Vimscript. A `vimrc` is not a numeric kernel; it is mostly setup, sourcing, and configuration, run once, where the JIT has nothing to bite on and does not need to. The JIT earns its keep on the computational inner loops — the string-processing pass over a large buffer's worth of lines, the numeric transform over a big list — and it is those the machine is built to accelerate.

Per-operator checking: fast without unfaithful

The tension the previous chapter named — a general fast machine offering a fast path that skips the language's guarantees — resolves, for the hot loop, into a specific discipline the book calls per-operator checking, and it is what lets the JIT be fast without being unfaithful. When the loop's `+` is compiled, it is not compiled to a bare machine add that would silently do the wrong thing on the values Vimscript's `+` treats specially. It is compiled to native code that still performs the checks Vimscript's `+` requires — the ones that decide Number-versus-Float behaviour, that raise where the language raises, that coerce where the language coerces — so that the compiled operator produces exactly the result the interpreted operator would, only faster. The speedup comes from removing the *dispatch* overhead — the decode-and-jump that the interpreter pays per instruction — not from removing the *semantic* checks that make the operator correct. This is the crucial distinction, and it is what separates a faithful JIT from a fast-but-wrong one: the interpreter's cost is dispatch plus semantics, the compiled code's cost is semantics alone, and the semantics are kept intact because dropping them would make the loop compute a different Vimscript than the one the reference describes.

The consequence is that the fast path and the faithful path are the same path. There is not a fast mode that trades correctness for speed and a slow mode that is correct — there is one behaviour, delivered at interpreter speed when cold and at native speed when hot, and the hot version is the same Vimscript as the cold version by construction. This is why the book can claim both faithfulness and speed without qualification and mean them jointly rather than as a trade-off: `fib.vim` computes the same Fibonacci at the bottom of a deep recursion, where the JIT has long since taken over, as it does on the first call, where the interpreter is still running, because the compiled arithmetic keeps every check the interpreted arithmetic had. The shape of a hot loop, then, is the shape of the whole project's ambition in miniature — a real language, taken out of its editor and lowered onto a borrowed machine, made fast by compiling its hot loops to native code, and kept faithful by refusing to let the compilation drop a single one of the semantics that make the language what it is.

Chapter 39 – A Tour Through the Examples

The crate ships over a hundred and fifty example scripts under `examples/`, and every one is a self-test run in CI by `tests/examples.rs`. That dual nature — documentation and regression suite at once — makes them the best material in the repository for seeing the language work end to end, because each is a complete program that asserts its own correctness. This chapter walks several of them line by line, not to re-explain features covered earlier but to show how they combine into real programs and to model how to read a vimlrs script. We will read the way the interpreter does: top to bottom, watching state accumulate, ending at the self-test epilogue every example shares.

fib.vim: two algorithms and the JIT

`fib.vim` computes Fibonacci two ways, and its header tells you what it is for: “`:function/:return`, recursion with `a:/l:` scopes, an accumulator while-loop that trace-JIT-compiles, and the built-in assert framework.” It opens with the recursive definition:

```
function! Fib(n) abort
  if a:n < 2
    return a:n
  endif
  return Fib(a:n - 1) + Fib(a:n - 2)
endfunction
```

Everything here has appeared in earlier chapters, now working together. The `function!` bang means the definition can be re-sourced; `abort` stops the function at the first error; the parameter `n` is read as `a:n` from the argument scope; the base case returns directly; and the recursive case calls `Fib` twice, proving a function name is in scope within its own body. This is a tree-walking recursion — exponential, slow for large `n` — and it is here as the correctness reference, the obviously-right implementation against which the fast one is checked.

The fast one is the iterative loop, and it is the JIT’s home turf:

```
function! FibIter(n) abort
  let a = 0
  let b = 1
  let i = 0
  while i < a:n
    let t = a + b
    let a = b
    let b = t
    let i += 1
  endwhile
  return a
endfunction
```

Read this through the lens of Chapter 14. The locals `a`, `b`, `i`, and `t` are bare names, which means they are `l:` locals, which means the compiler’s `slot_plan` can prove they hold Numbers and give them VM slots. The `while` condition `i < a:n` is an integer comparison; the body is integer additions and copies; `let i += 1` desugars to `let i = i + 1`, a provably-numeric slot update. Every precondition for trace-JIT compilation is

met, so this loop compiles to native machine code, and the file's header invites you to confirm it: “Run with `VIMLRS_JIT_STATS=1` to see the loop trace get compiled.”

Then come the unit tests, which are the heart of why the file exists:

```
let expected = [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
for i in range(len(expected))
  call assert_equal(expected[i], Fib(i))
  call assert_equal(expected[i], FibIter(i))
endfor
call assert_equal(55, FibIter(10))
call assert_equal(12586269025, FibIter(50))
```

The `for i in range(len(expected))` is itself the native-counter loop from Chapter 11 — `range()` builds no list, the loop traces — and each iteration asserts that *both* implementations agree with the known value. This is a classic testing pattern: a slow reference and a fast optimization, checked against each other and against ground truth. The `FibIter(50)` assertion is the telling one — `12586269025` is larger than a 32-bit integer, which is why `has('num64')` being true matters: vimlrs's Numbers are 64-bit, so the fiftieth Fibonacci number is exact, not overflowed. A demo section then shows the results with `map` and `printf`:

```
echo 'fib(0..9):' map(range(10), 'Fib(v:val)')
echo printf('fib(50) = %d', FibIter(50))
```

`map(range(10), 'Fib(v:val)')` uses the string-expression form of `map`, with `v:val` bound to each element — applying the recursive `Fib` across `0..9`. And the file ends with the universal epilogue: inspect `v:errors`, print each failure, throw if non-empty. Run `vimlrs examples/fib.vim` and, if all assertions pass, it prints `OK: fib assertions passed` and exits 0. That single program demonstrates functions, both scopes, recursion, the JIT, `range`, `map`, `printf`, 64-bit integers, and the assert framework — and proves it all works, every time CI runs.

The deepest lesson of `fib.vim` is in why it computes Fibonacci *twice*, with two functions that agree on every value, and it is worth drawing out because the two-implementation structure is a testing pattern this book has pointed at repeatedly and `fib.vim` is its clearest instance. The recursive `Fib` is the obviously-correct reference: it is a direct transcription of the mathematical definition, $Fib(n) = Fib(n-1) + Fib(n-2)$, slow and exponential but transparently right — you can read it and believe it without running it. The iterative `FibIter` is the fast, optimized version, the one that traces to native code, and it is *not* obviously correct: an off-by-one in the loop bound, a swapped assignment, a wrong initial value would all produce a function that looks plausible and computes wrong numbers. The example checks the fast one against the slow one, asserting that `Fib(i)` and `FibIter(i)` agree for every `i` in a range, and it checks both against a hard-coded table of the first ten Fibonacci numbers. So there are two independent correctness arguments — the fast version matches the obviously-correct reference, and both match a known table — and a bug in the fast version would have to also be a bug in the reference and in the table to escape, which is vanishingly unlikely. This is the slow-reference, fast-optimization pattern: keep an obviously-correct implementation as an oracle, write the fast one beside it, and assert they agree across a range of inputs, so the fast one's correctness is continuously checked against the slow one's transparency.

The pattern matters here specifically because the fast version is the one that exercises the JIT, and the JIT is exactly the kind of optimization that could silently produce wrong results. A trace-compiled loop that mis-handled the integer arithmetic, or that deopted incorrectly, or that got the 64-bit width wrong, would produce a `FibIter` that diverges from `Fib` at some input — and the example would catch it, because it asserts they agree. The `FibIter(50)` assertion is the pointed one: `12586269025` exceeds a 32-bit integer, so it is a direct test that the native arithmetic the JIT emits preserves 64-bit width, which `has('num64')` being true promises and which a 32-bit slip would violate. So `fib.vim` is not just a demonstration that vimlrs can compute Fibonacci; it is a continuous test that the JIT's fast path produces the same answers as the interpreter's slow path, checked against an obviously-correct reference and a known table, run every day in CI. That is the strongest kind of confidence you can have in an optimization: not that it was carefully written, but that it is continuously verified against a transparent reference, so the day it diverges, the build goes red. The two-algorithm structure is the example's quiet genius, and it is the pattern that makes the JIT trustworthy rather than merely fast.

wordfreq.vim: a text-processing pipeline

`wordfreq.vim` is the example that most looks like a real script someone would write, because it does a real task: count word frequencies in a file. Its header lists the cast — “`writefile()`/`readfile()` round-trip, `split()` tokenizing, a dict frequency table, `sort()` with a Funcref comparator, `printf()` formatting.” It opens by seeding a corpus so the example is self-contained, then reads it back:

```
let tmp = tempname()
call writefile(['the quick brown fox', 'the lazy dog', 'the fox jumps'], tmp)
let lines = readfile(tmp)
call delete(tmp)
```

`tempname()` gives a unique temporary path, `writefile()` writes a list of lines to it, `readfile()` reads them back as a list of lines, and `delete()` cleans up. This is the file-I/O round-trip that the editor-absent contract makes work standalone — these are real filesystem operations, ported faithfully, not stubs. Then it tokenizes every line into one flat word list:

```
let words = []
for line in lines
  call extend(words, split(line))
endfor
```

`split(line)` with no pattern splits on whitespace; `extend(words, ...)` appends each line’s words into the accumulating flat list. (Note `extend` mutating `words` in place — the in-place-and-return convention from Chapter 7.) Building the frequency table is the dict idiom from Chapter 4:

```
let freq = {}
for w in words
  let freq[w] = get(freq, w, 0) + 1
endfor
```

`get(freq, w, 0)` reads the current count or 0 if the word is new, and the `let freq[w] = ...` writes it back — index assignment into a dict. After the loop, `freq` maps each word to its count. Now the script sorts the (`word`, `count`) pairs by descending count, using a named comparator:

```
function! ByCountDesc(a, b) abort
  return a:b[1] - a:a[1]
endfunction
let pairs = []
for w in keys(freq)
  call add(pairs, [w, freq[w]])
endfor
call sort(pairs, function('ByCountDesc'))
```

`keys(freq)` gives the words; the loop builds a list of [`word`, `count`] pairs; and `sort(pairs, function('ByCountDesc'))` orders them, with the comparator returning `a:b[1] - a:a[1]` — the second pair’s count minus the first’s, which sorts descending. `function('ByCountDesc')` is the Funcref construction from Chapter 8. The assertions then check the whole pipeline:

```
call assert_equal(10, len(words))
call assert_equal(7, len(uniq(sort(copy(words)))))
call assert_equal(3, freq['the'])
call assert_equal(['the', 3], pairs[0])
```

Ten words total, seven unique (`uniq(sort(copy(words)))`) — sort a copy, dedupe adjacent duplicates, `the` appears three times, and the most common pair is `['the', 3]` at the front after the descending sort. The demo prints the summary, and the epilogue closes it out. This is a genuinely useful forty-line program — read a file, tokenize, count, rank, report — written entirely in faithfully-porting Vimscript and running as a standalone executable. It is the clearest answer to “what is vimlrs for”: exactly this kind of text-processing task that you

would otherwise reach for `awk` or a quick Python script to do.

testing.vim: the framework on itself

`testing.vim` is the example that documents vimlrs’s testing facilities by using all of them, and it is worth reading because it shows the error-path testers that the other examples only lean on lightly. It starts with a small function under test — `parse KEY=VALUE`, throwing on malformed input:

```
function! ParseKV(s) abort
  let m = matchlist(a:s, '^(\w\+=\(.*\))$')
  if empty(m)
    throw 'E0: not KEY=VALUE: ' . a:s
  endif
  return [m[1], m[2]]
endfunction
```

`matchlist` runs the regex and returns the whole match plus the submatch groups; `m[1]` and `m[2]` are the two captured groups (the key and the value). If the pattern does not match, `matchlist` returns an empty list, `empty(m)` is true, and the function throws an `E0`-prefixed message. The happy-path assertions are ordinary:

```
call assert_equal(['name', 'ada'], ParseKV('name=ada'))
call assert_equal(['x', ''], ParseKV('x='))
```

`ParseKV('x=')` returns `['x', '']` — the value group `(.*)` matches the empty string after the equals. Then the error-path tests, which are the point. First, the `:try/catch` plus `assert_exception` form:

```
try
  call ParseKV('garbage')
  call assert_report('ParseKV should have thrown on bad input')
catch
  call assert_exception('E0:')
endtry
```

This calls `ParseKV('garbage')` expecting it to throw. If it does *not*, control reaches `assert_report`, which unconditionally records a failure — the test asserts that the throw happened by failing loudly if it did not. If it does throw, the bare `:catch` catches it and `assert_exception('E0:')` checks that the thrown text (now in `v:exception`) contains `E0:`. Second, the more compact `assert_fails` form, which packages the same idea into one call:

```
call assert_fails("call ParseKV('nope')", 'E0:')
call assert_fails('call no_such_function()', 'E117:')
call assert_fails('let x = [] + 1')
```

`assert_fails(cmd, error)` runs the command, requires it to error, and optionally matches the error code: `ParseKV('nope')` must raise something matching `E0:`, `no_such_function()` must raise `E117:` (Vim’s “unknown function” error, faithfully numbered), and `let x = [] + 1` must error at all (you cannot add a number to a list). The value-asserts round it out:

```
call assert_inrange(1, 10, 7)
call assert_match('\d+', 'abc123')
call assert_notmatch('^d', 'abc123')
```

`assert_inrange(lower, upper, actual)` checks `1 <= 7 <= 10`; `assert_match` checks a pattern matches; `assert_notmatch` checks it does not. This file is the whole testing vocabulary in one place — value equality, range, pattern, the exception path two ways — and reading it is the fastest way to learn how to write tests *in* vimlrs, which matters because vimlrs’s own example suite is exactly these tools used at scale.

closures.vim: capture semantics as a test

`closures.vim` deserves a second look in this tour because it is the example whose *correctness* is the most subtle, and reading it as a program (rather than as the feature illustration of Chapter 8) shows how a test file can pin down a semantic distinction that is easy to get wrong. It builds up from the simplest capture to the trickiest:

```
function! Adder(n) abort
  return {x -> x + a:n}
endfunction
let Add5 = Adder(5)
call assert_equal(15, Add5(10))
call assert_equal(5, Adder(2)(3))
```

`Adder(5)` returns a lambda that captured `a:n` as 5, so `Add5(10)` is 15. The second assertion, `Adder(2)(3)`, calls `Adder` and immediately invokes the returned closure — a closure that is never even stored in a variable. Then independence:

```
function! Const(v) abort
  return {-> a:v}
endfunction
let one = Const(1)
let two = Const(2)
call assert_equal(1, one())
call assert_equal(2, two())
```

`one` and `two` are closures from separate calls, each having captured a different `a:v` by value, so they yield 1 and 2 with no interference — the proof that capture is by value at creation time, not by shared reference. Nesting:

```
function! Curry(a) abort
  return {b -> {c -> a:a + b + c}}
endfunction
call assert_equal(123, Curry(100)(20)(3))
```

`Curry(100)(20)(3)` is 123: the outer lambda captures `a:a` as 100, the inner captures the outer's parameter `b` as 20, and the innermost adds `c` as 3. This is capture *through* layers, each closure closing over the one outside it. And finally the late-binding global, the distinction that separates a real closure implementation from a fake one:

```
let g:counter = 10
function! ReadCounter() abort
  return {-> g:counter}
endfunction
let GetIt = ReadCounter()
let g:counter = 99
call assert_equal(99, GetIt())
```

`GetIt` returns `g:counter`, and because globals are *not* captured (Chapter 8), it reads the current value 99, not the 10 in place when the lambda was made. A naive closure implementation that captured everything would assert 10 here and be wrong. `closures.vim` asserts 99, and it passes, which is the test-as-specification proof that vimlrs captures locals by value and resolves dynamic scopes late — exactly Vim's semantics. This is the example to point to when someone asks whether vimlrs's closures are *really* faithful: the answer is a CI-checked assertion, not a claim.

lists_dicts.vim: the data layer in one file

`lists_dicts.vim` is the broadest single demonstration of the container types, and reading it shows how

naturally lists, dicts, the higher-order functions, and the two dict literals compose. It defines one helper and then exercises everything:

```
function! Add(acc, x) abort
    return a:acc + a:x
endfunction
let nums = range(1, 10)
let evens = filter(copy(nums), 'v:val % 2 == 0')
let squared = map(copy(nums), 'v:val * v:val')
let total = reduce(nums, function('Add'), 0)
let desc = reverse(sort(copy(nums), 'n'))
```

`range(1, 10)` builds `[1, 2, ..., 10]`. Then four transformations, each protecting `nums` with `copy()` because `filter`, `map`, `sort`, and `reverse` mutate in place: `evens` keeps the even values via a string-expression predicate, `squared` squares each via a string-expression body, `total` folds with the named `Add` Funcref starting at 0, and `desc` sorts numerically then reverses. Three different ways to pass a function — string expression, Funcref, and (elsewhere in the file) lambda — appear in five lines, which is the point: they are interchangeable. The example then builds the frequency dict and asserts the lot:

```
call assert_equal([2, 4, 6, 8, 10], evens)
call assert_equal([1, 4, 9, 16, 25, 36, 49, 64, 81, 100], squared)
call assert_equal(55, total)
call assert_equal([3, 4, 5], nums[2:4])
call assert_equal('ada', user.name)
call assert_equal('viml', user.langs[0])
```

The slice `nums[2:4]` is the inclusive-bound slice from Chapter 7 — three elements, not two. `user.name` and `user.langs[0]` show dotted member access chaining into list indexing. The file then exercises both dict literals and `:const`:

```
call assert_equal({'id': 1, 'tag': 'x'}, #{id: 1, tag: 'x'})
call assert_equal({'z': 1}, #{z:1})
call assert_equal({'a': 1, 'b': 2}, #{a:1, b:2})
const PI = 3
const CFG = #{width: 80}
call assert_equal(80, CFG.width)
```

The `#{z:1}` and `#{a:1, b:2}` cases are the ones that prove the lexer reads scope-letter keys as keys, not sigils, inside a `#{}` literal. The `const` bindings assign and read back, with the caveat from Chapter 4 that vimlrs does not yet enforce the immutability lock. One file, and you have seen lists, dicts, both literals, slicing, member access, the three callable forms, `range`, `filter`, `map`, `reduce`, `sort`, `reverse`, `copy`, and `const` — all asserted correct.

json.vim and msgpack.vim: serialization round-trips

The two serialization examples are worth reading together because they make the same claim — lossless round-trip — at two levels of strictness. `json.vim` builds a config dict (using line continuation), encodes it, decodes it, and asserts both the exact encoded string and the reconstructed value:

```
let encoded = json_encode(config)
let decoded = json_decode(encoded)
let expected = '{"name":"vimlrs","version":[0,1,0],',
    \ . '"features":["jit","lsp","dap"],"standalone":true}'
call assert_equal(expected, encoded)
call assert_equal([0, 1, 0], decoded['version'])
call assert_equal('0.1.0', join(map(copy(decoded['version']), 'string(v:val)'), '.'))
call assert_equal(config, decoded)
```

The last assertion, `assert_equal(config, decoded)`, is the round-trip in one line: decode of encode

equals the original. The middle line is a small pipeline — take the decoded version list, map each element through `string()`, join with dots — producing `'0.1.0'` from `[0, 1, 0]`, which is the kind of glue code real scripts are full of. That the exact encoded string can be asserted at all is the insertion-ordered-dict payoff: key order is deterministic, so the output is predictable.

`msgpack.vim` makes the stricter, byte-exact version of the same claim. Where JSON asserts a string equals a string, MessagePack asserts a Blob equals specific bytes:

```
call assert_equal(0z81A16101, msgpackdump([{'a': 1}], 'B'))
call assert_equal(0zCB3FF0000000000000, msgpackdump([1.0], 'B'))
let objs = [42, -7, 3.5, [1, [2, 3]], {'k': 'v', 'n': 10}, v:null, v:true]
call assert_equal(objs, msgpackparse(msgpackdump(objs, 'B')))
```

`0z81A16101` is the exact MessagePack encoding of a one-key map — fixmap of size 1, the key `'a'` as a one-byte fixstr, the value 1 as a positive fixint — and the example asserts vimlrs produces precisely those four bytes, “identical to Neovim’s.” The float `1.0` packs to the eight-byte IEEE-754 `0zCB3FF0...`. The round-trip line takes a heterogeneous list — integers, a float, nested lists, a dict, `v:null`, `v:true` — dumps it to bytes and parses it back, asserting equality. The example even documents the lossy edges honestly (a String dumps as BIN and returns as a Blob; a Funcref raises `E5004`). Reading these two examples back to back teaches the difference between *interoperable* fidelity (JSON, where matching the spec is enough) and *byte-exact* fidelity (MessagePack, where matching another implementation’s exact output is the bar), and shows vimlrs clearing both.

system.vim and env_vars.vim: the script as a shell citizen

These two examples are the ones that most justify the standalone premise, because they show a vimlrs script doing what you would otherwise open a shell or a Python REPL to do. `system.vim` drives external commands:

```
let greeting = system('echo hello')
let lines = systemlist('printf "one\ntwo\nthree\n"')
let shout = system('tr a-z A-Z', 'quiet')
call system('exit 7')
let code = v:shell_error
let env = environ()
call assert_equal("hello\n", greeting)
call assert_equal(['one', 'two', 'three'], lines)
call assert_equal('QUIET', shout)
call assert_equal(7, code)
call assert_true(has_key(env, 'PATH'))
```

Line by line: `system('echo hello')` captures stdout including the trailing newline; `systemlist` splits output into a list of lines; `system('tr a-z A-Z', 'quiet')` feeds `'quiet'` on stdin and gets `'QUIET'`; `system('exit 7')` runs a command for its status, which `v:shell_error` reports as 7; and `environ()` returns the process environment as a dict in which `PATH` is present. Every one of these is a genuine OS interaction, ported faithfully, working standalone — there is no editor in sight, and there does not need to be.

`env_vars.vim` is the environment-access companion, and its sharpest lesson is the `v:null`-versus-`''` distinction:

```
let $VIMLRS_DEMO = 'hello'
call assert_equal('hello', getenv('VIMLRS_DEMO'))
call assert_equal('hello', expand('$VIMLRS_DEMO'))
call assert_equal(v:null, getenv('VIMLRS_NOT_SET_XYZ'))
call assert_equal('', $VIMLRS_NOT_SET_XYZ)
call setenv('VIMLRS_DEMO', v:null)
call assert_equal(v:null, getenv('VIMLRS_DEMO'))
```

A variable set via `:let $X` reads back through `getenv()`, through `expand()`, and through the `$X` sigil — three surfaces agreeing. But an *unset* variable reads as `v:null` through `getenv()` and as `'` through `$X`: the functional form can distinguish unset from empty, the sigil cannot, and vimlrs reproduces both. `setenv(name, v:null)` removes the variable. These two files together show a vimlrs script taking its place in a pipeline — reading the environment, shelling out, capturing output and status — which is precisely the role `awk` and Perl one-liners have always played and which vimlrs can now play in Vimscript.

scopedvars.vim: the editor-absent contract as a program

The last walkthrough is the example that most directly tests Chapter 18's contract, because its whole purpose is to pin down what the editor-tied builtins return when there is no editor. It opens with the always-real part — a bare scope sigil is a dict:

```
let g:alpha = 10
call assert_equal(4, type(g:))
call assert_equal(10, get(g:, 'alpha', -1))
call assert_true(has_key(g:, 'beta'))
```

Then the scoped-variable getters, each returning its `{def}` fallback because the variable is absent, and the setters as accepted no-ops:

```
call assert_equal('DEF', getbufvar(1, 'missing', 'DEF'))
call assert_equal(42, getwinvar(1, 'missing', 42))
call assert_equal('t', gettabvar(1, 'missing', 't'))
call assert_equal(0, setbufvar(1, 'x', 1))
call assert_equal(0, setwinvar(1, 'x', 1))
```

Then the async builtins, each failing cleanly with its documented value:

```
call assert_equal(-1, jobstart('ls'))
call assert_equal(0, jobpid(1))
call assert_equal([], jobwait([1, 2]))
call assert_equal(0, chanclose(1))
call assert_equal(-1, wait(100, 1))
call assert_equal(0, sockconnect('tcp', 'localhost:80'))
```

The header states the rationale outright: with no buffers, windows, or event loop, “the scoped-var getters return their `{def}` argument ... the setters are no-ops, and jobs/channels/sockets fail (-1) or are no-ops (0). Pinning these contracts lets editor/async scripts load.” That is the entire editor-absent contract, written as a test: a script that calls a dozen editor functions, every one of which returns the value Vim returns for an absent thing, so the script loads and runs to completion instead of crashing. When you wonder whether some Vim plugin could even *parse* under vimlrs, this is the example that tells you the shape of the answer — the editor functions are present, take their arguments, and return the documented empty value, which is exactly what a defensively-written plugin already expects.

varargs.vim: variadic functions and :unlet

`varargs.vim` is the focused demonstration of `...` parameters and the `a:000` / `a:0` machinery, plus the `:unlet` command. It opens with a pure-varargs function:

```
function! Sum(...) abort
  let total = 0
  for x in a:000
    let total += x
  endfor
  return total
endfunction
call assert_equal(0, Sum())
call assert_equal(10, Sum(1, 2, 3, 4))
```

`Sum` takes no fixed parameters, so every argument lands in `a:000`, the List of extras, which the `for` loop folds into `total`. `Sum()` with no arguments yields 0 (empty list, loop body never runs); `Sum(1, 2, 3, 4)` yields 10. Then the mixed form, fixed parameter then varargs:

```
function! Tag(label, ...) abort
  return a:label . ': ' . a:0 . ' items'
endfunction
call assert_equal('x: 0 items', Tag('x'))
call assert_equal('y: 3 items', Tag('y', 1, 2, 3))
```

Here `label` consumes the first argument and `a:000` holds only what follows, so `a:0` — its count — is 0 for `Tag('x')` and 3 for `Tag('y', 1, 2, 3)`. The example then shows a vararg function used as a lambda body's callee and exercises `:unlet`:

```
let g:tmp = 99
call assert_true(exists('g:tmp'))
unlet g:tmp
call assert_false(exists('g:tmp'))
let g:a = 1 | let g:b = 2
unlet g:a g:b
unlet! g:missing g:keep
```

`unlet g:tmp` removes the variable, after which `exists('g:tmp')` is false. `unlet g:a g:b` removes two at once. And `unlet! g:missing g:keep` removes `g:keep` without erroring on the absent `g:missing` — the `!` form's whole reason to exist. This file is the complete varargs-and-unlet contract in thirty lines, each clause asserted.

blobs.vim and compare.vim: the edges

Two shorter examples are worth a quick read because they exercise corners the larger files skip. `blobs.vim` is the byte-buffer type end to end:

```
let b = 0z00112233
call assert_equal(17, b[1])
call assert_equal(0z001122, 0z00.11.22)
call assert_equal(0z1122, slice(b, 1, 3))
call assert_equal([0, 17, 34, 51], blob2list(b))
call assert_equal(0zFF0102, insert(0z0102, 255))
call assert_fails('call insert(0z0102, 300)', 'E475')
```

Indexing a Blob gives the byte value (`b[1]` is decimal 17); `.` groups hex pairs for readability; `slice` uses byte indices with an exclusive end; `blob2list` converts to byte-value Numbers; `insert` prepends a byte; and inserting 300 — not a byte — fails with `E475`, the faithful error code. `compare.vim` exercises the case-flag comparison operators from Chapter 3:

```

call assert_equal(1, 'ABC' ==? 'abc')
call assert_equal(0, 'ABC' ==# 'abc')
call assert_equal(1, 'FooBar' =~? 'foobar')
let a = [1, 2]
let b = a
let c = [1, 2]
call assert_equal(1, a is b)
call assert_equal(1, a isnot c)
call assert_equal(1, a == c)

```

`==?` is case-insensitive, `==#` case-sensitive, `==?` a case-insensitive regex match. The identity block is the distinction worth remembering: `a is b` is true because `b` was assigned *from* `a` (same list), `a isnot c` is true because `c` is an independent list, yet `a == c` is also true because their contents are equal. Identity and equality are different questions, and Vimscript — faithfully, in vimlrs — lets you ask both.

heredoc.vim and oneline.vim: literal text and one-line blocks

Two final examples show syntax that does not fit the expression/command split cleanly. `heredoc.vim` exercises the `<<` here-document, which reads literal lines into a List:

```

let basic << END
hello
world
END
call assert_equal(['hello', 'world'], basic)
function! Trimmed() abort
  let l << trim END
  root
  child
  root2
END
return l
endfunction
call assert_equal(['root', '  child', 'root2'], Trimmed())

```

The plain form collects the lines verbatim up to the `END` marker. The `trim` form strips the first body line's indentation from every line while preserving relative indent — so `root` and `root2` lose their leading spaces but `child` keeps the two-space step that made it deeper than `root`. The body is literal text: quotes, bars, and punctuation are all just characters, which is exactly why a heredoc is the clean way to embed data that would otherwise need escaping.

`oneline.vim` exercises the one-line block bars from Chapter 3 — `if`, `while`, and `for` with their bodies and terminators all on one physical line, separated by `|`:

```

if 1 | let g:msg = 'yes' | endif
if x > 3 | let g:size = 'big' | else | let g:size = 'small' | endif
let total = 0 | for n in [1, 2, 3, 4] | let total += n | endfor
for n in range(1, 6) | if n % 2 == 0 | call add(evens, n) | endif | endfor

```

The first line is a complete `if/endif` on one line. The second adds an `else` arm. The third puts a leading `let` and then a whole `for` loop on one line. The fourth nests a one-line `if` inside a one-line `for`. Every one of these depends on the lexer's string-/operator-/comment-aware bar splitting, which knows that the `|` between `endif` and `endfor` is a command separator while a `|` inside a string or part of `||` is not. The example asserts each produces the expected result, which is the test that the bar handling is correct in all these nested and leaf-command positions.

default_args.vim: optional parameters, evaluated late

`default_args.vim` is the focused test of optional function arguments, and it repays a close read because the defaults are more powerful than they first appear. The simplest case is a constant default:

```
function! Greet(name, greeting = 'Hello') abort
    return a:greeting . ', ' . a:name . '!'
endfunction
call assert_equal('Hello, Ada!', Greet('Ada'))
call assert_equal('Hi, Ada!', Greet('Ada', 'Hi'))
```

When the caller omits `greeting`, the default `'Hello'` fills in; when they supply it, theirs wins. The header states the rule that makes the next examples work: “the default expression is evaluated at call time (in the argument scope built so far, so a later default may reference an earlier argument).” That clause — *evaluated at call time, in the scope built so far* — is what separates a real implementation from a toy. A default can reference an earlier parameter:

```
function! Span(lo, hi = a:lo + 10) abort
    return [a:lo, a:hi]
endfunction
call assert_equal([5, 15], Span(5))
call assert_equal([5, 9], Span(5, 9))
```

`Span(5)` evaluates `a:lo + 10` with `a:lo` already bound to 5, yielding 15. And a default can be any expression — a List literal, a function call, a ternary — because it is genuinely evaluated, not pattern-matched:

```
function! Mag(n, m = abs(a:n)) abort
    return a:m
endfunction
call assert_equal(7, Mag(-7))
function! Label(n, s = a:n == 1 ? 'one' : 'many') abort
    return a:s
endfunction
call assert_equal('one', Label(1))
call assert_equal('many', Label(4))
```

`Mag(-7)` runs `abs(a:n)` to get 7; `Label(1)` evaluates the ternary to `'one'`. Defaults even compose with varargs — `function! Acc(base, step = 1, ...)` mixes a fixed parameter, an optional one, and `a:000` in one signature. This is Vim's `optional-function-argument` feature in full, and Chapter 21 explained the mechanism: each default is compiled to its own chunk that runs in the partially-bound `a:` scope when the argument is omitted, which is exactly why a later default can see an earlier argument.

funcref_call.vim and concat_dot.vim: two adjacency puzzles

These two short examples both turn on the byte-span adjacency rules from Chapter 20, and reading them together shows how much the parser decides from spacing alone. `funcref_call.vim` calls a funcref-valued expression directly by writing `(args)` right after it:

```
call assert_equal('HI', function('toupper')('hi'))
call assert_equal(42, {x -> x * 2}(21))
let fns = [function('toupper'), function('tolower')]
call assert_equal('HIhi', fns[0]('Hi') . fns[1]('Hi'))
```

The result of `function('toupper')` is a Funcref, and the abutting `('hi')` calls it — that is the `CallExpr` node from Chapter 20. A lambda literal can be called the same way, `{x -> x * 2}(21)`, and so can a funcref pulled out of a list element, `fns[0]('Hi')`. The header states the rule that makes it unambiguous: “the (

must about the expression (no space), so 'a' (x) stays two values.” The abutting paren is what tells the parser this is a call rather than a separate parenthesized argument.

`concat_dot.vim` is the dot’s version of the same puzzle. The `.` is overloaded — `a . b` concatenates, `d.key` reads a member — and the disambiguation is again adjacency plus a Vim9-vs-legacy subtlety:

```
call assert_equal('AB', toupper('a').toupper('b'))
call assert_equal('aabbcc', substitute('abc', '.', '\=submatch(0).submatch(0)', 'g'))
let d = {'k': 5, 'f': 1, 'g': 2}
call assert_equal(5, d.k)
let nested = {'a': {'b': 7}}
call assert_equal(7, nested.a.b)
```

`toupper('a').toupper('b')` concatenates two call results, because `.name()` is concatenation-with-a-call, not a member call — legacy Vimscript has no `dict.key(args)` syntax. But `d.k` and `nested.a.b` are member reads, because the dot abuts a bare name with no following `()`. The example’s header names the rule exactly: “`.name()` is always concatenation with a function call ... so `f(x).g(y)` concatenates the two results.” Both files are small, both pass in CI, and both are really tests of the parser’s adjacency logic from Chapter 20 working on real code.

fuzzy.vim: a faithful fuzzy matcher

`fuzzy.vim` tests `matchfuzzy()` and `matchfuzzypos()`, and it is worth a tour because fuzzy matching is exactly the kind of feature a casual reimplementer would get *approximately* right and vimlrs gets *exactly* right, by porting Neovim’s `search.c` scorer. The basic behaviour keeps and ranks subsequence matches:

```
call assert_equal(['help', 'hello', 'shell'], matchfuzzy(['hello', 'world', 'help',
  ↪ 'shell'], 'hl'))
call assert_equal([], matchfuzzy(['world'], 'xz'))
call assert_equal(['clip', 'cardiologist'], matchfuzzy(['cardiologist', 'clip'], 'cli'))
```

`matchfuzzy(..., 'hl')` keeps only the items where `h` then `l` appear as a subsequence, best score first; a non-subsequence pattern like `'xz'` drops the item entirely; and an exact prefix (`clip` for `'cli'`) outscores a scattered match. The scoring is the faithful part — the header lists the bonuses it ports: “sequential/camel/separator/first-letter bonuses.” `matchfuzzypos` exposes the machinery, returning items, the 0-based character positions that matched, and the integer scores:

```
call assert_equal(['help', 'hello'], [[0, 2], [0, 2]], [113, 112]),
  ↪ matchfuzzypos(['hello', 'help'], 'hl'))
```

Those scores, `113` and `112`, are `search.c`’s actual integer scores, not vimlrs’s invention — which is why the example can assert them exactly. The example also tests the options that tune matching: a space in the pattern splits it into independently-matched words by default (so `'f b'` matches `foobar`), the `matchseq` option forces one contiguous sequence, the `key` option fuzzy-matches a Dict field instead of the item, and `limit` caps the result count. Reproducing all of this — the bonuses, the scores, the word-splitting, the options — to the point of asserting exact integer scores is a strong signal of how deep the port reaches into even the discretionary corners of the builtin surface.

matchstrlist.vim and string_repr.vim: exact structure

`matchstrlist.vim` tests a function that reports every match across a list of strings, and its precision is the lesson. Each result is a Dict, and a multi-match input produces multiple results:

```

call assert_equal(
  \ [{ 'idx': 0, 'byteidx': 1, 'text': 'X'}, { 'idx': 0, 'byteidx': 3, 'text': 'X'}],
  \ matchstrlist(['aXbXc'], 'X'))
call assert_equal(
  \ [{ 'idx': 0, 'byteidx': 1, 'text': 'oo',
    \ 'submatches': ['o', 'o', '', '', '', '', '', '', '']}],
  \ matchstrlist(['foobar'], '\(o\) \(o\)'), { 'submatches': v:true}))

```

Both matches of `X` in `aXbXc` are reported with their byte offsets, and the `submatches` option adds the nine `\1..\9` group slots — padded to nine even when fewer groups matched. Asserting the exact Dict structure, key by key, is only possible because the dict is insertion-ordered (Chapter 5) and the submatch list is exactly nine long, both of which are faithful details.

`string_repr.vim` is the round-trip companion, testing that `string()` produces reparsable Vimscript and that `eval(string(x))` reconstructs `x`:

```

call assert_equal("'it's'", string("it's"))
call assert_equal("{ 'a': 1, 'b': 2 }", string({b: 2, a: 1}))
for v in [42, 'text', [1, 2, 3], #{x: 1}]
  call assert_equal(v, eval(string(v)))
endfor

```

A single quote inside a string is doubled in the output; a dict renders with its keys *sorted* (so `#{b: 2, a: 1}` becomes `{ 'a': 1, 'b': 2 }`); and the loop at the end is the round-trip property stated as a test — for each value, `eval(string(v))` equals `v`. This is the `eval(string(x)) == x` backbone Chapter 5 promised, here asserted directly over scalars and containers. It is also the property that makes the insertion-ordered-dict and the careful encoder pay off: without a deterministic renderer, neither the exact string assertions nor the round-trip could be written.

glob.vim, slicing.vim, and expand.vim: the filesystem and the edges

These three round out the standalone-as-shell-citizen story Chapter 24 began. `glob.vim` builds a throwaway directory and asserts exact wildcard expansion:

```

call assert_equal([dir . '/alpha.txt', dir . '/beta.txt'], glob(dir . '/*.txt', 0, 1))
call assert_equal([dir . '/gamma.log'], glob(dir . '/gamm?.log', 0, 1))
call assert_equal([], glob(dir . '/none.*', 0, 1))
call assert_equal($HOME, glob('$HOME'))

```

The list form returns sorted matches, `?` matches exactly one character, no match yields an empty list (or empty string in the string form), and `$VAR` is expanded before matching. These are real filesystem operations on a real temp directory, cleaned up afterward — `glob` is one of the genuinely-implemented functions from the Files family, not a stub. `slicing.vim` contrasts the two slice conventions that trip everyone up:

```

call assert_equal([1, 2], slice(nums, 1, 3))
call assert_equal([1, 2, 3], nums[1:3])

```

`slice(nums, 1, 3)` has an *exclusive* end (elements 1 and 2), while the `[1:3]` index form has an *inclusive* end (1, 2, and 3). The same file shows `slice` on a string using character indices and on a blob using byte indices, plus `strcharlen` folding a composing mark into its base and `strtrans` rendering a tab as `^I`. `expand.vim` is the edges file — `expand('$HOME')` and `expand('~')` resolve, file wildcards expand, but editor specials like `expand('%')` (the current file) are empty standalone because there is no current file, and the window-view and server builtins return their documented inactive values. Reading these three together shows the precise seam: the filesystem and environment functions are real, and the editor-context functions return emptiness, exactly as Chapters 18 and 24 framed it.

search.vim, marks.vim, and registers.vim: the buffer in motion

The last group are the virtual-buffer examples from Chapter 24, read as programs. `search.vim` drives the cursor over a buffer with the regex engine:

```
call setline(1, ['foo bar', 'baz foo', 'qux', 'foo end'])
call cursor(1, 1)
call assert_equal(2, search('foo'))
call assert_equal(4, search('foo'))
call assert_equal(1, search('foo', 'c'))
```

`search('foo')` from line 1 skips the match at the cursor and moves to line 2, then to line 4 on the next call; the `c` flag accepts a match at the cursor, so it returns line 1. The `n` flag finds without moving, `b` searches backward, `e` moves to the match end — every flag Vim’s `search()` documents, working on the in-memory buffer. `searchpair('\<if\>', '', '\<endif\>')` finds the matching `endif` of a nested `if`, and `searchcount()` reports the position among matches. `marks.vim` sets named marks with `:mark a`, reads them with `getpos("'a")`, uses them in ranges (`:a, 'bd` deletes the marked range), and lists them with `getmarklist()` — marks being buffer data, they work standalone. And `registers.vim` shows registers shared between the `@a` expression syntax and the `:yank/put` commands:

```
let @a = 'hello'
call setline(1, ['one', 'two', 'three'])
:2y c
call assert_equal("two\n", @c)
```

`let @a = 'hello'` writes a register through the expression layer; `:2y c` yanks buffer line 2 into register `c` through the command layer; and `@c` reads back `"two\n"` — the two views of the same register state staying consistent because there is one register store underneath, in the per-run state Chapter 22 described. Reading these three is the most direct way to feel that vimlrs’s virtual buffer is not a stub but a small, coherent, faithfully-ported editing substrate.

literal_dict.vim and index_get.vim: dict literals and safe reads

`literal_dict.vim` isolates the `#{}` literal-key dictionary, and it is worth a read because the distinction it pins down is genuinely confusing the first time you meet it. In a `#{}` dict the keys are bare literals — not quoted, and not evaluated as expressions — so `#{foo: 1}` is exactly `{'foo': 1}`:

```
let foo = 'EXPANDED'
let lit = #{foo: 10}
let expr = {foo: 10}
call assert_equal(10, lit.foo)
call assert_equal(10, expr['EXPANDED'])
call assert_false(has_key(expr, 'foo'))
```

The contrast is the lesson: in the `#{}` form the key is the literal text `foo`, but in the ordinary `{}` form `foo` is an *expression* that evaluates to `'EXPANDED'`, so `expr` has a key `'EXPANDED'` and no key `'foo'` at all. A `#{}` key can lead with a digit (`#{200: 'ok'}` has the string key `'200'`) precisely because it is literal text, not an identifier. The example also nests them — `#{name: 'root', kids: [{name: 'a'}]}` — proving the literal form composes. This is the `get_literal_key` path from `eval.c`, and the example is the cleanest statement of why `#{}` exists: it is the form for when your keys are known words and you do not want to quote them or risk them being evaluated.

`index_get.vim` is the companion for reading data safely, walking `index()` and `get()` over Lists and Blobs:

```
call assert_equal(1, index(['a', 'B'], 'b', 0, 1))
call assert_equal(-1, index(['a', 'B'], 'b'))
call assert_equal(99, get({'a': 1}, 'b', 99))
call assert_fails("call get('hello', 1)", 'E1531')
```

`index()` takes a `{start}` (negative counts from the end) and, for lists, an `{ic}` ignore-case flag — so `index(['a', 'B'], 'b', 0, 1)` finds the 'B' case-insensitively while the bare form returns -1. `get()` reads with a default, returning 99 for a missing dict key. The faithful edge is the last line: `get()` on a *String* errors with E1531 rather than silently returning the default, because a string is not a keyed collection — a from-scratch implementation would likely accept it and return the default, and vimlrs declines because the ported `get` does.

strindex.vim and match_startcount.vim: indexing and the match family

`strindex.vim` revisits the byte-versus-character indexing of Chapter 9 on a multibyte string, asserting the exact offsets where 'é' spans two bytes:

```
call assert_equal(1, byteidx('héllo', 1))
call assert_equal(3, byteidx('héllo', 2))
call assert_equal(2, charidx('héllo', 3))
call assert_equal('hé', strcharpart('héllo', 0, 2))
call assert_equal("'it'\\"'s'", shellescape("it's"))
```

`byteidx('héllo', 2)` is 3 because the first two characters occupy three bytes; `charidx` inverts it; `strcharpart` slices by character so `(0, 2)` is 'hé'. The example also checks `shellescape("it's")`, which single-quotes the argument and escapes the embedded quote as `'it'\''s'` — the exact shell-quoting Vim produces, faithful down to the backslash. And `matchstrpos('foobar', 'o\+')` returns `['oo', 1, 3]`, the matched text with its start and end positions.

`match_startcount.vim` is the focused test of the `{start}` and `{count}` arguments that every match-family function accepts, and it is the kind of completeness a casual port skips:

```
call assert_equal(4, match('abcabc', 'b', 3))
call assert_equal(4, match('abcabc', 'b', 0, 2))
call assert_equal('Y', matchstr('aXbYc', '[A-Z]', 0, 2))
call assert_equal(1, match(['foo', 'bar', 'baz'], 'ba'))
call assert_equal(['a', 1, 1, 2], matchstrpos(['foo', 'bar'], 'a'))
```

`{start}` says where to begin and `{count}` picks the Nth match, so `match('abcabc', 'b', 0, 2)` finds the *second* b at index 4. The genuinely intricate part is the List-subject form: when the subject is a List, `match` returns the *item index* of the first matching item, `matchstr` returns the whole matching item, and `matchstrpos` returns `[match, item_idx, start_col, end_col]` — four elements, not three. Reproducing the scalar-versus-list polymorphism of the whole match family, each with `{start}` and `{count}`, is exactly the depth that distinguishes a port from an approximation, and the example asserts every variant.

matches.vim, history.vim, and quickfix.vim: in-memory editor lists

Three examples exercise editor subsystems that are pure in-memory bookkeeping standalone — a match-highlight list, the command-line history rings, and the quickfix and location lists — and reading them shows how much of “the editor” is really just data structures. `matches.vim` walks the highlight-match list:

```

call assert_equal(42, matchadd('Error', 'foo', 20, 42))
call assert_equal([{'group': 'Error', 'pattern': 'foo', 'priority': 20, 'id': 42}],
  ~ getmatches())
call assert_equal(0, matchdelete(42))
call assert_equal(3, matchaddpos('Visual', [[1, 2, 3], 5], 10, 3))

```

`matchadd` records a (group, pattern, priority, id) match and returns its id; `getmatches` lists them as dicts; `matchdelete` removes one and returns 0 (or -1 quietly for a missing id); `matchaddpos` records line/column positions instead of a pattern. There is no screen to highlight on, but the list itself is real, ported from `window.c`, and `setmatches/clearmatches` restore and empty it. `history.vim` is the command-line history rings from `cmdhist.c`:

```

call assert_equal(1, histadd('cmd', 'ls'))
call assert_equal('pwd', histget('cmd'))
call assert_equal('ls', histget('cmd', -1))
call assert_equal(1, histdel('cmd', 1))

```

`histadd` appends to a named ring (cmd, search, expr, input), de-duplicating by moving an existing entry to newest; `histnr` counts; `histget` reads by absolute or negative index; `histdel` removes one or clears the ring. The named histories are independent, an empty item fails, and an invalid name returns -1 — the full faithful contract. `quickfix.vim` is the richest of the three, a real error-list implementation from `quickfix.c`:

```

call assert_equal(0, setqflist([{'lnum': 10, 'text': 'oops', 'bufnr': 1}, {'text': 'no
  ~ line'}]))
let qf = getqflist()
call assert_equal(1, qf[0].valid)
call assert_equal(0, qf[1].valid)
call assert_equal(0, setqflist([{'text': 'three'}], 'a'))

```

`setqflist` normalizes each entry to the full quickfix schema — an entry with a real buffer and line is `valid`, a bare-text entry is not, and every entry carries `end_lnum`, `type`, and the rest. The action argument ('a' append, 'r' replace, 'f' free) and the {what} dict for title and size all work, and the location list is independent of the quickfix list. These three examples together make a point the catalogue chapters argued abstractly: a large fraction of “the editor” is data structures that a standalone interpreter can keep faithfully, with only the display absent.

highlight.vim, editor_compat.vim, and windows.vim: the absent-editor contract

The flip side is the subsystems that genuinely need a live editor, and three examples pin down exactly what they return when there is none. `highlight.vim` checks the highlight and syntax queries, all of which miss because no syntax or highlight table is loaded:

```

call assert_equal(0, hllID('Comment'))
call assert_equal(hllID('Comment'), highlightID('Comment'))
call assert_equal(0, diff_hllID(1, 1))
call assert_equal('', synIDattr(synIDtrans(hllID('Comment')), 'bg'))

```

Every group name resolves to id 0, the deprecated `highlightID` alias agrees, and the idiomatic chain `hllID()->synIDtrans()->synIDattr()` folds cleanly to the empty string — so a script that reads a highlight attribute gets '' rather than crashing. `editor_compat.vim` is the precise statement of the buffer-is-real-but-screen-is-not boundary:

```

call assert_equal([0, 1, 1, 0], getpos('.'))
call assert_equal([0, 1, 1, 0, 1], getcurpos())
call assert_equal(0, search('foo'))
call assert_equal(-1, screenchar(1, 1))
call assert_equal([], synstack(1, 1))
call assert_equal(0, cursor(1, 1))

```

The cursor is real — `getpos('.')` is `[0, 1, 1, 0]`, line 1 column 1, and `cursor()` succeeds — because the virtual buffer has a cursor; but `screenchar` is `-1` and `synstack` is `[]` because there is no rendered screen or syntax engine. `windows.vim` draws the same line for windows and tabs: the virtual buffer's line functions all succeed (`getline`, `setline`, `getbufinfo()[0].bufnr` is 1), while `getwininfo()` is `[]`, `win_getid()` is 0, and `getwinpos()` is `[-1, -1]`. The example even models the adaptive idiom directly — `if win_getid() == 0 | echo 'running headless' | endif` — which is exactly the pattern a well-written plugin uses, and exactly why the contract returns a checkable value rather than an error.

map_commands.vim, equalfile.vim, sfile.vim, and undo.vim: commands, files, and paths

Four shorter examples round out the tour. `map_commands.vim` exercises the `:map` family Ex commands — the command-syntax counterpart to the `mapset()` function of `mappings.vim`:

```

nnoremap <silent> <leader>s :nohlsearch<CR>
let m = maparg('<leader>s', 'n', 0, 1)
call assert_equal(1, m.silent)
nunmap <[-a>
imapclear

```

`:nmap`, `:inoremap`, `:nnoremap`, `:vmap` all define mode-scoped mappings observable through `maparg/maplist`; the `<silent>/<expr>/<nowait>` prefixes set their flags; `:nnoremap` variants set the `noremap` flag while plain `:map` leaves it 0; and `:unmap`/`:mapclear` remove. `equalfile.vim` tests `assert_equalfile`, writing scratch files and comparing them — identical files pass, differing or missing files record one error each (the example captures and clears `v:errors` for the expected failures so they do not fail the script). `sfile.vim` resolves `expand('<sfile>')` to the running script's own path, with filename-modifiers:

```

call assert_equal('sfile.vim', expand('<sfile>:t'))
call assert_equal('vim', expand('<sfile>:e'))
call assert_equal(expand('<sfile>'), expand('<script>'))

```

`<sfile>` is the current script, and inside a `:source`'d file it is *that* file, restored afterward — the sourcing-name stack made observable. And `undo.vim` shows `undofile()` computing the undo-file path Vim would use (`/tmp/.notes.txt.un~`) even though there is no undo history, while `undotree()` reports an empty, synced tree. Each is a small, complete contract, and each is a CI-checked assertion that vimlrs honours it.

buffer.vim: the virtual buffer in full

`buffer.vim` is the most complete single demonstration of the in-memory buffer of Chapter 24, walking the line-editing functions end to end. It fills the buffer, inserts, replaces, deletes, and tracks positions:

```

call setline(1, ['alpha', 'beta', 'gamma'])
call assert_equal(3, line('$'))
call append(1, 'inserted')
call assert_equal(['alpha', 'inserted', 'beta', 'gamma'], getline(1, '$'))
call append(0, 'top')
call cursor(1, 7)
call assert_equal(12, col('$'))
call assert_equal([0, 1, 7, 0, 7], getcurpos())

```

`setline` fills the buffer and `line('$')` reports the count; `append(1, ...)` inserts after line 1 while `append(0, ...)` inserts before the first line; `setline` with a List replaces several lines at once; `deletebufline('', 2, 3)` deletes a range. The position functions are exact: after `cursor(1, 7)`, `col('$')` is 12 because 'hello world' is 11 bytes and the end-of-line column is one past, and `getcurpos` returns the five-element form with curswant. `indent()` measures leading whitespace with tabs expanded to 'tabstop' (a tab indents to 8), and `line2byte/byte2line` convert between line numbers and byte offsets — `line2byte(2)` is 5 because line 1 'abc' plus its newline is 4 bytes. `nextnonblank/prevnonblank` skip whitespace-only lines. Every one of these is a real operation on the one virtual buffer, ported from `buffer.c` and `memline.c`, and reading the file is the fastest way to internalize that the buffer is genuinely there.

sign_place.vim: placed signs as bookkeeping

`sign_place.vim` complements the `sign_define` table of `signs.vim` with the *placement* list — where signs sit in buffers — and it is a good example of a subsystem that is real data even with no buffer column to draw in:

```

call assert_equal(1, sign_place(0, 'g1', 'err', 1, {'lnum': 10, 'priority': 20}))
call assert_equal(5, sign_place(5, 'g1', 'warn', 1, {'lnum': 12}))
let all = sign_getplaced()
call assert_equal({'id': 1, 'group': 'g1', 'name': 'err', 'bufnr': 1, 'lnum': 10,
  ~ 'priority': 20}, all[0].signs[0])
call assert_equal(10, all[0].signs[1].priority)

```

`sign_place` with id 0 auto-assigns an id and returns it; an explicit id is kept; the default priority is 10 when omitted. `sign_getplaced` groups placed signs by buffer and returns the full per-sign dict, and it can filter by buffer and then by group. `sign_placelist/sign_unplacelist` operate on lists of placements, and `sign_unplace('g2')` removes a whole group. Only `sign_jump` returns -1, because jumping needs an editor cursor to move to. The list itself — ids, groups, names, lines, priorities — is faithful in-memory bookkeeping ported from `sign.c`, which is why a script that places signs and reads them back works exactly as in Neovim.

completion.vim and editor_queries.vim: well-defined empty answers

These two examples are the careful specification of what the still-more-editor-tied queries return standalone, and the lesson is that the answers are *well-defined*, not arbitrary. `completion.vim` shows `getcompletion` doing real work for the contexts that need no editor:

```

call setenv('VIMLRS_COMPLETION_TESTVAR', 'yes')
let env = getcompletion('VIMLRS_COMPLETION_', 'environment')
call assert_true(index(env, 'VIMLRS_COMPLETION_TESTVAR') >= 0)
call assert_true(len(getcompletion('', 'file')) > 0)
call assert_equal([], getcompletion('x', 'this_is_not_a_real_type'))

```

`getcompletion('...', 'environment')` matches real environment variable names and `getcompletion('', 'file')` lists the current directory — genuine completion against the environment and filesystem — while an unsupported completion type yields an empty list rather than an error. The

input functions (`getchar`, `getcharstr`) return “no input,” the indent helpers `cindent/lispindent` return -1 with no buffer context, and the insert-completion functions are inactive. `editor_queries.vim` pins the snapshot dict that inactive subsystems return — a `searchcount()` with all-zero counts but `maxcount` of 99, and a `complete_info()` with an empty `mode`, `pum_visible` 0, empty `items`, and `selected` -1:

```
let ci = complete_info()
call assert_equal('', ci.mode)
call assert_equal(0, ci.pum_visible)
call assert_equal(-1, ci.selected)
```

The reason these matter is that a plugin often reads such a dict and branches on a field; returning the documented inactive snapshot — with every key present and set to its empty value — lets that branch take the no-op path rather than crashing on a missing key. It is the editor-absent contract at the level of dict *shape*, not just scalar return values.

printf_exists.vim and interactive.vim: formatting depth and input

`printf_exists.vim` covers two corners: `printf`'s `%5` conversion and argument-supplied width/precision, and `exists('*name')` for callables:

```
call assert_equal('    3', printf('%*d', 5, 3))
call assert_equal('3    ', printf('%-*d', 5, 3))
call assert_equal('    3.14', printf('%*.*f', 8, 2, 3.14159))
call assert_equal(1, exists('*substitute'))
call assert_equal(0, exists('*no_such_function_xyz'))
```

A `*` in the width or precision takes that value from the next argument, so `printf('%*d', 5, 3)` right-justifies 3 in a field of width 5, `%-*d` left-justifies, and `%*.*f` takes both width and precision from arguments. And `exists('*name')` reports whether a callable is defined — true for builtins like `substitute` and `printf`, true for a user `:function`, false otherwise — the feature-detection idiom plugins use to guard optional dependencies. `interactive.vim` is the one example that genuinely reads from the user: `input()` reads a line, `inputlist()` offers a numbered menu, and `confirm()` a button choice. Standalone they write the prompt to stdout and read a line from stdin — “the role `read` plays in a shell script” — and CI feeds canned answers through a fixture so the asserts can check the parsed values. It is the clearest demonstration that vimlrs can play the interactive-script role too, not just the batch one.

table_complete.vim and providers.vim: completing the table

The last two examples are the ones that exist to drive the final entries of the `eval.lua` function table to a defined value, and reading them is a tour of the furthest-from-standalone surface. `providers.vim` walks the context stack, the language providers, RPC, and msgpack, each returning its inactive value:

```
call assert_equal({}, ctxget())
call assert_equal(v:null, py3eval('1 + 1'))
call assert_equal(0, rpcnotify(1, 'event'))
call assert_equal([], msgpackdump([]))
call assert_equal('', submatch(0))
```

The editor context stack is empty, the Python and Perl providers return `v:null`, the RPC functions return 0 with no channel, and `submatch(0)` outside a substitution is the empty string. `table_complete.vim` is the same in spirit, covering the provider evals, the mouse and screen positions (all-zero dicts), the script and stack introspection (empty lists), and the deprecated job aliases `jobsend/jobclose` (aliases of `chansend/chanclose`).

Neither example teaches a new feature; both exist to assert that the *entire* function table — even its most editor-bound, most provider-dependent corners — returns a defined, documented value rather than an undefined-function error. That completeness is the quiet achievement these two files certify: there is no implemented builtin that crashes or is missing, only ones that do real work and ones that honestly return emptiness, and the line between them is exactly the line between data and editor activity this book has traced throughout.

autocommands.vim and beeps.vim: events and the bell

`autocommands.vim` is the fullest demonstration of the autocommand subsystem of Chapter 24, and reading it as a program shows how a feature that *seems* to need an event loop is made testable standalone. The trick is `:doautocmd`, the manual trigger:

```
let g:log = []
autocmd User Foo call add(g:log, 'foo1')
autocmd User Foo call add(g:log, 'foo2')
autocmd BufRead *.txt call add(g:files, 'txt')
doautocmd User Foo
call assert_equal(['foo1', 'foo2'], g:log)
doautocmd BufRead notes.txt
```

Handlers register against an event (and optional pattern) and fire in registration order when `:doautocmd` names a matching event; a `BufRead *.txt` handler fires for `notes.txt` but not `script.vim`, comma-separated events register one handler each, `:augroup` groups them, and `:autocmd!` removes them. Nothing fires on its own — there is no editor generating events — but the registry and the firing logic are real, so a script that registers handlers and triggers them by hand exercises the whole subsystem. `beeps.vim` is a smaller, cleverer example: it tests `assert_beeps` and `assert_nobeep` by modelling a “beep” as a command that errors, since in Vim an erroring Ex command rings the bell:

```
call assert_equal(0, assert_beeps('call no_such_function_xyz()'))
call assert_equal(0, assert_nobeep('let g:beeps_demo = 1'))
```

`assert_beeps` passes when its command errors (rings the bell), `assert_nobeep` when it runs clean, and both capture the errors they produce rather than leaking them to `v:errors`. It is a neat illustration of how the testing framework maps an editor concept — the bell — onto the standalone reality of an error, faithfully enough that the assertion behaves as it would in Neovim.

sourcing.vim, autoload.vim, and heredoc.vim: modules and literal data

Three examples demonstrate how a multi-file vimlrs program is structured, and reading them shows Vimscript’s module system working standalone. `sourcing.vim` sources a tiny library and then uses it:

```
source examples/lib/mathlib.vim
call assert_equal(1, g:mathlib_loaded)
call assert_equal(16, Square(4))
call assert_equal([1, 4, 9], map([1, 2, 3], {i, v -> Square(v)}))
```

The library, `lib/mathlib.vim`, is six lines — two `function!` definitions and a `let g:mathlib_loaded = 1` — and after the `source`, `Square` and `Cube` are ordinary callable functions and `g:mathlib_loaded` is set, because the sourced file ran in the current scope. The last assertion proves the sourced functions compose with lambdas and the higher-order builtins exactly as if they had been defined inline. `autoload.vim` is the lazy version: it writes a throwaway `autoload/demo.vim` at runtime, then calls `demo#double(5)`, which finds no such function, derives `autoload/demo.vim` from the `demo#` prefix, sources it, and dispatches — `g:demo_automloaded` confirming the file ran. Subsequent calls reuse the loaded function. Together they show the two

module forms: explicit `:source` for code you always need, `autoload` for code you might not.

`heredoc.vim` is the literal-data companion, and it walks the `=<<` here-document more fully than Chapter 4 did:

```
let raw =<< EOF
it's got a | pipe
a "quoted" word
END is only the marker when alone
EOF
call assert_equal(["it's got a | pipe", 'a "quoted" word',
    \ 'END is only the marker when alone'], raw)
let doc = {}
let doc.body =<< END
first
second
END
call assert_equal(['first', 'second'], doc.body)
```

The body is verbatim — the bar, the quotes, and a line that merely *contains* `END` are all just text, because the heredoc is captured before any splitting (Chapter 4). A heredoc can target a Dict entry (`let doc.body =<< END`) like any other `:let` lvalue, the `trim` form strips the first body line's indent from every line while preserving relative indent, and an empty heredoc is the empty list. These three examples are the structural toolkit — source a library, autoload a package, embed literal data — that lets a real `.vim` codebase be more than a single file, which matters because the whole premise of vimlrs is running real `.vim` code, and real `.vim` code is rarely one file.

What this tour left out

This chapter walked perhaps a quarter of the example suite closely, and it is worth naming the rest by theme so you know what is there when you go looking. The *language-core* examples pin down syntax and semantics: `numbers.vim`, `float_literals.vim`, `numeric_edge.vim`, and `floatmath.vim` for the numeric tower (Chapter 23); `string_index.vim`, `slicing.vim`, `width.vim`, and `strindex.vim` for string indexing and width (Chapter 9); `list_unpack.vim`, `literal_dict.vim`, `index_get.vim`, and `special_none.vim` for the data shapes; `compare.vim`, `scopedvars.vim`, and `concat_dot.vim` for scopes and operators. The *regex* examples — `regex_verymagic.vim`, `regex_zs.vim`, `regex_backref.vim`, `regex_optseq.vim` — each isolate one corner of the dialect (Chapter 10). The *virtual-buffer* examples — `excommands.vim`, `streamedit.vim`, `normal.vim`, `search.vim`, `marks.vim`, `registers.vim`, `cmdline.vim`, `options.vim` — exercise the Ex layer (Chapter 24).

The largest group is the *editor-contract* examples, which pin down what the standalone interpreter returns for editor-shaped queries: `editor_absent.vim`, `editor_queries.vim`, `editor_compat.vim`, `buffers.vim`, `windows.vim`, `mappings.vim`, `autocommands.vim`, `signs.vim`, `sign_place.vim`, `marks.vim`, `highlight.vim`, `quickfix.vim`, `completion.vim`, `providers.vim`, `undo.vim`, `history.vim`, `digraphs.vim`, `stdpath.vim`, and more — together they are the executable specification of Chapter 18's contract and Chapter 24's data subsystems, each asserting either a real answer or the documented empty value. And the *interop* examples — `json.vim`, `msgpack.vim`, `system.vim`, `env_vars.vim`, `sysinfo.vim` — cover the outside world (Chapter 16). Every one of these is a self-test run in CI, so each is a true statement about how vimlrs behaves; the ones walked above were chosen because they teach the most per line, but any of the ninety can be read the same way, and reading a few you have not seen is the best exercise this book can leave you with.

How to read any example

The files walked above share a shape, and once you see it you can read any of the ninety. Each opens with a comment header naming the features it demonstrates and the command to run it. Each defines

whatever functions it needs. Each runs a block of assertions — value-asserts for the happy path, `assert_fails/assert_exception` for the error path — that accumulate failures into `v:errors` rather than aborting. Most include a small demo section that `echos` results for a human reader. And every one ends with the same epilogue: if `v:errors` is non-empty, print each failure and `:throw`, which sets the error flag and makes the process exit non-zero. To read an example, read the header to know what it is about, read the functions to know the machinery, read the assertions to know the exact contract being claimed, and trust the epilogue to enforce it. Because CI runs all of them, every assertion in every example is a true statement about how vimlrs behaves — which is why this book could quote them so freely. They are not illustrations of intended behaviour; they are the behaviour, tested daily.

Chapter 40 – One Script, End to End

The internals chapters took the machine apart stage by stage — lexer, parser, compiler, bridge, JIT, cache. This closing chapter puts it back together by following a single small script through every stage, from the bytes on disk to the native code that runs and the cache entry that persists. The script is the iterative Fibonacci function of Chapter 4, chosen because it touches almost every interesting path: a function definition, a numeric loop, the slot planner, the JIT, and the assert framework. Tracing it is the best way to see how the pieces this book described separately actually compose into one run.

The bytes and the splitter

The run begins when the driver (`cli.rs`) is handed `fib.vim` and routes it through the cache. Suppose this is a cold run — the cache misses, so the full pipeline fires. The source text is read and handed to the `Lines` cursor, which does the three pre-passes of Chapter 20 before any lexing: it folds any heredoc assignments (there are none here), joins line-continuation backslashes, and splits `|`-separated commands. For `fib.vim` the splitter mostly passes lines through unchanged, but it is already doing the comment-aware work — the file's `"`-prefixed header lines are recognized as comments and stripped, and the function body's lines are separated into logical lines for the parser. What reaches the parser is a clean sequence of logical lines, each one statement's worth of text, with the command-line concerns the lexer refused already resolved.

Lexing and parsing FibIter

Each logical line is lexed to tokens. The line `let i += 1` lexes to an `Ident("i")`, a compound-assign operator, and a `Number(1)` — the lexer having already decoded the literal and recorded byte spans. The parser's statement dispatcher sees the leading `let` and parses a `Let` statement, desugaring the `+=` into `i = i + 1` as it goes (Chapter 21). The line `while i < a:n` becomes a `While` whose condition is a `Compare { lhs: Var("i"), op: Less, rhs: Var("a:n") }`, parsed through the `eval4` rung of the ladder. The whole `function! FibIter(n) abort ... endfunction` block is parsed by `parse_function`, which finds the matching `)` of the parameter list, reads the body statements until `endfunction`, and produces a `Function` statement with the name, the parameter `n`, an empty defaults list, the body, and the bang flag. By the end of parsing, `fib.vim` is a `Vec<Stmt>` — the two function definitions, the assertion loop, and the epilogue — each statement a node in the AST the next stage lowers.

Lowering and the slot plan

The compiler walks the AST to a `CompiledProgram`. For `FibIter` it first runs `slot_plan` over the body. The locals `a`, `b`, `i`, and `t` are all bare names — so `l:` locals — and every assignment to each has a numeric right-hand side: `let a = 0` is a number, `let t = a + b` is an arithmetic of numerics, `let i = i + 1` is the desugared increment. The body contains no function call, no `:execute`, no `:try`, no non-bare `:let` target — so no bail condition fires — and the fixpoint sweep adds all four names to the slot set, with `a`, `b`, `i`, and `t` in `int_slots` since they only ever hold integers. With the slots assigned, the compiler lowers the `while` in its rotated form: a jump to the condition at the bottom, the body emitting `Op::GetSlot/Op::Add/Op::SetSlot` for the integer updates, and the condition emitting a native `Op::NumLt` with a `JumpIfTrue` backedge. The recursive `Fib`, by contrast, contains a function call (`Fib(a:n - 1)`), so its slot plan bails and its body lowers

to `VIML_*` builtin calls — correct but not traced. The output is a `main` chunk plus the `FibIter` and `Fib` chunks, ready for fusevm.

The bridge, the VM, and the JIT

The driver installs the bridge (`fusevm_bridge.rs`): it registers every `VIML_*` handler against its id, seeds the `v:` variables, installs the filter/map and substitute hooks, and — unless `VIMLR5_NO_JIT` is set — enables fusevm's tracing JIT. Then it runs the `main` chunk. The assertion loop `for i in range(len(expected))` lowers to a native counter (Chapter 21), and each iteration calls `FibIter(i)`. The first few calls run `FibIter`'s rotated loop in the interpreter, but the loop is hot — the backedge is taken many times — so the tracing JIT records a trace of the body, sees that every operation is a native integer op over slots, and compiles the trace to machine code. From then on, `FibIter`'s loop runs as native code, the `Op::Add` and `Op::NumLt` becoming the actual machine instructions the CPU would run for the equivalent Rust. `VIMLR5_JIT_STATS=1` would report the trace compiled; `FibIter(50)` returning `12586269025` exercises the 64-bit integer path the native ops preserve. The recursive `Fib` runs entirely on the builtin path, its `VIML_ADD` calls reaching `b_add` and the ported arithmetic — correct, faithful, and slower, exactly as the deopt boundary predicts.

The assertions, the exit, and the cache

Each `assert_equal(expected[i], FibIter(i))` runs the ported `f_assert_equal`, which compares the two values with `typval_compare` and, on a mismatch, appends a message to `v:errors` rather than throwing. With all assertions passing, `v:errors` stays empty, the epilogue's `if !empty(v:errors)` is false, the `:throw` is skipped, and the final `echo 'OK: fib assertions passed'` runs. The driver checks `did_emsg` — never set, since nothing errored — and exits zero. On the way, the compiled `CompiledProgram` was written to the rkyv script cache: the next run of `fib.vim` will `mmap` the shard, validate its header, check the source and binary mtimes, and — finding both unchanged — decode the cached chunk and skip lexing, parsing, and lowering entirely, while the JIT disk cache supplies the native code for `FibIter`'s loop so even the re-JIT is skipped.

What the trace shows

Trace the same script with `VIMLR5_NO_JIT=1` and `--disasm` and the whole machine is laid bare: the disassembler prints `FibIter`'s rotated loop as its `Op::GetSlot`, `Op::Add`, `Op::NumLt`, and `Op::JumpIfTrue` instructions, the `Fib` body as its `CallBuiltin VIML_*` sequence, and the `<lambda>`-free program as a `main` chunk with the two function sub-chunks nested beneath it. Drop the `VIMLR5_NO_JIT` and the same `FibIter` loop runs fifteen to a hundred times faster, because those `Op::Adds` became native adds. That single small script has exercised the lexer's literal decoding, the parser's ladder and block parsing, the compiler's slot planner and loop rotation, the bridge's handler dispatch, the JIT's trace compilation, the ported arithmetic and comparison, the assert framework, the exit-code discipline, and the cache — every stage this book described, composed into one run. That composition is vimlr5: faithful semantics ported from Neovim's C, lowered by a synthesis frontend to a borrowed bytecode machine, cached and JIT-compiled for free, running a `.vim` file as an ordinary program. A language that spent its whole life inside an editor, lexed, parsed, compiled, traced to native code, and exited with a status — and every step of it grounded in real source you can open and check.

What the script proves

It is worth ending on what that single traced run actually demonstrates, because it is the thesis of the whole book in miniature. It proves, first, that the eval engine is separable from the editor: a `.vim` file ran with no buffer drawn, no status line, no insert mode — just a process that read a file and exited with a code. It proves, second, that the separation can be faithful: the arithmetic was `num_divide` and `typval_arith`, the

comparison `typval_compare`, the assertions `f_assert_equal` with `eval.lua` wording, every operation a ported function you can open in `csrc/` and check against. It proves, third, that the faithful semantics can be fast: the loop the slot planner proved numeric compiled to native machine code through a JIT vimlrs does not own, fifteen to a hundred times faster than the interpreter, with 64-bit integers exact to the last digit. And it proves, fourth, that the result is durable: the compiled program went into a cache that a later run will restore without recompiling, and could be baked into a self-contained binary that carries no interpreter at all. Separable, faithful, fast, durable — the four claims of Chapter 1, demonstrated by one small Fibonacci script run end to end. Everything between this chapter and that one was the elaboration: the language's two layers, its scopes and types and functions, its regex dialect, its faithful-port discipline, its borrowed machine, its standalone contract, its catalogue, and its examples. But the core is this one run, and the core is enough: Vimscript, taken out of the editor it was captive to, ported honestly from the C that defines it, lowered to a shared machine that makes it fast, and run as an ordinary program — cached, compiled, debuggable, and grounded, every step, in real source you can open and check.

What it took, and what it means

Standing back from the whole book, it is worth naming what kind of achievement vimlrs is, because it is an unusual one and the unusualness is instructive. It is not a research breakthrough — it invents no new language, no new compilation technique, no new type system. It is not a feat of raw engineering scale — it is the work of one person, building on a shared engine they also wrote. What it is, is a feat of *discipline* applied to a clear idea. The idea is simple: Vimscript's eval engine is separable from the editor, so separate it, faithfully, and run it on a machine that already exists. The discipline is what makes the simple idea into a working thing: the refusal to reimplement from documentation, porting from the C instead; the citations that make every port checkable; the drift gate that enforces the zone boundary; the EXTENSION banners that keep synthesis from masquerading as port; the self-testing examples that turn every claim into a CI-checked assertion; the per-operator checking that lets the JIT be fast without being unfaithful. None of these is clever in the sense of being hard to think of. All of them are hard to *sustain*, because each is a constraint that makes the easy path unavailable and the faithful path mandatory, day after day, function after function. vimlrs is what you get when a clear idea is pursued with that much discipline, and the discipline is the achievement.

What it means is harder to state but worth attempting. A language that lived its whole life inside one application has been shown to be a real language — not a configuration format, not editor glue, but a language with a frontend, a compiler, a JIT, a debugger, a deployment story, and a standard library of hundreds of functions, all running outside the application that birthed it. The showing is the meaning. It demonstrates that the captivity was contingent, not essential; that the eval engine was always a real interpreter wearing an editor as a costume; that the line between “the language” and “the editor” was always there to be drawn, and could be drawn cleanly. For the people who know Vimscript — and there are a great many of them — it means their knowledge was knowledge of a real language all along, and now they can use it as one. For the broader question of what a captive language is, it is a worked example of liberation: take the engine out, port it honestly, lower it to a shared machine, and the captive turns out to have been a real thing the whole time. That is what the one small Fibonacci run, traced end to end, finally proves — and it is grounded, every step, in real source you can open and check, which is the only kind of proof this book ever offered or this project ever wanted.

Appendix A – Variable Scopes and Sigils

Vimscript names carry their scope and kind in a prefix sigil. This appendix collects the ones vimlrs recognizes, drawn from the lexer (which absorbs the single-letter scope prefixes), the AST (which enumerates the `:let` targets), and the examples that exercise them. A bare scope sigil — `g:`, `a:`, and so on with nothing after it — is itself a Dict of every variable in that scope.

The eight variable scopes (recognized by the lexer as name prefixes):

- `g:` — **global**. Visible everywhere; always real in standalone vimlrs.
- `l:` — **function-local**. Exists only inside a function. A bare name inside a function is `l:`, which is the precondition for trace-JIT-compiling a numeric loop over it.
- `a:` — **arguments**. Inside a function, `a:n` is the parameter `n`; `a:000` is the List of varargs; `a:0` is their count. Bare `a:` is the Dict of arguments.
- `s:` — **script-local**. Private to the file that declared it; meaningful in a standalone interpreter because a sourced file has a script identity.
- `b:` — **buffer-local**. Editor-tied; reached via the scoped-var getters, which return their `{def}` fallback when standalone.
- `w:` — **window-local**. Editor-tied; same getter/setter contract as `b:`.
- `t:` — **tab-page-local**. Editor-tied; same contract.
- `v:` — **Vim-predefined**. Holds the special predefined variables (Appendix B).

The non-scope sigils (other lvalue and rvalue prefixes):

- `&name` — an **option**, e.g. `&ignorecase`. A `:let &opt = ...` target and an `&opt` rvalue.
- `$NAME` — an **environment variable**. `$X` reads it as a string (empty if unset), `:let $X = v` sets it, `getenv()/setenv()` are the functional forms.
- `@x` — a **register**. A `:let @x = ...` target and an `@x` rvalue.

The `:let` lvalue forms (from the AST's `LetTarget`):

- `let x = ...` / `let g:x = ...` — a plain (possibly scoped) variable.
- `let &opt = ...` — an option.
- `let $ENV = ...` — an environment variable.
- `let @r = ...` — a register.
- `let l[i] = ...` / `let d['k'] = ...` / `let d.k = ...` — an element or member, with the container being any expression (so `d['a']['b']` nests).
- `let [a, b] = list` / `let [a, b; rest] = list` — list-unpack, optionally with a trailing remainder.
- `let l[i1:i2] = list` — a list-range assignment.

Compound assignment desugars to `target = target op rhs`: `+=`, `-=`, `*=`, `/=`, `%=`, and the string-concatenation `.=`.

Appendix B — Special v: Variables

The `v:` scope holds variables the engine predefines or maintains. vimlrs's *documented v:* set in the reference is small — three constants — but the engine maintains several more dynamically, which the examples read. This appendix lists both, separating what the reference documents from what the source and examples show the engine using. Where a variable is engine-maintained rather than reference-documented, it is marked.

Documented constants (the three in [reference.html](#) / [builtin_docs.rs](#)):

- `v:true` — Boolean true. Numeric value 1, string value "`v:true`".
- `v:false` — Boolean false. Numeric value 0, string value "`v:false`".
- `v:null` — Null value. Behaves like an empty string in most contexts.

Engine-maintained specials and dynamics (present in the source and exercised by the examples; not all are in the documented three-entry reference list):

- `v:none` — a Special “empty” value distinct from `v:null`. `string(v:none)` is '`v:none`', `type(v:none)` is 7, `empty(v:none)` is 1 (see [special_none.vim](#)).
- `v:val` — inside `map()/filter()` string-expression bodies, the current element (e.g. '`v:val % 2 == 0`').
- `v:key` — inside `map()/filter()` over a Dict, the current key.
- `v:errors` — the List the `assert_*` builtins append failure messages to; the basis of the self-test epilogue.
- `v:exception` — inside a `:catch`, the text of the exception being handled.
- `v:throwpoint` — the location an in-flight exception was thrown from.
- `v:shell_error` — the exit status of the last `system()/systemlist()`.
- `v:colornames` — a Dict mapping a colour name to its "`#rrggbb`" hex, empty at startup and populated at runtime by the `colors/lists/*.vim` files. The binding is read-only (you cannot reassign it), but its contents are writable via indexed assignment or `extend()` (see [colornames.vim](#)).
- `v:count`, `v:version`, `v:hlsearch`, `v:searchforward` — further engine values referenced in the source.

A practical note for accuracy: the *reference manual* documents only `v:true`, `v:false`, and `v:null`. The others above are real — the examples assert on them — but they are maintained by the engine rather than listed as documented constants, so treat the three-entry reference list and this fuller working set as two different things.

Appendix C – The Builtin Function Catalogue

vimlrs's builtin surface is ported incrementally from Neovim's `eval/funcs.c`, and the published [reference.html](#) — generated from `builtin_docs.rs`, the same corpus `vimlrs --lsp` serves — groups the implemented functions into categories. As of v0.1.2 the reference reports **463 entries across 11 chapters**: 439 builtin functions in eight functional categories, plus 6 ex-commands, 15 options, and 3 documented `v:` constants. This appendix lists the function names by category, so you can see the shape of the implemented surface at a glance. Names are reproduced exactly, including mixed case (`hlID`, `synIDattr`, `diff_hlID`). Many editor-touching entries are marked “stub in standalone” in the reference, meaning they return the documented empty value (Chapter 18) rather than driving an editor.

String manipulation (56): `byteidx`, `byteidxcomp`, `char2nr`, `charclass`, `charidx`, `digraph_get`, `digraph_getlist`, `digraph_set`, `digraph_setlist`, `escape`, `expandcmd`, `fnameescape`, `gettext`, `iconv`, `join`, `keytrans`, `list2str`, `match`, `matchbuflin`, `matchend`, `matchfuzzy`, `matchfuzzypos`, `matchlist`, `matchstr`, `matchstrlist`, `matchstrpos`, `nr2char`, `printf`, `repeat`, `sha256`, `shellescape`, `soundfold`, `split`, `str2float`, `str2list`, `str2nr`, `strcharlen`, `strcharpart`, `strchars`, `strdisplaywidth`, `strgetchar`, `stridx`, `string`, `strlen`, `strpart`, `strridx`, `strtrans`, `strutf16len`, `strwidth`, `submatch`, `substitute`, `tolower`, `toupper`, `tr`, `trim`, `utf16idx`.

List manipulation (22): `add`, `blob2list`, `copy`, `count`, `deepcopy`, `flatten`, `flattennew`, `foreach`, `index`, `indexof`, `insert`, `list2blob`, `mapnew`, `max`, `min`, `range`, `reduce`, `remove`, `reverse`, `slice`, `sort`, `uniq`. (Note `map` and `filter` are catalogued under Functional & Misc; this category lists `mapnew`, the non-mutating variant.)

Dictionary manipulation (9): `dictwatcheradd`, `dictwatcherdel`, `extend`, `extendnew`, `get`, `has_key`, `items`, `keys`, `values`.

Variables & Type (8): `empty`, `exists`, `funcref`, `function`, `garbagecollect`, `islocked`, `len`, `type`. (`type({expr})` returns the numeric type code: 0 Number, 1 String, 2 Func, 3 List, 4 Dict, and so on.)

Number & Float (13): `abs`, `and`, `atan2`, `float2nr`, `fmod`, `invert`, `isinf`, `isnan`, `or`, `pow`, `rand`, `srand`, `xor`. (The bitwise builtins `and/or/xor/invert` live here; their integer forms lower to native fusevm ops so bit-manipulation loops trace-JIT.)

Files, Buffers & Lines (77): `append`, `appendbuflin`, `browse`, `browsedir`, `bufadd`, `bufexists`, `buflisted`, `bufload`, `bufloaded`, `bufname`, `bufnr`, `bufwinid`, `bufwinnr`, `byte2line`, `changenr`, `charcol`, `chdir`, `cindent`, `col`, `delete`, `deletebuflin`, `executable`, `exepath`, `expand`, `filecopy`, `filereadable`, `filewritable`, `finddir`, `findfile`, `fnamemodify`, `getbufinfo`, `getbuflin`, `getbufoneline`, `getbufvar`, `getchangelist`, `getcwd`, `getfontname`, `getfperm`, `getfsize`, `getftime`, `getftype`, `getline`, `glob`, `glob2regpat`, `globpath`, `haslocaldir`, `indent`, `isabsolutepath`, `isdirectory`, `last_buffer_nr`, `line`, `line2byte`, `lispindent`, `mkdir`, `nextnonblank`, `pathshorten`, `prevnonblank`, `readblob`, `readdir`, `readfile`, `rename`, `resolve`, `setbuflin`, `setbufvar`, `setfperm`, `setline`, `simplify`, `swapfilelist`, `swapinfo`, `swapname`, `tempname`, `undofile`, `undotree`, `virtcol`, `virtcol2col`, `winbufnr`, `writefile`. (The filesystem functions — `readfile`, `writefile`, `delete`, `tempname`, `glob`, `mkdir`, `getcwd` — are real; the buffer-numbered functions are largely stubs in standalone.)

System & Environment (44): `chanclose`, `chansend`, `environ`, `getenv`, `getpid`, `hostname`, `jobpid`, `jobresize`, `jobstart`, `jobstop`, `jobwait`, `json_decode`, `json_encode`, `libcall`, `libcallnr`, `localtime`, `luaeval`, `msgpackdump`, `msgpackparse`, `perleval`, `py3eval`, `pyeval`, `pyxeval`, `reltime`,

reltimefloat, reltimestr, rpcnotify, rpcrequest, rpcstart, rpcstop, rubyeval, serverlist, serverstart, serverstop, setenv, sockconnect, stdioopen, stdpath, strftime, strptime, system, systemlist, termopen, windowsversion. (system/systemlist, environ, getenv/setenv, json_*, msgpack*, reltime*, strftime/strptime, and sha256 are real; the rpc/job/server functions are standalone stubs.)

Functional & Misc (210, the catch-all): includes the assert family (`assert_equal`, `assert_notequal`, `assert_true`, `assert_false`, `assert_match`, `assert_notmatch`, `assert_report`, `assert_inrange`, `assert_exception`, `assert_equalfile`); the higher-order `map`, `filter`, `call`, `eval`, `execute`, `foldtext`; argument-list (`argc`, `argidx`, `argv`, `arglistid`); `has`, `exists`-adjacent introspection; the `get*/set*` editor-state queries (`getpos`, `getcurpos`, `getreg`, `setreg`, `getwinvar`, `setwinvar`, `gettabvar`, `settabvar`, `getbufvar`-family); search (`search`, `searchpos`, `searchpair`, `searchcount`); marks, signs (`sign_define`, `sign_place`, ...), matches (`matchadd`, `matchdelete`, `getmatches`), folds, syntax (`synID`, `synIDattr`, `synIDtrans`), windows (`winnr`, `winheight`, `winwidth`, `win_getid`, ...), tabs, timers (`timer_start`, `timer_stop`, ...), history (`histadd`, `histget`, ...), completion (`complete`, `complete_info`, `getcompletion`), input (`input`, `inputlist`, `confirm`), and the test helpers (`test_garbagecollect_now`, `test_write_list_log`). Most of the editor-state functions here are stubs returning the documented empty value standalone; the assert family, `map/filter/call/eval/execute`, `has`, and the argument-list functions are fully real.

Ex Commands (6): `:echo`, `:echon`, `:echomsg`, `:let`, `:call`, `:eval`. (These are the ex-commands the reference documents with hover text; the parser recognizes many more — `:if`, `:while`, `:for`, `:function`, `:delfunction`, `:try`, `:execute`, `:source`, `:runtime`, `:unlet`, `:set`, `:const`, the `:map` family, `:command`, `:autocmd`, the vim9script `def/var/const` forms, and the no-op editor commands (`:nohlsearch`, the `:fold` family, `:python/:``ruby/...`) — but the LSP-doc corpus covers the six above.)

Options (15): `ignorecase` (ic), `smartcase` (scs), `magic`, `expandtab` (et), `number` (nu), `relativenumber` (rnu), `wrap`, `hlsearch` (hls), `incsearch` (is), `autoindent` (ai), `tabstop` (ts), `shiftwidth` (sw), `softtabstop` (sts), `textwidth` (tw), `scrolloff` (so). Each is settable via `:set` or the `<opt>` sigil; `'ignorecase'` is wired into the regex engine so a bare `=~` honours it.

Special v: Variables (3): `v:true`, `v:false`, `v:null` (see Appendix B for the fuller engine-maintained set).

Appendix D — Glossary

AOT (ahead-of-time build). `vimlrs --build` bakes one or more `.vim` scripts into a copy of the binary as a zstd-compressed trailer (magic `VIMLRAOT`), producing a self-contained executable; `--build --native` instead AOT-compile to native machine code via `fusevm::aot` and the `libvimlrs.a` staticlib.

block-JIT. Compilation of an integer expression block to native machine code (per the README; the report names only “three-tier Cranelift JIT” without naming tiers).

Blob. A byte-buffer value, written `0z` followed by hex pairs (`0z00112233`). Indexes to byte values, slices by byte, converts to/from a List via `blob2list/list2blob`.

carve-out / synthesis zone. Crate-root modules with no `csrc/` counterpart — the lexer, parser, AST, bytecode compiler, fusevm bridge, regex engine, JSON parser — each marked “EXTENSION — NO `csrc/` COUNTERPART.” The opposite of the `ported` zone.

csrc/. The vendored Neovim C eval sources (`eval.c`, `ex_eval.c`, ...) that the port mirrors. “The C source is the spec.” Excluded from the crate build.

Dict. An insertion-ordered map (backed by `indexmap`), matching Vim’s observable hashtable iteration and giving deterministic `string()/echo` output.

did_emsg. The ported Vim flag set when an error is raised; the CLI driver exits non-zero when it is set after a run.

drift gate. The test `ported_fn_names_match_c.rs` that fails the build if any `fn` in the `ported` zone has a name not traceable to vendored Neovim C (or an explicit allowlist). The “immune system against porting drift.”

eval1...eval7. The Neovim precedence-ladder functions that define expression grammar, from the ternary (`eval1`) down to the unary leaders and primary atoms (`eval7`). `vimlrs`’s parser reproduces the ladder.

Funcref / Partial. A reference to a function as a value (`function('name')`), and a Funcref with leading arguments pre-bound (`function('name', [args])`).

fusevm. The shared, language-agnostic bytecode VM with a three-tier Cranelift JIT that hosts `vimlrs` (as the 4th language), `zshrs`, `stryke`, and `awkrs`. `vimlrs` has no VM or JIT of its own.

lambda. An anonymous function `{args -> body}`, desugared to a generated `<lambda>N` function; captures enclosing locals (`a:/l:/bare`) by value and resolves dynamic scopes (`g:`, `s:`, ...) at call time.

magic / very-magic. Vim’s default regex mode (some metacharacters bare, some backslashed) and the `\v` mode (all punctuation is an operator unless backslashed).

ported zone. `src/ported/` — strict 1:1 ports of Neovim C, with exact names and `// c:NNN` citations, mirroring `csrc/` file by file.

rkyv script cache. The versioned, mmap-zero-copy bytecode cache at `~/.cache/vimlrs/scripts.rkyv` that lets the second run of a script skip lex/parse/compile.

slot_plan. The compiler pass that assigns VM slots to locals it can prove are Numbers, the precondition for a numeric loop to trace-JIT-compile.

trace-JIT. Compilation of a hot numeric loop (provably-Number locals, condition rotated to the backedge) to native machine code; the main source of `vimlrs`’s loop speed.

typval_T. The ported Vim value type, the substrate the engine was built on first. The fusevm bridge converts between `typval_T` and `fusevm::Value`.

v:errors. The List the `assert_*` builtins append failure messages to; inspected by every example's self-test epilogue, which throws if it is non-empty.

VIML_* builtins. The handlers the bytecode compiler emits for every operator; each pops operands and calls the canonical port in `crate::ported::*`, keeping faithful semantics out of the synthesis-zone compiler.

call_func. The generic bridge wrapper that dispatches every builtin-function id to its ported `f_*`, pre-initializing the return value to 0 and arity-checking before the call — the reason hundreds of builtins need no per-function bridge code.

case flag. The `#/?` suffix on a comparison or regex-match operator forcing case-sensitive or case-insensitive matching; carried by the lexer's `CaseFlag` from token to AST to the ported comparison.

deopt fallback. The slow path the JIT falls back to when an operation is not provably numeric: the bytecode keeps it as a `CallBuiltin` to the `VIML_*` builtin, which routes to the faithful port.

heredoc. The `:let var =<< [trim] MARKER` form that reads verbatim lines into a List of strings, captured before bar-splitting so the body may contain bars and quotes freely (mirrors `heredoc_get()`).

indexmap. The crate `vimlrs` uses for its Dict so that iteration follows insertion order, matching Vim's observable hashtable order and keeping `echo/string()` output deterministic.

refpool. The thread-local pool in the bridge that holds compound `typval_T` values; the VM stack carries a small reference handle in their place, so a List or Dict can flow through `fusevm` without the VM understanding Vim's value types.

rotated loop. The compiler's loop layout — jump to the condition, body first, condition as the conditional backedge — chosen so the only backward branch is the test, the single-backed shape `fusevm`'s tracing JIT records.

slot. A fast VM register a provably-Number bare local is assigned to by the slot planner; slot reads and writes lower to native `Op::GetSlot/SetSlot` the JIT can compile, where a dict-backed variable cannot.

sourcing-name stack. The stack of currently-sourced script paths the engine maintains, read by `expand('<sfile>')` and bound to the `s:` script scope, so nested `:source` calls each have their own identity.

stub. A not-yet-ported Neovim C function scaffolded under `src/ported/stubs/` with its real C name and a `csrc/` citation; also, an editor builtin that returns the documented empty value standalone, labelled “stub in standalone” in the reference.

Appendix E – Ex-Command and Statement Reference

The reference manual documents six ex-commands with hover text, but vimlrs's parser recognizes a much larger statement vocabulary, enumerated in the AST's `Stmt` type and the README's status table. This appendix lists the statements vimlrs parses and runs, grouped by purpose, so you can see the command surface that backs the expression language. Each maps to a `Stmt` variant.

Output:

- `:echo {expr}...` — evaluate and display space-separated expressions, with a trailing newline.
- `:echon {expr}...` — like `:echo` but with no trailing newline.
- `:echormsg {expr}...` — display and save in the message history.

Binding and unbinding:

- `:let {target} = {expr}` — assign to a variable, option (`&`), environment variable (`$`), register (`@`), element/member, list-unpack, or list-range target (Appendix A). Supports the compound operators `+= -= *= /= %=`, `.`, `=` and the heredoc form `=<< [trim] MARKER`.
- `:const {name} = {expr}` — assign like `:let` (the immutability lock is not yet enforced in vimlrs).
- `:unlet[!] {name}...` — remove one or more variables, list items, or dict entries; the `!` form does not error on a missing name.

Calling and evaluating:

- `:call {func}({args})` — call a function, discarding the return value.
- `:eval {expr}` — evaluate an expression for its side effects.
- `:execute {expr}...` — concatenate the values and run the result as an ex command line.

Control flow:

- `:if {cond} / :elseif {cond} / :else / :endif` — conditional.
- `:while {cond} / :endwhile` — loop while a condition holds.
- `:for {var} in {expr} / :endfor` — iterate; `{var}` may be a name or a `[a, b]` unpack.
- `:break / :continue` — loop control.
- `:try / :catch [/pat/] / :finally / :endtry` — structured exceptions.
- `:throw {expr}` — raise an exception value.
- `:return [{expr}]` — return from a function.

Definition and sourcing:

- `:function[!] {name}({args}) / :endfunction` — define a function; `!` replaces an existing definition; parameters may include defaults and `...` varargs. A `g:`-prefixed definition (`:function g:Foo()`) registers under the bare name, and a `<SID>/<SNR>` marker (case-insensitive) names a script-local function.
- `:delfunction[!] {name}` — remove a user function from the registry; the `!` form does not error on a missing name. Accepts a bare, `g:-`, or `s:`-scoped name and dispatches as a statement so it works inside an `:if` block.
- `:source {file}` — read and run another `.vim` file in its own private `s:` script scope, so a nested source's script-local variables neither leak into nor clobber the sourcing file's.

- `:runtime[!]` `{file}` — source `{file}` found on the `'runtimepath'` option; the plain form sources the first match, the `!` form (with a wildcard) sources every match, honouring a live `set runtimepath+=DIR`.
- `:set {args}` — set options (Appendix C lists the 15 supported).

Editor-facing commands (parsed; behaviour follows the editor-absent contract, and some are planned rather than complete):

- The `:map` family — `:nmap`, `:inoremap`, `:vunmap`, `:mapclear`, and so on.
- `:command[!]` / `:delcommand` — define and delete user commands.
- `:autocmd[!]` / `:augroup` / `:doautocmd` — autocommands (the README marks `:command`/`:autocmd` as planned).
- A `:/%-`prefixed Ex command line with a line range (`:%s/...`, `:1,3d`) — parsed and resolved against the (virtual) buffer at run time.

Parsed as no-ops (recognized as Ex commands so a runtime file loads clean, but with no editor to act on they do nothing — the alternative is a fall-through to expression evaluation that raises [E121/E492](#)):

- `:nohlsearch` (down to `:noh`) — clear search-match highlighting.
- `:fold` / `:foldopen[!]` / `:foldclose[!]` — the fold-view commands, over a range.
- `:python[3]` / `:py3` / `:ruby` / `:perl` / `:lua` / `:tcl` / `:mzscheme` and their `do/file` variants — the script-language interface commands; the embedded interpreter is absent, and a `{:lang} << MARKER ... MARKER` heredoc body is skipped entirely rather than mis-parsed as Vimscript.

vim9script (the newer Vim 9 dialect, parsed — see Chapter 6):

- `:vim9script` — mark the file as vim9script, switching the parser into vim9 mode (bare literals, bare-key dicts, automatic line continuation). `has('vim9script')` returns 1.
- `def {name}({params}): {type} / enddef` — a vim9 function; parameters are bare (no `a:`) and `:type` annotations are parsed and discarded.
- `var / final / const {name}[: type] = {expr}` — vim9 declarations; a type-only `var x: T` default-opts to `T`'s zero value, and a bare `x = expr` / `x += expr` / `x .= expr` reassigns without a keyword.
- `export def` / `export var` / `export const` — the `export` modifier; editor-less the marker has no runtime effect, but the definition still registers.

Appendix F – The Vim Regex Dialect

vimlrs's regex engine (`viml_regex.rs`) reproduces Vim's documented pattern behaviour rather than porting Neovim's regex VM. This appendix is a quick reference to the dialect as supported, drawn from the engine's own header and the regex example files. All patterns are conventionally written in single quotes so backslashes survive string parsing.

Modes:

- Default (“magic”) mode — `.`, `*`, `[...]`, `^`, `$` are special bare; `\+`, `\?`, `\=`, `\(`, `\)`, `\|`, `\{n,m}` need a backslash.
- `\v` (“very magic”) — every ASCII punctuation character is an operator without a backslash; a backslash makes it literal. Applies to the rest of the pattern.
- `\c` / `\C` — force case-insensitive / case-sensitive matching, overriding the caller's `'ignorecase'`.

Atoms:

- `.` — any character. `^` / `$` — start / end of line.
- `[...]` / `[^...]` — character set / negated set, with ranges.
- Class atoms: `\d` `\D` (digit), `\w` `\W` (word), `\s` `\S` (space), `\a` (alpha), `\l` (lower), `\u` (upper), `\x` (hex) and their negations.
- `\<` `\>` — start-of-word / end-of-word boundaries (in `\v`, bare `<` / `>`).

Quantifiers:

- `*` — zero or more. `\+` — one or more. `\?` / `\=` — zero or one.
- `\{n,m}` — between *n* and *m*. `\{-n,m}` — the non-greedy form.

Groups and alternation:

- `\(...\)` — capturing group (bare `(...)` in `\v`).
- `\%(...\)` — non-capturing group.
- `\|` — alternation (bare `|` in `\v`).
- `\1..\9` — backreference to a captured group (verified working by `regex_backref.vim`).

Zero-width match-boundary and sequence atoms:

- `\zs` — set where the *reported* match starts (consumes nothing before it).
- `\ze` — set where the *reported* match ends.
- `\%[atoms]` — optional sequence: match the longest in-order prefix of the atoms, each only if the previous matched (the abbreviatable-keyword idiom, e.g. `r\%[ead]`).

The engine backs: `=~`, `!~` (and case-flagged `=~#/=~?`); `match`, `matchend`, `matchstr`, `matchlist`; `substitute` (with the `\=` expression replacement and the `\u/\l/\U/\L` case escapes); pattern `split`; and `:catch /pat/`.

Appendix G – Command-Line Flags and Environment

This appendix collects the `vimlrs` binary's command-line interface (from `cli.rs`) and the environment variables that affect its behaviour (from the README and source). It is the operational quick-reference to complement the language reference.

Input modes (mutually exclusive ways to supply a program):

- `vimlrs FILE.vim [FILE...]` — source one or more script files in order, through the bytecode cache. The files seed `argv()/argc()`.
- `vimlrs -e EXPR / vimlrs --expr EXPR` — evaluate one expression and print its value (like `:echo`).
- `vimlrs -c CMD / vimlrs --cmd CMD` — run one ex command line.
- `vimlrs` (no arguments) — start the REPL on stdin, prompt `vimlrs>`.

Mode-changing flags:

- `--lsp` — run the Language Server Protocol server on stdio (diagnostics, completion, hover, document symbols).
- `--dap` — run the Debug Adapter Protocol server on stdio (breakpoints, stepping, variables, evaluate).
- `--disasm` — print the fusevm bytecode listing before running.
- `-b OUT / --build OUT` — AOT build: bake the input scripts into a self-contained executable at `OUT`.
- `-n / --native` — with `--build`, AOT-compile to native machine code instead of embedding the source.
- `--clear-cache` — delete the rkyv bytecode script cache and exit.

Environment variables:

- `VIMLRS_JIT_STATS=1` — report how many loop traces the JIT compiled, for the run. The way to confirm a loop you expected to trace did.
- `VIMLRS_NO_JIT=1` — force the interpreter baseline, disabling the JIT, for comparison.

Paths and artifacts:

- `~/.cache/vimlrs/scripts.rkyv` — the rkyv bytecode script cache shard (magic "`VIML`", current `format_version` 3).
- `autoload/` — resolved relative to the current directory for `pkg#func()` autoload sourcing.
- `VIMLRAOT` — the 8-byte trailer magic on an AOT-built executable; the original binary bytes before it still run as ordinary `vimlrs`.

Exit status: the process exits non-zero when any error was raised during the run (Vim's `did_emsg` flag set), which is what makes every self-testing example a CI-checkable regression test.

Appendix H – The Source File Map

This appendix maps the crate’s source files to their roles, so you can find the code behind any chapter. The two zones (Chapter 13) are kept visually separate.

Synthesis zone (crate-root carve-outs, each with an `EXTENSION – NO csrc/ COUNTERPART` banner):

- `viml_lexer.rs` — the tokenizer (Chapter 19): source text to a `Tok` stream, with literal decoding, case-flagged comparison tokens, and byte-span recording.
- `viml_ast.rs` — the `Expr` and `Stmt` types (Chapter 20), shaped by the `eval1...eval7` precedence ladder.
- `viml_parser.rs` — the recursive-descent parser and the source splitter (Chapter 20): the precedence ladder, the statement dispatcher, and the heredoc/continuation/ bar pre-passes.
- `compile_viml.rs` — the bytecode compiler (Chapter 21): the slot planner, the loop rotation, the native-range lowering, the lambda generation, and the `VIML_*` emission.
- `fusevm_bridge.rs` — the runtime bridge (Chapter 22), the largest file: value marshalling, the `VIML_*` handlers, `call_func`, the per-run state, and the re-entry hooks.
- `fusevm_disasm.rs` — the `--disasm` pretty-printer (Chapter 22).
- `viml_regex.rs` — the Vim-magic regex engine (Chapter 10): the `Node` AST, the backtracking matcher, and the `\=` substitute hook.
- `viml_json.rs` — the JSON decoder (Chapter 16), a recursive-descent parser yielding the ported `typval_T` tree.
- `aot.rs` — the ahead-of-time builder (Chapter 15): the source-embedding trailer and the native build path.
- `script_cache.rs` — the rkyv bytecode cache (Chapter 15).
- `builtin_docs.rs` — the single-source `(name, signature, doc)` corpus (Chapter 25).
- `cli.rs` — the command-line driver (Chapter 2).
- `lsp.rs` — the Language Server (Chapter 17).
- `dap.rs` — the Debug Adapter (Chapter 17).
- `lib.rs` / `ported.rs` / `main.rs` — the crate root, the ported-zone module, and the binary entry point.

Ported zone (`src/ported/`, strict 1:1 ports with exact C names and `// c:NNN` citations, mirroring `csrc/`):

- `typval.rs` — `eval/typval.c`, the value type and its helpers (the substrate ported first).
- `list.rs` / `dict.rs` / `blob.rs` — the three container files (`dict` insertion-ordered via `indexmap`).
- `eval.rs` / `eval_h.rs` — `eval.c` and `eval.h`, the comparison, arithmetic, and expression-type logic.
- `ex_eval.rs` — `ex_eval.c`, the try/catch/throw and abort state machine.
- `funcs.rs` — `eval/funcs.c`, the builtin table.
- `encode.rs` / `decode.rs` — the `string()`/JSON/msgpack encode and decode paths.
- `charset.rs` / `strings.rs` / `message.rs` / `ops.rs` / `option.rs` / `path.rs` / `profile.rs` / `sha256.rs` / `typval_defs_h.rs` — the supporting C files, each ported as far as the builtins that need it.
- `stubs/` — the generated, cited placeholders for not-yet-ported C functions.

Appendix I – Index of Example Scripts

The crate ships over a hundred and fifty self-testing examples under `examples/`, each run in CI by `tests/examples.rs`. This appendix indexes them by name with a one-line gloss, so you can find the example that pins down any behaviour. They are grouped by theme.

Language core: `fizzbuzz.vim` (control flow + assert), `fib.vim` (recursion + JIT loop), `numbers.vim` (float `%g` + max/min + integer bases), `float_literals.vim` (the dot-required float grammar), `numeric_edge.vim` (`%` is integer-only, charidx safety), `floatmath.vim` (the transcendental library), `string_repr.vim` (`string()` round-trip), `special_none.vim` (`v:null` vs `v:none`), `compare.vim` (case flags + identity), `scopedvars.vim` (the scope dicts + the editor-absent getters), `concat_dot.vim` (the `.` overload), `default_args.vim` (optional parameters), `varargs.vim` (`.../a:000` + `:unlet`), `funcrefs.vim` and `funcref_call.vim` (first-class functions), `lambdas.vim` and `closures.vim` (lambdas + capture).

Data shapes: `lists_dicts.vim` (the container tour), `blobs.vim` (the byte buffer), `literal_dict.vim` (`#{}), list_unpack.vim (:let [a; rest]), index_get.vim (index/get + E1531), slicing.vim (exclusive slice), string_index.vim (str[i]/str[a:b]), strindex.vim (byte/char indexing), strings.vim (the dense string surface), width.vim (display width + charclass), repeat.vim, indexof.vim, reverse.vim.`

Pattern matching: `regex_verymagic.vim` (`\v`), `regex_zs.vim` (`\zs/\ze`), `regex_backref.vim` (`\1`), `regex_optseq.vim` (`\%[...]`), `match_startcount.vim` (the match family's `{start}/{count}`), `matches.vim` (the highlight-match list), `matchstrlist.vim`, `fuzzy.vim` (`matchfuzzy`), `glob2regpat.vim`.

The virtual buffer and Ex layer: `buffer.vim` (line editing), `excommands.vim` (`:s/:g/:d/:m/:t/:j`), `streamedit.vim` (`:sort/:!/:e/:w`), `normal.vim` (a bounded `:normal`), `search.vim` (cursor search), `marks.vim`, `registers.vim`, `cmdline.vim`, `options.vim` (`fopt` + `:%s`), `user_commands.vim` (`:command`), `map_commands.vim` and `mappings.vim` (mappings), `autocommands.vim`, `signs.vim` and `sign_place.vim`, `quickfix.vim`, `history.vim`, `fullcommand.vim`.

The editor-absent contract: `has.vim` (feature detection), `editor_absent.vim`, `editor_queries.vim`, `editor_compat.vim`, `buffers.vim`, `windows.vim`, `highlight.vim`, `completion.vim`, `providers.vim`, `table_complete.vim`, `undo.vim`, `stdpath.vim`, `sysinfo.vim`, `digraphs.vim`, `expand.vim`, `sfile.vim`.

Interop and the outside world: `json.vim`, `msgpack.vim`, `system.vim`, `env_vars.vim`. **Metaprogramming and structure:** `execute_capture.vim`, `sourcing.vim`, `autoload.vim`, `heredoc.vim`, `online.vim`, `exceptions.vim`, `testing.vim`, `beeps.vim`, `equalfile.vim`, `printf_exists.vim`, `builtin_arity.vim`, `error_output.vim`, `interactive.vim`, `wordfreq.vim`, `arglist.vim`. Each is a complete program that asserts its own correctness; reading any one is a guided lesson in the feature it names, and CI checks every one of them on every change.